

파이썬 프로그래밍

(인공지능)

• 머신러닝(Machine Learning) 개요

• 인공지능 (Artificial Intelligence)

사람의 지적능력을 컴퓨터를 통해 구현하는 기술로 하위 개념으로 딥러닝과 머신러닝이 있다.

• 머신러닝 (Machine Learning)

머신러닝은 인간이 데이터 분석의 힌트를 알려준 후 학습해서 추론할 수 있게 하는 기술이다. 비교적 간단한 문제에 적합하며 이미지 분류, 스팸 메일 감지, 주식 시장 예측 등 다양한 분야에 적용된다. 딥러닝에 비해 적은 양의 데이터로도 학습이 가능하다.

• 딥러닝 (Deep Learning)

딥러닝은 인공 신경망 구조를 통해 처음부터 기계가 학습하는 구조로 머신러닝이 쉽게 할 수 없는 이미지 인식, 음성 인식, 자연어 처리 등 주로 복잡하고 비정형 데이터를 분석하는 분야에 적용된다. 머신러닝에 비해 더 많은 양의 필요하며 데이터가 많을수록 성능이 향상된다.

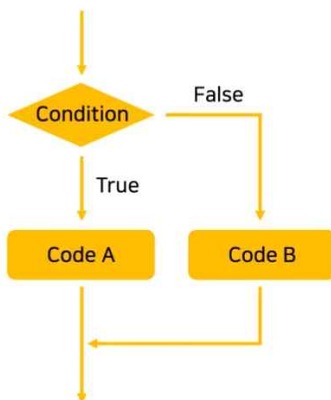
• 머신러닝 구현 예제

예를 들어 중고 스마트폰을 파는 경우 판매자와 구매자 사이의 적절한 가격을 정해야 한다. 가격을 정하는 기준은 기기의 스펙(제조사/모델명/제조 연월/화면크기/CPU 코어/내장메모리/램(RAM)/색상) 또는 상태(파손 여부/스크래치/변인)에 따라 달라진다.

- 사람이 직접 작성한 프로그램으로 가격을 책정할 경우 프로그램 함수는 아래와 같다. 프로그램 함수에 의해 변인이 조금 있고(-50,000) 찍힘은 없지만 생활 스크래치가 있고(-5,000) 배터리는 아직 오래 간다면 245,000원에 가격을 책정할 수 있다.

```
def get_price(최대금액, 액정파손, 변인, 찍힘, 스크래치, 색상, ....):  
    적정금액 = 최대금액 #300000  
  
    if 액정파손 == True:  
        적정금액 -= 150000  
    elif 변인 == True:  
        적정금액 -= 50000  
  
    if 찍힘 >= 3: #3군데 이상  
        적정금액 -= 30000  
    elif 스크래치 == True:  
        적정금액 -= 5000  
  
    if 배터리 < 10 #10시간미만  
        적정금액 -= 20000  
  
    ...  
    return 적정금액
```

- 순서도로 표현하면 아래와 같다. 사람이 정해진 조건에 의해 만족하면 Code A를 실행하고 조건을 만족하지 않으면 Code B를 실행한다.

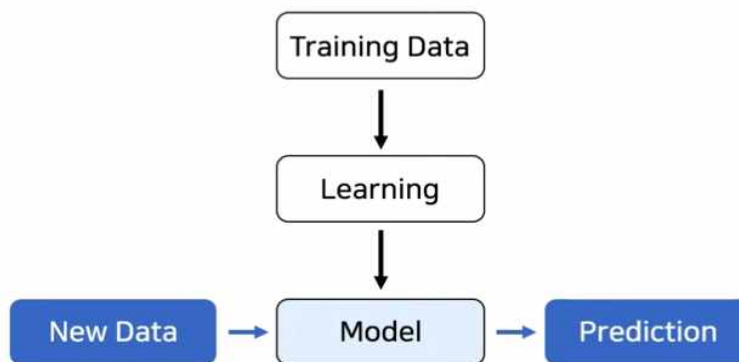


Traditional Approach

- 머신러닝을 통해 가격을 책정할 경우 기계가 아래 주어진 거래 데이터를 분석하여 핸드폰 상태와 거래가격에 상관관계를 찾는 학습을 통해 거래 가격을 스스로 결정한다. 학습 결과에 따라 내 핸드폰은 이러이러한데 적정 판매 가격이 얼마일까? 물어보면 예측 가격을 알려준다.

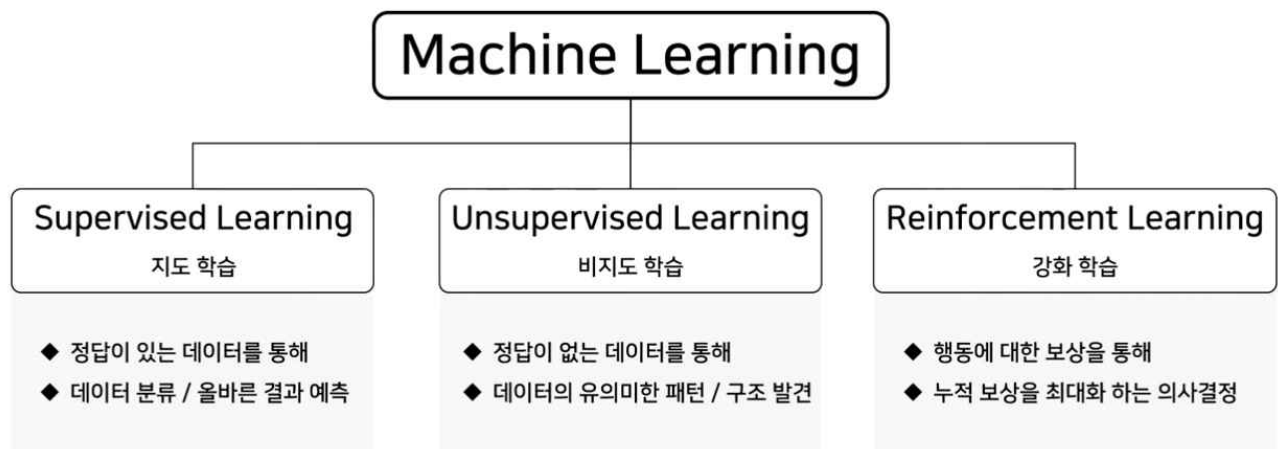
제조사	모델명	제조 연월	내장메모리	램 (RAM)	액정파손	번인	찍힘	생활기스	...	거래가격
A	A1	2021	128	8	X	O	1	O		280,000
A	A1	2021	256	12	X	X	3	O		310,000
A	A1	2021	256	12	O	O	5	O		235,000
B	B1	2020	256	8	X	O	2	O		560,000
B	B2	2022	512	12	X	X	1	X		820,000
...	...	...	...	...	...	...	...	...	...	...

- 그림으로 표현하면 아래와 같다. 훈련 데이터를 제공하면 기계는 데이터를 통해 스스로 학습하여 새로운 모델(프로그램인 경우 함수에 해당)을 생성한다. 이 모델을 이용하여 새로운 데이터를 입력하여 결과를 예측한다.



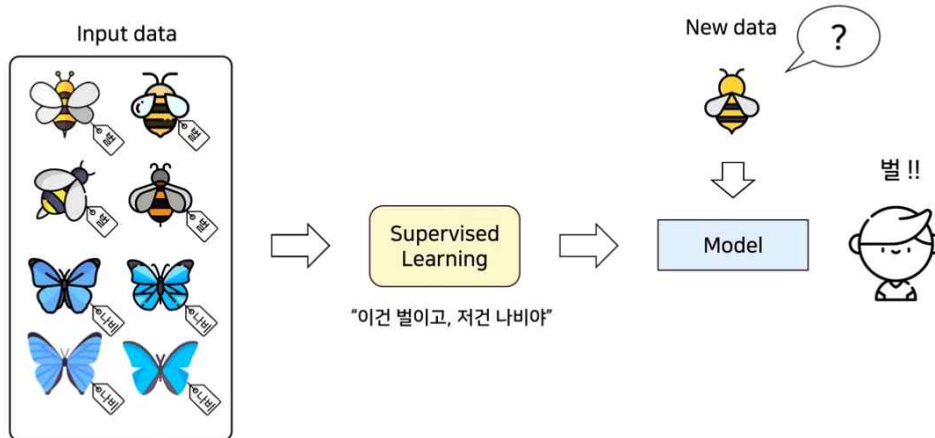
Machine Learning

- 머신러닝은 아래와 같이 지도 학습, 비지도 학습 강화 학습 등 세 가지로 분류할 수 있다.



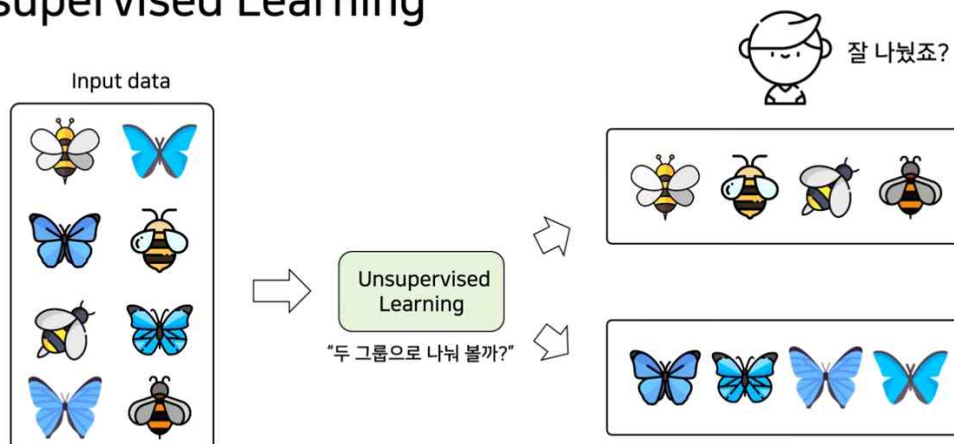
- 지도 학습은 '이건 벌이고, 저건 나비야'라고 이미지와 정답을 입력 데이터로 미리 주면 기계가 스스로 지도 학습을 통해 모델을 생성한다. 새로운 데이터가 입력되면 지도 학습으로 생성된 모델을 통해서 이진 '벌'이라고 예측할 수 있다.

Supervised Learning



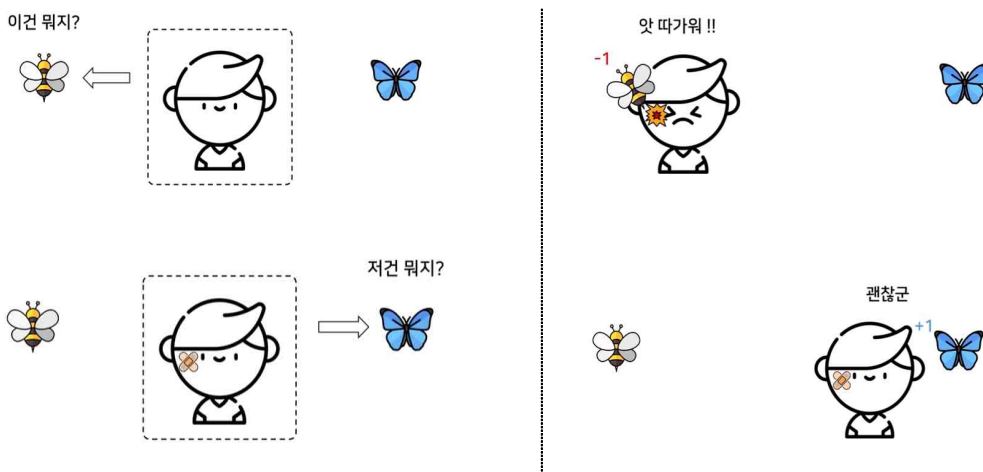
- 비지도 학습은 정답이 없는 그림만 입력 데이터로 주면 기계가 스스로 패턴을 찾아 패턴별로 분류한 후 새로운 데이터를 예측한다.

Unsupervised Learning



- 강화 학습은 입력 데이터에 대해 왼쪽으로 향한 경우 따가워서 벌점으로 -1점, 오른쪽으로 향한 경우 괜찮아서 +1점을 부여하는 행동(경험)을 통해 누적 보상을 최대화하는 방향으로 학습을 한다.

Reinforcement Learning



• 지도학습(Supervised Learning) 종류

지도 학습의 종류로 회귀(Regression) 모델과 분류(Classification) 모델이 있다. 예측할 데이터가 연속 데이터인 경우 회귀모델을 사용하고 범주 데이터(몇 개의 항목 형태로 나뉘어진 자료)인 경우는 분류모델을 사용한다.

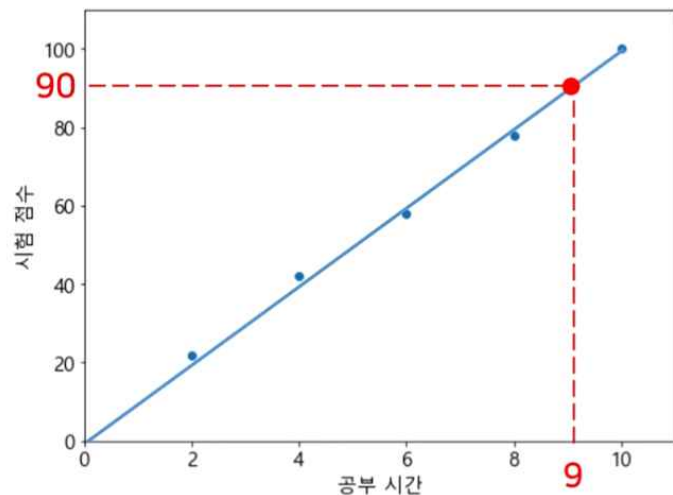
• 회귀(Regression)

회귀란 변수들 사이의 상관관계를 찾는 것, 연속적인 데이터로부터 결과를 예측하는 것으로 예측 결과가 숫자일 때 적합하다. 예를 들어 근무 연수에 따른 임금, 키에 따른 몸무게, 사용 기간에 따른 핸드폰 가격 등이 있다.

- 아래 예제는 주어진 데이터를 분석한 후 상관관계를 갖는 직선의 방정식을 구하여 공부 시간에 대한 점수를 예측하는 회귀 모델이다.

9시간을 공부했을 때 예상 시험 점수는? 90점

공부 시간	시험 점수
2 시간	22 점
4 시간	42 점
6 시간	58 점
8 시간	78 점
10 시간	100 점



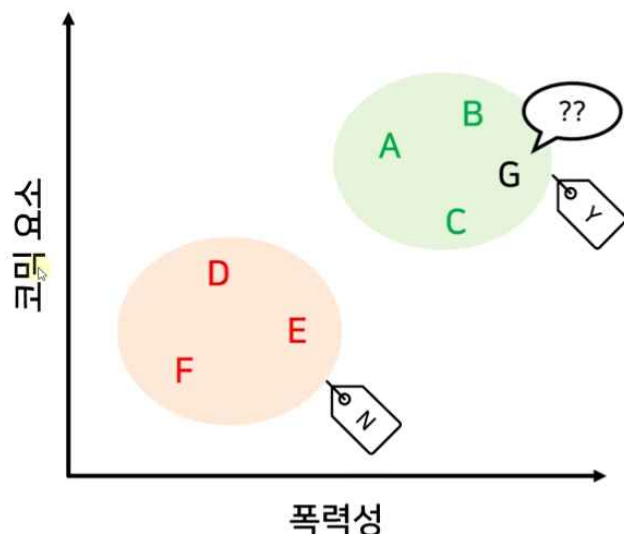
• 분류(Classification)

주어진 데이터를 정해진 범주(category)에 따라 분류하여 예측하는 것으로 예측 결과가 숫자가 아닐 때 적합하다. 예를 들어 스팸 메일 필터링, 시험 합격 여부, 재활용 분리수거 품목, 악성 종양 여부 등이 있다.

- 아래는 코믹과 폭력성을 기준으로 A~F 영화들을 두 그룹으로 나누어 G가 재미있는지? 없는지? 예측해 보는 예제이다. 두 그룹으로 분류한 결과 '좋아요'는 폭력성이 높고, 코믹요소가 많은 영화이고 '좋아요'가 아닌 영화는 폭력성도 낮고, 코믹요소가 적은 영화이다. G 영화는 폭력성이 높고, 코믹요소가 많은 '좋아요' 그룹에 위치하므로 재미있다고 예측할 수 있다.

영화	좋아요
A	Y
B	Y
C	Y
D	N
E	N
F	N

새로운 영화 G는 재미있을까?



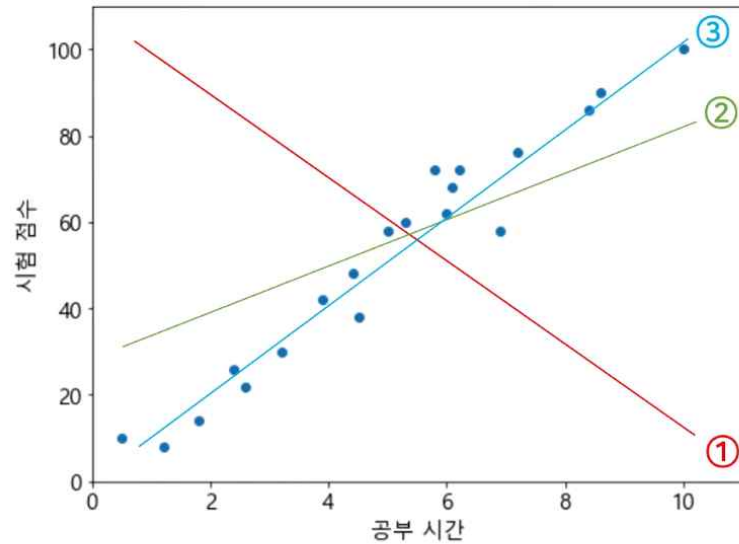
• 단순 선형회귀 (Linear Regression)

• 최소 제곱법

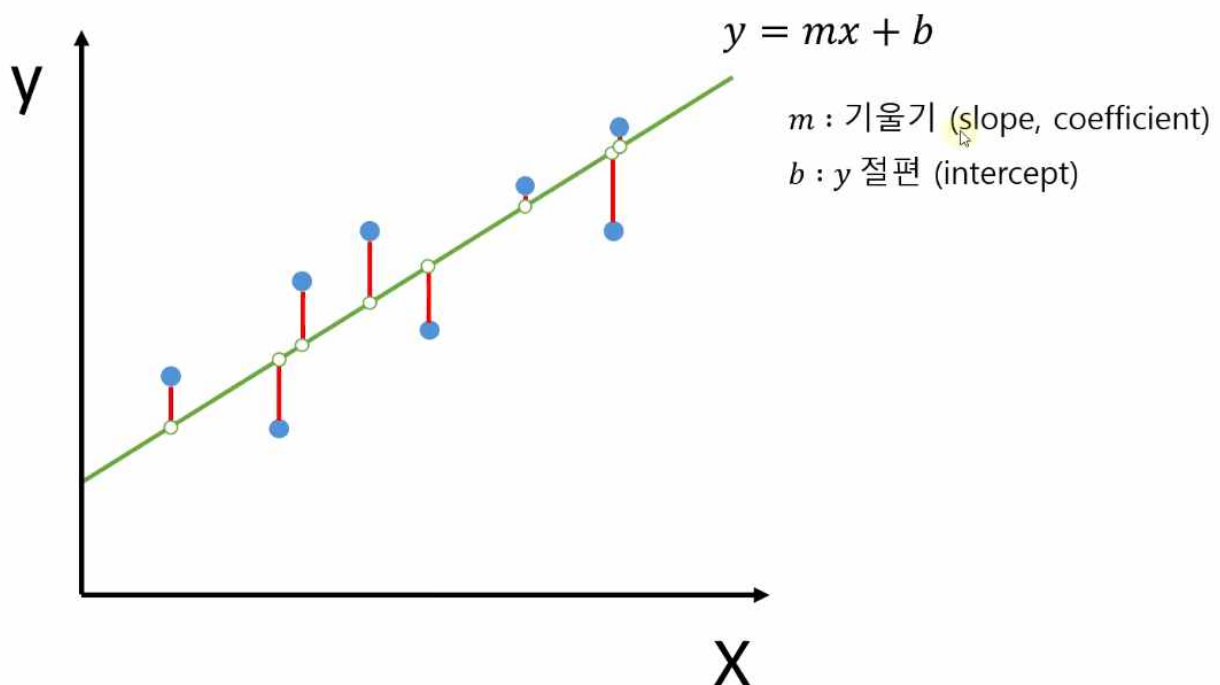
선형회귀란 종속변수(y)와 한 개 이상의 독립변수(X)와의 선형 상관관계를 모델링하는 회귀분석 기법이다.

공부 시간	시험 점수
0.5	10
1.2	8
1.8	14
2.4	26
2.6	22
3.2	30
3.9	42
4.4	48
4.5	38
5	58
5.3	60
5.8	72
6	62
6.1	68
6.2	72
6.9	58
7.2	76
8.4	86
8.6	90
10	100

데이터를 가장 잘 표현하는 직선은?



- 공부 시간: Independent variable(독립변수) 원인(X) = 입력변수, feature
- 시험 점수: Dependent Variable(종속변수) 결과 (y) = 출력변수, target, label
- 최적의 직선: 잔차(실제 값: 파랑 - 예측한 값: 초록)의 제곱의 합이 최소인 직선



- **단순 선형회귀 모델링**

- 새로운 '01.단순선형회귀.ipynb' 파일을 생성한 후 터미널에서 아래의 두 개의 라이브러리를 설치한다.

```
pip install matplotlib
pip install pandas
```

- 시각화를 위한 matplotlib 과 데이터 분석을 위한 pandas 라이브러리를 import 한다.

```
import matplotlib.pyplot as plt
import pandas as pd
```

- 공부 시간에 따른 시험 점수를 저장한 csv 파일을 읽어 저장한다.

```
dataset = pd.read_csv('data/LinearRegressionData.csv')
```

- head() 함수를 이용하여 상위 5개의 데이터가 출력되는지 확인해 본다.

```
dataset.head()
```

	hour	score
0	0.5	10
1	1.2	8
2	1.8	14
3	2.4	26
4	2.6	22

- 데이터 세트에서 독립변수와 종속변수의 값을 저장한다.

```
#독립변수(원인)에 모든 행의 처음 칼럼부터 마지막 칼럼 직전까지의 데이터를 X에 저장한다.
X = dataset.iloc[:, :-1].values
#종속변수(결과)에 모든 행의 마지막 칼럼 데이터를 y에 저장한다.
y = dataset.iloc[:, -1].values
```

- X변수와 y변수 값들을 출력하여 확인 본다. X 값들은 2차원 배열로, y 값들은 1차원 배열로 출력된다.

```
X, y
```

```
(array([[ 0.5],
        [ 1.2],
        [ 1.8],
        [ 2.4],
        [ 2.6],
        ...,
        [ 6.9],
        [ 7.2],
        [ 8.4],
        [ 8.6],
        [10. ]]),
 array([ 10,  8, 14, 26, 22, 30, 42, 48, 38, 58, 60, 72, 62,
        68, 72, 58, 76, 86, 90, 100], dtype=int64))
```

- 인공지능 선형회귀 모델을 생성하기 위하여 아래 라이브러리를 설치한다.

```
pip install scikit-learn
```

- LinearRegression 클래스로 선형회귀 객체 생성한 후 fit 함수로 학습하여 모델을 생성한다.

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression() #모델 객체 생성
reg.fit(X, y) #학습하여 모델생성
```

```
▼ LinearRegression
LinearRegression()
```

- 독립변수(X)에 대한 예측 값(y_pred)을 predict() 함수를 이용하여 예측한 후 출력해 본다.

```
y_pred = reg.predict(X)
y_pred
```

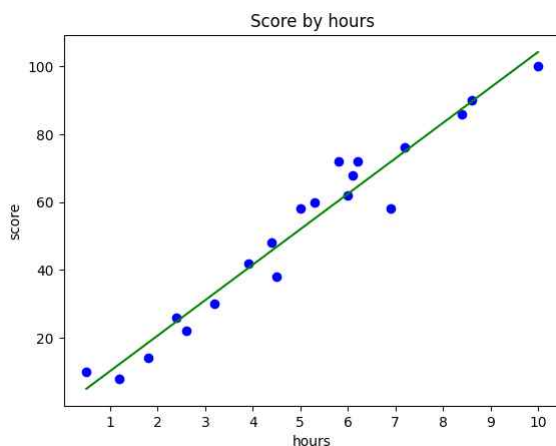
```
array([ 5.00336377, 12.31395163, 18.58016979, 24.84638795,
        26.93512734, 33.20134551, 40.51193337, 45.73378184,
        46.77815153, 52.          , 55.13310908, 60.35495755,
        62.44369694, 63.48806663, 64.53243633, 71.84302419,
        74.97613327, 87.5085696 , 89.59730899, 104.2184847 ])
```

• 단순 선형회귀 시각화 및 결과 예측

matplotlib의 pyplot 패키지를 사용하여 실제 값과 예측 값에 대한 그래프 출력으로 시각화 작업을 해본다.

- 실제 값들은 scatter() 함수를 이용해 산점도 그래프로 예측 값들은 plot() 함수를 이용하여 선 그래프로 출력해 본다.

```
plt.scatter(X, y, color='blue')
plt.plot(X, y_pred, color='green')
plt.title('Score by hours')
plt.xticks([x for x in range(1, 11)])
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 독립변수(X) 데이터가 이차원 배열 형태이므로 값을 2차원 배열 형태로 입력해 준다.

```
print('9시간 공부했을 때 예상 점수: ', reg.predict([[9]]))
```

```
9시간 공부했을 때 예상 점수: [93.77478776]
```

```
print('9, 8, 7시간 공부했을 때 예상 점수: ', reg.predict([[9], [8], [7]]))
```

```
9, 8, 7시간 공부했을 때 예상 점수: [93.77478776 83.33109082 72.88739388]
```

- 선형회귀 모델($y=mx+b$)의 기울기(coefficient) m을 출력한다.

```
reg.coef_
```

```
array([10.44369694])
```

- 선형회귀 모델($y=mx+b$)의 절편(intercept) b를 구한 후 출력한다.

```
reg.intercept_
```

```
-0.218484702867201
```

```
y = 10.4436 * 9 - 0.2184
```

```
y
```

```
93.774
```

• 모델 평가

다음 시험까지 기다리지 않고 기존 데이터를 가지고 모델 평가를 위해 일어난 데이터를 훈련 세트(Train set):80%, 테스트 세트(Test set):20%로 나눈다. 그리고 훈련 세트로 모델을 만들고 테스트 세트로 훈련 모델이 잘 만들었는지 테스트한다.

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
dataset = pd.read_csv('data/LinearRegressionData.csv')
dataset
```

	hour	score
0	0.5	10
1	1.2	8
2	1.8	14
3	2.4	26
...		
16	7.2	76
17	8.4	86
18	8.6	90
19	10.0	100

- **훈련 데이터와 테스트 데이터 분리**

- 독립변수(X)와 종속변수(y)에 데이터 저장한다.

```
X = dataset.iloc[:, :-1].values  
y = dataset.iloc[:, -1].values
```

- `train_test_split()` 함수를 이용해 훈련 세트와 테스트 세트 80%:20%로 분리한다.

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

- 전체 데이터 세트와 개수를 출력해 본다.

X, len(X)

```
(array([[ 0.5],  
       [ 1.2],  
       [ 1.8],  
       [ 2.4],  
       [ 2.6],  
       [ 3.2],  
       [ 3.9],  
       [ 4.4],  
       [ 4.5],  
       [ 5. ],  
       [ 5.3],  
       [ 5.8],  
       [ 6. ],  
       [ 6.1],  
       [ 6.2],  
       [ 6.9],  
       [ 7.2],  
       [ 8.4],  
       [ 8.6],  
       [10. ]]),  
20)
```

- 훈련 세트 데이터와 개수를 출력해 본다. (원본 데이터 중에서 80%를 Random 하게 선택한다.)

X_train, len(X_train)

```
(array([[5.3],  
       [8.4],  
       [3.9],  
       [6.1],  
       [2.6],  
       [1.8],  
       [3.2],  
       [6.2],  
       [5. ],  
       [4.4],  
       [7.2],  
       [5.8],  
       [2.4],  
       [0.5],  
       [6.9],  
       [6. ]]),  
16)
```

- 테스트 세트 데이터와 개수를 출력해 본다. (원본 데이터 중에서 20%를 Random 하게 선택한다.)

X_test, len(X_test)

```
(array([[ 8.6],  
       [ 1.2],  
       [10. ],  
       [ 4.5]]),  
4)
```

- 전체 데이터 종속변수(y)를 출력해 본다.

```
y, len(y)
```

```
(array([ 10,  8, 14, 26, 22, 30, 42, 48, 38, 58, 60, 72, 62,
        68, 72, 58, 76, 86, 90, 100], dtype=int64),
20)
```

- 훈련 세트(y_train)를 출력해 본다.

```
y_train, len(y_train)
```

```
(array([60, 86, 42, 68, 22, 14, 30, 72, 58, 48, 76, 72, 26, 10, 58, 62],
      dtype=int64),
16)
```

- 테스트 세트(y_test)를 출력해 본다.

```
y_test, len(y_test)
```

```
(array([ 90,  8, 100, 38], dtype=int64), 4)
```

• 모델 생성 및 시각화

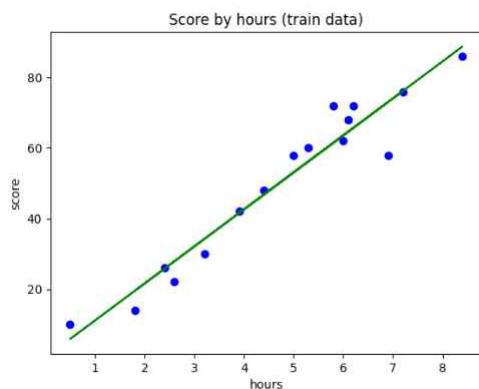
- 분리된 데이터(훈련 세트)를 통해 LinearRegression 객체를 생성하고 학습 후 선형회귀 모델을 생성한다.

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
reg.fit(X_train, y_train)
```

```
▼ LinearRegression
LinearRegression()
```

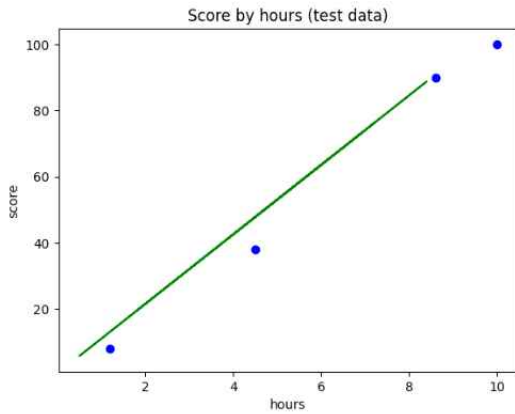
- 훈련 세트의 데이터의 실제 값과 예측 값을 시각화 한다.

```
plt.scatter(X_train, y_train, color='blue')
plt.plot(X_train, reg.predict(X_train), color='green')
plt.title('Score by hours (train data)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 훈련 세트를 이용한 테스트 세트의 실제 값과 예측 값을 시각화 한다.

```
plt.scatter(X_test, y_test, color='blue')
plt.plot(X_train, reg.predict(X_train), color='green')
plt.title('Score by hours (test data)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 모델 기울기와 y절편 결과 확인

- 훈련 모델의 기울기를 출력해 본다.

```
reg.coef_
```

```
array([10.49161294])
```

- 훈련 모델의 y 절편을 출력해 본다.

```
reg.intercept_
```

```
0.6115562905169369
```

- 모델 평가 결과 확인

- 테스트 세트를 통한 모델을 평가해 본다.

```
reg.score(X_test, y_test)
```

```
0.9727616474310156
```

- 훈련 세트를 통한 모델을 score() 함수를 사용해 모델을 평가해 본다.

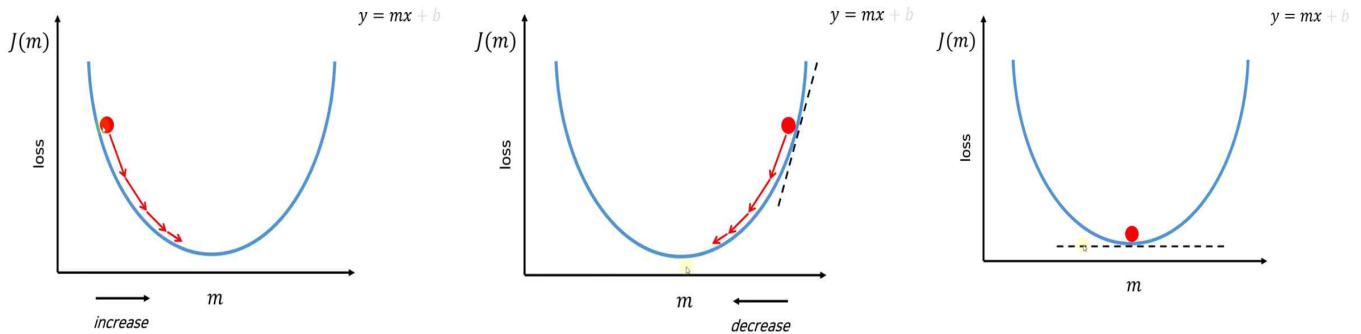
```
reg.score(X_train, y_train)
```

```
0.9356663661221668
```

경사하강법 (Gradient Descent)

경사하강법은 선형회귀 최소제곱법(OLS: Ordinary Least Squares)의 noise(이상 값)에 취약점 보완을 위한 Modeling 기법으로 실제 값과 예측 값과의 loss를 최소화하는 기울기(m)에 값을 찾는 것을 목표로 한다.

- 학습률(Learning rate): 기울기의 보폭으로 너무 크면 가장 낮은 지역을 지나치고 너무 작으면 내려오기 전에 중간에 끝난다. 0.001, 0.003, 0.01, 0.03, 0.1, 0.3 주로 사용한다.
- 에포크(Epoch): 훈련 세트의 모든 데이터를 한 번씩 다 사용해 보는 과정을 의미하며 데이터의 수가 많은 경우 시간이 오래 걸린다.



확률적 경사하강법(Stochastic Gradient Descent)

경사하강법은 전체 데이터에 사용해 기울기를 적용하기 때문에 데이터가 많을수록 시간이 오래 걸리는 단점이 있다. 이런 점을 보완하는 여러 알고리즘 중 하나가 확률적 경사하강법(SGD)이다. SGD는 매번 단계마다 무작위로 추출한 하나의 데이터에만 기울기를 계산한다. 즉, 속도가 훨씬 빠르나 정확도는 전체 데이터에 대하여 가중치를 적용하는 경사하강법에 비하여 떨어진다.

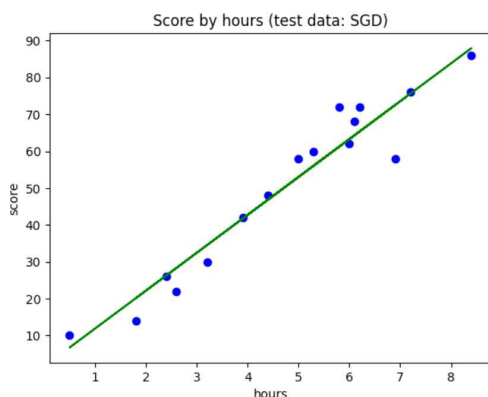
- 확률적 경사 하강(SGD: Stochastic Gradient Descent) 클래스로 객체를 생성하고 훈련 후 모델을 생성한다.

```
from sklearn.linear_model import SGDRegressor
sr = SGDRegressor()
sr.fit(X_train, y_train)
```

```
SGDRegressor()
SGDRegressor()
```

- 훈련 데이터로 시각화 하면 LinearRegression이용한 직선과 거의 유사하다.

```
plt.scatter(X_train, y_train, color='blue')
plt.plot(X_train, sr.predict(X_train), color='green')
plt.title('Score by hours (test data: SGD)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 확률적 경사 하강법 모델의 기울기(10.49161294), y절편(0.6115562905169369)과 비교해 본다.

```
sr.coef_, sr.intercept_
```

```
(array([10.21667615]), array([1.32363048]))
```

- 테스트 세트를 통한 경사 하강 모델을 평가한다.

```
sr.score(X_test, y_test)
```

```
0.9770432600935411
```

- 훈련 세트를 통한 경사 하강 모델을 평가한다. (데이터가 많은 경우 훈련 세트 평가 점수가 테스트 세트 평가 점수보다 더 높다)

```
sr.score(X_train, y_train)
```

```
0.9343604100805729
```

- 확률적 경사 하강 옵션 사용

- 1) max_iter: 훈련 세트 반복 횟수 (Epoch 횟수)
- 2) eta0 : 학습률(learning rate)로 지수 표기법(0.001 : 1e-3, 1000 : 1e+3)이 가능하다.
- 3) verbose : 생략하면 아래 결과 출력이 사라진다.

- 최대 학습 반복 횟수가 1000이고 학습률이 0.001인 경우 학습 횟수가 84에서 loss 값이 가장 작다.

```
from sklearn.linear_model import SGDRegressor
sr = SGDRegressor(max_iter=1000, eta0=0.001, random_state=0, verbose=1)
sr.fit(X_train, y_train)
```

```
-- Epoch 1
Norm: 2.40, NNZs: 1, Bias: 0.442470, T: 16, Avg. loss: 1181.034371
Total training time: 0.00 seconds.
...
-- Epoch 84
Norm: 10.28, NNZs: 1, Bias: 1.783802, T: 1344, Avg. loss: 17.015926
Total training time: 0.00 seconds.
Convergence after 84 epochs took 0.00 seconds
```

- 최대 학습 반복 횟수가 1000이고 학습률이 0.0001인 경우 학습 횟수가 873에서 loss 값이 가장 작다.

```
from sklearn.linear_model import SGDRegressor
sr = SGDRegressor(max_iter=1000, eta0=1e-4, random_state=0, verbose=1)
sr.fit(X_train, y_train) #학습률이 적어지만 학습 반복횟수는 늘어난다.
```

```
-- Epoch 1
Norm: 0.27, NNZs: 1, Bias: 0.048869, T: 16, Avg. loss: 1484.241876
Total training time: 0.00 seconds.
...
-- Epoch 873
Norm: 10.19, NNZs: 1, Bias: 1.776030, T: 13968, Avg. loss: 17.042407
Total training time: 0.06 seconds.
Convergence after 873 epochs took 0.06 seconds
```

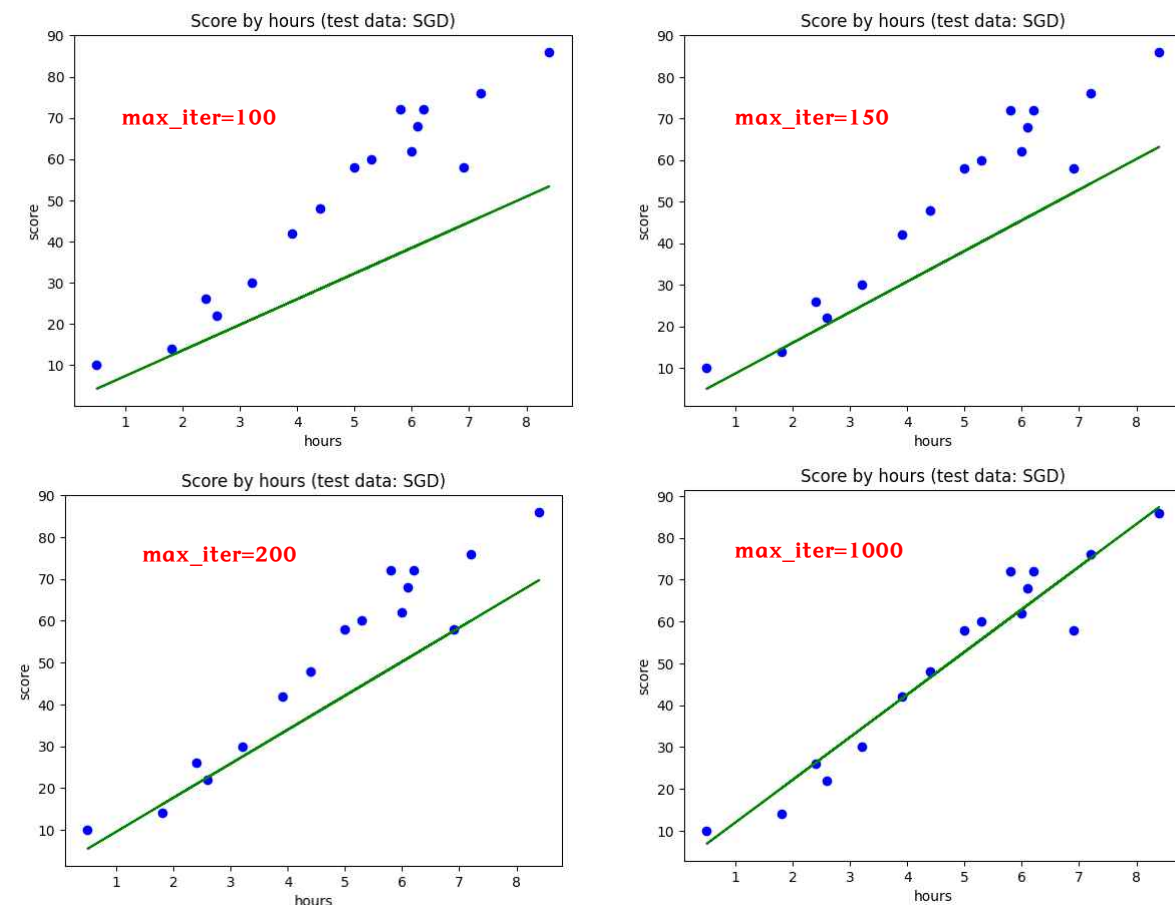
- 학습 반복 횟수가 100이고 학습률이 0.0001인 경우 Epoch 100에서 loss가 너무 크므로 경고가 나온다.

```
from sklearn.linear_model import SGDRegressor
sr = SGDRegressor(max_iter=100, eta0=1e-4, random_state=0, verbose=1)
sr.fit(X_train, y_train)
```

```
-- Epoch 1
Norm: 0.27, NNZs: 1, Bias: 0.048869, T: 16, Avg. loss: 1484.241876
Total training time: 0.00 seconds.
-- Epoch 2
Norm: 0.47, NNZs: 1, Bias: 0.083896, T: 32, Avg. loss: 1419.741822
Total training time: 0.00 seconds.
...
-- Epoch 100
Norm: 6.22, NNZs: 1, Bias: 1.098974, T: 1600, Avg. loss: 253.278959
Total training time: 0.01 seconds.
```

- 학습 반복 횟수에 따라 그래프를 그려보면 반복 횟수가 많을수록 정확한 예측 모델이 생성됨을 확인할 수 있다.

```
plt.scatter(X_train, y_train, color='blue')
plt.plot(X_train, sr.predict(X_train), color='green')
plt.title('Score by hours (test data: SGD)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



• 다중 선형회귀 (Multiple Linear Regression)

다중 선형회귀란 종속변수(y) 1개와 독립변수(X) 2개 이상의 선형 상관관계를 모델링(Modeling) 하는 회귀분석 기법이다. 독립 변수 중 공부 장소는 문자로 입력되어있으므로 수식을 만들기 위해 One-Hot-Encoding 방식을 사용해 숫자로 변환한다.

공부 시간	결석 횟수	공부 장소	시험 점수
0.5	3	Home	10
1.2	4	Library	8
1.8	2	Cafe	14
2.4	0	Cafe	26
2.6	2	Home	22
3.2	0	Home	30
3.9	0	Library	42
4.4	0	Library	48
4.5	5	Home	38
5	1	Cafe	58
5.3	2	Cafe	60
5.8	0	Cafe	72
6	3	Library	62
6.1	1	Cafe	68
6.2	1	Library	72
6.9	4	Home	58
7.2	2	Cafe	76
8.4	1	Home	86
8.6	1	Library	90
10	0	Library	100

$$y = b + m_1x_1 + m_2x_2 + m_3x_3$$

- One-Hot Encoding: 표현하고 싶은 단어의 인덱스에 1의 값을 부여하고, 다른 인덱스에는 0을 부여하는 벡터 표현 방식

One-Hot Encoding

다중공선성

※ Dummy Column 이 n 개면? n-1 개만 사용

공부 장소	Home	Library	Cafe
Home	1	0	0
Library	0	1	0
Cafe	0	0	1

→

공부 장소	Home	Library	Cafe
Home	1	0	0
Library	0	1	0
Cafe	0	0	1

→

공부 장소	Home	Library	Cafe
Home	1	0	0
Library	0	1	0
Cafe	0	0	1

※ Dummy Column 이 n 개면? n-1 개만 사용

공부 시간	결석 횟수	Home	Library	Cafe	시험 점수
0.5	3	1	0	0	10
1.2	4	0	1	0	8
1.8	2	0	0	1	14
2.4	0	0	0	1	26
2.6	2	1	0	0	22
3.2	0	1	0	0	30
3.9	0	0	1	0	42
4.4	0	0	1	0	48
4.5	5	1	0	0	38
5	1	0	0	1	58
5.3	2	0	0	1	60
5.8	0	0	0	1	72
6	3	0	1	0	62
6.1	1	0	0	1	68
6.2	1	0	1	0	72
6.9	4	1	0	0	58
7.2	2	0	0	1	76
8.4	1	1	0	0	86
8.6	1	0	1	0	90
10	0	0	1	0	100

• 데이터 전처리 작업

- '02.다중선형회귀.ipynb' 파일을 생성한 후 데이터 세트를 읽어 독립변수와 종속변수 값을 지정한다.

```
import pandas as pd
dataset = pd.read_csv('data/MultipleLinearRegressionData.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

```
array([[0.5, 3, 'Home'],
       [1.2, 4, 'Library'],
       [1.8, 2, 'Cafe'],
       [2.4, 0, 'Cafe'],
       [2.6, 2, 'Home'],
       ...,
       [6.9, 4, 'Home'],
       [7.2, 2, 'Cafe'],
       [8.4, 1, 'Home'],
       [8.6, 1, 'Library'],
       [10.0, 0, 'Library']], dtype=object)
```

- 원-핫 인코딩(One-Hot Encoding)을 위해 필요한 라이브러리를 import 한 후 인코딩 작업을 한다.

- 1) drop='first' : 첫 번째 자리 값은 삭제한다. (10:Home, 01:Library, 00:Caffe)
- 2) [2] : index 2 칼럼을 One-Hot Encoding 한다. 예) [0.5, 3, 'Home']
- 3) remainder = 'passthrough' : 인코딩을 적용 안 할 나머지는 칼럼들은 그냥 Passing 한다.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(drop='first'), [2])],
    remainder='passthrough')
X = ct.fit_transform(X)
X
```

```
array([[1.0, 0.0, 0.5, 3],
       [0.0, 1.0, 1.2, 4],
       [0.0, 0.0, 1.8, 2],
       [0.0, 0.0, 2.4, 0],
       [1.0, 0.0, 2.6, 2],
       [1.0, 0.0, 3.2, 0],
       [0.0, 1.0, 3.9, 0],
       ...,
       [1.0, 0.0, 6.9, 4],
       [0.0, 0.0, 7.2, 2],
       [1.0, 0.0, 8.4, 1],
       [0.0, 1.0, 8.6, 1],
       [0.0, 1.0, 10.0, 0]], dtype=object)
```

• 훈련데이터와 테스트 데이터 분리 및 모델링

- 데이터 세트를 훈련 데이터 세트와 테스트 세트로 분리한다.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

- 훈련 세트를 입력받아 학습 후 선형회귀 객체를 생성하고 학습 후 선형회귀 모델을 생성한다.

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
reg.fit(X_train, y_train)
```

```
▼ LinearRegression
LinearRegression()
```

• 결과 예측 및 모델 평가

- 테스트 세트의 예측(y_{pred})값과 실제(y_{test})값을 비교한다.

```
y_pred = reg.predict(X_test)
y_pred
```

```
array([ 92.15457859, 10.23753043, 108.36245302, 38.14675204])
```

```
y_test
```

```
array([ 90, 8, 100, 38], dtype=int64)
```

- 훈련 세트의 기울기(m_1, m_2, m_3, m_4)를 출력한다. 카페의 기울기가 가장 크므로 카페에서 공부한 경우 가장 성적 향상에 도움이 된다. $home(-5.82712824), library(-1.04450647), Cafe(0), 공부 시간(10.40419528), 결석 횟수(-1.64200104)$

```
reg.coef_
```

```
array([-5.82712824, -1.04450647, 10.40419528, -1.64200104])
```

- 훈련 세트의 y 절편 b 를 출력한다.

```
reg.intercept_
```

```
5.3650067065447615
```

- 훈련 세트로 모델을 평가해 본다.

```
reg.score(X_train, y_train)
```

```
0.9623352565265527
```

- 테스트 세트로 모델을 평가해 본다.

```
reg.score(X_test, y_test)
```

```
0.9859956178877446
```

- 카페(0, 0)에서 9시간 공부하고 1일 결석하는 경우 예측 점수를 출력하시오.

```
reg.predict([[0, 0, 9, 1]])
```

```
array([97.36076317])
```

• 회귀 모델 평가

모델 생성 후 모델의 신뢰성을 평가하여 더 좋은 모델을 선택할 수 있다. 평가 방법은 아래 4가지의 방법들이 있다.

Evaluation

y	$\hat{y}$	$ y - \hat{y} $
15	25	10
55	42	13
50	59	9
95	76	19
80	93	13

MAE (Mean Absolute Error)

: 실제 값과 예측 값 차이의 절대값들의 평균

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{ex) } MAE = \frac{10 + 13 + 9 + 19 + 13}{5} = 12.8$$

Evaluation

y	$\hat{y}$	$(y - \hat{y})^2$
15	25	100
55	42	169
50	59	81
95	76	361
80	93	169

MSE (Mean Squared Error)

: 실제 값과 예측 값 차이의 제곱한 값들의 평균

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{ex) } MSE = \frac{100 + 169 + 81 + 361 + 169}{5} = 176$$

Evaluation

y	$\hat{y}$	$(y - \hat{y})^2$
15	25	100
55	42	169
50	59	81
95	76	361
80	93	169

RMSE (Root Mean Squared Error)

: MSE 에 루트를 적용

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\begin{aligned} \text{ex) } RMSE &= \sqrt{\frac{100 + 169 + 81 + 361 + 169}{5}} \\ &= \sqrt{176} = 13.27 \end{aligned}$$

Evaluation

y	$\hat{y}$	$(y - \hat{y})^2$	$(y - \bar{y})^2$
15	25	100	1936
55	42	169	16
50	59	81	81
95	76	361	1296
80	93	169	441

R^2 (R Square)

: 결정계수 (데이터의 분산을 기반으로 한 평가 지표)

$$R^2 = 1 - \frac{SSE}{SST} = \frac{SSR}{SST}$$

$$R^2 = 1 - \frac{(\text{실제 값} - \text{예측 값})^2 \text{의 합}}{(\text{실제 값} - \text{평균 값})^2 \text{의 합}}$$

$$\text{ex) } R^2 = 1 - \frac{880}{3770} = 1 - 0.233 = 0.767$$

$$\begin{aligned} \bar{y} &= 59 & SSE &= 880 \\ SST &= 3770 \end{aligned}$$

※ MAE, MSE, RMSE 3가지는 평가 방법은 0에 가까울수록 우수하고 R2 방법은 1에 가까울수록 우수하다.

- 다양한 평가 지표를 계산하기 위해 회귀 모델을 생성한다.

```
import pandas as pd
dataset = pd.read_csv('data/MultipleLinearRegressionData.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct=ColumnTransformer(transformers=[('encoder', OneHotEncoder(drop='first'), [2])], remainder='passthrough')
X = ct.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
reg.fit(X_train, y_train)
```

- MAE(Mean Absolute Error) 지표로 회귀 모델을 평가한다.

```
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_test, y_pred)
```

3.225328518828796

- MSE(Mean Squared Error) 지표로 회귀 모델을 평가한다.

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_test, y_pred)
```

19.900226981514965

- RMSE(Root Mean Squared Error) 지표로 회귀 모델을 평가한다.

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_test, y_pred, squared=False)
```

4.460967045553572

- R2(결정 계수) 지표로 회귀 모델을 평가한다.

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

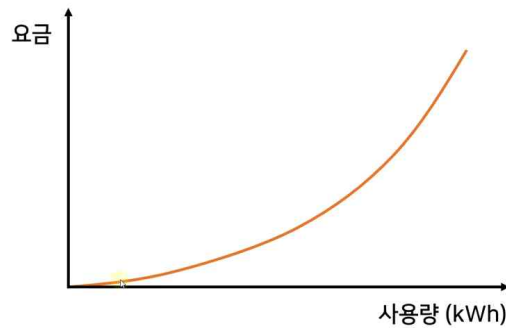
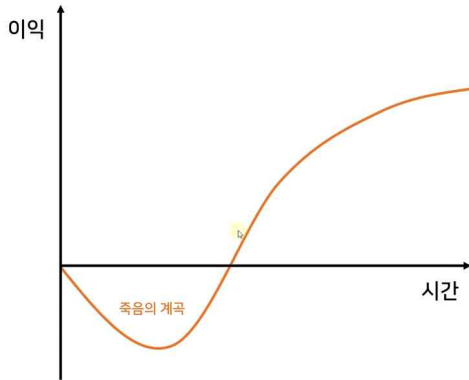
0.9859956178877446

• 다항회귀 (Polynomial Regression)

기업의 성장 단계나 주택 전기 요금 같은 경우에는 일차 방정식으로 표현하기 힘들다. 이런 경우 다항회귀 모델을 사용한다.

예1) 기업의 성장 단계

예2) 주택 전기 요금



Simple
Linear Regression
단순 선형 회귀

$$y = mx + b$$

Multiple
Linear Regression
다중 선형 회귀

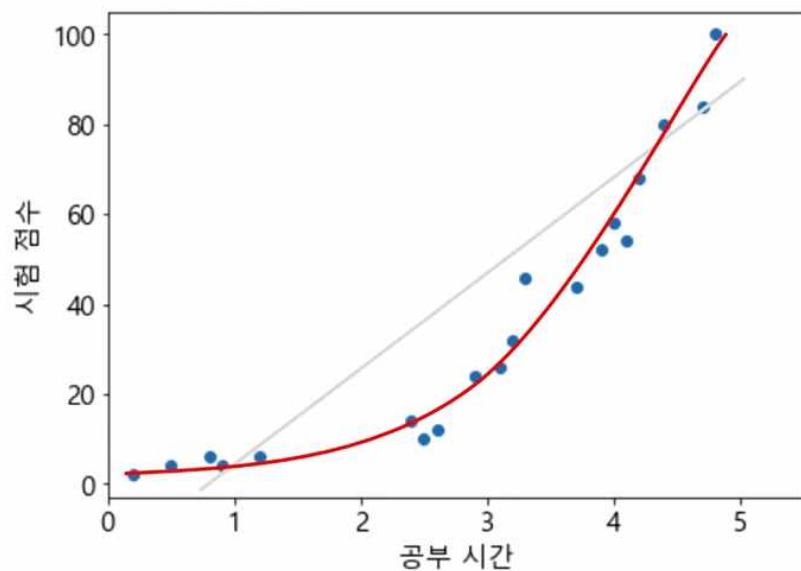
$$y = b + m_1x_1 + m_2x_2 + \dots + m_nx_n$$

Polynomial
Regression
다항 회귀

$$y = b + m_1x + m_2x^2 + \dots + m_nx^n$$

공부 시간	시험 점수
0.2	2
0.5	4
0.8	6
0.9	4
1.2	6
2.4	14
2.5	10
2.6	12
2.9	24
3.1	26
3.2	32
3.3	46
3.7	44
3.9	52
4	58
4.1	54
4.2	68
4.4	80
4.7	84
4.8	100

데이터를 가장 잘 표현하는 선?



- 단순 선형회귀 모델링

Google에서 polynomial regression Data fit 검색 후 첫 번째 링크로 이동 후 다양한 옵션으로 다항회귀를 연습할 수 있다.

- 새로운 파일 '03.다항회귀.ipynb'를 생성한 후 필요한 라이브러리들을 import 한다.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

- 우등생들 시험 점수를 읽어 데이터 세트 생성 후 독립변수(X)와 종속변수(y) 값들을 각각의 변수에 저장한다.

```
dataset = pd.read_csv('data/PolynomialRegressionData.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

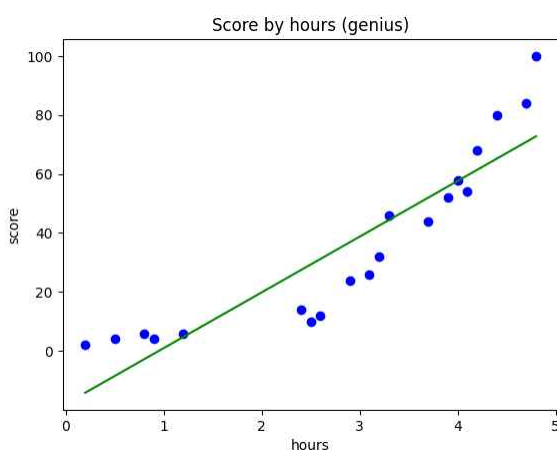
- 전체 데이터로 학습한 후 선형회귀 모델(LinearRegression)을 생성한다.

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
reg.fit(X, y)
```

```
▼ LinearRegression
LinearRegression()
```

- 전체 데이터를 그래프 출력으로 시각화한다.

```
plt.scatter(X, y, color='blue')
plt.plot(X, reg.predict(X), color='green')
plt.title('Score by hours (genius)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 전체 데이터를 통한 단순 선형회귀 모델을 평가해 본다. 평가 점수는 0.8169로 아주 우수한 점수는 아니다.

```
reg.score(X, y)
```

```
0.8169296513411765
```

- 다항회귀(Polynomial Regression) 모델링

- sklearn는 다항회귀 클래스가 제공되지 않으므로 독립변수(X)를 다항회귀에 필요한 독립변수로 변환해 주어야 한다.

```
from sklearn.preprocessing import PolynomialFeatures
poly=PolynomialFeatures(degree=2) #2차 방식 객체 생성
X_poly=poly.fit_transform(X)
X_poly[:5] #X의 0승, X의 1승, X의 2승으로 변환
```

```
array([[1. , 0.2 , 0.04],
       [1. , 0.5 , 0.25],
       [1. , 0.8 , 0.64],
       [1. , 0.9 , 0.81],
       [1. , 1.2 , 1.44]])
```

- get_feature_names_out() 메서드를 호출하면 X 특성이 각각 어떤 입력의 조합으로 만들어졌는지 알려준다.

```
poly.get_feature_names_out()
```

```
array(['1', 'x0', 'x0^2'], dtype=object)
```

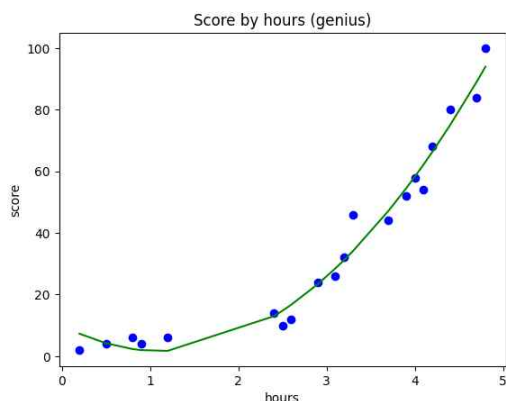
- 변환된 독립변수(X_poly)와 종속변수(y)를 이용하여 선형회귀 객체를 생성한 후 학습하여 다항회귀 모델을 생성한다.

```
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
```

```
▼ LinearRegression
LinearRegression()
```

- 변환된 독립변수(X_poly)와 종속변수(y)를 이용하여 다항회귀 모델을 시각화한다.

```
plt.scatter(X, y, color='blue')
X_poly = poly.fit_transform(X)
plt.plot(X, lin_reg.predict(X_poly), color='green')
plt.title('Score by hours (genius)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 곡선을 부드럽게 출력하기 위해 X(독립변수)을 0.1단위로 일정하게 증가시켜 새로운 값을 생성해 준다. (Numpy 이용)

```
X_range = np.arange(np.min(X), np.max(X), 0.1) #X축을 최솟값~최댓값 0.1 단위로 잘라서 데이터를 생성
X_range

array([0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7,
       2.8, 2.9, 3. , 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.8, 3.9, 4. , 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7])
```

- 새로운 X_range의 형태를 출력하면 1차원 배열이다.

```
X_range.shape
```

```
(46,)
```

- 원본 X의 형태를 출력하면 을 출력하면 2차원 배열이다.

```
X.shape
```

```
(20, 1)
```

- 독립변수(X_range)를 reshape 함수를 이용하여 2차원 배열로 변환한다.

```
X_range=X_range.reshape(-1, 1) #-1은 전체 행(len(X_range)), 1열로 변경
X_range.shape
```

```
(46, 1)
```

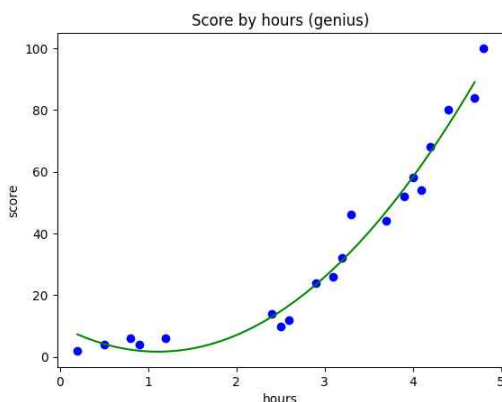
- 독립변수(X_range) 값을 5개 출력해 보면 2차원 배열로 변경되었다.

```
X_range[:5]
```

```
array([[0.2], [0.3], [0.4], [0.5],[0.6]])
```

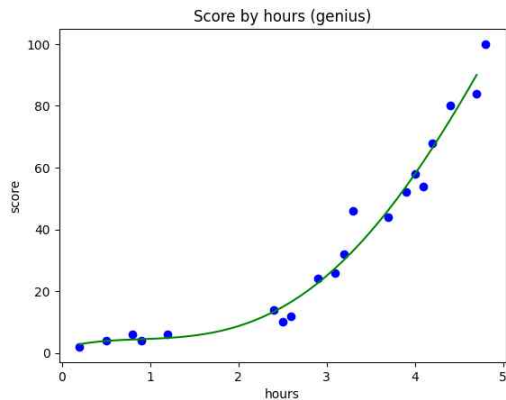
- X값을 X_range로 변경한 후 그래프를 출력하면 부드러운 곡선이 출력된다.

```
plt.scatter(X, y, color='blue')
X_poly = poly.fit_transform(X_range)
plt.plot(X_range, poly_reg.predict(X_poly), color='green')
plt.title('Score by hours (genius)')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- degree를 4로 변경 후 Run All을 실행하면 예측 값이 조금 더 정확한 곡선이 출력된다.

```
...
poly_reg = PolynomialFeatures(degree=4)
```



- 2시간을 공부했을 때 선형회귀 모델의 예측 값을 출력해 본다.

```
reg.predict([[2]])
```

```
array([19.85348988])
```

- 2시간을 공부했을 때 다항회귀 모델의 예측 값을 출력해 본다.

```
lin_reg.predict(poly.fit_transform([[2]]))
```

```
array([8.70559135])
```

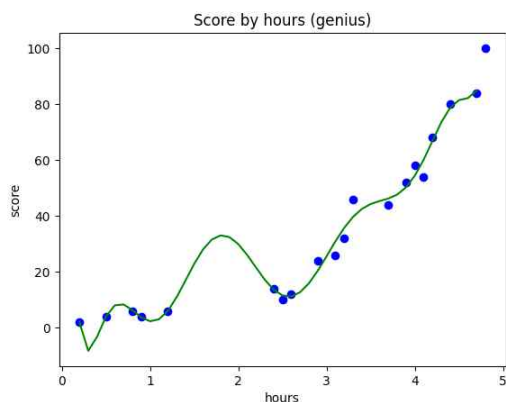
- 다항회귀 모델의 예측 점수를 출력해 본다. 선형회귀 모델보다 점수가 높은 것을 확인 할 수 있다.

```
reg.score(X, y), lin_reg.score(poly.fit_transform(X), y)
```

```
(0.8169296513411765, 0.9782775579000046)
```

- degree를 10으로 변경 후 Run All을 실행하여 새로운 다항회귀 모델을 생성해 본다. 훈련에서는 높은 예측률을 가지지만 실제(2시간)의 예측 값은 엉터리 값(과대 적합, 과소 적합)이 예측될 수 있다.

```
...
poly = PolynomialFeatures(degree=10)
```

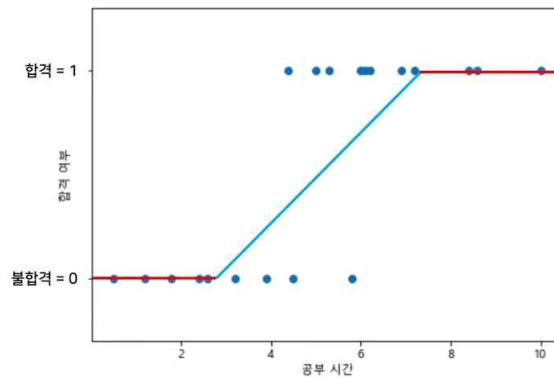
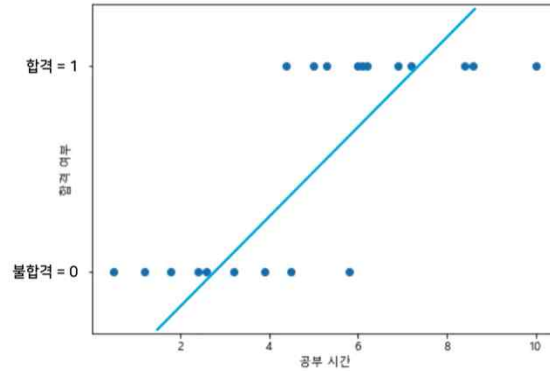


• 로지스틱회귀 (Logistic Regression)

분류(Classification)는 주어진 데이터를 정해진 범주(Category)에 따라 분류하고 예측 결과가 숫자가 아닐 때 사용한다. 분류의 대표적인 알고리즘이 로지스틱 회귀이다. 로지스틱 회귀는 선형회귀 방식을 지도학습의 분류에 적용한 알고리즘으로 데이터가 어떤 범주(Category)에 속할 확률을 0~1 사이의 값으로 예측해서 더 높은 범주에 속하는 쪽으로 분류한다.

- 범주: True/False, Yes/No, 합격/불합격
- 적용예제: 스팸 메일, 은행 대출, 중앙 약성 여부, 고객의 제품 구매 의사 등이 있다.
- 아래는 공부 시간에 따른 자격증 시험 합격 여부를 표현하는 그래프다. 합격이면 1에 수렴하고 불합격 경우 0에 수렴한다.

공부 시간	합격 여부
0.5	불합격
1.2	불합격
1.8	불합격
2.4	불합격
2.6	불합격
3.2	불합격
3.9	불합격
4.4	합격
4.5	불합격
5	합격
5.3	합격
5.8	불합격
6	합격
6.1	합격
6.2	합격
6.9	합격
7.2	합격
8.4	합격
8.6	합격
10	합격



- 첫 번째는 선형회귀 함수, 두 번째는 시그모이드(Sigmoid)함수 또는 로지스틱(Logistic)함수 그래프이다.

$$y = mx + b$$

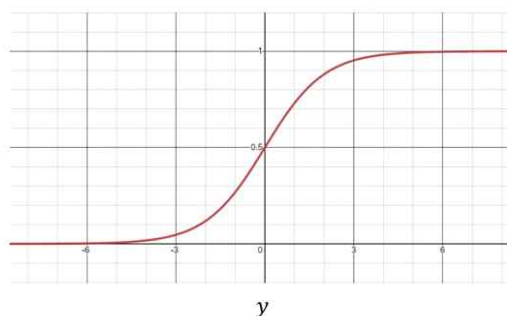
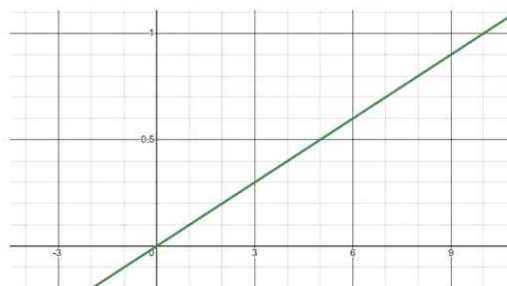


Sigmoid
Function

$$P = \frac{1}{1 + e^{-y}}$$



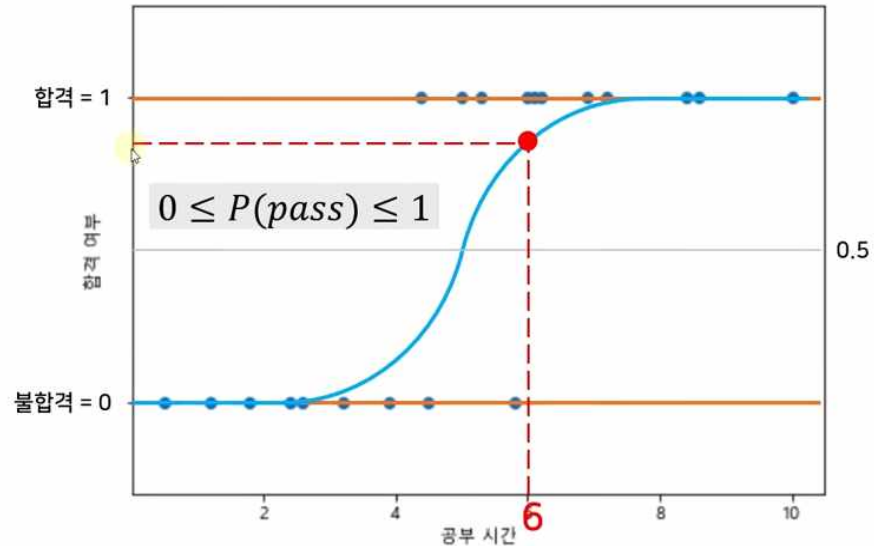
$$\ln\left(\frac{P}{1-P}\right) = mx + b$$



- 6시간 공부했을 때 합격 여부는 0.5보다 크므로 합격으로 예측한다.

공부 시간	합격 여부
0.5	불합격
1.2	불합격
1.8	불합격
2.4	불합격
2.6	불합격
3.2	불합격
3.9	불합격
4.4	합격
4.5	불합격
5	합격
5.3	합격
5.8	불합격
6	합격
6.1	합격
6.2	합격
6.9	합격
7.2	합격
8.4	합격
8.6	합격
10	합격

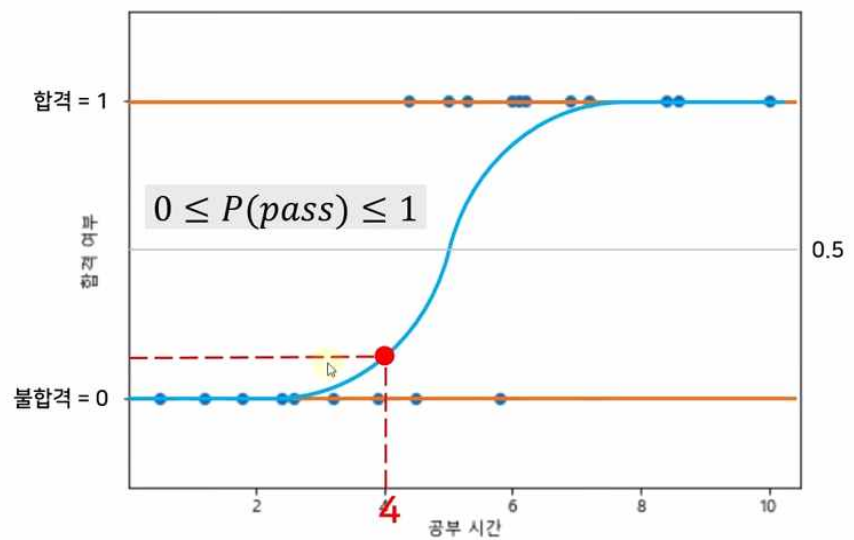
6시간을 공부했을 때 자격증을 딸 수 있을까?



- 4시간 공부했을 때 합격 여부는 0.5보다 작으므로 불합격으로 예측한다.

공부 시간	합격 여부
0.5	불합격
1.2	불합격
1.8	불합격
2.4	불합격
2.6	불합격
3.2	불합격
3.9	불합격
4.4	합격
4.5	불합격
5	합격
5.3	합격
5.8	불합격
6	합격
6.1	합격
6.2	합격
6.9	합격
7.2	합격
8.4	합격
8.6	합격
10	합격

4시간을 공부했을 때 자격증을 딸 수 있을까?



- 로지스틱회귀 모델링

공부 시간에 따른 자격증 시험 합격 가능성을 로지스틱회귀 모델을 사용하여 예측해 본다.

- 새로운 파일 '04.로지스틱회귀.ipynb'를 생성한 후 필요한 라이브러리를 import 한다.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

- csv(공부 시간에 따른 자격증 합격 여부) 파일을 데이터 세트로 읽어 독립변수(X)와 종속변수(y) 값을 저장한다.

```
dataset = pd.read_csv('data/LogisticRegressionData.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

- 실제 데이터를 훈련 데이터 세트와 테스트 데이터 세트로 분리한다.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

- 로지스틱 회귀 객체 생성한 후 훈련 데이터를 이용하여 학습한 로지스틱 회귀 모델을 생성한다.

```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression()
classifier.fit(X_train, y_train)
```

```
▼ LogisticRegression
LogisticRegression()
```

- 6시간 공부했을 때 1이 출력되므로 합격이 예측된다. 합격 확률을 보기 원하면 predict_proba() 함수를 사용한다.

```
classifier.predict([[6]])
```

```
array([1], dtype=int64)
```

```
classifier.predict_proba([[6]])
```

```
array([[0.14150735, 0.85849265]]) #불합격 확률: 14%, 합격 확률:85%
```

- 4시간 공부했을 때 0이 출력되므로 불합격이 예측된다.

```
classifier.predict([[4]])
```

```
array([0], dtype=int64)
```

```
classifier.predict_proba([[4]])
```

```
array([[0.6249966, 0.3750034]]) #불합격 확률: 62%, 합격 확률:37%
```

- 테스트 세트 데이터를 가지고 결과(y_pred)를 예측해 본다.

```
y_pred = classifier.predict(X_test)
y_pred
array([1, 0, 1, 1], dtype=int64)
```

- 실제 데이터 결과(y_test)를 출력해 본다. (4번째 데이터 값은 예측 결과가 다르다.)

```
y_test
array([1, 0, 1, 0], dtype=int64)
```

- 테스트 데이터 세트의 공부 시간(X_test)을 출력해 본다. (4.5시간 예측 결과가 틀리게 출력되었다.)

```
X_test
array([[ 8.6], [ 1.2], [10. ], [ 4.5]])
```

- 테스트 세트의 모델을 평가해 본다. (전체 테스트 세트 4개 중에서 분류 예측을 맞힌 개수 3개 → $3/4 = 0.75$)

```
classifier.score(X_test, y_test)
0.75
```

• 로지스틱회귀 훈련세트 시각화

- 부드러운 곡선을 위해 X 범위를 세분화하여 생성한다. (X의 최솟값~최댓값을 0.1 단위로 잘라서 생성)

```
X_range = np.arange(np.min(X), np.max(X), 0.1)
X_range
array([0.5, 0.6, 0.7, 0.8, 0.9, 1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7 ,... 9.6, 9.7, 9.8, 9.9])
```

- 로지스틱 함수 공식에 X_range 값을 대입하여 p의 값을 구한다. np.exp() 함수는 자연 상수 e에 대한 지수함수이다.

```
y=classifier.coef_ * X_range + classifier.intercept_
p = 1/(1 + np.exp(-y)) #p = 1/(1+e^-y), y=mx + b
p
array([[0.01035705, 0.01161247, 0.01301807, 0.0145913 , 0.01635149,...
        0.99713321, 0.99744558, 0.997724 , 0.99797213, 0.99819325]])
```

- p의 shape를 출력해 보면 2차원 배열이다. p는 y축 값이므로 1차원 배열로 변경한다.

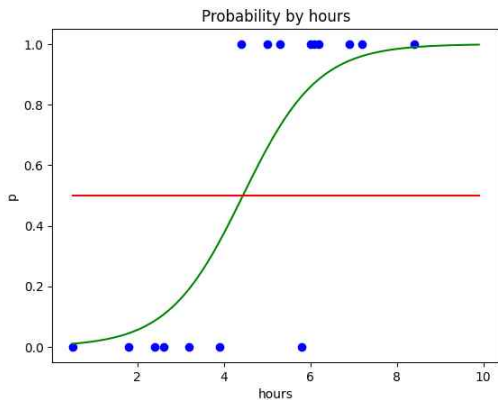
```
p.shape
(1, 95)
```

- p를 2차원 배열에서 reshape 함수를 이용하여 1차원 배열로 변경한다. 옵션 값 -1은 len(p)을 의미한다.

```
p = p.reshape(-1)
(95,)
```

- 로지스틱함수를 이용하여 훈련세트를 시각화 한다.

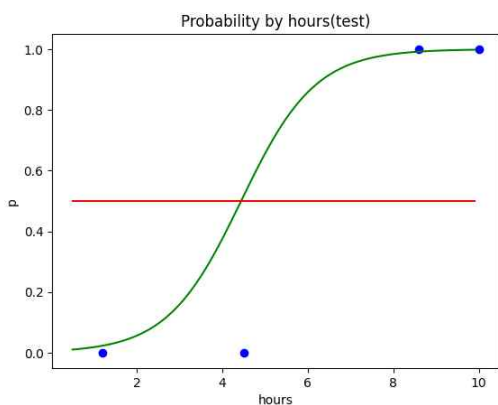
```
plt.scatter(X_train, y_train, color='blue')
plt.plot(X_range, p, color='green')
plt.plot(X_range, np.full(len(X_range), 0.5), color='red') #X_range의 개수만큼 0.5로 가득 찬 배열
plt.title('Probability by hours')
plt.xlabel('hours')
plt.ylabel('p')
plt.show()
```



- 로지스틱회귀 테스트 세트 시각화

- 로지스틱함수를 이용하여 훈련세트를 시각화 한다.

```
plt.scatter(X_test, y_test, color='blue')
plt.plot(X_range, p, color='green')
plt.plot(X_range, np.full(len(X_range), 0.5), color='red')
plt.title('Probability by hours(test)')
plt.xlabel('hours')
plt.ylabel('p')
plt.show()
```



- 4.5시간 공부했을 때 모델을 이용하면 합격 확률 51.6%로 합격을 예측하지만 실제 값은 불합격이다.

```
classifier.predict_proba([[4.5]])
array([[0.48310686, 0.51689314]])
```

- 혼동 행렬 (Confusion Matrix)

혼동 행렬은 분류 모델의 예측 결과를 평가하는 데 사용되는 표이다. 결과표를 보고 맞게 예측한 결과수와 틀린 결과수를 확인할 수 있다.

- confusion_matrix 함수를 이용하여 테스트 세트(X_test)와 예측 결과(y_pred)를 혼동 행렬로 변경한다.

```
from sklearn.metrics import confusion_matrix
y_pred = classifier.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[1, 1],
       [0, 2]], dtype=int64)
```

True Negative (TN)
불합격을 예측했는데 실제로 불합격한 수 (1)

False Positive (FP)
합격으로 예측했는데 실제로 불합격한 수 (1)

False Negative (FN)
불합격을 예측했는데 실제로 합격한 수 (0)

True Positive (TP)
합격으로 예측했는데 실제로 합격한 수 (2)

- confusion_matrix 함수를 이용하여 훈련 세트(X_train)와 예측 결과(y_pred)를 혼동 행렬로 변경한다.

```
from sklearn.metrics import confusion_matrix
y_pred = classifier.predict(X_train)
cm = confusion_matrix(y_train, y_pred)
cm
```

```
array([[6, 1],
       [1, 8]], dtype=int64)
```

True Negative (TN)
불합격을 예측했는데 실제로 불합격한 수 (6)

False Positive (FP)
합격으로 예측했는데 실제로 불합격한 수 (1)

False Negative (FN)
불합격을 예측했는데 실제로 합격한 수 (1)

True Positive (TP)
합격으로 예측했는데 실제로 합격한 수 (8)



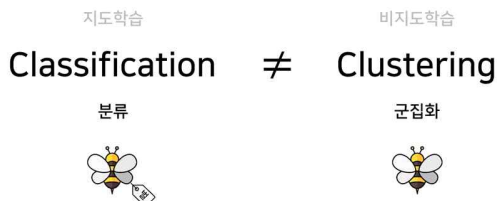
• K-평균 (K-Means)

• 비지도 학습 (Unsupervised Learning)

정답이 없는 데이터를 통해 유의미한 패턴/구조를 발견하여 유사한 특징을 가지는 데이터들을 그룹화 하여 학습하는 방법으로 대표적인 학습으로 군집화(Clustering)가 있다. 대표적인 군집화 예제로 아래와 같다.

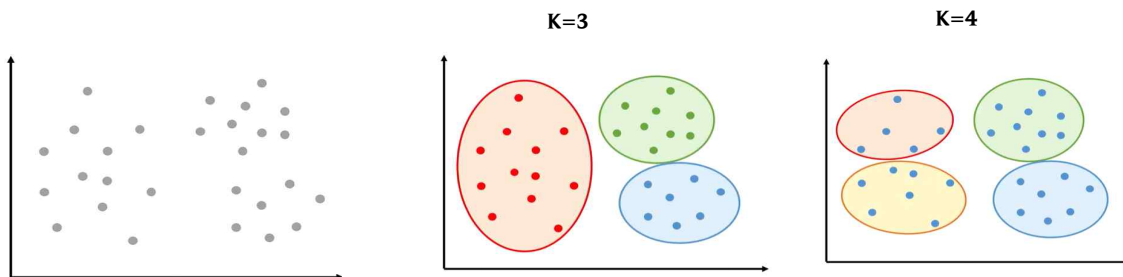
- 1) 고객 세분화: 고객의 성별, 거주 지역, 나이, 소득, 지금까지 구매한 목록에 따라 군집화 하여 소비성향 파악 후 마케팅에 활용한다.
- 2) 소셜 네트워크 분석: 소셜 네트워크를 통해 나와 관련 있는 친구들을 찾는데 활용한다.
- 3) 기사 그룹 분류: 매일 매일 수없이 나오는 기사들을 비슷한 그룹(과학기술, 정치, 경제, 스포츠)등으로 군집화 하는데 활용한다.

분류(Classification)는 지도학습으로 정답이 존재하며 군집화(Clustering)는 비 지도학습으로 정답이 존재하지 않는다.



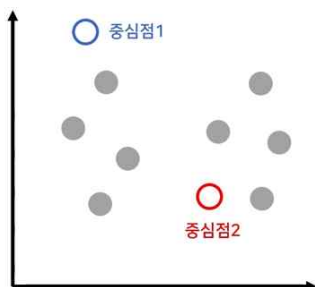
• K-Means

K-Means 군집화(clustering)은 비지도 학습 방법 중 군집화의 대표적인 알고리즘으로 주어진 데이터를 K개의 군집(클러스터)으로 나누는 방법이다. 여기서 K는 사용자가 지정하는 군집의 개수를 의미한다. K-Means는 각 데이터 포인트를 가장 가까운 중심점(centroid)에 할당하고 각 중심점을 데이터 포인트들의 평균으로 업데이트하는 과정을 반복하여 군집을 형성한다.

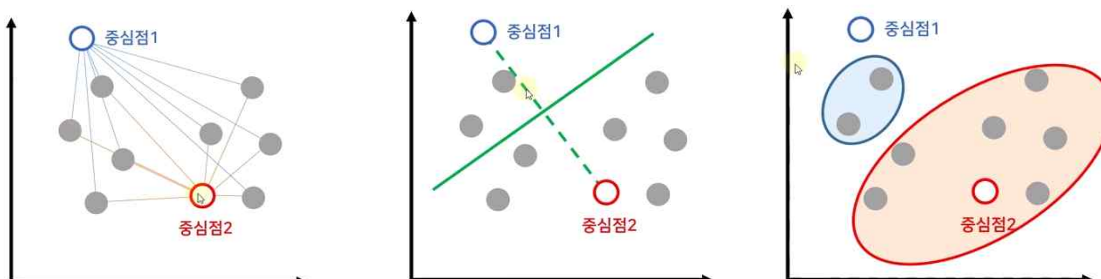


• K-Means 동작 순서

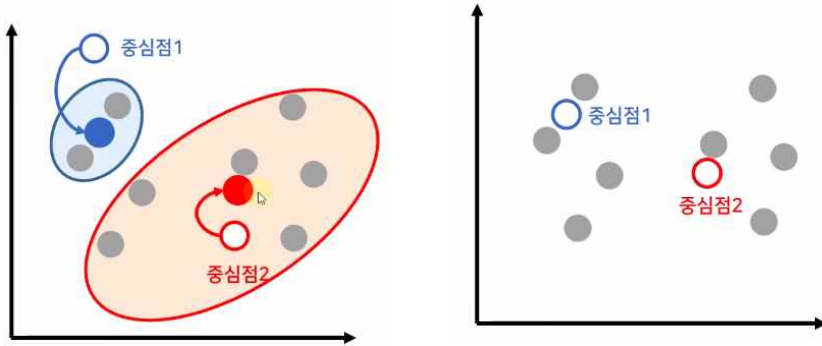
1. K값 설정 (K=2로 설정)
2. 지정한 K개만큼의 random 중심점 좌표 설정



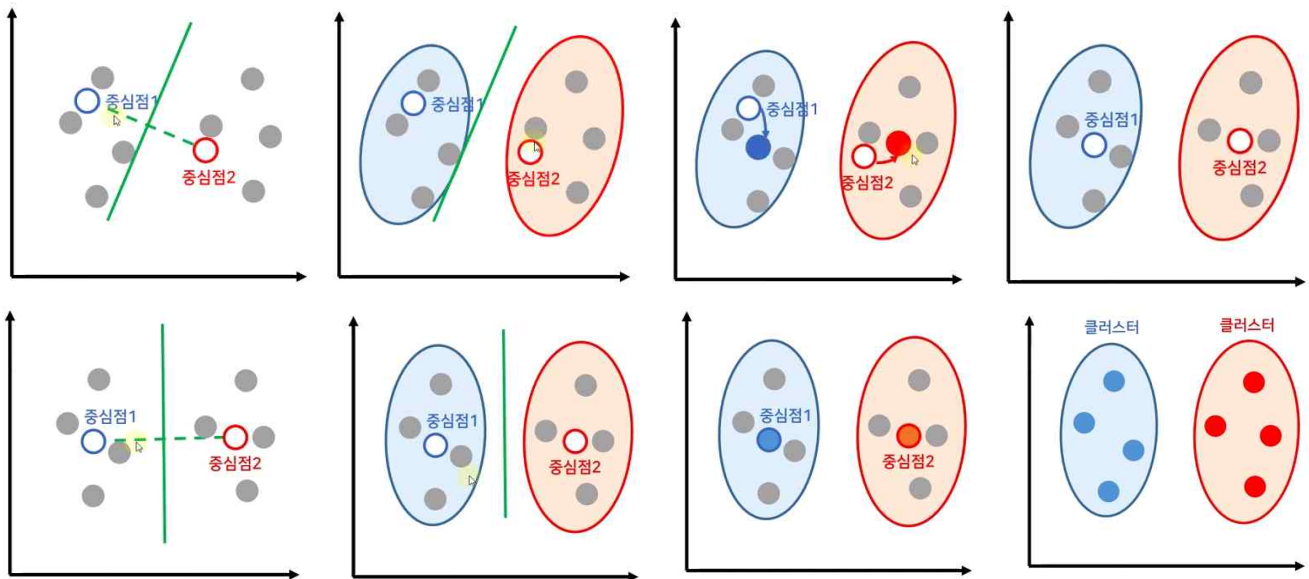
3. 모든 데이터로부터 가장 가까운 중심점 선택



4. 데이터들의 평균 중심으로 중심점 이동

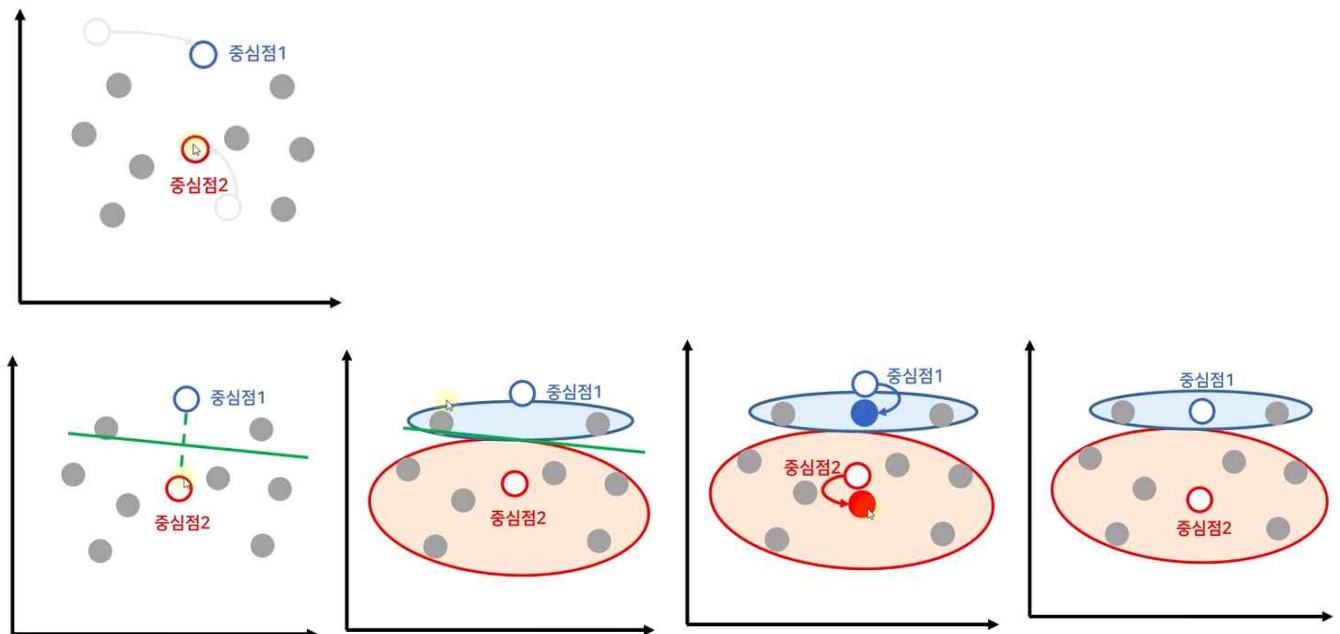


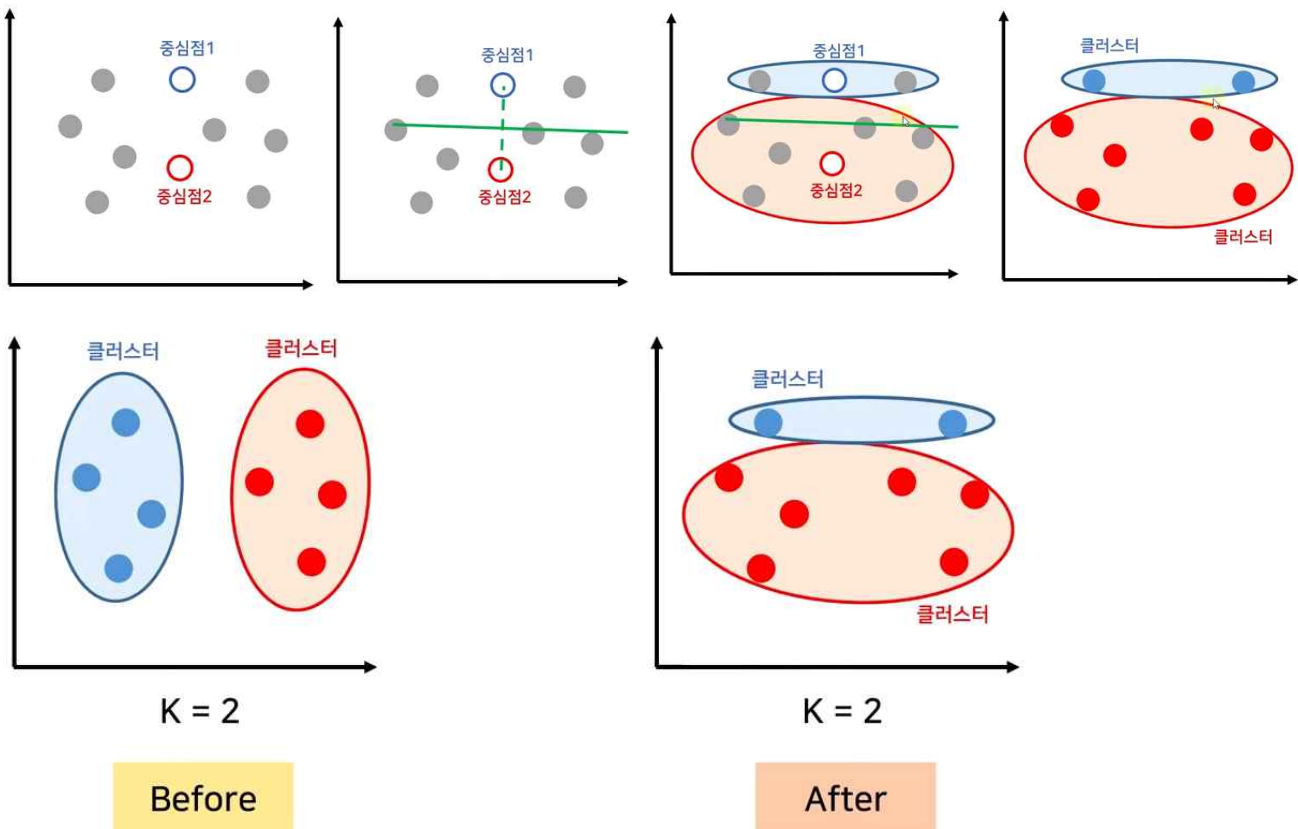
5. 중심점이 더 이상 이동되지 않을 때까지 3번(모든 데이터로부터 가장 가까운 중심점 선택) 반복



• Random Initialization Trap (중심점 무작위 선정 문제)

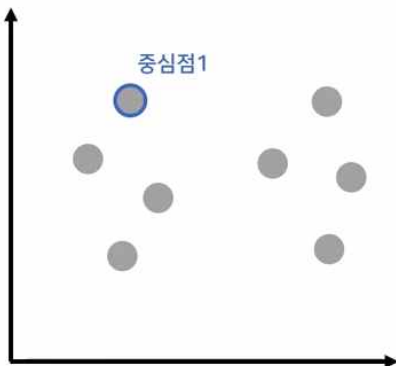
random으로 중심점 좌표를 설정하면 클러스터 작업이 제대로 이뤄지지 않는 경우가 있다.





• K-Means++

1. 데이터 중에서 random으로 1개를 중심점으로 선택

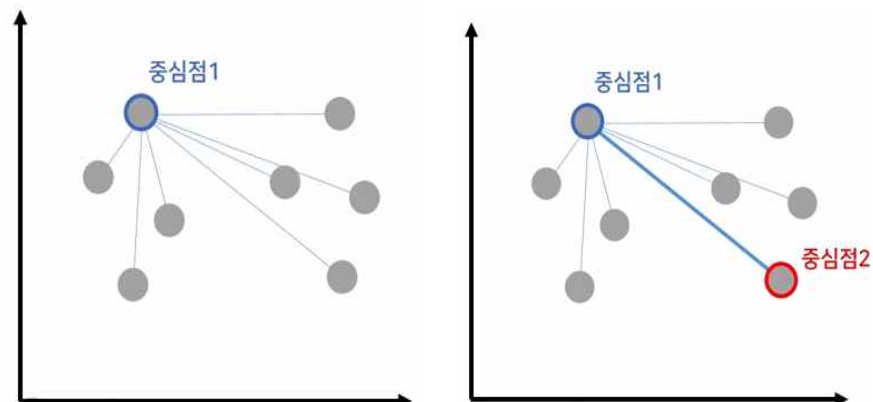


2. 나머지 데이터로부터 중심점까지의 거리 계산

3. 중심점과 가장 먼 지점의 데이터를 다음 중심점으로 선택

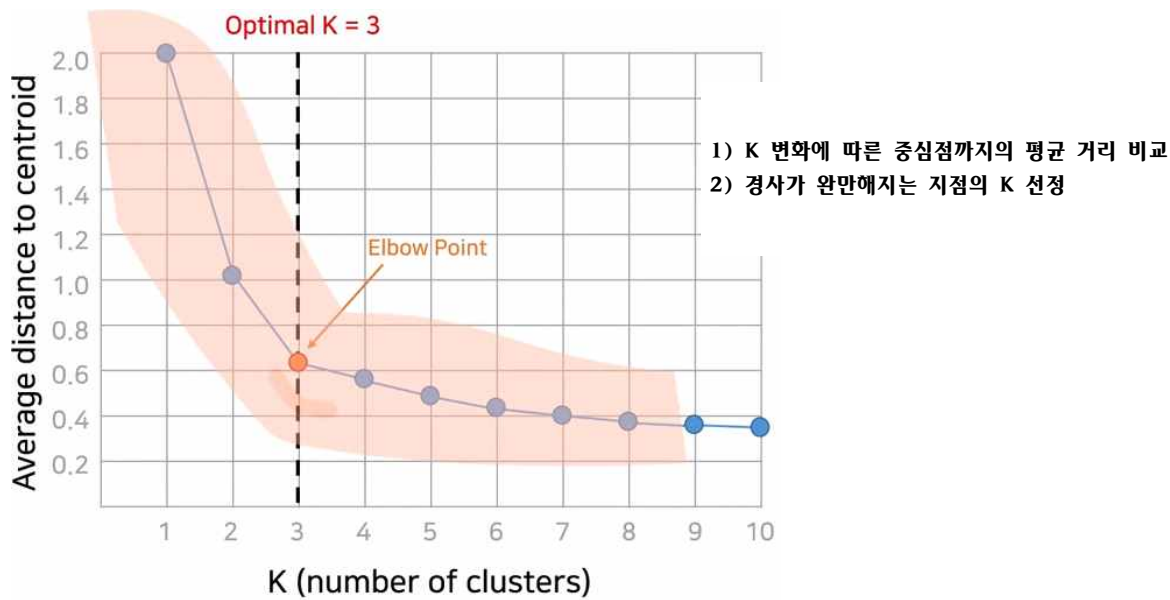
4. 중심점이 K개가 될 때까지 반복

5. K-Means 전통적인 방식으로 진행



- **Optimal K (최적의 K)**

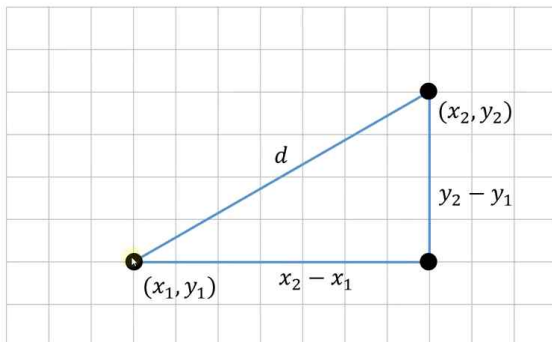
K값 지정을 위해 최적의 K값을 구할 때는 Elbow Method 을 이용하여 계산한다.



- 데이터 유사도 구하는 방법

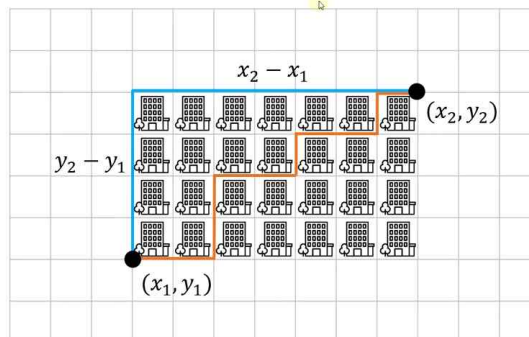
Euclidean Distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



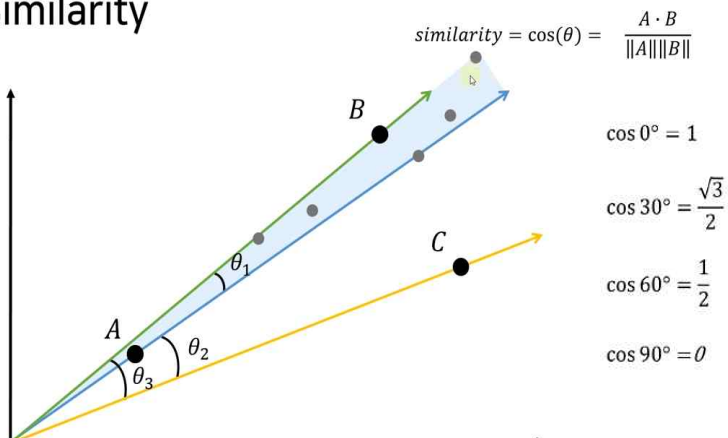
Manhattan Distance

$$d = |x_2 - x_1| + |y_2 - y_1|$$



- 각도가 커질수록 Cosine 값은 작아진다. (각도가 작을수록 값이 커지고 유사도는 높다.)

Cosine Similarity



- K-평균을 위한 데이터 생성

- 공부 시간별 성적 데이터들을 몇 개의 그룹으로 군집화하기 '05.K-평균.ipynb'를 생성한 후 필요한 라이브러리를 import 한다.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

- KMeanData.csv 파일을 읽어 데이터 세트에 저장 후 3개의 데이터(시간과 점수)를 출력한다.

```
dataset = pd.read_csv('data/KMeansData.csv')
dataset[:3]
```

	hour	score
0	7.33	73
1	3.71	55
2	3.43	55

- K-Means는 정답이 없는 비지도 학습이므로 종속변수(y)가 없다. 그러므로 독립변수(X)의 모든 값만 가져온다.

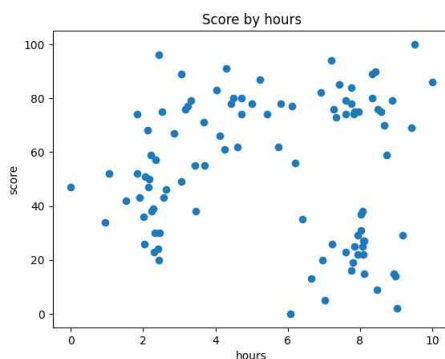
```
X = dataset.iloc[:, :].values
#X = dataset.values 위 결과와 동일하다.
X[:5]
```

```
array([[ 7.33, 73. ],
       [ 3.71, 55. ],
       [ 3.43, 55. ],
       [ 3.06, 89. ],
       [ 3.33, 79. ]])
```

- 전체 데이터 분포 시각화

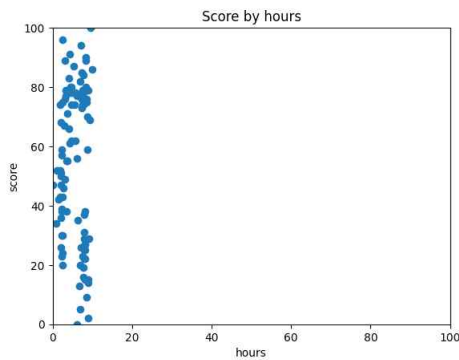
- 전체 데이터 분포를 X축을 시간 Y축을 점수로 시각화한다. K-Means로 군집화를 하려면 중심점과 데이터들 사이의 거리를 구해 가까운 데이터들 끼리 그룹을 지어야 하는데 X축, Y축 범위가 다르므로 실제 두 점 사이의 거리와 시각적으로 보여 지는 거리가 다를 수 있다.

```
plt.scatter(X[:,0], X[:,1])
plt.title('Score by hours')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



- 실제 두 점 사이의 거리와 실제 보여 지는 거리를 같게 하기에서는 X축과 Y축의 범위 limit 값을 0~100으로 같게 한다.

```
plt.scatter(X[:, 0], X[:, 1])
plt.title('Score by hours')
plt.xlabel('hour')
plt.ylabel('score')
plt.xlim(0, 100)
plt.ylim(0, 100)
```



• 피쳐 스케일링 후 시각화

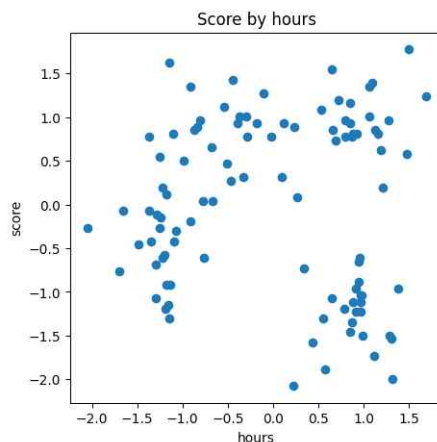
- 피쳐 스케일링(Feature Scaling) 함수를 이용하여 데이터 피쳐(feature)인 독립변수 들이 서로 다른 범위(scale)를 가질 때 동일한 스케일로 맞추는 작업을 한다. 서로 다른 스케일을 가진 데이터들은 모델의 성능을 떨어뜨리기 때문에 피쳐 스케일링은 중요한 기법이다.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
X[:1]

(array([[0.68729921, 0.73538376]]), array([[ 7.33, 73. ]]))
```

- 스케일링 한 X_scale 변수로 시각화하면 데이터 분포는 X축, Y축 limit 설정하기 전과 같으며 X축, Y축의 범위가 같게 된다.

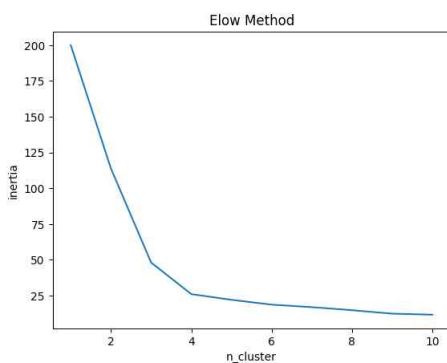
```
plt.figure(figsize=(5, 5))
plt.scatter(X[:, 0], X[:, 1])
plt.title('Score by hours')
plt.xlabel('hour')
plt.ylabel('score')
```



• 최적의 K 값 찾기

- 몇 개의 cluster로 나눌지 Elbow Method를 이용해 곡선이 완만해지는 지점의 cluster의 수 최적의 K값을 찾는다. (최적의 K=4)

```
from sklearn.cluster import KMeans
inertia_list = [] #inertia(각 지점에서 클러스터 중심(centroid)까지의 거리 제곱의 합) 값을 저장할 배열
for i in range(1, 11): #클러스터 개수를 1 ~11까지 변경하며 반복한다.
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=0)
    kmeans.fit(X_scale) #생성된 kmeans 객체모델로 데이터(X_scale)를 학습한다.
    inertia_list.append(kmeans.inertia_) #inertia 값을 배열에 추가한다.
plt.plot(range(1, 11), inertia_list)
plt.title('Elbow Method')
plt.xlabel('n_cluster')
plt.ylabel('inertia')
plt.show()
```



Thread 경고가 발생하는 경우 상위에 아래 코드 등록
import os
os.environ['OMP_NUM_THREADS'] = '1'

• 최적의 K(4) 값으로 KMeans 학습 후 결과 확인

- 최적의 K값으로 모델을 생성하여 학습한 후 예측하면 어느 클러스터에 속하는지 알 수 있다.

```
K = 4
kmeans = KMeans(n_clusters=K, random_state=0) #init 생략 시 'k-means++'로 지정된다.
y_kmeans = kmeans.fit_predict(X) #X_scale의 점들이 어느 cluster에 속하는지 예측해 본다.
y_kmeans
array([1, 0, 3, 0, 0, 2, 2, 0, 1, 0, 0, 3, 2, 3, 3, 0, 2, 1, 3, 0, 2, 0,
       3, 2, 1, 1, 3, 3, 3, 3, 2, 2, 3, 0, 1, 1, 3, 0, 0, 0, 3, 2, 1, 3,
       3, 1, 2, 0, 2, 2, 1, 0, 2, 2, 0, 0, 0, 0, 3, 2, 2, 1, 1, 1, 1, 2,
       2, 0, 2, 1, 3, 1, 1, 1, 3, 3, 3, 3, 0, 1, 2, 1, 2, 2, 1, 0, 3, 2, 1, 3, 0, 2, 0, 1, 3, 0, 1, 0, 2, 3])
```

- dataset에 cluster 칼럼 추가하고 y_kmeans(cluster 그룹) 결과를 저장한다.

```
dataset['cluster'] = y_kmeans
dataset.head()
```

	hour	score	cluster
0	7.33	73	1
1	3.71	55	0
2	3.43	55	3
3	3.06	89	0
4	3.33	79	0

- index 칼럼 이름을 'index'로 수정하고 'kmenas.csv' 파일로 저장한다.

```
dataset.index.name='index'
dataset.to_csv('data/kmeans.csv')
```

- y_kmeans 목록을 출력하고 y_kmeans 중 cluster가 0인 그룹에 속한 index 값들만을 출력한다.

```
y_kmeans
```

```
array([1, 0, 3, 0, 0, 2, 2, 0, 1, 0, 0, 3, 2, 3, 3, 0, 2, 1, 3, 0, 2, 0,
       3, 2, 1, 1, 3, 3, 3, 2, 2, 3, 0, 1, 1, 3, 0, 0, 0, 3, 2, 1, 3,
       3, 1, 2, 0, 2, 2, 1, 0, 2, 2, 0, 0, 0, 0, 3, 2, 2, 1, 1, 1, 1, 2,
       2, 0, 2, 1, 3, 1, 1, 1, 3, 3, 3, 3, 0, 1, 2, 1, 2, 2, 1, 0, 3, 2,
       1, 3, 0, 2, 0, 1, 3, 0, 1, 0, 2, 3])
```

```
indices = np.where(y_kmeans==0)
indices
```

```
(array([ 1,  3,  4,  7,  9, 10, 15, 19, 21, 33, 37, 38, 39, 47, 51, 54, 55,
        56, 57, 67, 78, 85, 90, 92, 95, 97], dtype=int64),)
```

- cluster가 0인 그룹의 X 좌표인 공부시간들을 출력한다.

```
X[indices, 0]
```

```
array([[ -0.66687438, -0.9100271 , -0.8090252 ,  0.09251026, -0.28531166,
        -0.18430976, -0.50976032, -0.68183762, -0.98484332, -0.37135031,
        -0.02345489, -0.39379518, -0.86887817, -1.37014685, -1.10829008,
        -0.33020139, -0.54342762, -0.46487058, -1.14195738, -0.29279328,
        -0.44990734, -0.8426925 , -1.25792252,  0.11495512,  0.26832837,
        -0.10201192]])
```

- cluster가 0인 그룹의 Y 좌표인 성적들을 출력한다.

```
X[indices, 1]
```

```
array([[0.04198891, 1.35173473, 0.96651537, 0.31164246, 0.77390569,
        0.92799344, 0.4657302 , 0.65833988, 0.50425214, 1.00503731,
        0.77390569, 0.92799344, 0.85094956, 0.77390569, 0.81242763,
        0.31164246, 1.12060312, 0.27312053, 1.62138828, 1.00503731,
        1.4287786 , 0.8894715 , 0.54277408, 0.92799344, 0.08051085,
        1.27469086]])
```

- y-kmenas 중 클러스터가 0~3인 그룹들의 index 값들을 출력한다.

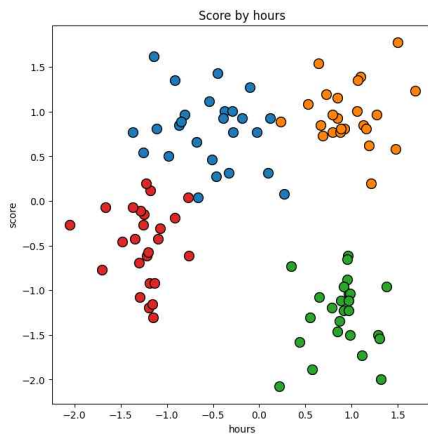
```
for cluster in range(4):
    indices = np.where(y_kmeans==cluster)
    print(cluster, indices)
```

```
0 (array([ 1,  3,  4,  7,  9, 10, 15, 19, 21, 33, 37, 38, 39, 47, 51, 54, 55,
        56, 57, 67, 78, 85, 90, 92, 95, 97], dtype=int64),)
1 (array([ 0,  8, 17, 24, 25, 34, 35, 42, 45, 50, 61, 62, 63, 64, 69, 71, 72,
        73, 79, 81, 84, 88, 93, 96], dtype=int64),)
2 (array([ 5,  6, 12, 16, 20, 23, 30, 31, 41, 46, 48, 49, 52, 53, 59, 60, 65,
        66, 68, 80, 82, 83, 87, 91, 98], dtype=int64),)
3 (array([ 2, 11, 13, 14, 18, 22, 26, 27, 28, 29, 32, 36, 40, 43, 44, 58, 70,
        74, 75, 76, 77, 86, 89, 94, 99], dtype=int64),)
```

- K-Means 모델 그룹별 시각화

- 최적의 K를 이용하여 각 cluster의 점들을 서로 다른 색상으로 출력해 본다.

```
plt.figure(figsize=(7, 7))
for cluster in range(4):
    indices = np.where(y_kmeans==cluster)
    plt.scatter(X[indices, 0], X[indices, 1], s=100, ec='black')
...
```



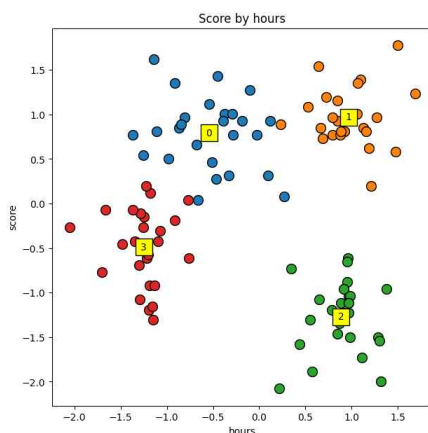
- cluster 그룹 번호를 출력할 위치 cluster의 중심점(centroid) 좌표들을 구한다.

```
centers = kmeans.cluster_centers_

array([[ -0.54299598,  0.79316666], [ 0.96910697,  0.97133061], [ 0.8837666 , -1.26929779], [-1.24939347, -0.48807293]])
```

- 중심점(centroid)을 사각형으로 출력하고 cluster 번호도 텍스트로 출력한다.

```
plt.figure(figsize=(7, 7))
for cluster in range(4):
    indices = np.where(y_kmeans==cluster)
    plt.scatter(X[indices, 0], X[indices, 1], s=100, ec='black')
    plt.scatter(centers[cluster, 0], centers[cluster, 1], s=300, ec='black', c='yellow', marker='s')
    plt.text(centers[cluster, 0], centers[cluster, 1], cluster, va='center', ha='center')
...
```



- Feature Scaling 한 독립변수(X)의 값들을 다시 원래 값으로 복원한다.

```
X_org = scaler.inverse_transform(X)
X_org[:5]

array([
  [ 7.33, 73. ],
  [ 3.71, 55. ],
  [ 3.43, 55. ],
  [ 3.06, 89. ],
  [ 3.33, 79. ]
])
```

- 중심점(Centroid)의 값들도 원래 데이터로 복원하여 centers_org에 저장한다.

```
centers_org = scaler.inverse_transform(centers)
centers_org

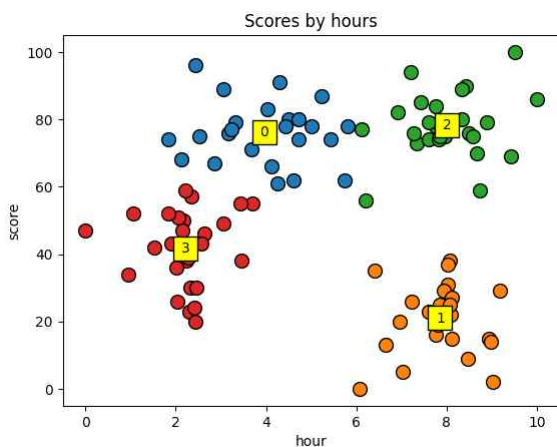
array([
  [ 3.96458333, 76.08333333],
  [ 7.8552    , 20.96    ],
  [ 8.0084    , 78.2    ],
  [ 2.21269231, 41.76923077]
])
```

- 원래대로 복원된 데이터들로 시각화 작업을 한다.

```
plt.figure(figsize=(7, 7))

for cluster in range(4):
    indices = np.where(y_kmeans==cluster)
    plt.scatter(X[indices, 0], X[indices, 1], s=100, ec='black')
    plt.scatter(centers[cluster, 0], centers[cluster, 1], s=300, ec='black', c='yellow', marker='s')
    plt.text(centers[cluster, 0], centers[cluster, 1], cluster, va='center', ha='center')

plt.title('Score by hours')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



• 그룹별 데이터의 index 출력

데이터들의 index 번호를 출력하면 각 그룹에 속해있는 데이터들의 index 번호를 알 수 있다. 결과적으로 아래와 같이 예측할 수 있다.

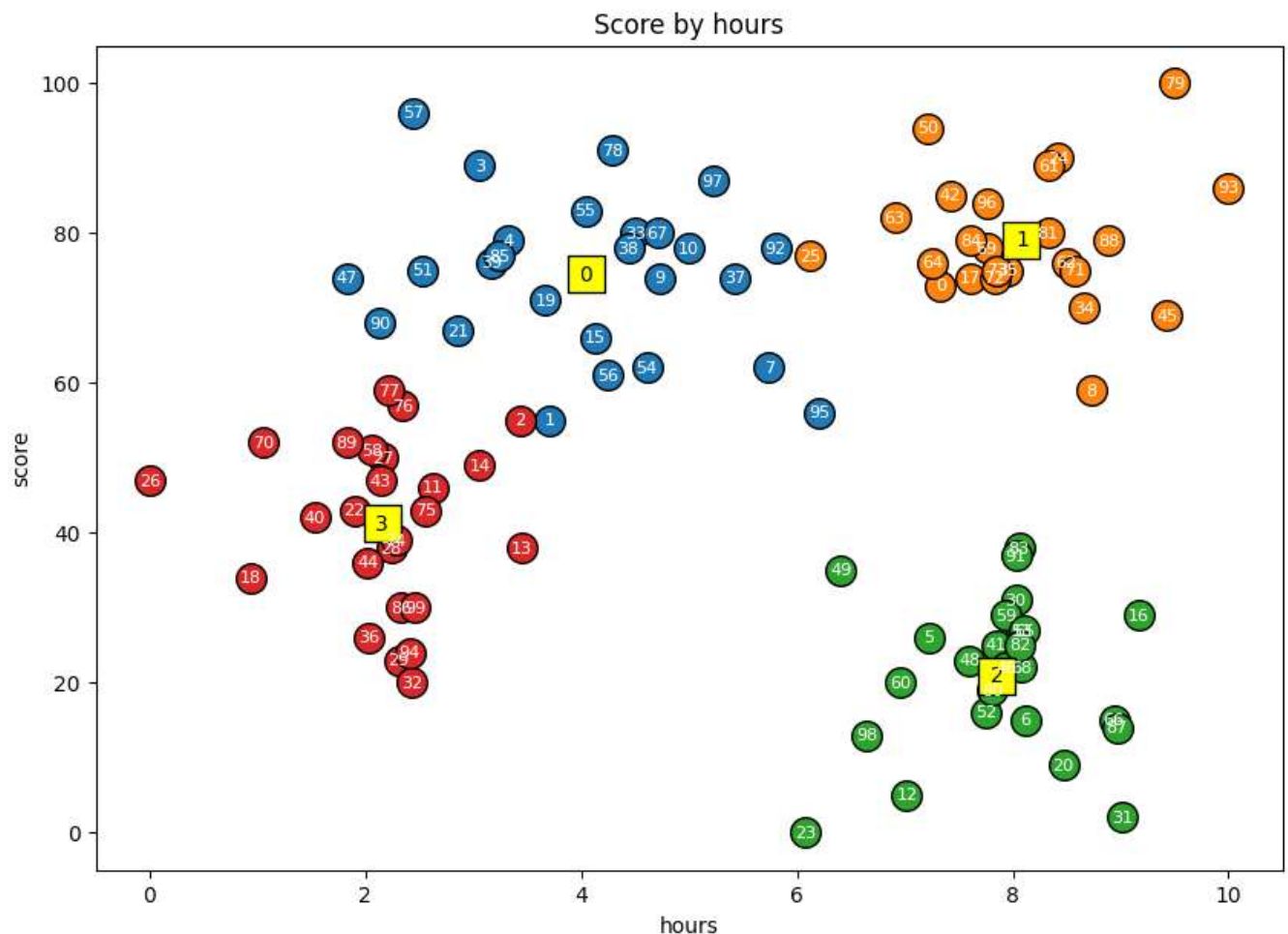
- 1) 0번 클러스터: 공부시간은 비교적 적지만 성적은 높은 그룹이므로 이과목외 다른 과목을 공부하라고 가이드 할 수 있다.
- 2) 1번 클러스터: 공부시간은 많은데 성적이 낮은 그룹이므로 집중력을 높이는 방법이나 효율적인 공부 방법을 가이드 할 수 있다.
- 3) 2번 클러스터: 공부시간도 많고 성적도 높은 그룹으로 0번 클러스터처럼 시간을 단축할 수 있는 방법을 가이드 할 수 있다.
- 4) 3번 클러스터: 공부시간도 적고 성적도 낮은 그룹으로 공부시간을 늘려서 성적을 높일 수 있도록 가이드 할 수 있다.

```
plt.figure(figsize=(7, 7))

for cluster in range(4):
    indices = np.where(y_kmeans==cluster)
    plt.scatter(X_org[indices, 0], X_org[indices, 1], s=200, ec='black')
    plt.scatter(centers_org[cluster, 0], centers_org[cluster, 1],
                s=300, ec='black', c='yellow', marker='s')
    plt.text(centers_org[cluster, 0], centers_org[cluster, 1], cluster, va='center', ha='center')

for idx, x in enumerate(X_org):
    plt.text(x[0], x[1], f'{idx}', size=8, color='white', ha='center', va='center')

plt.title('Score by hours')
plt.xlabel('hours')
plt.ylabel('score')
plt.show()
```



• 퀴즈

결혼식장에서 피로연의 식수 인원을 올바르게 예측하지 못하여 버려지는 음식으로 고민이 많다고 합니다. 현재까지 진행된 결혼식에 대한 결혼식 참석 인원과 그중에서 식사하는 인원의 데이터가 제공될 때, 아래 각 문항에 대한 코드를 작성하시오.

• 퀴즈 실습

- 새로운 파일 '06.퀴즈.ipynb'를 생성한 후 필요한 라이브러리를 import 한다.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

- 데이터 파일로부터 데이터를 읽어서 결혼식 참석 인원(total), 식수 인원(reception)을 각각의 변수로 저장한다.

```
dataset = pd.read_csv('data/QuizData.csv')
dataset.head()
```

```
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
X[:5], y[:5]
```

```
(array([[118],
        [253],
        [320],
        [ 94],
        [155]], dtype=int64),
 array([ 62, 148, 201,  80,  92], dtype=int64))
```

- 전체 데이터를 훈련 세트와 테스트 세트로 분리한다. 이때 비율은 75:25로 한다.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
```

- 독립변수(X_train), 종속변수(y_train) 값들을 출력한다.

```
X_train, y_train
```

```
(array([[262],
        ...
        [ 86]], dtype=int64),
 array([183, 147,  68,  92, 201, 131,  76, 152, 187, 152, 199,  80,  62,
        149,  58], dtype=int64))
```

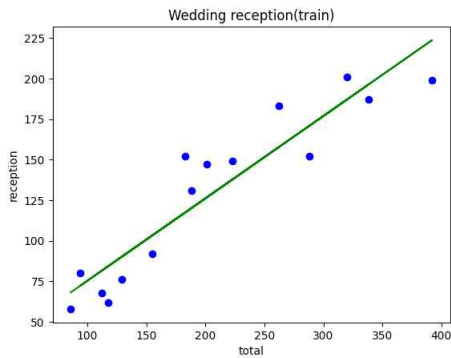
- 훈련 세트를 이용하여 단순 선형 회귀(Simple Linear Regression) 객체를 생성하여 학습 후 모델을 생성한다.

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
reg.fit(X_train, y_train)
```

```
▼ LinearRegression
LinearRegression()
```

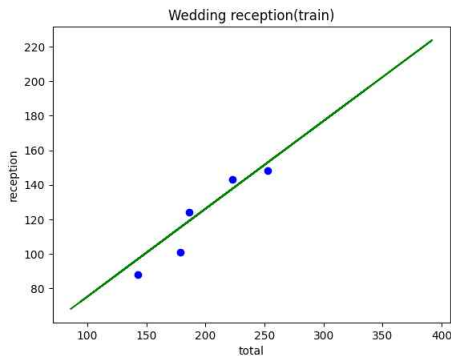
- 훈련세트 실제 값을 산점도(scatter) 그래프로 훈련세트 예측 값을 꺾은 선(plot) 그래프로 시각화한다.

```
plt.scatter(X_train, y_train, color='blue')
plt.plot(X_train, reg.predict(X_train), color='green')
plt.title('Wedding reception(train)')
plt.xlabel('total')
plt.ylabel('reception')
plt.show()
```



- 테스트세트 실제 값을 산점도(scatter) 그래프로 훈련세트 예측 값을 꺾은 선(plot) 그래프로 시각화한다.

```
plt.scatter(X_test, y_test, color='blue')
plt.plot(X_train, reg.predict(X_train), color='green')
plt.title('Wedding reception(test)')
plt.xlabel('total')
plt.ylabel('reception')
plt.show()
```



- 훈련 세트, 테스트 세트에 대해 각각 모델 평가 점수를 구한다.

```
reg.score(X_train, y_train), reg.score(X_test, y_test)

(0.8707088403321211, 0.8634953212566615)
```

- 결혼식 참석 인원이 300명일 때 예상되는 식수 인원을 구한다.

```
total = 300
y_pred = reg.predict([[total]])
print(f'결혼식 인원 {total}명에 대한 예상 식수 인원은 {np.round(y_pred[0]).astype(int)}입니다.')
```

결혼식 인원 300명에 대한 예상 식수 인원은 177입니다.

• 영화 추천 시스템

추천 시스템은 영화나 노래 등을 추천하는데 사용되며 주로 관심사나 이용 내역을 기반으로 추천한다. 이번 영화 추천 프로젝트에서 사용할 데이터는 Kaggle 사이트의 TMDB 5000을 사용한다.

• Kaggle이란?

Kaggle은 데이터 분석 및 머신러닝에 대한 학습 플랫폼이자, 경쟁할 수 있는 플랫폼입니다. 기업, 기관 또는 특정 사용자가 데이터를 첨부해서 문제를 제출하면 Kaggle 사용자 누구나 문제에 대한 답을 제출할 수 있다.

• TMDB 5000 Movie Dataset이란?

TMDB 5000 영화 데이터 세트는 유명한 영화 데이터 정보 사이트인 IMDB의 많은 영화 중 주요 5000개 영화에 대한 메타 정보를 새롭게 가공해 Kaggle에서 제공하는 데이터 세트이다.

• Demographic Filtering (인구 통계학적 필터링)

많은 사람이 일반적으로 좋아하는 영화를 추천하는 방식이다. 예를 들어 영화가 개봉했을 경우 인기가 많아 천만 관객을 돌파하는 등 누가 봐도 좋아할 만한 영화를 추천하는 방식이다. (평점, 평가 수로 추천한다.)

• TMDB 5000 데이터 다운로드

- 1) [구글]-[TMDB 5000]-[Kaggle 사이트이동]-[상단 Code Tab]-[우측 Hotness 메뉴]-[Most Votes] 선택한다.
- 2) [Getting Started with a Movie Recommendation System]-[Input] 탭을 선택한다.
- 3) 'tmdb_5000_credits.csv', 'tmdb_5000_movies.csv' 두 파일을 다운로드 후 프로젝트 폴더로 이동한다.

- 새로운 '07.영화추천(인구통계학적).ipynb' 파일을 생성한 후 필요한 라이브러리를 import 한다.

```
import pandas as pd
import numpy as np
```

- csv 데이터 파일을 읽어 데이터프레임 변수에 각각 저장한다.

```
df1 = pd.read_csv('data/tmdb_5000_credits.csv')
df2 = pd.read_csv('data/tmdb_5000_movies.csv')
```

• 데이터분석을 위한 데이터 전처리

- 1) 데이터프레임(df1)과 데이터프레임(df2)를 병합을 위하여 구조를 확인한다.

- 데이터프레임(df1)의 move_id와 데이터프레임(df2)의 id가 일치하므로 두 컬럼을 키로 지정하여 병합이 가능하다.

df1.head(3)

	movie_id	title	cast	crew
0	1995	Avatar	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	Pirates of the Caribbean: At World's End	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	Spectre	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...

df2.head(1)

	budget	genres	homepage	id	keywords	original_language	original_title	overview	popularity
0	237000000	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}]	http://www.avatarmovie.com/	1995	[{"id": 1463, "name": "culture clash"}, {"id": ...	en	Avatar	In the 22nd century, a paraplegic Marine is di...	150.437577

- 데이터프레임(df1)은 4803행, 4열, 데이터프레임(df2)는 4803행, 20열로 구성되어 있다.

```
df1.shape, df2.shape
```

```
((4803, 4), (4803, 20))
```

- 데이터프레임(df1)의 'title' 칼럼과 데이터프레임(df2)의 'title' 칼럼의 값이 같은지 비교해 본다.

```
df1['title'].equals(df2['title'])
```

```
True
```

- 데이터프레임(df1)의 칼럼 목록을 출력한다.

```
df1.columns
```

```
Index(['movie_id', 'title', 'cast', 'crew'], dtype='object')
```

- 2) df1과 df2의 데이터프레임을 병합하기 위하여 df1의 칼럼 중 'movie_id'를 'id'로 변경한다.

```
df1.rename(columns={'movie_id':'id'}, inplace=True)
```

```
df1.columns
```

```
Index(['id', 'title', 'cast', 'crew'], dtype='object')
```

- 데이터프레임(df1)의 칼럼 중 'movie_id'가 'id'로 변경되었는지 데이터프레임을 출력해 본다.

```
df1.head()
```

	id	title	cast
0	19995	Avatar	[{"cast_id": 242, "character": "Jake Sully", "... [{"credit_id":
1	285	Pirates of the Caribbean: At World's End	[{"cast_id": 4, "character": "Captain Jack Spa... [{"credit_id":
2	206647	Spectre	[{"cast_id": 1, "character": "James Bond", "cr... [{"credit_id": "
3	49026	The Dark Knight Rises	[{"cast_id": 2, "character": "Bruce Wayne / Ba... [{"credit_id": "
4	49529	John Carter	[{"cast_id": 5, "character": "John Carter", "c... [{"credit_id":

- 3) 'title' 칼럼은 df1과 df2에 중복되므로 'title' 칼럼을 제외한 나머지 칼럼으로 데이터프레임(df_temp)을 만든다.

```
df_temp = df1[['id', 'cast', 'crew']]
```

```
df_temp.head()
```

	id	cast	crew
0	19995	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...
3	49026	[{"cast_id": 2, "character": "Bruce Wayne / Ba...	[{"credit_id": "52fe4781c3a36847f81398c3", "de...
4	49529	[{"cast_id": 5, "character": "John Carter", "c...	[{"credit_id": "52fe479ac3a36847f81398c3", "de...

4) df2 데이터프레임과 위에서 만든 df_temp의 데이터프레임을 id가 같은 것끼리 병합하여 df2 데이터프레임에 저장한다.

```
df2 = df2.merge(df_temp, on='id')
df2.head(1)
```

status	tagline	title	vote_average	vote_count	cast	crew
Released	Enter the World of Pandora.	Avatar	7.2	11800	[[{"cast_id": 242, "character": "Jake Sully", "...	[[{"credit_id": "52fe48009251416c750aca23", "de...

• 평점 가중치 구하기

영화를 평가하기 위해서는 평점도 중요하고 평가 수도 중요하므로 평점에 가중치를 주는 아래 공식을 이용해 영화의 평점 가중치를 구한다.

$$\text{Weighted Rating (WR)} = \left(\frac{v}{v+m} \cdot R \right) + \left(\frac{m}{v+m} \cdot C \right)$$

where,

- v is the number of votes for the movie;
- m is the minimum votes required to be listed in the chart;
- R is the average rating of the movie; And
- C is the mean vote across the whole report

1) 데이터프레임(df2)의 전체 영화들의 평점(vote_average)의 평균을 구한다.

```
C = df2['vote_average'].mean()
C
```

```
6.092171559442016
```

2) 평가 수(vote_count)가 적은 경우를 제외하기 위하여 하위 90%, 상위 10% 지점의 값을 구한다.

```
m = df2['vote_count'].quantile(0.9)
m
```

```
1838.4000000000015
```

3) 평가 수(vote_count)가 상위 10%보다 크거나 같은 데이터를 필터링해서 새로운 데이터프레임(df_movies)에 저장한다.

```
df_movies = df2.copy()
filt = df2['vote_count'] >= m
df_movies = df_movies[filt]
```

```
df2.shape, df_movies.shape
```

```
((4803, 22), (481, 22))
```

- 가장 적은 평가 수(1840) ~ 가장 많은 평가 수(13752)까지의 데이터만 처리한다.

```
df_movies['vote_count'].sort_values()
```

```
2585      1840
195       1851
2454      1859
597       1862
1405      1864
...
788      10995
16       11776
0        11800
65       12002
96       13752
Name: vote_count, Length: 481, dtype: int64
```

- 평점 가중치를 구하는 함수를 작성한다.

```
def weighted_rating(x, m = m, C = C):
```

```
    v = x['vote_count']
```

```
    R = x['vote_average']
```

```
    return (v / (v + m) * R) + (m / (v + m) * C)
```

$$\text{Weighted Rating (WR)} = \left(\frac{v}{v+m} \cdot R\right) + \left(\frac{m}{v+m} \cdot C\right)$$

- weighted_rating 함수를 row 단위(axis=1) 로 적용하여 새로운 'score' 컬럼에 저장한다.

```
df_movies['score'] = df_movies.apply(weighted_rating, axis=1)
```

```
df_movies.head(1)
```

status	tagline	title	vote_average	vote_count	cast	crew	score
Released	Enter the World of Pandora.	Avatar	7.2	11800	[[{"cast_id": 242, "character": "Jake Sully", "...	[[{"credit_id": "52fe48009251416c750aca23", "de...	7.050669

- 영화정보 데이터프레임을 'score' 순으로 내림차순 정렬 후 10개의 데이터를 출력한다.

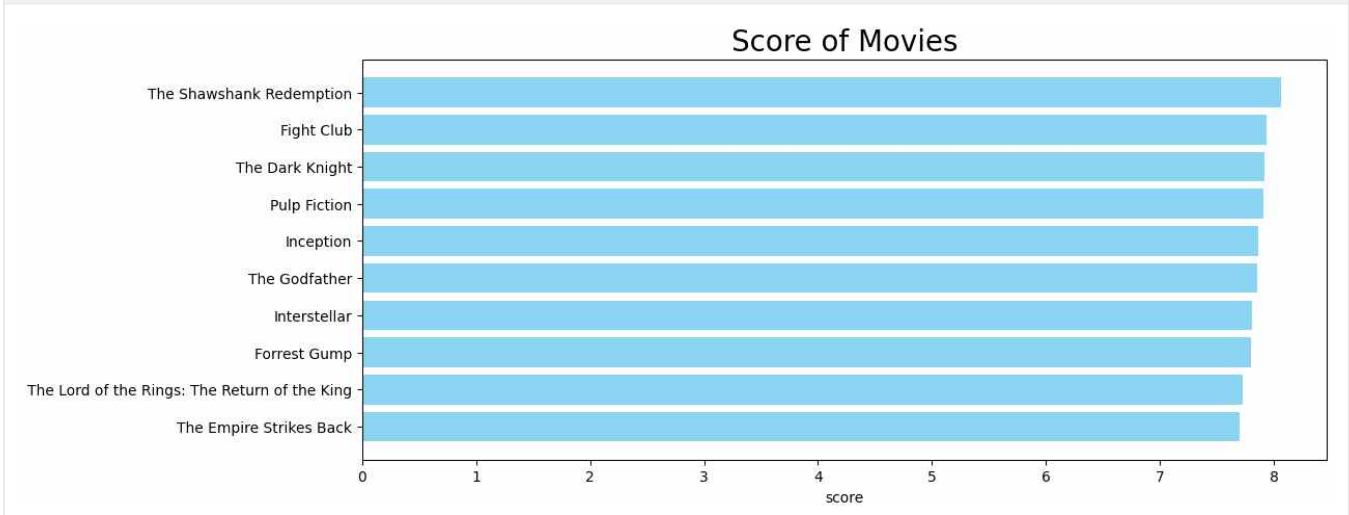
```
df_movies = df_movies.sort_values('score', ascending=False)
```

```
df_movies[['title', 'vote_count', 'vote_average', 'score']].head(10)
```

	title	vote_count	vote_average	score
1881	The Shawshank Redemption	8205	8.5	8.059258
662	Fight Club	9413	8.3	7.939256
65	The Dark Knight	12002	8.2	7.920020
3232	Pulp Fiction	8428	8.3	7.904645
96	Inception	13752	8.1	7.863239
3337	The Godfather	5893	8.4	7.851236
95	Interstellar	10867	8.1	7.809479
809	Forrest Gump	7927	8.2	7.803188
329	The Lord of the Rings: The Return of the King	8064	8.1	7.727243
1990	The Empire Strikes Back	5879	8.2	7.697884

7) 데이터프레임(df_movies)에서 score 컬럼 값이 큰 순으로 정렬하여 시각화 한다. '쇼생크 탈출' 영화 점수가 가장 높다.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 4))
plt.barh(df_movies['title'].head(10), df_movies['score'].head(10), color='skyblue')
plt.gca().invert_yaxis()
plt.xlabel('score')
plt.title('Score of Movies', size=20)
plt.show()
```



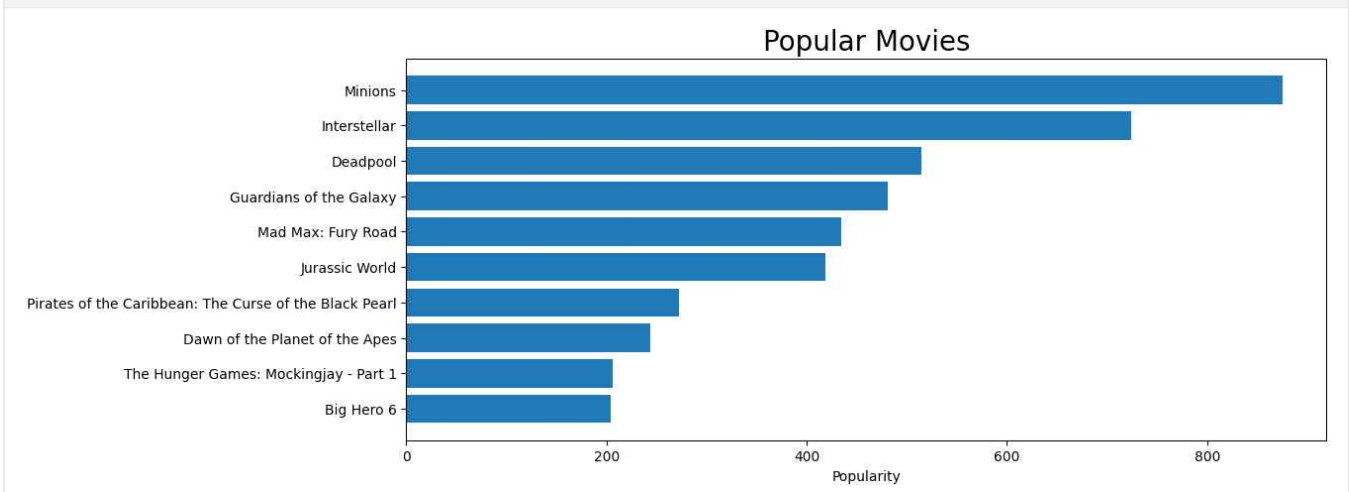
8) 데이터프레임(df2)를 'popularity' 컬럼의 값이 큰 순으로 정렬하여 데이터프레임(df_pop)에 저장한다.

```
df_pop = df2.sort_values('popularity', ascending=False)
```

9) 데이터프레임(df_pop)에서 'popularity' 컬럼 값이 큰 순으로 정렬하여 시각화 한다. '미니언즈' 영화 인기도가 가장 높다.

위에서 계산한 영화점수 Top10과 인기 있는 영화 Top 10을 비교하면 겹치는 영화는 '인터스텔라' 하나이다. 이유는 인기를 계산하는 방식이 위에서 계산한 영화 점수 계산 방식과 다르기 때문이다.

```
plt.figure(figsize=(12, 4))
plt.barh(df_pop['title'].head(10), df_pop['popularity'].head(10))
plt.gca().invert_yaxis()
plt.xlabel('Popularity')
plt.title('Popular Movies', size=20)
plt.show()
```



• Content Based Filtering (콘텐츠 기반 필터링)

이 방식은 사용자가 특정 item을 선호하는 경우 그 item과 비슷한 콘텐츠를 가진 다른 item을 추천해 주는 방식이다.

1) 줄거리(overview) 기반 추천

영화의 줄거리('overview') 분석하여 사용자가 재밌게 관람했던 영화와 비슷한 줄거리 영화를 필터링하여 추천한다.

• 데이터 생성

- 새로운 '08.영화추천(콘텐츠기반).ipynb' 파일을 생성한 후 데이터 파일을 읽어 데이터프레임 변수(df1, df2)에 저장한다.

```
import pandas as pd
import numpy as np
df1 = pd.read_csv('data/tmdb_5000_credits.csv')
df2 = pd.read_csv('data/tmdb_5000_movies.csv')
```

- 데이터프레임(df1)의 'movie_id' 컬럼 이름을 'id' 컬럼 이름으로 변경한다.

```
df1.rename(columns={'movie_id':'id'}, inplace=True)
df1.columns
```

```
Index(['id', 'title', 'cast', 'crew'], dtype='object')
```

- 'title' 컬럼을 제외한 'id', 'cast', 'crew' 컬럼들만 추출하여 데이터프레임(df_temp)에 저장한다.

```
df_temp = df1[['id', 'cast', 'crew']]
df_temp.head()
```

	id	cast	crew
0	19995	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...
3	49026	[{"cast_id": 2, "character": "Bruce Wayne / Ba...	[{"credit_id": "52fe4781c3a36847f81398c3", "de...
4	49529	[{"cast_id": 5, "character": "John Carter", "c...	[{"credit_id": "52fe479ac3a36847f813eaa3", "de...

- 데이터프레임(df2)와 데이터프레임(df_temp)을 'id' 컬럼이 같은 것 끼리 병합하여 데이터프레임(df2)에 저장한다.

```
df2 = df2.merge(df_temp, on='id')
df2.head()
```

	status	tagline	title	vote_average	vote_count	cast	crew
	Released	Enter the World of Pandora.	Avatar	7.2	11800	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
	Released	At the end of the world, the adventure begins.	Pirates of the Caribbean: At World's End	6.9	4500	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...

- 분석에 사용할 데이터프레임(df2)의 줄거리 'overview'칼럼을 출력해 본다. 줄거리 텍스트를 분석하여 영화를 추천한다.

```
df2['overview'].head(5)
```

```
0    In the 22nd century, a paraplegic Marine is di...
1    Captain Barbossa, long believed to be dead, ha...
2    A cryptic message from Bond' s past sends him o...
3    Following the death of District Attorney Harve...
4    John Carter is a war-weary, former military ca...
Name: overview, dtype: object
```

• Bag Of Words(BOW)로 피쳐 벡터화 후 유사도 매트릭스 생성

문장의 단어들을 분리하여 해당 문장에 각 단어들이 몇 번이 나오는지 분리하여 vector로 표현한 것을 BOW라고 하고 이러한 작업을 피쳐 벡터화(Feature Vectorization)라고 한다.

문장	I	am	a	boy	girl	결과
I am a body	1	1	1	1	0	(1, 1, 1, 1, 0)
I am a girl	1	1	1	0	1	(1, 1, 1, 0, 1)

문장이 100개이고 모든 문장에서 나온 단어 10,000개인 경우
 $100 * 10,000 = 100만$

	단어 1	단어 2	단어 3	단어 4	...	단어 10000
문장 1	1	1	3	0	...	1
문장 2	0	4	2	1	...	0
문장 3	1	1	3	0	...	1
...	0	0	0	1	...	2
문장 100	1	1	3	0	...	1

• sklearn의 BOW 벡터를 만든 클래스

- 1) CounterVectorizer : 단어의 빈도를 Count 하여 BOW 벡터(Vector)로 만든다.
- 2) TfidfVectorizer : CounterVectorizer와 비슷하나 TF-IDF 방식으로 단어의 가중치를 조정한 BOW 벡터를 만든다.

- 단어의 가중치를 조정하여 BOW 벡터를 생성하는 TfidfVectorizer 클래스를 import 하여 객체를 생성한다.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(stop_words='english') #stop_words='english' : the, a와 같은 의미 없는 단어를 제외한다.
```

- ENGLISH STOP WORDS에 어떤 단어들이 있는지 확인해 본다.

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
ENGLISH_STOP_WORDS
```

```
frozenset({'a',
          'about',
          'above',
          'across',
          'after',
          'afterwards',
          'again',
          ...,
          'yours',
          'yourself',
          'yourselves'})
```

- 'overview'에 null 값이 하나라도 있는지 확인해 본다.

```
df2['overview'].isnull().any()

True
```

- 'overview'에 null이 있으면 모두 비어있는 값('')으로 Update 한다. (vector 작업에 null이 있는 경우 오류발생)

```
df2['overview'] = df2['overview'].fillna('')
```

- 'overview'를 BOM vector 데이터(행렬)로 변경하여 저장한다. (문서:4,803개, 단어:20,978개)

```
tfidf_matrix = tfidf.fit_transform(df2['overview'])
tfidf_matrix.shape

(4803, 20978)
```

- 전체 데이터 수를 출력한다.

```
tfidf_matrix

<4803x20978 sparse matrix of type '<class 'numpy.float64''
with 125840 stored elements in Compressed Sparse Row format>
```

- cosine 유사도(방향) 값을 구하기 위한 함수를 import하고 유사도 매트릭스(cosine_sim)를 생성한 후 결과를 출력해 본다.

```
from sklearn.metrics.pairwise import linear_kernel
cosine_sim = linear_kernel(tfidf_matrix, tfidf_matrix)
cosine_sim

array([[1.         , 0.         , 0.         , ..., 0.         , 0.         ,
        0.         ],
       [0.         , 1.         , 0.         , ..., 0.02160533, 0.         ,
        0.         ],
       ...,
       [0.         , 0.02160533, 0.01488159, ..., 1.         , 0.01609091,
        0.00701914],
       [0.         , 0.         , 0.         , ..., 0.01609091, 1.         ,
        0.01171696],
       [0.         , 0.         , 0.         , ..., 0.00701914, 0.01171696,
        1.         ]])
```

```
cosine_sim.shape

(4803, 4803)
```

	문장 1	문장 2	문장 3	...	문장 4801	문장 4802	문장 4803
문장 1	1.	0.	0.	...	0.	0.	0.
문장 2	0.	1.	0.	...	0.02160533	0.	0
...	...	...	...	...	...	...	...
문장 4803	0.	0.	0.	...	0.00701914	0.01171696	1

- 유사도 검색을 위한 영화 제목으로 영화 index 구하기

영화 이름을 입력하면 이 영화와 가장 줄거리가 비슷한 영화들을 위에서 생성한 유사도 매트릭스(cosine_sim)을 참고하여 추천한다. 유사도 매트릭스에서 유사도 값들을 검색하려면 영화 제목이 몇 번째 영화인지를 알 수 있는 index 값을 구해야 한다.

- 영화 제목을 index로 index를 value값으로 변경하는 시리즈를 생성하면 영화 제목으로 몇 번째 영화인지 index를 구할 수 있다.

```
indices = pd.Series(df2.index, index=df2['title'])
indices
```

title	
Avatar	0
Pirates of the Caribbean: At World's End	1
Spectre	2
The Dark Knight Rises	3
John Carter	4
...	
Shanghai Calling	4801
My Date with Drew	4802
Length: 4803, dtype: int64	

- 위에서 생성한 indices 시리즈를 사용하여 'Avatar'와 'The Dark Knight Rises' index 값을 출력한다.

```
indices['Avatar']
```

0

```
filt = df2['title'] == 'Avatar'
df2[filt].index[0]
```

0

- 데이터프레임(df2)에 index 0의 데이터 값을 일차원, 이차원 배열로 출력하여 제목이 맞는지 확인해 본다.

```
df2.iloc[0]
```

budget	237000000
genres	[{"id": 28, "name": "Action"}, {"id": 12, "nam...
homepage	http://www.avatarmovie.com/
id	1995
keywords	[{"id": 1463, "name": "culture clash"}, {"id":...
title	Avatar
vote_average	7.2
vote_count	11800
cast	[{"cast_id": 242, "character": "Jake Sully", "...
crew	[{"credit_id": "52fe48009251416c750aca23", "de...
Name: 0, dtype: object	


```
df2.iloc[[0]]
```

spoken_languages	status	tagline	title	vote_average	vote_count	cast	crew
[{"iso_639_1": "en", "name": "English"}, {"iso...	Released	Enter the World of Pandora.	Avatar	7.2	11800	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...

▪ **영화 제목으로 유사도가 높은 상위 10개의 영화 목록 반환 함수 작성**

영화 제목을 입력하면 몇 번째 영화인지 index 값을 구하여 유사도 매트릭스에서 index 번째 유사도 값들을 구한 후 내림차순 정렬하여 상위 10개의 영화 index들을 반환하는 함수를 작성한다.

1. 영화 제목을 통해서 전체 데이터(indices)에서 그 영화의 index(해당 영화의 위치) 값을 얻어온다.

```
indices['The Dark Knight Rises']
```

```
3
```

2. 유사도 매트릭스(cosine_sim)에서 index에 해당하는 유사도 데이터를 (index, 유사도) 형태로 얻어와 list에 저장한다.

```
cosine_sim[3]
```

```
array([0.02499512, 0.          , 0.          , ..., 0.03386366, 0.04275232,
       0.02269198])
```

```
test_sim_scores=list(enumerate(cosine_sim[3]))
```

```
test_sim_scores
```

```
[(0, 0.0249951158376727),
 (1, 0.0),
 (2, 0.0),
 (3, 1.0),
 (4, 0.010433403719159354),
 (5, 0.0051446018158107934),
 (6, 0.01260063243546246),
 (7, 0.026954270578912674),
 (8, 0.02065221688538951),
 (9, 0.1337400906655523),
 (10, 0.0),
 (11, 0.0),
 (12, 0.0),
 (13, 0.0),
 (14, 0.0),
 ...
 (996, 0.0),
 (997, 0.0),
 (998, 0.0),
 (999, 0.0),
 ...]
```

3. cosine 유사도가 높은 순으로 정렬한 후 자기 자신(index=0)을 제외한 상위 10개의 추천 영화를 Slicing 한다.

▪ 아래는 일반 함수와 람다식(익명의 작은 함수)을 사용하여 cosine 유사도 값을 구하는 예제이다.

```
def get_second(x):
    return x[1]
get_second(test_sim_scores[0])
```

```
0.024995115837672686
```

```
(lambda x: x[1])(test_sim_scores[0])
```

```
0.024995115837672686
```

- 람다식을 이용하여 유사도 값을 구한 후 내림차순 정렬하여 자신을 제외한 10개의 추천 영화를 출력한다.

```
test_sim_scores = sorted(test_sim_scores, key=lambda x : x[1], reverse=True)
test_sim_scores = test_sim_scores[1:11]
test_sim_scores
```

```
[(65, 0.30151176591665485),
 (299, 0.29857045255396825),
 (428, 0.2878505467001694),
 (1359, 0.264460923827995),
 (3854, 0.18545003006561456),
 (119, 0.16799626199850706),
 (2507, 0.16682891043358278),
 (9, 0.1337400906655523),
 (1181, 0.13219702138476813),
 (210, 0.13045537014449818)]
```

4. 위에서 생성한 10개의 추천 영화 유사도 목록(test_sim_scores)에서 영화 인덱스 정보만 추출한다.

```
test_movie_indices = [i[0] for i in test_sim_scores]
test_movie_indices
```

```
[65, 299, 428, 1359, 3854, 119, 2507, 9, 1181, 210]
```

5. 전체 원본 데이터프레임(df2)에서 위 영화 인덱스 정보를 통해 영화 제목을 추출한다.

```
df2.loc[test_movie_indices, 'title'] #df2['title'].loc[test_movie_indices] 같은 결과가 출력된다.
```

```
65          The Dark Knight
299      Batman Forever
428      Batman Returns
1359          Batman
3854  Batman: The Dark Knight Returns, Part 2
119      Batman Begins
2507          Slow Burn
9      Batman v Superman: Dawn of Justice
1181          JFK
210      Batman & Robin
Name: title, dtype: object
```

6. 최종적으로 위 작업을 함수로 작성한다.

```
#영화 추천 함수
```

```
def get_recommendations(title, cosine_sim=cosine_sim):
    filt = df2['title']==title
    idx = df2[filt].index[0] #영화 제목을 통해 영화의 index 값을 얻기
    sim_scores = list(enumerate(cosine_sim[idx])) #cosine 유사도에서 idx에 해당하는 데이터를 [idx, 유사도] 형태 list로 얻기
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True) #cosine 유사도 기준으로 내림차순 정렬
    sim_scores = sim_scores[1: 11] #자기 자신을 제외한 10개의 추천 영화를 Slicing
    movie_indices = [i[0] for i in sim_scores] #추천 영화 목록 10개의 index 정보 추출
    titles = df2.loc[movie_indices, 'title'] #index 정보를 통해 영화 제목 추출
    return titles
```

- 작성한 함수(get_recommendations)를 적용하여 추천 영화 목록을 추출

원본 데이터프레임(df2)에서 영화제목을 20개 출력한 후 영화제목을 참고해서 줄거리가 비슷한 추천 영화 목록을 추출한다.

1. 원본 데이터(df2)에서 영화 제목 20개를 출력한다.

df2['title'][:20]	
0	Avatar
1	Pirates of the Caribbean: At World's End
2	Spectre
3	The Dark Knight Rises
4	John Carter
5	Spider-Man 3
6	Tangled
7	Avengers: Age of Ultron
8	Harry Potter and the Half-Blood Prince
9	Batman v Superman: Dawn of Justice
10	Superman Returns
11	Quantum of Solace
12	Pirates of the Caribbean: Dead Man's Chest
13	The Lone Ranger
14	Man of Steel
15	The Chronicles of Narnia: Prince Caspian
16	The Avengers
17	Pirates of the Caribbean: On Stranger Tides
18	Men in Black 3
19	The Hobbit: The Battle of the Five Armies
Name: title, dtype: object	

2. 영화 제목 'Avengers: Age of Ultron'과 줄거리가 비슷한 10개의 영화를 출력한다.

get_recommendations('Avengers: Age of Ultron')	
16	The Avengers
79	Iron Man 2
68	Iron Man
26	Captain America: Civil War
227	Knight and Day
31	Iron Man 3
1868	Cradle 2 the Grave
344	Unstoppable
1922	Gettysburg
531	The Man from U.N.C.L.E.
Name: title, dtype: object	

3. 영화 제목 'The Avengers'와 줄거리가 비슷한 10개의 영화를 출력한다.

get_recommendations('The Avengers')	
7	Avengers: Age of Ultron
3144	Plastic
1715	Timecop
4124	This Thing of Ours
3311	Thank You for Smoking
3033	The Corruptor
588	Wall Street: Money Never Sleeps
2136	Team America: World Police
1468	The Fountain
1286	Snowpiercer
Name: title, dtype: object	

2) 다양한 요소 기반 추천 (genre, keywords, cast, crew 기반 추천)

내가 좋아하는 장르(genre), 키워드(keywords), 주연배우(cast), 감독(director)을 합쳐 줄거리 기반 추천처럼 영화를 추천한다.

• 데이터를 분석 및 처리하기 위한 전처리

1. genre, keywords, cast, crew 칼럼의 string 타입을 list 타입으로 변환한다.

```
df2.head(2)
```

	budget	genres	homepage	id	keywords	cast	crew
0	237000000	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}]	http://www.avatarmovie.com/	19995	[{"id": 1463, "name": "culture clash"}, {"id": 726, "name": "ocean"}]	[{"cast_id": 242, "character": "Jake Sully", "credit_id": "52fe48009251416c750aca23"}]	[{"credit_id": "52fe48009251416c750aca23"}]
1	300000000	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}]	http://disney.go.com/disneypictures/pirates/	285	[{"id": 270, "name": "ocean"}, {"id": 726, "name": "ocean"}]	[{"cast_id": 4, "character": "Captain Jack Sparrow", "credit_id": "52fe4232c3a36847f800b579"}]	[{"credit_id": "52fe4232c3a36847f800b579"}]

```
df2.loc[0, 'genres']
```

```
'[{"id":28,"name":"Action"}, {"id":12,"name":"Adventure"}, {"id":14,"name":"Fantasy"}, {"id":878,"name":"Science Fiction"}]'
```

```
type(df2.loc[0, 'genres'])
```

```
str
```

▪ literal_eval 함수를 이용하여 주어진 문자열이 표현하는 데이터 구조를 Python 객체로 변환해 준다.

```
from ast import literal_eval
```

```
df2['genres'] = df2['genres'].apply(literal_eval)
```

```
([{"id": 28, "name": "Action"}], list)
```

```
df2.loc[0, 'genres']
```

```
[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 878, "name": "Science Fiction"}]
```

```
type(df2.loc[0, 'genres'])
```

```
list
```

▪ features 목록(cast, crew, keywords)의 모든 칼럼의 타입을 str에서 list로 변경한다. (한번 실행)

```
features = ['cast', 'crew', 'keywords']
```

```
for feature in features:
```

```
    df2[feature] = df2[feature].apply(literal_eval)
```

2. 데이터프레임(df2)에서 'crew' 칼럼 값에서 감독 명(job이 Director)만 추출한다.

```
df2.loc[0, 'crew']
```

```
[
  ...
  {'credit_id': '52fe48009251416c750ac9c3',
   'department': 'Directing',
   'gender': 2,
   'id': 2710,
   'job': 'Director',
   'name': 'James Cameron'}
  ...
]
```

- director(감독)의 name(이름)을 추출하는 함수(get_director())를 작성한다.

```
def get_director(x):
    for i in x:
        if i['job'] == 'Director':
            return i['name']
    return np.nan #x값이 없는 경우 np.nan를 return 한다.
```

- 데이터프레임(df2)에 'director' 칼럼을 생성한 후 get_director() 함수를 적용하여 감독 이름을 추가한다.

```
df2['director'] = df2['crew'].apply(get_director)
df2['director']
```

```
0          James Cameron
1          Gore Verbinski
2          Sam Mendes
3    Christopher Nolan
4    Andrew Stanton
...
4798    Robert Rodriguez
4799    Edward Burns
4800    Scott Smith
4801    Daniel Hsia
4802    Brian Herzlinger
Name: director, Length: 4803, dtype: object
```

- 'director' 칼럼에 null이 있는 데이터를 출력해 확인 후 null값을 ''으로 수정한다.

```
filt = df2['director'].isnull() #df2['director'].isnull().sum() null값이 몇 개인지 출력된다.
df2[filt]
df2.fillna({'director':''}, inplace=True)
```

...	runtime	spoken_languages	status	tagline	title	vote_average	vote_count	cast	crew	director
...	95.0	[{"iso_639_1": "en", "name": "English"}]	Released	It's about the music	Flying By	7.0	2	[{"cast_id": 1, "character": "George", "credit..."}]	[]	NaN
...	88.0	[]	Released	NaN	Running Forever	0.0	0	[]	[]	NaN

3. 'cast', 'genres', 'keywords' list에서 여러 개의 항목 중 처음 3개의 항목만 중요하다 유추하고 추출한다.

- 원본 데이터프레임(df2)의 'cast' 컬럼에서 상위 3명만 추출한다.

```
df2.loc[0, 'cast']
```

```
[{'cast_id': 242,
  'character': 'Jake Sully',
  'credit_id': '5602a8a7c3a3685532001c9a',
  'gender': 2,
  'id': 65731,
  'name': 'Sam Worthington',
  'order': 0},
 {'cast_id': 3,
  'character': 'Neytiri',
  'credit_id': '52fe48009251416c750ac9cb',
  'gender': 1,
  'id': 8691,
  'name': 'Zoe Saldana',
  'order': 1},
 {'cast_id': 25,
  'character': 'Dr. Grace Augustine',
  'credit_id': '52fe48009251416c750aca39',
  'gender': 1,
  'id': 10205,
  'name': 'Sigourney Weaver',
  ... ]
```

- 원본 데이터프레임(df2)의 'genres' 컬럼의 상위 3개만 추출한다.

```
df2.loc[0, 'genres']
```

```
[{'id': 28, 'name': 'Action'},
 {'id': 12, 'name': 'Adventure'},
 {'id': 14, 'name': 'Fantasy'},
 {'id': 878, 'name': 'Science Fiction'}]
```

- 원본 데이터프레임(df2)의 'keywords' 컬럼에서 상위 3개만 추출한다.

```
df2.loc[0, 'keywords']
```

```
[{'id': 1463, 'name': 'culture clash'},
 {'id': 2964, 'name': 'future'},
 {'id': 3386, 'name': 'space war'},
 {'id': 3388, 'name': 'space colony'},
 ... ]
```

- list를 받아서 상위 3개만 추출하는 함수를 작성한다. list 타입이 아닌 경우 빈 배열을 return 한다.

```
def get_list(x):
    if isinstance(x, list):
        names = [i['name'] for i in x]
        if len(names) > 3:
            names = names[:3]
        return names
    return []
```

- 'cast', 'keywords', 'genres' 컬럼에 get_list 함수를 적용하여 각 컬럼을 업데이트한다.

```
features = ['cast', 'keywords', 'genres']

for feature in features:
    df2[feature] = df2[feature].apply(get_list)
```

- 원본 데이터프레임(df2) 데이터 세트의 상위 3개의 모든 컬럼을 출력한다.

```
df2[['title', 'director', 'cast', 'keywords', 'genres']].head(3)
```

	title	director	cast	keywords	genres
0	Avatar	James Cameron	[Sam Worthington, Zoe Saldana, Sigourney Weaver]	[culture clash, future, space war]	[Action, Adventure, Fantasy]
1	Pirates of the Caribbean: At World's End	Gore Verbinski	[Johnny Depp, Orlando Bloom, Keira Knightley]	[ocean, drug abuse, exotic island]	[Adventure, Fantasy, Action]
2	Spectre	Sam Mendes	[Daniel Craig, Christoph Waltz, Léa Seydoux]	[spy, based on novel, secret agent]	[Action, Adventure, Crime]

4. 값을 입력받아 빈칸을 없애고 대문자를 모두 소문자로 치환한다.

- list 또는 string을 받아서 빈칸을 없애고 대문자를 모두 소문자로 치환하는 함수를 작성한다.

```
def clean_data(x):
    if isinstance(x, list):
        return [str.lower(i.replace(' ', '')) for i in x] #빈칸을 없앤다.
    else:
        if isinstance(x, str):
            return str.lower(x.replace(' ', ''))
        else:
            return ''
```

- clean_data() 함수를 'cast', 'keywords', 'genres', 'director' 컬럼에 적용한다.

```
features = ['director', 'cast', 'keywords', 'genres']

for feature in features:
    df2[feature] = df2[feature].apply(clean_data)
```

- clean_data() 함수가 잘 적용되었는지 상위 3개의 데이터를 출력한다.

```
df2[['title', 'director', 'cast', 'keywords', 'genres']].head(3)
df2[features].head(3)
```

	title	director	cast	keywords	genres
0	Avatar	jamescameron	[samworthington, zoesaldana, sigourneyweaver]	[cultureclash, future, spacewar]	[action, adventure, fantasy]
1	Pirates of the Caribbean: At World's End	goreverbinski	[johnnydepp, orlandobloom, keiraknightley]	[ocean, drugabuse, exoticisland]	[adventure, fantasy, action]
2	Spectre	sammendes	[danielcraig, christophwaltz, léaseydoux]	[spy, basedonnovel, secretagent]	[action, adventure, crime]

5. 'keywords', 'cast', 'director', 'genres' 각 칼럼 데이터를 빈칸으로 연결한 값을 'soup' 칼럼에 추가한다.

```
def create_soup(x):
    str = x['director'] + ' '
    str += ' '.join(x['keywords']) + ' '
    str += ' '.join(x['cast']) + ' '
    str += ' '.join(x['genres'])
    return str
```

```
df2['soup'] = df2.apply(create_soup, axis=1)
df2['soup']
```

```
0      jamescameron cultureclash future spacewar samw...
1      goreverbinski ocean drugabuse exoticisland joh...
2      sammendes spy basedonnovel secretagent danielc...
3      christophernolan dcomics crimefighter terrori...
...
4801    danielhsia danielhenney elizacoupe billpaxton
4802    brianherzlinger obsession camcorder crush drew...
Name: soup, Length: 4803, dtype: object
```

▪ 'overview' 칼럼과 'soup' 칼럼을 비교해 보면 유사한 형태로 저장 되어있다.

```
df2['overview']
```

```
0      In the 22nd century, a paraplegic Marine is di...
1      Captain Barbossa, long believed to be dead, ha...
...
4800    "Signed, Sealed, Delivered" introduces a dedic...
4801    When ambitious New York attorney Sam is sent t...
4802    Ever since the second grade when he first saw ...
Name: overview, Length: 4803, dtype: object
```

• BOW 벡터 및 유사도 매트릭스 생성

▪ 각 단어가 모두 중요하므로 단어의 빈도를 Count 하여 BOW 벡터(Vector)로 만드는 CountVectorizer 클래스를 사용한다.

```
from sklearn.feature_extraction.text import CountVectorizer
count = CountVectorizer(stop_words='english')
count_matrix = count.fit_transform(df2['soup'])
count_matrix
```

```
<4803x11520 sparse matrix of type '<class 'numpy.int64'>'
with 42935 stored elements in Compressed Sparse Row format>
```

▪ CountVectorizer로 만든 벡터 객체의 cosine 유사도 매트릭스를 구한다.

```
from sklearn.metrics.pairwise import cosine_similarity
cosine_sim2 = cosine_similarity(count_matrix, count_matrix)
cosine_sim2
```

```
array([
  [1. , 0.3, 0.2, ..., 0. , 0. , 0. ],
  [0.3, 1. , 0.2, ..., 0. , 0. , 0. ],
  ...,
  [0. , 0. , 0. , ..., 0. , 1. , 0. ],
  [0. , 0. , 0. , ..., 0. , 0. , 1. ]
])
```

- 유사도 매트릭스를 이용한 상위 10개 영화 추천

- 제목으로 index 값을 추출하는 배열(indices)을 생성한다.

```
df2 = df2.reset_index()
indices = pd.Series(df2.index, index=df2['title'])
indices
```

```
title
Avatar                                0
Pirates of the Caribbean: At World's End  1
Spectre                                2
The Dark Knight Rises                    3
John Carter                             4
...
El Mariachi                           4798
Newlyweds                             4799
Signed, Sealed, Delivered              4800
Shanghai Calling                       4801
My Date with Drew                       4802
Length: 4803, dtype: int64
```

- 'The Dark Knight Rises' 제목의 index 값을 출력한다.

```
indices['The Dark Knight Rises']
```

```
3
```

- 'The Dark Knight Rises'과 유사한 영화 10개를 추천한다.

```
get_recommendations('The Dark Knight Rises', cosine_sim2)
```

```
65          The Dark Knight
119          Batman Begins
4638      Amidst the Devil's Wings
1196          The Prestige
3073      Romeo Is Bleeding
3326      Black November
1503          Takers
1986          Faster
303          Catwoman
747      Gangster Squad

Name: title, dtype: object
```

- 'The Dark Knight'과 유사한 영화 10개를 추천한다.

```
get_recommendations('The Dark Knight', cosine_sim2)
```

```
3          The Dark Knight Rises
119          Batman Begins
4638      Amidst the Devil's Wings
2398          Hitman
1720          Kick-Ass
1740      Kick-Ass 2
3326      Black November
1503          Takers
1986          Faster
303          Catwoman

Name: title, dtype: object
```

- 'The Martian'과 유사한 영화 10개를 추천한다.

```
get_recommendations('The Martian', cosine_sim2)
```

```
4          John Carter
95         Interstellar
365        Contact
256        Allegiant
1326       The 5th Wave
1958       On the Road
3043       End of the Spear
3373       The Other Side of Heaven
3392       Gerry
3698       Moby Dick
```

Name: title, dtype: object

```
indices['The Martian']
```

270

- 'The Martian'과 'John Carter' 두 영화정보를 비교해 본다.

```
df2.loc[270]
```

```
budget          108000000
genres          [drama, adventure, sciencefiction]
homepage        http://www.foxmovies.com/movies/the-martian
id              286217
keywords        [basedonnovel, mars, nasa]
original_language en
status          Released
tagline         Bring Him Home
title           The Martian
vote_average    7.6
vote_count      7268
cast            [mattdamon, jessicachastain, kristenwiig]
crew            [['credit_id': '5607a7e19251413050003e2c', 'de...
director        ridleyscott
soup            basedonnovel mars nasa mattdamon jessicachasta...
```

Name: 270, dtype: object

```
df2.loc[4]
```

```
budget          260000000
genres          [action, adventure, sciencefiction]
homepage        http://movies.disney.com/john-carter
id              49529
keywords        [basedonnovel, mars, medallion]
original_language en
original_title   John Carter
overview        John Carter is a war-weary, former military ca...
vote_average    6.1
vote_count      2124
cast            [taylorkitsch, lynncollins, samanthamorton]
crew            [['credit_id': '52fe479ac3a36847f813eaa3', 'de...
director        andrewstanton
soup            basedonnovel mars medallion taylorkitsch lynnc...
```

Name: 4, dtype: object

• 영화 추천 웹사이트

• 데이터 생성 및 라이브러리 설치

위 프로젝트에서 작성한 코사인 유사도(cosine_sim2) 파일을 이용하여 영화 제목을 입력하면 10개의 영화를 추천하는 웹페이지를 만든다.

- df2에서 'id', 'title' 컬럼들을 복사하여 movies 변수에 저장한다.

```
movies = df2[['id', 'title']].copy()
```

- 영화 추천에서 사용할 영화정보('movies') 데이터와 유사도('cosine_sim2') 데이터를 pickle 파일로 저장한다.

```
import pickle
pickle.dump(movies, open('data/movies.pickle', 'wb'))
pickle.dump(cosine_sim2, open('data/cosine_sim.pickle', 'wb'))
```

- 터미널에서 필요한 라이브러리 streamlit(Padas, Matplotlib, sklearn의 결과를 웹에서 확인), TMDb를 설치한다.

```
pip install streamlit #브라우저에 출력할 UI를 쉽게 제작할 수 있는 라이브러리
```

```
pip install tmdbv3api #TMDb에서 영화 상세 정보를 제공하는 라이브러리
```

• 웹페이지 실행 결과

Deploy

영화추천

Choose a movie you like

Kung Fu Panda

Recommend



쿵푸팬더 3



쿵푸팬더 2



작은 영웅 데스페로



발리언트



아틀란티스: 잃어버린 제국



가디언의 전설



에픽: 숲속의 전설



다이노소어 어드벤처 3D



정글 히어로



Running Forever

- streamlit을 이용한 웹페이지 작성

- [구글]-[TMDb] 사이트에서 회원가입 후 [오른쪽 상단]-[설정]에서 [API] 키를 생성한다.
- 웹 프로그램을 작성한 후 실행한다. [cmd창]에서 streamlit run app.py를 입력하여 실행한다.

[movie] app.py

```
import pickle
import streamlit as st
from tmdbv3api import Movie, TMDb

def get_recommendations(title):
    # 영화 제목을 통해 영화의 index 값을 얻기
    idx = movies[movies['title']==title].index[0]
    # cosine 유사도 매트릭스에서 idx에 해당하는 데이터를 (idx, 유사도) 형태로 얻기
    sim_scores = list(enumerate(cosine_sim[idx]))
    # cosine 유사도 기준으로 내림차순 정렬
    sim_scores = sorted(sim_scores, key=lambda x:x[1], reverse=True)
    # 자기 자신을 제외한 10개의 추천 영화를 슬라이싱
    sim_scores = sim_scores[1:11]
    # 추천 영화 목록 10개의 index 정보 추출
    movie_indices = [i[0] for i in sim_scores]

    # index 정보를 통해 영화 제목과 이미지 추출
    images = []
    titles = []
    for idx in movie_indices:
        id = movies.loc[idx, 'id']
        details = tmMovie.details(id) # [구글]-[get TMDb Movie details]-[Search & Query For Details]
        image_path = details['poster_path']
        if image_path:
            image_path = 'https://image.tmdb.org/t/p/w500' + details['poster_path']
        else:
            image_path = 'no_image.jpg' # 제목 Identity 검색
        images.append(image_path)
        # details title은 TMDb 한글 설정 시 한글 제목을 가져올 수 있다.
        titles.append(details['title'])
    return images, titles

# 프로그램 시작
tmMovie = Movie() # 영화 상세정보
tmdb = TMDb() # TMDb 설정
tmdb.api_key = 'your api key'
tmdb.language = 'ko-KR'

# 영화 제목과 코사인 유사도 파일을 읽어 저장한다.
movies = pickle.load(open('../data/movies.pickle', 'rb'))
cosine_sim = pickle.load(open('../data/cosine_sim.pickle', 'rb'))

st.set_page_config(layout='wide')
st.header('영화추천')
movie_list = movies['title'].values
title = st.selectbox('Choose a movie you like', movie_list)

# 추천 버튼을 클릭한 경우
if st.button('Recommend'):
    with st.spinner('Please wait...'):
        images, titles = get_recommendations(title)
        idx = 0
        for i in range(0, 2):
            cols = st.columns(5)
            for col in cols:
                col.image(images[idx])
                col.write(titles[idx])
            idx += 1
```

• Collaborative Filtering(협업 필터링: 사용자 리뷰 기반)

협업 필터링(Collaborative Filtering)은 사용자들의 행동 패턴이나 선호도를 분석하여, 비슷한 취향을 가진 사용자들이 선호하는 항목을 추천하는 방식이다. 즉, 다른 사용자들이 특정 항목에 대해 어떻게 반응했는지(예: 평가, 구매 등)를 바탕으로 해당 항목을 추천하는 방법이다. 예를 들어, A라는 영화를 좋아했던 사람들이 다른 영화 B를 좋아했다면, A 영화를 좋아했던 다른 사용자들에게 B 영화를 추천하는 방식이다.

• The Movies Dataset 다운로드

- 1) [구글]-[TMDB 5000]-[Kaggle 사이트이동]-[상단 Code Tab]-[우측 Hotness 메뉴]-[Most Votes] 선택한다.
- 2) [Getting Started with a Movie Recommendation System]-[The Movie Dataset]
- 3) 'ratings_small.csv' 파일을 다운로드하여 프로젝트 폴더로 이동한다.

- 협업 필터링에 필요한 라이브러리 'pip install scikit-surprise'를 설치한다. [구글]-[scikit surprise] 검색

```
pip install scikit-surprise
```

- 설치 시 Microsoft Visual C++ Build Tools가 필요하다는 에러가 발생하면 아래 웹 사이트에 방문하여 'VisualStudioSetup.exe' 파일을 다운로드 한다. 파일을 실행하여 'C++를 사용한 데스크톱 개발'을 check하여 설치한다.

```
https://visualstudio.microsoft.com/ko/vs/community/
```

- 아래 사이트로 접속하면 scikit surprise 홈페이지로 예제 및 모델별 성능 평가 결과도 확인할 수 있다. 성능 평가 결과 SVD++ 모델이 평가 점수는 높지만 시간이 많이 걸리는 것을 확인할 수 있다.

```
https://surpriselib.com/
```

- Surprise에서 자주 사용되는 추천 알고리즘 클래스는 아래와 같으며, 이밖에도 SVD++, NMF 등 다양한 알고리즘을 제공한다.

클래스 명	설명
SVD	행렬 분해를 통한 잠재 요인 협업 필터링을 위한 SVD 알고리즘
KNNBasic	최근접 이웃 협업 필터링을 위한 KNN 알고리즘
BaselineOnly	사용자 Bias와 아이템 Bias를 고려한 SGD 베이스라인 알고리즘

- '09.영화추천(협업필터링).ipynb' 파일을 생성한 후 필요한 Pandas 라이브러리를 import 한다.

```
import pandas as pd
```

- 사용자들의 영화별로 평점을 저장한 'ratings_small.csv' 파일을 읽어 데이터프레임(ratings)에 저장한다.

```
ratings = pd.read_csv('data/ratings_small.csv')
ratings.head()
```

	userId	movieId	rating	timestamp
0	1	31	2.5	1260759144
1	1	1029	3.0	1260759179
2	1	1061	3.0	1260759182
3	1	1129	2.0	1260759185
4	1	1172	4.0	1260759205

• Surprise 데이터 세트 로딩

Surprise는 사용자 아이디, 아이템 아이디, 평점 데이터가 로우 레벨로 된 데이터 세트만 적용할 수 있다. 그래서 데이터의 첫번째 칼럼을 사용자 아이디, 두번째 칼럼을 아이템 아이디, 세번째 칼럼을 평점으로 가정해 데이터를 로딩 하여야 한다.

- Surprise 패키지로부터 필요한 라이브러리를 import 한다.

```
from surprise import Reader, Dataset, SVD, accuracy
```

- Reader 객체의 Rating scale은 ratings의 최소값, 최대값으로 지정한다.

```
min = ratings['rating'].min()
max = ratings['rating'].max()
min, max
```

```
(0.5, 5.0)
```

- 데이터세트 로딩을 위한 Reader 객체를 생성한다.

```
reader = Reader(rating_scale=(min, max))
```

- 데이터프레임(ratings)에서 반드시 '사용자ID', '영화ID', '평점' 칼럼 순으로 추출한다. 추출한 칼럼으로 생성된 데이터프레임을 reader 객체를 사용하여 surprise의 load_from_df() 함수로 Dataset을 로딩 한다.

```
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader=reader)
data
```

```
<surprise.dataset.DatasetAutoFolds at 0x277224aaad0>
```

• 훈련세트와 테스트세트 분리 후 모델평가

훈련세트와 테스트세트를 75:25 비율로 분리한다. SVD 객체를 생성하여 훈련세트로 학습한 후 테스트세트로 평점을 예측하고 예측한 값과 실제 값과 비교하여 RMSE(Root Mean Squared Error:평균 제곱근 오차)로 모델을 평가한다.

- 훈련 세트와 테스트 세트를 75:25 비율로 분리한다.

```
from surprise.model_selection import train_test_split
trainset, testset = train_test_split(data, test_size=0.25, random_state=0)
```

- SVD 모델 객체를 생성한다. 수행시마다 동일한 결과 도출을 위해 random_state 설정한다.

```
svd = SVD(random_state=0)
```

- 훈련 세트로 학습 후 테스트 세트로 예측한다. RMSE로 모델을 평가한다. RMSE는 값이 낮을수록 좋은 모델이다.

```
svd.fit(trainset)
predictions = svd.test(testset)
accuracy.rmse(predictions)
```

```
RMSE: 0.8933
```

```
0.8933337294505111
```

• K-Fold 교차 검증을 통한 모델 평가

K-Fold 교차 검증이란 모델을 평가할 때 데이터를 여러 세트로 나누어서 교체로 모델을 평가하는 방법이다. K-Fold 교차 검증에 사용되는 함수는 `surprise.model_selection` 클래스의 `cross_validate()` 함수이다. 예를 들어 100개의 데이터이고 데이터 세트 개수(`cv`)가 5인 경우 4개의 세트로 훈련하고 1개의 세트로 테스트하는 작업을 5번하고 결과의 평균을 구하여 성능을 검증한다.

No	Group	Train Set	Test Set
1	1~20(A)	ABCD	E
2	21~40(B)	ABCE	D
3	41~60(C)	ABDE	C
4	61~80(D)	ACDE	B
5	81~100(E)	BCDE	A

- Rating scale을 데이터프레임(`ratings`)의 평점의 최소, 최대로 지정하여 Reader 객체를 생성한다.

```
min = ratings['rating'].min()
max = ratings['rating'].max()
reader = Reader(rating_scale=(min, max))
```

- Reader 객체를 이용하여 사용자 아이디, 영화 아이디, 평점 순으로 데이터세트로 로딩한다.

```
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader=reader)
data
```

```
<surprise.dataset.DatasetAutoFolds at 0x277224aaad0>
```

- 위에서 생성한 SVD 모델 객체를 `cross_validate()` 함수로 모델을 평가해 본다. `cv=5`는 Fold의 개수 이다.

```
from surprise.model_selection import cross_validate
cross_validate(svd, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

```
Evaluating RMSE, MAE of algorithm SVD on 5 split(s).
```

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.8999	0.8937	0.8987	0.8958	0.9002	0.8977	0.0025
MAE (testset)	0.6926	0.6898	0.6910	0.6894	0.6944	0.6914	0.0019
Fit time	1.37	1.45	1.42	1.41	1.47	1.42	0.04
Test time	0.19	0.17	0.27	0.18	0.27	0.22	0.05

```
{'test_rmse': array([0.8999247, 0.89371236, 0.89873246, 0.89581395, 0.90015218]),
'test_mae': array([0.69263929, 0.68980035, 0.69101543, 0.68937457, 0.69440648]),
'fit_time': (1.3661470413208008,
1.4471731185913086,
1.422184944152832,
1.4082145690917969,
1.4729771614074707),
'test_time': (0.1948843002319336,
0.1679229736328125,
0.2748243808746338,
0.17587780952453613,
0.2651553153991699)}
```

- 영화 평점 예측

- 전체 데이터셋(dataset)에 대하여 훈련세트(trainset)를 생성한다. SVD객체를 훈련세트로 학습한다.

```
trainset = data.build_full_trainset()
svd.fit(trainset)
```

```
<surprise.prediction_algorithms.matrix_factorization.SVD at 0x27746adfa30>
```

- 사용자 아이디 '1'의 영화 아이디별 평점 목록을 출력한다.

```
filt = ratings['userId'] == 1
ratings[filt][:3]
```

	userId	movieId	rating	timestamp
0	1	31	2.5	1260759144
1	1	1029	3.0	1260759179
2	1	1061	3.0	1260759182

- 사용자 아이디 '1'의 영화 아이디 '31'번 영화 평점을 예측한다. est는 예측 평점이고 r_ui는 실제 평점이다.

```
svd.predict(1, 31)
```

```
Prediction(uid=1, iid=31, r_ui=None, est=2.4162799702909346, details={'was_impossible': False})
```

```
svd.predict(1, 31, 2.5)
```

```
Prediction(uid=1, iid=31, r_ui=2.5, est=2.4162799702909346, details={'was_impossible': False})
```

- 사용자 아이디 '1'의 영화 아이디 '1029'번 예측 영화 평점은 2.881점이고 실제 영화 평점은 3점이다.

```
svd.predict(1, 1029, 3)
```

```
Prediction(uid=1, iid=1029, r_ui=3, est=2.8814455446761933, details={'was_impossible': False})
```

- 사용자 아이디 '100'의 영화 아이디별 평점 목록을 출력한다.

```
filt = ratings['userId'] == 100
ratings[filt][:3]
```

	userId	movieId	rating	timestamp
15273	100	1	4.0	854193977
15274	100	3	4.0	854194024
15275	100	6	3.0	854194023

- 사용자ID '100'의 영화ID '1029'번 예측 점수는 3.770점이다. 같은 영화(1029)에 대해서도 사용자들에 따라 영화 평점들에 대한 취향 즉 이력들이 다르기 때문에 예측 점수가 다를 수 있다.

```
svd.predict(100, 1029)
```

```
Prediction(uid=100, iid=1029, r_ui=None, est=3.7705476478414846, details={'was_impossible': False})
```

• 하이퍼 파라미터 튜닝 후 모델 평가

하이퍼 파라미터는 모델 학습 과정에서 사용자가 직접 설정해야 하는 값들을 의미한다. 하이퍼 파라미터 튜닝은 머신 러닝 모델의 성능을 최적화하기 위해 모델 학습 전에 설정되는 하이퍼 파라미터 값을 조정하는 과정이다. 이러한 하이퍼 파라미터 조정은 모델의 학습 속도, 정확도 등 성능에 큰 영향을 미치므로, 적절한 튜닝을 통해 모델의 성능을 극대화할 수 있다.

Surprise는 GridSearchCV를 통해 하이퍼 파라미터 최적화를 수행할 수 있다. 하이퍼 파라미터 최적화는 알고리즘 유형에 따라 다를 수 있지만, SVD의 경우 주로 점진적 하강법 (Stochastic Gradient Descnt)의 반복 횟수를 지정하는 `n_pochs`와 SVD의 잠재 요인 `K`의 크기를 지정하는 `n_factor`를 튜닝 한다.

- 하이퍼 파라미터 최적화를 수행하는 Surprise의 GridSearchCV 클래스를 `impor` 한다. `n_epochs` 하이퍼 파라미터 값과 `n_factor` 하이퍼 파라미터 값을 변경하면서 CV가 3일 때의 K-Fold 교차 검증을 통한 최적의 하이퍼 파라미터 값을 도출한다.

```
from surprise.model_selection import GridSearchCV

epochs = [20, 40, 60]
factors = [50, 100, 200]
param_grid = {'n_epochs':epochs, 'n_factors':factors }
gs = GridSearchCV(SVD, param_grid, measures=['rmse', 'mae'], cv=3)
gs.fit(data)
```

- `n_epochs: 20, n_factor: 50` 일 때 3개 Fold의 검증 데이터 세트에서 최적 RMSE가 0.894로 도출되었다.

```
gs.best_score['rmse'], gs.best_params['rmse']

(0.8985721031982608, {'n_epochs': 20, 'n_factors': 50})
```

- 훈련세트와 테스트세트 분리 후 하이퍼 매개변수를 바꿔가면서 모델을 RMSE로 평가해 본다.

```
from surprise.model_selection import train_test_split
trainset, testset = train_test_split(data, test_size=0.25, random_state=0)

for epochs in [20, 40, 60]:
    for factors in [50, 100, 200]:
        svd = SVD(n_epochs=epochs, n_factors=factors, random_state=0)
        svd.fit(trainset)
        predictions = svd.test(testset)
        accuracy.rmse(predictions)
        print(f'n_epochs:{epochs}, n_factors:{factors}')
```

RMSE: 0.8908
n_epochs:20, n_factors:50
RMSE: 0.8933
n_epochs:20, n_factors:100
RMSE: 0.8977
n_epochs:20, n_factors:200
RMSE: 0.9024
n_epochs:40, n_factors:50
RMSE: 0.9004
n_epochs:40, n_factors:100
RMSE: 0.8988
n_epochs:40, n_factors:200
RMSE: 0.9174
n_epochs:60, n_factors:50
RMSE: 0.9062
n_epochs:60, n_factors:100
RMSE: 0.8973
n_epochs:60, n_factors:200

- Surprise를 이용한 영화 추천시스템

build_full_trainset() 메서드를 이용해 학습데이터를 생성하고, SVD 알고리즘을 이용해 학습을 수행한 후 특정사용자에게 아직 보지 않은 영화 목록(평점을 매기지 않은 영화)에서 예측 평점이 높은 영화를 추천한다.

- Reader 객체를 사용해 사용자 아이디, 영화 아이디, 평점 순으로 데이터프레임 생성 후 데이터셋으로 로딩 한다.

```
min = ratings['rating'].min()
max = ratings['rating'].max()
reader = Reader(rating_scale=(min, max))
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)
```

- 위에서 구한 최적의 하이퍼 파라미터를 매개변수로 SVD 학습 모델을 생성한다.

```
svd = SVD(n_epochs=20, n_factors=50, random_state=0)
trainset = data.build_full_trainset()
svd.fit(trainset)
```

- 사용자 아이디 9, 영화 아이디 42의 예측 평점을 계산

- 1) 사용자 아이디 9가 평점을 매긴 영화 수와 영화 목록을 출력한다.

```
userId = 9
filt = ratings['userId'] == userId
rated_movies = ratings[filt]['movieId']
rated_movies = rated_movies.values
print(f'평점을 매긴 영화 수: {len(rated_movies)}개')
print(f'평점을 매긴 영화목록: {rated_movies}')
```

```
평점을 매긴 영화 수: 45개
평점을 매긴 영화목록: [ 1  17  26  36  47 318 497 515 527 534 593 608 733 1059
1177 1357 1358 1411 1541 1584 1680 1682 1704 1721 1784 2028 2125 2140
2249 2268 2273 2278 2291 2294 2302 2391 2396 2427 2490 2501 2539 2571
2628 2762 2857]
```

- 2) 평점을 매긴 영화목록(rated_movies)에 영화 아이디 42가 있는지 확인한다.

```
movieId = 42
if not movieId in rated_movies:
    print(f'사용자 아이디 {userId}는(은) 영화 아이디 {movieId}의 평점이 없음')
```

```
사용자 아이디 9는(은) 영화 아이디 42의 평점이 없음
```

- 2) 사용자 아이디 9, 영화 아이디 42의 평점을 예측하고 예측 평점점수를 반올림하여 출력한다.

```
pred = svd.predict(userId, movieId)
pred
```

```
Prediction(uid=9, iid=42, r_ui=None, est=2.638932480440445, details={'was_impossible': False})
```

```
print(f'사용자 아이디 {userId}의 영화아이디 {movieId} 예측 평점은 {round(pred.est, 2)}')
```

```
사용자 아이디 9의 영화아이디 42 예측 평점은 2.64
```

- 특정 사용자가 '평점 매긴 영화 수', '전체 영화 수', '추천 대상 영화 수'를 출력

1) 특정 사용자가 평점 매긴 영화 목록을 `rated_movies`에 저장한 후 영화 수를 출력한다.

```
filt = ratings['userId'] == userId
rated_movies = ratings[filt]['movieId'].to_list()
len(rated_movies)
```

45

2) 평점 매긴 전체 영화 목록을 추출한다. 사용자 아이디별 영화 아이디가 중복되므로 중복을 제거한 후 list 타입으로 변환한다.

```
total_movies = ratings['movieId'].drop_duplicates().to_list()
len(total_movies)
```

9066

3) 전체 영화 중 사용자가 평점을 매기지 않은 추천할 영화 목록을 추출한다.

```
import numpy as np
unrated_movies = np.setdiff1d(total_movies, rated_movies)
len(unrated_movies)
```

9021

4) 위 작업(1~3)을 함수(`get_unrated_movies`)로 작성한다.

```
def get_unrated_movies(ratings, userId):
    filt = ratings['userId'] == userId
    rated_movies = ratings[filt]['movieId'].to_list()
    total_movies = ratings['movieId'].drop_duplicates().to_list()
    unrated_movies = np.setdiff1d(total_movies, rated_movies)
    print(f'평점 매긴 영화 수: {len(rated_movies)}')
    print(f'추천대상 영화 수: {len(unrated_movies)}')
    print(f'전체 영화 수: {len(total_movies)}')
    return unrated_movies
```

- 사용자 아이디 9의 추천 영화 목록을 추출한다.

```
unrated_movies = get_unrated_movies(ratings, 9)
```

평점 매긴 영화 수: 45
추천 대상 영화 수: 9021
평점 매긴 모든 영화 수: 9066

- 사용자 아이디 1의 추천 영화 목록을 추출한다.

```
unrated_movies = get_unrated_movies(ratings, 1)
```

평점 매긴 영화 수: 20
추천대상 영화 수: 9046
전체 영화 수: 9066

- 예측 평점을 구한 후 높은 순으로 Top10을 출력하는 함수 작성

1) 특정 사용자에게 추천할 영화들을 SVD 모델을 이용하여 평점을 예측한다.

```
userId = 9
predictions = [svd.predict(userId, movieId) for movieId in unrated_movies]
predictions

[
  Prediction(uid=9, iid=1, r_ui=None, est=3.6723740633974558, details={'was_impossible': False}),
  Prediction(uid=9, iid=2, r_ui=None, est=3.2605300875559977, details={'was_impossible': False}),
  Prediction(uid=9, iid=3, r_ui=None, est=3.0231151855003455, details={'was_impossible': False}),
  ...]
```

2) 예측 평점이 높은 순으로 정렬하여 예측 목록을 업데이트한다.

```
predictions.sort(key=lambda prediction:prediction.est, reverse=True)
predictions

[
  Prediction(uid=9, iid=318, r_ui=None, est=4.4672412633139755, details={'was_impossible': False}),
  Prediction(uid=9, iid=527, r_ui=None, est=4.376939378735491, details={'was_impossible': False}),
  Prediction(uid=9, iid=905, r_ui=None, est=4.370540045092638, details={'was_impossible': False}),
  ...]
```

3) 내림차순 정렬한 목록 중 상위 5개의 데이터만 추출한다.

```
top_predictions = predictions[:5]
top_predictions

[ Prediction(uid=9, iid=318, r_ui=None, est=4.4672412633139755, details={'was_impossible': False}),
  Prediction(uid=9, iid=527, r_ui=None, est=4.376939378735491, details={'was_impossible': False}),
  Prediction(uid=9, iid=905, r_ui=None, est=4.370540045092638, details={'was_impossible': False}),
  Prediction(uid=9, iid=926, r_ui=None, est=4.363849637541674, details={'was_impossible': False}),
  Prediction(uid=9, iid=912, r_ui=None, est=4.336024874543918, details={'was_impossible': False})]
```

4) 앞의 평점 예측 목록(top_predictions)에서 영화 아이디와 예측 평점만 추출하여 저장한다.

```
top_movies = [(prediction.iid, prediction.est) for prediction in top_predictions]
top_movies

[(318, 4.4672412633139755),
 (527, 4.376939378735491),
 (905, 4.370540045092638),
 (926, 4.363849637541674),
 (912, 4.336024874543918)]
```

5) 위 작업(1~4)을 함수로 정의한다.

```
def get_recommendation(svd, userId, unrated_movies, top_n):
    predictions = [svd.predict(userId, movieId) for movieId in unrated_movies]
    predictions.sort(key=lambda prediction:prediction.est, reverse=True)
    top_predictions = predictions[:top_n]
    top_movies = [(prediction.iid, prediction.est) for prediction in top_predictions]
    return top_movies
```

- `get_recommendation()` 함수를 적용하여 추천 영화 리스트 출력

- `get_recommendation()` 함수를 실행 후 특정 사용자(아이디:9)에게 추천할 영화 리스트 Top-10을 출력한다.

```
#사용자 아이디 9에게 추천할 영화 목록
userId = 9
#추천할 영화 목록 10개
top_n = 10
#평점을 매기지 않은 영화 목록
unrated_movies = get_unrated_movies(ratings, userId)
#추천 영화 목록
top_movies = get_recommendation(svd, userId, unrated_movies, top_n)
top_movies
```

```
평점 매긴 영화 수: 45
추천대상 영화 수: 9021
전체 영화 수: 9066
[(1172, 4.400579409001147),
 (905, 4.370540045092638),
 (926, 4.363849637541674),
 (912, 4.336024874543918),
 (3462, 4.321015160843752),
 (913, 4.309431147405599),
 (50, 4.303661756312046),
 (858, 4.289780175135358),
 (969, 4.283687861376393),
 (953, 4.271531364943996)]
```

- 추천 영화 리스트에 영화 제목을 추가하여 출력

- 1) 영화 리스트 Top-10에서 영화 아이디만 추출하여 `top_movi_ids` 변수에 저장한다.

```
top_movie_ids = [pred[0] for pred in top_movies]
top_movie_ids
```

```
[1172, 905, 926, 912, 3462, 913, 50, 858, 969, 953]
```

- 2) 영화 리스트 Top-10에서 영화 아이디만 추출하여 `top_movi_ids` 변수에 저장한다.

```
top_movie_rating = [round(pred[1],2) for pred in top_movies]
top_movie_rating
```

```
[4.4, 4.37, 4.36, 4.34, 4.32, 4.31, 4.3, 4.29, 4.28, 4.27]
```

- 3) 영화 제목이 있는 'tmdb_5000_credits.csv' 파일을 데이터프레임(movies)에 저장한다.

```
movies = pd.read_csv('data/tmdb_5000_credits.csv')
movies.head()
```

	movie_id	title	cast	crew
0	19995	Avatar	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	Pirates of the Caribbean: At World's End	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	Spectre	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...
3	49026	The Dark Knight Rises	[{"cast_id": 2, "character": "Bruce Wayne / Ba...	[{"credit_id": "52fe4781c3a36847f81398c3", "de...
4	49529	John Carter	[{"cast_id": 5, "character": "John Carter", "c...	[{"credit_id": "52fe479ac3a36847f813eaa3", "de...

- 추천 영화 아이디로 영화 제목을 출력한다. 데이터프레임(movies)에 없는 영화 아이디도 있으므로 제목이 4개만 출력된다.

```
filt = movies['movie_id'].isin(top_movie_ids)
top_movie_titles = movies[filt]['title']
top_movie_titles = top_movie_titles.to_frame()
top_movie_titles
```

	title
509	Madagascar
1025	The Thomas Crown Affair
1053	Galaxy Quest
4457	Pandora's Box

- 4) 데이터프레임(movies)에 영화 제목을 추출하고 없으면 'No Title'로 저장한다.

```
top_movie_titles = []
for id in top_movie_ids:
    filt = movies['movie_id'] == id
    titles = movies[filt]['title'].values
    if len(titles)==0:
        title='No Title'
    else:
        title = titles[0]
    top_movie_titles.append(title)
top_movie_titles
```

```
['No Title',
 "Pandora's Box",
 'Galaxy Quest',
 'No Title',
 'No Title',
 'The Thomas Crown Affair',
 'No Title',
 'No Title',
 'No Title',
 'Madagascar']
```

- 5) 추천 영화 Top-10의 영화 제목, 영화 아이디, 평점을 출력한다.

```
print('#### Top-10 추천 영화 리스트 ####')
for i in range(10):
    print(f'{top_movie_titles[i]}({top_movie_ids[i]}) : {top_movie_rating[i]}점')
```

```
#### Top-10 추천 영화 리스트 ####
No Title(1172) : 4.4점
Pandora's Box(905) : 4.37점
Galaxy Quest(926) : 4.36점
No Title(912) : 4.34점
No Title(3462) : 4.32점
The Thomas Crown Affair(913) : 4.31점
No Title(50) : 4.3점
No Title(858) : 4.29점
No Title(969) : 4.28점
Madagascar(953) : 4.27점
```

• Titanic 생존자 예측(Feature Engineering & 모델링)

Feature Engineering은 머신러닝과 데이터 분석에서 중요한 단계로, 원시 데이터를 모델이 이해하고 처리할 수 있는 유용한 피처로 변환하는 과정이다. 이를 통해 모델의 성능을 향상시키고, 해석 가능성을 높이며, 데이터의 복잡한 관계를 보다 명확하게 만든다.

머신 러닝 모델링이란, 주어진 데이터를 학습하여 패턴을 파악하고 이를 바탕으로 예측이나 분류 등의 작업을 수행하는 컴퓨터 프로그램 또는 알고리즘을 개발하는 과정이다. 즉, 데이터를 학습하여 스스로 지식을 습득하고, 이를 통해 새로운 입력에 대해 정확한 결과를 도출할 수 있도록 훈련된 모델을 개발하는 과정을 의미한다.

• 피처 엔지니어링(Feature Engineering)

- '10.타이타닉.ipynb' 파일을 생성 후 훈련데이터와 테스트데이터를 읽어 상관계수, 결측치를 확인해 본다.

```
import pandas as pd
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
before_corr = train.corr(numeric_only=True) #각 칼럼 간의 상관관계(correlation)를 계산
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

```
train.isnull().sum()
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age            177
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked        2
dtype: int64
```

```
test.isnull().sum()
```

```
PassengerId    0
Pclass          0
Name            0
Sex             0
Age            86
SibSp           0
Parch           0
Ticket          0
Fare            1
Cabin          327
Embarked        0
dtype: int64
```

- 데이터프레임(Train)과 데이터프레임(Test)를 한 번에 변환하기 위해 list에 저장한다.

```
data_list = [train, test]
```

• Sex(성별)

categorical 칼럼을 숫자 타입으로 변경하고 싶은 경우 category 타입으로 강제 형 변환시킨 후 cat.codes값을 구한다.

- Sex(성별) 는 male(남성), female(여성)으로 나누어져 있다.

```
train['Sex'].head()
```

```
0    male
1  female
2  female
3  female
4    male
Name: Sex, dtype: object
```

- 성별 칼럼 값을 male, female에서 0, 1 숫자형 데이터로 수정한다.

```
for data in data_list:
    data['Sex'] = data['Sex'].astype('category').cat.codes
```

- Sex(성별) 는 0(남성), 1(여성)으로 수정되었다.

```
train['Sex'].head()
```

```
0    1
1    0
2    0
3    0
4    1
Name: Sex, dtype: int8
```

• Age(나이)

성별 칼럼의 남자와 여자는 0, 1로 차이가 극명하게 나뉘는 값이므로 category 타입으로 변경하여 처리할 수 있지만 요금이나 나이 칼럼은 극명하게 나뉘는 값이 아니다. 이를 전체적인 범주에 맞게 낮춰주기 위해 Category 타입으로 변경한다.

- Age 칼럼 결측치에 대해 남성과, 여성의 평균 나이를 구한다.

```
sex_mean = train.groupby('Sex')['Age'].mean()
```

```
Sex
0    27.915709
1    30.726645
Name: Age, dtype: float64
```

- 성별이 남자이면서 나이가 Null인 경우에는 나이 칼럼에 남자 평균나이를 여자이면서 나이가 Null인 경우는 여자 평균나일로 수정한다.

```
for data in data_list:
    filt = (data['Sex']==0) & (data['Age'].isnull())
    data.loc[filt, 'Age'] = sex_mean[0]
    filt = (data['Sex']==1) & (data['Age'].isnull())
    data.loc[filt, 'Age'] = sex_mean[1]
```

- 데이터프레임(train)의 'Age' 칼럼의 Null 수가 0으로 변경되었다.

```
train.isnull().sum()
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age             0
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked        2
dtype: int64
```

- 데이터프레임(test)의 'Age' 칼럼의 Null 수가 0으로 변경되었다.

```
train.isnull().sum()
```

```
PassengerId    0
Pclass          0
Name            0
Sex             0
Age             0
SibSp           0
Parch           0
Ticket          0
Fare            1
Cabin          327
Embarked        0
dtype: int64
```

- 'Age' 칼럼을 전체적인 범주에 맞게 5개 구간으로 나누기 위해 pandas의 cut()함수를 사용한다.

```
train['AgeRange'] = pd.cut(train['Age'], 5)
train['AgeRange'].value_counts()
```

```
AgeRange
(16.336, 32.252]    523
(32.252, 48.168]    188
(0.34, 16.336]      100
(48.168, 64.084]     69
(64.084, 80.0]       11
Name: count, dtype: int64
```

- 16, 32, 48, 64 값들을 기준으로 분류하여 새로운 칼럼 'Age2'에 0, 1, 3, 4 값을 저장한다.

```
for data in data_list:
    data.loc[(data['Age']<=16), 'Age2'] = 0
    data.loc[(data['Age']>16) & (data['Age']<=32), 'Age2'] = 1
    data.loc[(data['Age']>32) & (data['Age']<=48), 'Age2'] = 2
    data.loc[(data['Age']>48) & (data['Age']<=64), 'Age2'] = 3
    data.loc[(data['Age']>64), 'Age2'] = 4
```

- 'AgeRange' 컬럼은 더 이상 사용되지 않으므로 삭제한다.

```
if 'AgeRange' in train.columns:
    train.drop(columns='AgeRange', inplace=True)
```

- 'Age2' 컬럼의 값이 'Age' 연령대에 따라 0~4로 분류되어졌다.

```
train['Age2'].value_counts()
```

```
Age2
1.0    523
2.0    188
0.0    100
3.0     69
4.0     11
Name: count, dtype: int64
```

- 'Age2' 컬럼 값을 'Age' 값으로 수정하고 'Age2' 컬럼을 삭제한다.

```
for data in data_list:
    if 'Age2' in data.columns: data['Age'] = data['Age2']
    if 'Age2' in data.columns:
        data.drop(columns='Age2', inplace=True)
```

- 'Sex' 컬럼과 'Age' 컬럼의 값이 몇 개의 범주로 구분되었다.

```
train[['Name', 'Sex', 'Age']].head()
```

	Name	Sex	Age
0	Braund, Mr. Owen Harris	1	1.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	2.0
2	Heikkinen, Miss. Laina	0	1.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	2.0
4	Allen, Mr. William Henry	1	2.0

• Name(성명)

탑승객들의 이름 앞을 추출해보면 Mr, Mrs, Miss 등의 명칭 등을 뽑아낼 수 있다. 이름 전체를 사용하는 것은 생존자 예측에 큰 도움이 되지 않을 것으로 예상되므로 이름을 필요한 단어만 간소화한다. 이름 컬럼에 Mr, Miss, Mrs, Master가 가장 많이 존재하므로 이 타이틀로 새로운 컬럼에 저장하고 나머지는 적게 존재하므로 모두 'Other'로 저장해 준다.

- 1) 정규식을 이용하여 빈칸으로 시작하고 마침표전까지 영문자만을 추출하여 새로운 컬럼 'Title'에 저장한다.

```
train['Title'] = train['Name'].str.extract(' ([A-Za-z]+)\.')
titles = train['Title'].value_counts()
titles.head(5)
```

```
Title
Mr      517
Miss    182
Mrs     125
Master   40
Dr        7
Name: count, dtype: int64
```

2) Title이 Mr, Miss, Mrs, Master를 제외한 호칭을 추출하여 other 변수에 저장한다.

```
others = [i for i in titles.index if i not in ['Mr', 'Miss', 'Mrs', 'Master']]
print(others)

['Dr', 'Rev', 'Mlle', 'Major', 'Col', 'Countess', 'Capt', 'Ms', 'Sir', 'Lady', 'Mme', 'Don', 'Jonkheer']
```

3) 'Title' 칼럼 값이 others에 속하는 값이면 값을 'Other' 값으로 변경한다.

```
train['Title'] = train['Title'].replace(others, 'Other')
titles = train['Title'].value_counts()
titles.head(5)
```

```
Title
Mr      517
Miss    182
Mrs     125
Master   40
Other    27
Name: count, dtype: int64
```

4) 'title_name' 새로운 칼럼에 title을 카테고리 수치화 한 값으로 저장한다.

```
train['Title_name'] = train['Title'].astype('category').cat.codes
train[['Title', 'Title_name']].value_counts()
```

```
Title  Title_name
Mr      2           517
Miss    1           182
Mrs     3           125
Master  0            40
Other   4            27
Name: count, dtype: int64
```

5) 1번~4번 작업은 두 개의 데이터프레임(data_list)에 적용한다.

```
for data in data_list:
    data['Title'] = data['Name'].str.extract('([A-Za-z]+)\.')
    titles = data['Title'].value_counts()
    others = [i for i in titles.index if i not in ['Mr', 'Miss', 'Mrs', 'Master']]
    data['Title'] = data['Title'].replace(others, 'Other')
    data['Title_name'] = data['Title'].astype('category').cat.codes
```

6) 데이터프레임(test)의 호칭별 인원수를 확인한다.

```
test[['Title', 'Title_name']].value_counts()
```

```
Title  Title_name
Mr      2           240
Miss    1            78
Mrs     3            72
Master  0            21
Other   4             7
Name: count, dtype: int64
```


7) Title_name 칼럼을 만들어줬으니 이제 필요 없는 'Name', 'Title' 칼럼을 지우고 확인해본다.

```
drop_list = ['Name', 'Title']
for data in data_list:
    for col in drop_list:
        if (col in data.columns):
            data.drop(columns=col, inplace=True)
train.head()
```

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Title_name
0	1	0	3	1	1.0	1	0	A/5 21171	7.2500	NaN	S	2
1	2	1	1	0	2.0	1	0	PC 17599	71.2833	C85	C	3
2	3	1	3	0	1.0	0	0	STON/O2. 3101282	7.9250	NaN	S	1
3	4	1	1	0	2.0	1	0	113803	53.1000	C123	S	3
4	5	0	3	1	2.0	0	0	373450	8.0500	NaN	S	2

- SibSp(동승한 형제자매, 배우자 수), Parch(동승한 부모, 자녀 수)

SibSp는 함께 탑승한 형제자매, 배우자의 총합이고 Parch는 함께 탑승한 부모, 자녀의 총합이다. 혼자 탑승한 탑승객과 가족들과 함께 탑승한 탑승객의 사망률에 차이가 있을 수 있기 때문에 가족이라는 새로운 칼럼을 만들어준다.

- 새로운 칼럼 'FamilySize'에 함께 동승한 가족의 합에 본인 1을 더한 값을 저장한다.

```
for data in data_list:
    data['FamilySize'] = data['SibSp'] + data['Parch'] + 1
test.head()
```

	PassengerId	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Title_name	FamilySize
0	892	3	1	2.0	0	0	330911	7.8292	NaN	Q	3	1.0
1	893	3	0	2.0	1	0	363272	7.0000	NaN	S	4	2.0
2	894	2	1	3.0	0	0	240276	9.6875	NaN	Q	3	1.0
3	895	3	1	1.0	0	0	315154	8.6625	NaN	S	3	1.0
4	896	3	0	1.0	1	1	3101298	12.2875	NaN	S	4	2.0

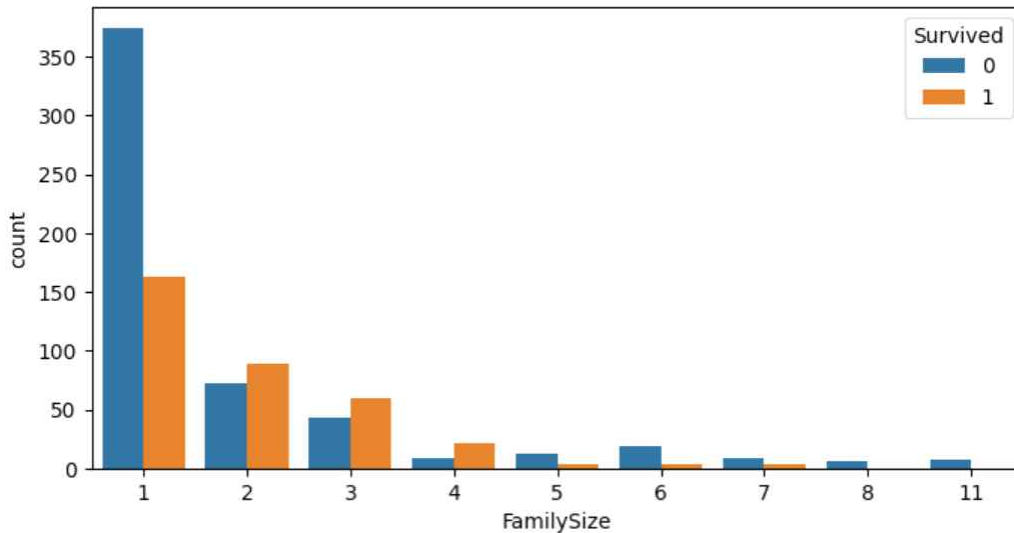
- 피벗 테이블을 이용하여 가족 수별 생존율을 확인해보고 시각화한다.

```
pd.pivot_table(train, index='FamilySize', values='Survived')
#train.groupby('FamilySize')[['Survived']].mean() 결과가 동일하다.
```

	Survived
FamilySize	
1	0.303538
2	0.552795
3	0.578431
4	0.724138
5	0.200000
6	0.136364
7	0.333333
8	0.000000
11	0.000000

- 데이터프레임(train)의 가족 수(FamilySize)별 생존자와 사망자의 인원수를 시각화 한다.

```
import matplotlib.pyplot as plt
import seaborn as sns
fig, ax = plt.subplots(figsize=(10, 5))
sns.countplot(data=train, x='FamilySize', hue='Survived', ax=ax)
plt.show()
```



- FamilySize라는 칼럼을 만들어주었으므로 이제 필요 없는 'SibSp', 'Parch' 칼럼을 제거한다.

```
drop_list = ['SibSp', 'Parch']
for data in data_list:
    for col in drop_list:
        if (col in data.columns):
            data.drop(columns=drop_list, inplace=True)
train.head()
```

	PassengerId	Survived	Pclass	Sex	Age	Ticket	Fare	Cabin	Embarked	Title_name	FamilySize
0	1	0	3	1	1.0	A/5 21171	7.2500	NaN	S	2	2
1	2	1	1	0	2.0	PC 17599	71.2833	C85	C	3	2
2	3	1	3	0	1.0	STON/O2. 3101282	7.9250	NaN	S	1	1
3	4	1	1	0	2.0	113803	53.1000	C123	S	3	2
4	5	0	3	1	2.0	373450	8.0500	NaN	S	2	1

• Embarked(탑승항구)

데이터프레임(train, test) 'Embarked' 칼럼의 결측치가 있는지 확인해보고 결측치를 가장 큰 비율을 차지하는 탑승항구로 채워준다.

- 데이터프레임(train)의 'Embarked' 칼럼에 Null 값의 수를 출력하여 몇 개인지 확인한다.

```
train['Embarked'].isnull().sum()
```

2

- 탑승항구(Embarked)별 탑승인원수를 출력하면 'S' 항구가 탑승객이 가장 많은 항구이다.

```
train['Embarked'].value_counts()
```

```
Embarked
S      644
C      168
Q       77
Name: count, dtype: int64
```

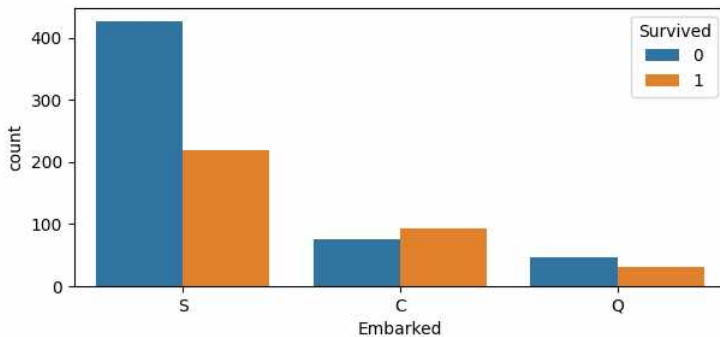
- 데이터프레임(data_list) 'Embarked' 칼럼의 값을 탑승객이 가장 많은 'S' 항구로 채워준다.

```
for data in data_list:
    data.fillna({'Embarked':'S'}, inplace=True)
train['Embarked'].value_counts()
```

```
Embarked
S      646
C      168
Q       77
Name: count, dtype: int64
```

- 항구별 생존, 사망자 수를 시각화한다.

```
fig, ax = plt.subplots(figsize=(7, 3))
sns.countplot(data=train, x='Embarked', hue='Survived', ax=ax)
plt.show()
```



- 데이터프레임(data_list) 'Embarked' 칼럼의 값 S, C, Q를 각각 0, 1, 2로 숫자로 Mapping 한다.

```
em_mapping = {'S':0, 'C':1, 'Q':2}
for data in data_list:
    if isinstance(data['Embarked'][0], str):
        data['Embarked'] = data['Embarked'].map(em_mapping)
train.head()
```

	PassengerId	Survived	Pclass	Sex	Age	Ticket	Fare	Cabin	Embarked	Title_name	FamilySize
0	1	0	3	1	1.0	A/5 21171	7.2500	NaN	0	2	2
1	2	1	1	0	2.0	PC 17599	71.2833	C85	1	3	2
2	3	1	3	0	1.0	STON/O2. 3101282	7.9250	NaN	0	1	1
3	4	1	1	0	2.0	113803	53.1000	C123	0	3	2
4	5	0	3	1	2.0	373450	8.0500	NaN	0	2	1

- **Cabin(객실)**

'Cabin' 컬럼은 객실을 대표하는 알파벳이 맨 처음 첫 글자로 저장되어있다. 첫 알파벳 한 글자만 추출하여 'Cabin' 컬럼 값으로 수정하고 Null인 경우는 'N' 값으로 수정한다. 수정된 알파벳 한 글자는 다시 category 수치형 값으로 수정한다.

- 데이터프레임(train)의 객실별 탑승객 수를 확인한다.

```
train['Cabin'].value_counts()

Cabin
B96 B98      4
G6            4
C23 C25 C27   4
C22 C26       3
F33           3
..
E34           1
C7            1
C54           1
E36           1
C148          1
Name: count, Length: 147, dtype: int64
```

- 데이터프레임(train) 'Cabin' 컬럼의 Null 수를 확인한다. 결측치가 687개 존재한다.

```
train.isnull().sum()

PassengerId      0
Survived          0
Pclass           0
Sex              0
Age              0
Ticket           0
Fare             0
Cabin            687
Embarked         0
Title_name       0
FamilySize       0
dtype: int64
```

- 결측치는 'N'으로 결측치가 아니면 앞자리의 첫 알파벳을 'Cabin' 컬럼에 저장한다. 'Cabin' 컬럼을 다시 카테고리 수치형으로 바꿔준다.

```
for data in data_list:
    data['Cabin'] = data['Cabin'].fillna('N')
    if isinstance(data['Cabin'][0], str):
        data['Cabin'] = data['Cabin'].apply(lambda x:x[0])
        data['Cabin'] = data['Cabin'].astype('category').cat.codes
train.head()
```

	PassengerId	Survived	Pclass	Sex	Age	Ticket	Fare	Cabin	Embarked	Title_name	FamilySize
0	1	0	3	1	1.0	A/5 21171	7.2500	7	0	2	2
1	2	1	1	0	2.0	PC 17599	71.2833	2	1	3	2
2	3	1	3	0	1.0	STON/O2. 3101282	7.9250	7	0	1	1
3	4	1	1	0	2.0	113803	53.1000	2	0	3	2
4	5	0	3	1	2.0	373450	8.0500	7	0	2	1

- Fare(요금)

'Fare' 컬럼도 'Age' 컬럼과 같은 작업을 한다. 4개의 그룹으로 나누어서 나눈 범위에 따라 0~3의 값을 저장한다.

1) 데이터프레임(data_list)의 'Fare' 컬럼을 4개의 범위로 나누어 새로운 컬럼 'Farerange'에 저장한다.

```
for data in data_list:
    data['Farerange'] = pd.cut(data['Fare'], 4)
```

```
train['Farerange'].value_counts()
```

```
Farerange
(-0.512, 128.082]    853
(128.082, 256.165]    29
(256.165, 384.247]     6
(384.247, 512.329]     3
Name: count, dtype: int64
```

2) 'Fare' 컬럼 값을 128이하, 128~256, 256~384, 380이사 기준으로 나누어 0, 1, 2, 3으로 저장한다.

```
for data in data_list:
    data.loc[(data['Fare']<=128), 'Fare'] = 0
    data.loc[(data['Fare']>128) & (data['Fare']<=256), 'Fare'] = 1
    data.loc[(data['Fare']>256) & (data['Fare']<=384), 'Fare'] = 2
    data.loc[(data['Fare']>384), 'Fare'] = 3
```

```
train['Fare2'].value_counts()
```

```
Fare2
0.0    853
1.0     29
2.0      6
3.0      3
Name: count, dtype: int64
```

3) 데이터프레임(data_list) 'Fare2' 컬럼 값을 'Fare' 컬럼에 저장하고 'Fare2', 'Farerange' 컬럼은 필요 없으므로 제거한다.

```
drop_list = ['Fare2', 'Farerange']
for data in data_list:
    if 'Fare2' in data.columns: data['Fare'] = data['Fare2']
    for col in drop_list:
        if col in data.columns: data.drop(columns=col, inplace=True)
```

```
train.head()
```

	PassengerId	Survived	Pclass	Sex	Age	Ticket	Fare	Cabin	Embarked	Title_name	FamilySize
0	1	0	3	1	1.0	A/5 21171	0.0	7	0	2	2
1	2	1	1	0	2.0	PC 17599	0.0	2	1	3	2
2	3	1	3	0	1.0	STON/O2. 3101282	0.0	7	0	1	1
3	4	1	1	0	2.0	113803	0.0	2	0	3	2
4	5	0	3	1	2.0	373450	0.0	7	0	2	1

- 기타 칼럼

- 'PassengerId', 'Ticket' 칼럼들은 필요 없으므로 제거한다. 'Cabin' 칼럼도 제거하는 것이 조금 더 성능이 좋아서 제거한다.

```
drop_list = ['PassengerId', 'Ticket', 'Cabin']

for data in data_list:
    for col in drop_list:
        if col in data.columns:
            data.drop(columns=col, inplace=True)
```

```
train.head()
```

	Survived	Pclass	Sex	Age	Fare	Embarked	Title_name	FamilySize
0	0	3	1	1.0	0.0	0	2	2
1	1	1	0	2.0	0.0	1	3	2
2	1	3	0	1.0	0.0	0	1	1
3	1	1	0	2.0	0.0	0	3	2
4	0	3	1	2.0	0.0	0	2	1

- 데이터프레임(test) 칼럼별 결측치가 있는지 확인한다. 'Fare' 칼럼에 1개의 결측치가 존재한다.

```
test.isnull().sum()
```

```
Pclass      0
Sex          0
Age          0
Fare         1
Embarked     0
Title_name   0
FamilySize   0
dtype: int64
```

- 데이터프레임(test)의 'Fare' 카테고리별 승객수를 출력하여 Null값을 승객수가 많은 카테고리 값으로 수정한다.

```
test['Fare'].value_counts()
```

```
Fare
0.0    388
1.0     21
2.0      7
3.0      1
Name: count, dtype: int64
```

```
test.fillna({'Fare':0}, inplace=True)
test['Fare'].value_counts()
```

```
Fare
0.0    389
1.0     21
2.0      7
3.0      1
Name: count, dtype: int64
```

- 피쳐 엔지니어링(Feature Engineering)이 끝났으므로 데이터프레임(train, test)의 데이터를 확인한다.

train.head()

	Survived	Pclass	Sex	Age	Fare	Embarked	Title_name	FamilySize
0	0	3	1	1.0	0.0	0	2	2
1	1	1	0	2.0	0.0	1	3	2
2	1	3	0	1.0	0.0	0	1	1
3	1	1	0	2.0	0.0	0	3	2
4	0	3	1	2.0	0.0	0	2	1

test.head()

	Pclass	Sex	Age	Fare	Embarked	Title_name	FamilySize
0	3	1	2.0	0.0	2	3	1.0
1	3	0	2.0	0.0	0	4	2.0
2	2	1	3.0	0.0	2	3	1.0
3	3	1	1.0	0.0	0	3	1.0
4	3	0	1.0	0.0	0	4	2.0

- 다음은 피쳐 엔지니어링(Feature Engineering) 이전 칼럼들의 상관관계이다.

before_corr

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

- 다음은 피쳐 엔지니어링 이후 칼럼들의 상관관계이다. 'Sex', 'Embarked', 'Title_name', 'FamilySize' 칼럼들이 추가되었다.

after_corr = train.corr(numeric_only=True)

after_corr

	Survived	Pclass	Sex	Age	Fare	Embarked	Title_name	FamilySize
Survived	1.000000	-0.338481	-0.543351	-0.043800	0.147466	0.106811	-0.052471	0.016639
Pclass	-0.338481	1.000000	0.131900	-0.358769	-0.298580	0.045702	-0.195910	0.065997
Sex	-0.543351	0.131900	1.000000	0.070220	-0.114771	-0.116569	0.040484	-0.200988
Age	-0.043800	-0.358769	0.070220	1.000000	0.063220	-0.051334	0.427999	-0.217063
Fare	0.147466	-0.298580	-0.114771	0.063220	1.000000	0.047596	-0.044644	0.098769
Embarked	0.106811	0.045702	-0.116569	-0.051334	0.047596	1.000000	-0.081928	-0.080281
Title_name	-0.052471	-0.195910	0.040484	0.427999	-0.044644	-0.081928	1.000000	-0.207530
FamilySize	0.016639	0.065997	-0.200988	-0.217063	0.098769	-0.080281	-0.207530	1.000000

• 다양한 모델링

여러 알고리즘(1.KNN, 2.Decision Tree, 3.Random Forest, 4.GradientBoosting, 5.HistGradientBoosting, 6.Naive Bayes, 7.Support Vector Machine) 모델의 성능 평가 점수가 가장 좋은 것을 최종 모델로 선정 한다.

• 훈련 데이터 준비

- 모델링에 필요한 훈련데이터(train_input)과 실제 생존 여부의 결과 데이터(train_target)을 생성한다.

```
train_input = train.drop('Survived', axis=1).values
train_target = train['Survived'].values
```

- train_input, train_target 데이터를 확인한다.

```
train_input.shape, train_target.shape
```

```
((891, 7), (891,))
```

```
train_input.values
```

```
array([[3., 1., 1., ..., 0., 2., 2.],
       [1., 0., 2., ..., 1., 3., 2.],
       [3., 0., 1., ..., 0., 1., 1.],
       ...,
       [3., 0., 1., ..., 0., 1., 4.],
       [1., 1., 1., ..., 1., 2., 1.],
       [3., 1., 1., ..., 2., 2., 1.]])
```

```
train_target[:5]
```

```
array([0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1,
       1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0], dtype=int64)
```

• 모델링 후 교차검증

- 교차 검증을 위해 필요한 라이브러리를 import 한다.

```
from sklearn.model_selection import cross_validate
from sklearn.model_selection import StratifiedKFold
```

1) KNN : train data의 정확도는 약 84%, 검증 데이터의 정확도는 약 80%이다.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier()
score = cross_validate(model, train_input, train_target,
                       return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()
```

```
(np.float64(0.8369821296310886), np.float64(0.8013495700207143))
```


2) Decision Tree : train data의 정확도는 약 87%, 검증 데이터의 정확도는 약 80%로 과대적합으로 보인다.

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
score = cross_validate(model, train_input, train_target,
                        return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()

(np.float64(0.872335203366059), np.float64(0.8013746783001695))
```

3) Random Forest : train data의 정확도는 약 87%, 검증 데이터의 정확도는 약 80%로 과대적합으로 보임

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
score = cross_validate(model, train_input, train_target,
                        return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()

(np.float64(0.8555009691602187), np.float64(0.8204632477559475))
```

4) GradientBoosting : train data의 정확도는 약 86%, 검증 데이터의 정확도는 약 82%

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier()
score = cross_validate(model, train_input, train_target,
                        return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()

(np.float64(0.8555009691602187), np.float64(0.8204632477559475))
```

5) HistGradientBoosting : train data의 정확도는 약 86%, 검증 데이터의 정확도는 약 81%로 과대적합으로 보인다.

```
from sklearn.ensemble import HistGradientBoostingClassifier
model = HistGradientBoostingClassifier()
score = cross_validate(model, train_input, train_target,
                        return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()

(np.float64(0.8597093307278945), np.float64(0.8126043562864854))
```

6) Naive Bayes : train data의 정확도는 약 81%, 검증 데이터의 정확도는 약 81%

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
score = cross_validate(model, train_input, train_target,
                        return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()

(np.float64(0.8117284145169169), np.float64(0.812560416797439))
```

7) Support Vector Machine : train data의 정확도는 약 84%, 검증 데이터의 정확도는 약 80%

```
from sklearn.svm import SVC
model = SVC()
score = cross_validate(model, train_input, train_target,
                        return_train_score=True, n_jobs=-1, cv=StratifiedKFold())
score['train_score'].mean(), score['test_score'].mean()

(np.float64(0.8366992609168413), np.float64(0.8349946644906158))
```

• 최종 모델 선택

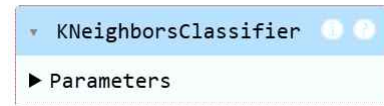
KNN, Gradient Boosting, SVM의 성능이 가장 좋았고 최종 모델로는 KNN으로 정하였다.

Model	train 데이터 정확도	검증 데이터 정확도
KNN	0.83698	0.80135
Gradient Boosting	0.8555	0.82046
SVM	0.836699	0.834995

• KNN 알고리즘을 이용한 생존 결과 예측

- KNN 알고리즘을 이용한 객체 생성 후 훈련데이터를 사용해 학습 모델을 생성한다.

```
model = KNeighborsClassifier()
model.fit(train_input, train_target)
```



- 생존 여부를 예측할 테스트 데이터프레임(test) 확인 후 모델링에 필요한 데이터(test_input)를 생성한다.

```
test.head(3)
```

	Pclass	Sex	Age	Fare	Embarked	Title_name	FamilySize
0	3	1	2.0	0.0	2	2	1
1	3	0	2.0	0.0	0	3	2
2	2	1	3.0	0.0	2	2	1

```
test_input = test.values
```

```
test_input
```

```
array([[3., 1., 2., ..., 2., 2., 1.],
       [3., 0., 2., ..., 0., 3., 2.],
       [2., 1., 3., ..., 2., 2., 1.],
       ...,
       [3., 1., 2., ..., 0., 2., 1.],
       [3., 1., 1., ..., 0., 2., 1.],
       [3., 1., 1., ..., 1., 0., 3.]])
```

```
test_input.shape
```

```
(418, 7)
```

- 테스트 데이터(test_input)을 입력으로 생존여부를 예측한다.

```
import warnings
warnings.filterwarnings('ignore')
prediction = model.predict(test_input)
```

- 제출할 파일을 읽어 데이터프레임(submission)에 저장한다. 테스트 파일의 탑승객 아이디와 생존 여부 칼럼으로 구성되어있다.

```
submission = pd.read_csv('data/gender_submission.csv')
submission.shape
```

```
(418, 2)
```

```
submission.head()
```

	PassengerId	Survived
0	892	0
1	893	1
2	894	0
3	895	0
4	896	1

- 예측한 결과(prediction)값을 제출 데이터프레임(submission)에 추가한 후 생존여부 성공률을 출력해 본다.

```
submission['Prediction'] = prediction
submission.head()
```

	PassengerId	Survived	Prediction
0	892	0	0
1	893	1	1
2	894	0	0
3	895	0	0
4	896	1	1

```
submission['Result'] =
    submission.apply(lambda x: True if x['Survived']==x['Prediction'] else False, axis=1)
```

```
filt = submission['Result'] == True
success = submission[filt]
len(success), len(success)/len(test)
```

```
(372, 0.8899521531100478)
```

```
submission.to_csv('data/submission_result.csv', index=False)
```

• 예측 Model의 성능을 올리기 위해 개선할 점

- 1) 더 자세한 EDA(Exploratory Data Analysis:탐색적 데이터 분석)을 한다.
- 2) 결측값을 다른 방법으로 처리해본다.
- 3) 사용하지 않은 Feature을 사용하거나, 사용한 Feature들 중 필요 없다고 생각하는 것들은 제거해본다.
- 4) Modeling 단계에서 파라미터 값을 튜닝해보며 최적화 값 찾는다.

• 시계열 분석

시계열 분석이란 시간의 흐름에 따라 기록된 데이터를 분석하여 패턴, 추세, 계절성 등을 파악하고, 이를 기반으로 미래 값을 예측하는 통계적 방법론이다. 주로 경제, 금융, 기상 등 다양한 분야에서 활용된다. 시계열 데이터 예측에 대표적인 모델은 ARIMA, LSTM, Prophet이다.

• ARIMA

ARIMA는 자기회귀모형(AR)과 이동평균모형(MA)을 고려하여 비정상성 데이터의 변환을 위해 관측치 간의 차분(I)을 사용하는 모델이다. 차분이란 현시점 데이터에서 d시점 이전 데이터를 뺀 것이다. 1차 차분은 1일전 데이터와의 차이, 2차 차분은 2일전 데이터와의 차이이다. AR(p), MA(q) 모형에 차분(d)을 이용해 비정상성을 제거하는 과정을 ARIMA(p, d, q)로 표현한다.

- AR: 자기회귀모형(Autoregressive) p시점만큼 앞선 시점까지의 값에 영향을 받는 모형
- I : integrated 누적을 의미하며 비정상성 데이터의 변환(평균의 정상화)을 위해 차분을 이용하는 시계열 모형에 사용하는 표현
- MA: 이동평균모형(Moving Average) q시점만큼 앞선 시점까지의 연속적인 오차 값들의 영향을 받는 모형

'11.구글주가(ARIMA).ipynb' 파일을 생성한 후 pmdarima 패키지 auto_arima 모듈을 사용하여 구글 주가를 예측해 본다.

- pmdarima 패키지를 설치한다.

```
pip install pmdarima==2.0.4
```

- numpy 오류가 발생하는 경우 특정 버전으로 업그레이드한다.

```
pip install --upgrade numpy==1.26.0
```

- 구글 10년 동안 주가 정보를 가지고와 데이터프레임(goog)에 저장 한다. High(최고가), Low(최저가), Open(시가), Close(종가), Volume(거래량), Adj Close(수정 종가: 기업의 배당 등 이익 배분활동에 대한 지급액을 재투자한다는 가정 하에 계산된 종가

```
import FinanceDataReader as fdr
import datetime
```

```
code = 'GOOG'
start = datetime.datetime(2012, 10, 31)
end = datetime.datetime(2022, 10, 31)
goog = fdr.DataReader(code, start, end)
goog.head(3), len(goog)
```

```
(
           Open      High      Low      Close  Volume  Adj Close
2012-10-31  16.933031  16.961424  16.811985  16.943991  61710442  16.843866
2012-11-01  16.924065  17.208000  16.904636  17.125559  82311371  17.024363
2012-11-02  17.304888  17.323816  17.120081  17.133778  93324497  17.032534,
2517)
```

- Step1. 데이터 전처리 & 원본시계열, 이동평균, 이동표준편차 시각화

1) 전처리 작업으로 거래량이 0인 행을 제거 하고 수정 종가 데이터만 가지고와 시리즈(data)에 저장한다.

```
data = goog['Adj Close'][goog['Volume'] != 0]
data.head(), len(data)
```

```
(2012-10-31    16.843866
 2012-11-01    17.024363
 2012-11-02    17.032534
 Name: Adj Close, dtype: float64,
 2517)
```

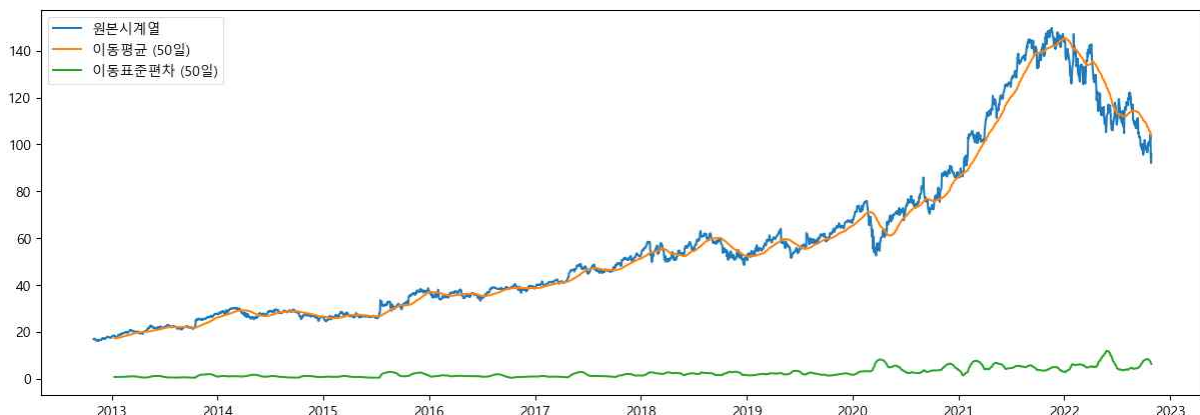
2) 원본시계열, 이동평균, 이동표준편차를 시각화하면 아래와 같이 해석된다.

- 해석1. 데이터가 평균이 일정하지 않은 비정상성의 특징(시간에 따라 평균 수준이 다르거나 추세, 계절성에 영향을 받는 데이터)을 가지는 것 같아 보이므로, 변환과정을 거친다면 ARIMA의 d차수가 1 이상일 것이다.
- 해석2. 계절성이나 특정 주기성은 크게 확인되지 않기 때문에 추후 ARIMA 분석에서 모수 seasonal이나 m은 auto_arima에 적용할 필요가 없다.

```
import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
plt.rc('font', size= 10)
plt.rc('axes', unicode_minus=False)
```

```
interval = 50
rolmean = data.rolling(interval).mean()
rolstd = data.rolling(interval).std()

plt.figure(figsize=(15, 5))
plt.plot(data, label='원본시계열')
plt.plot(rolmean, label=f'이동평균 ({interval}일)')
plt.plot(rolstd, label=f'이동표준편차 ({interval}일)')
plt.legend()
plt.show()
```



3) ARIMA 모델을 사용할 때는 차분 계수(차수)를 입력하여 데이터가 정상성을 띠게 하여 예측을 진행해야 하므로 이때 사용할 최적 차분 계수를 구한다. 위에서 설치한 pmdarima 라이브러리의 ndiffs() 함수를 이용하여 인자에 검정종류(kpss, adf)를 입력하면 ARIMA 모델에 적용할 최적의 차분 계수를 찾아준다. 실행한 결과 최적의 차분 계수는 1로 추정된다.

```
import warnings
warnings.filterwarnings('ignore')

from pmdarima.arima import ndiffs

kpss_diffs = ndiffs(data, alpha=0.05, test='kpss', max_d=6)
adf_diffs = ndiffs(data, alpha=0.05, test='adf', max_d=6)
n_diffs = max(adf_diffs, kpss_diffs)

print(f'추정된 차분 횟수 d = {n_diffs}')
```

추정된 차분 횟수 d=1

▪ Step2. ARIMA 학습 모델 생성 및 검정

1) data를 이용하여 train data와 test data를 9:1로 분리한다.

```
n_train = int(len(data) * 0.9)
train_data = data[:n_train]
test_data = data[n_train:]
len(data), len(train_data), len(test_data)

(2517, 2265, 252)
```

2) 훈련 데이터를 이용하여 학습한 모델을 생성한다. auto_arima 실행결과 최적의 모델 ARIMA(p, d, q)는 ARIMA(1, 1, 1) 모형으로 나왔다. 모델의 fitting이 좋을수록 AIC값이 작아지고 모델의 복잡도가 높아질수록 AIC값이 커진다.

```
import pmdarima as pm
model = pm.auto_arima(y=train_data, d=1, start_p=0, max_p=2, start_q=0, max_q=2,
                      m=1, seasonal=False, stepwise=True, trace=True)

Performing stepwise search to minimize aic
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=6100.783, Time=0.11 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=6092.276, Time=0.06 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=6092.636, Time=0.09 sec
ARIMA(0,1,0)(0,0,0)[0] : AIC=6107.188, Time=0.02 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AIC=6093.470, Time=0.08 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AIC=6091.929, Time=0.22 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AIC=6093.928, Time=0.33 sec
ARIMA(1,1,2)(0,0,0)[0] intercept : AIC=6093.928, Time=0.56 sec
ARIMA(0,1,2)(0,0,0)[0] intercept : AIC=6093.969, Time=0.11 sec
ARIMA(2,1,2)(0,0,0)[0] intercept : AIC=6093.668, Time=0.55 sec
ARIMA(1,1,1)(0,0,0)[0] : AIC=6099.179, Time=0.09 sec

Best model: ARIMA(1,1,1)(0,0,0)[0] intercept
Total fit time: 2.220 seconds
```

3) summary() 함수를 이용하여 몇 가지 방식으로 모델을 평가한 결과를 확인할 수 있다.

- Ljung-Box (Q)(용-박스 검정 통계량)은 잔차가 백색잡음(정상성을 만족인지)인지 검정한 통계량이다.
- Prob(Q)값이 0.99이므로 유의수준 0.05에서 귀무가설을 기각하지 못한다.
- Ljung-Box (Q) 통계량의 귀무가설은 '잔차(residual)가 백색잡음(white noise)시계열을 따른다'이므로 아래 결과를 통해 시계열 모형이 잘 적합 되었고 남은 잔차는 더 이상 자기상관을 가지지 않는 백색잡음임을 확인할 수 있다.

```
print(model.summary())
```

```
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:      2265
Model:                SARIMAX(1, 1, 1)      Log Likelihood:    -3041.964
Date:                Wed, 09 Jul 2025      AIC:                6091.929
Time:                14:22:42      BIC:                6114.828
Sample:              0      HQIC:                6100.284
                    - 2265
Covariance Type:      opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
intercept      0.0846      0.028       3.000      0.003       0.029       0.140
ar.L1      -0.4906      0.106      -4.649      0.000      -0.697      -0.284
ma.L1       0.4247      0.108       3.915      0.000       0.212       0.637
sigma2       0.8602      0.010      82.185      0.000       0.840       0.881
=====
Ljung-Box (L1) (Q):                0.00      Jarque-Bera (JB):          9707.38
Prob(Q):                          0.99      Prob(JB):                0.00
Heteroskedasticity (H):            11.78      Skew:                      0.20
Prob(H) (two-sided):              0.00      Kurtosis:                 13.14
=====
```

▪ Step3. 모델을 이용한 테스트 데이터 예측

1) 기간을 1일로 지정하여 하루 주가를 예측하는 함수를 정의한다.

```
def forecast_one_step():  
    #fc: 예측결과, conf_int: 하한, 상한 신뢰구간(confidence interval)  
    fc, conf_int = model.predict(n_periods=1, return_conf_int=True)  
    print(fc, conf_int)  
forecast_one_step()
```

```
2265      145.532325  
dtype: float64 [[143.71456597 147.35008455]]
```

```
def forecast_one_step():  
    fc, conf_int = model.predict(n_periods=1, return_conf_int=True)  
    print(fc.tolist()[0], conf_int[0])  
forecast_one_step()
```

```
145.53232526296222 [143.71456597 147.35008455]
```

```
def forecast_one_step():  
    fc, conf_int = model.predict(n_periods=1, return_conf_int=True)  
    return fc.tolist()[0], conf_int[0]  
fc, conf = forecast_one_step()  
print(fc, conf[0], conf[1])
```

```
95.8619100577736 93.4499426594174 98.2738774561298
```

2) 하루 주가를 예측하고 모형에 업데이트하여 다음날 주가를 예측하는 작업을 테스트 데이터 수만큼 반복한다.

```
y_pred = []  
upper_pred = []  
lower_pred = []  
  
#test_data의 데이터 값을 하나씩 꺼내 new_ob(new observation)에 저장하는 작업 반복한다.  
for new_ob in test_data:  
    fc, conf = forecast_one_step() #하루 예측한 값과 신뢰구간 값을 fc, conf에 저장한다.  
    y_pred.append(fc) #1개의 예측한 값을 y_pred 배열에 추가한다.  
    lower_pred.append(conf[0]) #하한 신뢰구간 값을(conf[0])을 lower_pred 배열에 추가한다.  
    upper_pred.append(conf[1]) #상한 신뢰구간 값을(conf[1])을 upper_pred 배열에 추가한다.  
    model.update(new_ob) #모델(model)에 new_ob(new observation)를 넣어 모델을 업데이트한다.
```

3) 세 개의 배열을(y_pred, lower_pred, upper_pred) 시각화하기 위해 시리즈(Series) 타입으로 변경한다.

```
import pandas as pd  
  
fc = pd.Series(y_pred, index=test_data.index)  
lower = pd.Series(lower_pred, index=test_data.index)  
upper = pd.Series(upper_pred, index=test_data.index)
```

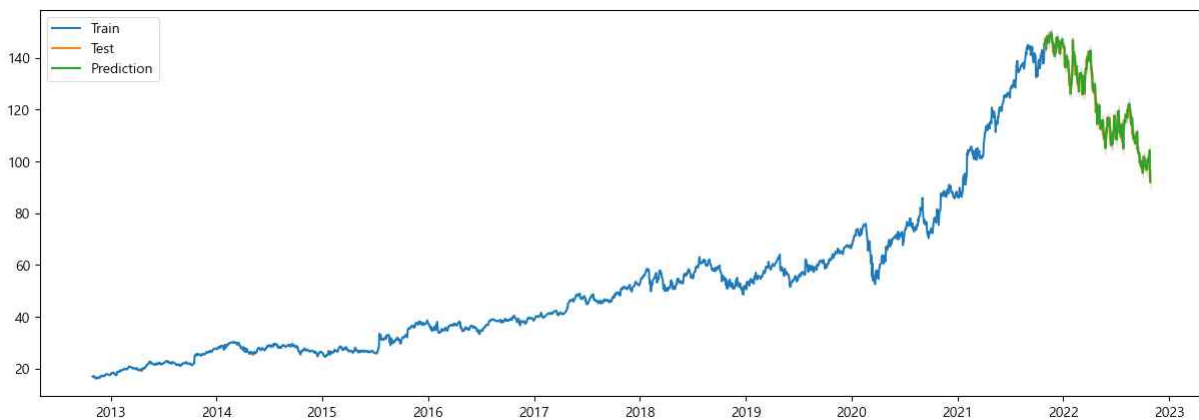
3) 모델의 오차율(성능평가)을 계산한다.

```
import numpy as np
def MAPE(y_test, y_pred):
    return np.mean(np.abs((y_test - y_pred) / y_test)) * 100
print(f"MAPE: {MAPE(test_data, fc):.3f}")
```

MAPE: 1.731

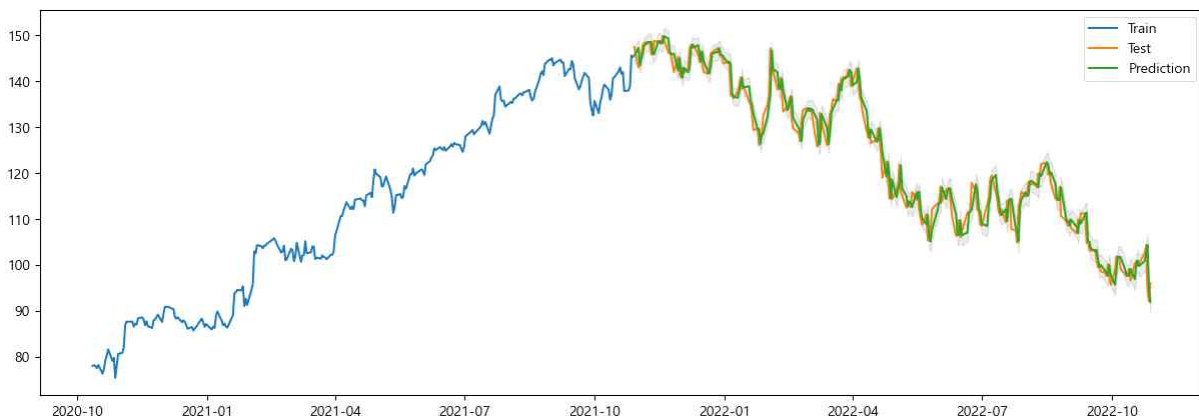
4) 테스트 데이터(test data)와 예측 데이터(fc)를 그래프로 시각화한다.

```
plt.figure(figsize=(15, 5))
plt.plot(train_data, label='Train')
plt.plot(test_data, label='Test')
plt.plot(fc, label='Prediction')
plt.fill_between(lower.index, lower, upper, color='k', alpha=0.1)
plt.legend()
```



5) 테스트 데이터(test data) 2000번째 이후 데이터와 예측 데이터(fc)를 그래프로 확대해서 시각화한다.

```
plt.figure(figsize=(15, 5))
plt.plot(train_data[2000:], label='Train')
plt.plot(test_data, label='Test')
plt.plot(fc, label='Prediction')
plt.fill_between(lower.index, lower, upper, color='black', alpha=0.1)
plt.legend()
```



- Step4. '2022-11-01'~'2023-10-31' 향후 1년 주가 예측
- 주식 개장일을 불러오는 패키지를 설치한다.

```
pip install exchange_calendars
```

- 1) 시작일과 종료일을 입력받아 주식 개장일 생성 후 데이트 타임의 index로 생성하는 함수를 정의한다.

```
import exchange_calendars as ecals

def get_open_dates(start, end):
    k = ecals.get_calendar('KRX')
    df = pd.DataFrame(k.schedule.loc[start:end])
    data_list=[]
    for i in df['open']:
        data_list.append(i.strftime('%Y-%m-%d'))
    date_index = pd.DatetimeIndex(data_list)
    return date_index
```

```
date_index = get_open_dates('2022-11-01', '2023-10-31')
date_index
```

```
DatetimeIndex(['2022-11-01', '2022-11-02', '2022-11-03', '2022-11-04',
                '2022-11-07', '2022-11-08', '2022-11-09', '2022-11-10',
                '2022-11-11', '2022-11-14',
                ...,
                '2023-10-18', '2023-10-19', '2023-10-20', '2023-10-23',
                '2023-10-24', '2023-10-25', '2023-10-26', '2023-10-27',
                '2023-10-30', '2023-10-31'],
               dtype='datetime64[ns]', length=247, freq=None)
```

- 2) for문을 통해 하루씩 미래를 예측하고 모델 업데이트를 반복한다.

```
y_pred2 = []
lower_pred2 = []
upper_pred2 = []

for i in range(len(date_index)):
    fc2, conf2 = forecast_one_step()
    y_pred2.append(fc2)
    lower_pred2.append(conf2[0])
    upper_pred2.append(conf2[1])
    model.update(fc2)
```

```
print(y_pred2[:5])
print(lower_pred2[:5])
print(upper_pred2[:5])
```

```
[95.8619389207178, 95.980098978169, 95.97120639352197, 96.01930687921151, 96.04321241857967]
[93.44997219347287, 93.5659201795017, 93.55977131018004, 93.60850553300817, 93.63279560150315]
[98.27390564796274, 98.39427777683629, 98.3826414768639, 98.43010822541486, 98.4536292356562]
```

3) 세 개의 배열을(y_pred, lower_pred, upper_pred) 시각화하기 위해 데이터프레임 타입으로 변경한다.

```
fc2 = pd.Series(y_pred2, index=date_index)
lower2 = pd.Series(lower_pred2, index=date_index)
upper2 = pd.Series(upper_pred2, index=date_index)
```

```
df_fc2 = pd.DataFrame(fc2)
df_lower2 = pd.DataFrame(lower2)
df_upper2 = pd.DataFrame(upper2)
merge = pd.merge(df_fc2, df_lower2, how='inner', left_index=True, right_index=True)
merge = pd.merge(merge, df_upper2, how='inner', left_index=True, right_index=True)
merge.columns = ['예측가', '하한가', '상한가']
```

merge.head(2)

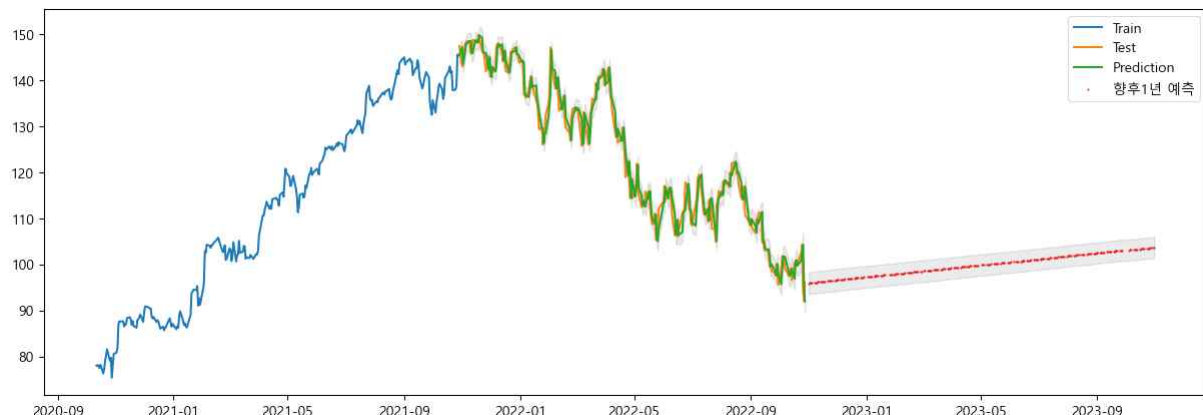
	예측가	하한가	상한가
2022-11-01	95.861993	93.450029	98.273958
2022-11-02	95.980139	93.565950	98.394329

merge.tail(2)

	예측가	하한가	상한가
2023-10-30	103.614335	101.311457	105.917213
2023-10-31	103.645745	101.343284	105.948206

4) 테스트 데이터(test data)와 예측 데이터(fc)과 향후1년 예측 데이터를 그래프로 시각화한다.

```
plt.figure(figsize=(15, 5))
plt.plot(train_data[2000:], label='Train')
plt.plot(test_data, label='Test')
plt.plot(fc, label='Prediction')
plt.fill_between(lower.index, lower, upper, color='black', alpha=0.1)
plt.scatter(fc2.index, fc2.values, s=0.5, color='red')
plt.fill_between(lower2.index, lower2, upper2, color='black', alpha=0.1)
plt.legend()
plt.show()
```



• LSTM(Long Short-Term Memory)

RNN(Recurrent Neural Network)은 이전의 입력 데이터를 기억해 다음 출력 값을 결정하는 모델이다. 하지만 RNN은 입력 데이터의 길이가 길어지면 그래디언트 소실 문제(Gradient Vanishing Problem)가 발생하여 이전의 정보를 제대로 기억하지 못하는 문제가 있다.

LSTM은 RNN(순환신경망)의 그래디언트 소실 문제를 해결하기 위해 고안된 모델이다. LSTM은 이전 정보를 오랫동안 기억할 수 있는 메모리 셀을 가지고 있으며 이를 통해 긴 시퀀스 데이터를 처리할 수 있다. 이를 통해 시계열 예측, 자연어 처리 등 다양한 분야에서 활용된다.

LSTM 모델은 tensorflow 패키지의 keras 모듈을 사용하여 구현한다. keras는 사용자 친화적이고 확장하기 쉽도록 설계된 모듈식 오픈 소스 신경망 라이브러리이다. 주요(기본) 백엔드는 TensorFlow이며 Google의 주요 지원을 받지만 Python으로 작성되었으며 여러 백엔드 신경망 계산 엔진을 지원한다.

'12.구글주가(LSTM).ipynb' 파일을 생성한 후 tensorflow 패키지 keras 모듈을 사용하여 구글 주가를 예측해 본다.

- tensorflow 패키지를 설치한다.

```
pip install tensorflow
```

• 데이터 수집

- 1) 필요한 모듈을 import하고 'GOOG' 종목 '2021-01-04'~'2024-08-01' 기간의 주가 데이터를 다운로드한다.

```
import FinanceDataReader as fdr

code = 'GOOG'
start = '2021-01-04'
end = '2024-08-01'
stock = fdr.DataReader(code, start, end)
stock.head(3)
```

	Open	High	Low	Close	Volume	Adj Close
2021-01-04	87.876999	88.032501	85.392502	86.412003	38038000	85.901398
2021-01-05	86.250000	87.383499	85.900749	87.045998	22906000	86.531639
2021-01-06	85.131500	87.400002	84.949997	86.764503	52042000	86.251808

- 2) index 이름을 'Date'로 수정한다.

```
stock.index.name = 'Date'
stock.tail(3)
```

	Open	High	Low	Close	Volume	Adj Close
Date						
2024-07-29	170.500000	172.160004	169.720001	171.130005	13768900	170.312393
2024-07-30	171.830002	172.949997	170.119995	171.860001	13681400	171.038910
2024-07-31	174.919998	175.910004	171.720001	173.149994	15650200	172.322708

- 3) 데이터프레임(stock)의 크기를 출력한다.

```
stock.shape
```

```
(899, 6)
```

• 데이터 준비 및 탐색

1) 일별 주가를 분석할 것이므로 분석에 필요한 날짜와 종가를 추출하여 데이터프레임(stock2)에 저장한다.

```
stock2 = stock[['Close']]
stock2.head(3)
```

	Close
Date	
2021-01-04	86.412003
2021-01-05	87.045998
2021-01-06	86.764503

2) 분석 정확도를 높이기 위해 정규화 작업을 진행한다. 스케일링 정규화 작업을 하여 종가를 0~1 범위로 조정한다.

▪ 정규화 작업을 하기 위해 값들만 stock_values에 저장한다.

```
stock_values = stock2.values
stock_values[0]
```

```
array([86.41200256])
```

▪ 스케일링 정규화 작업을 위해 MinMaxScaler 모듈을 import하고 stock_values에 대해 스케일링을 수행한다.

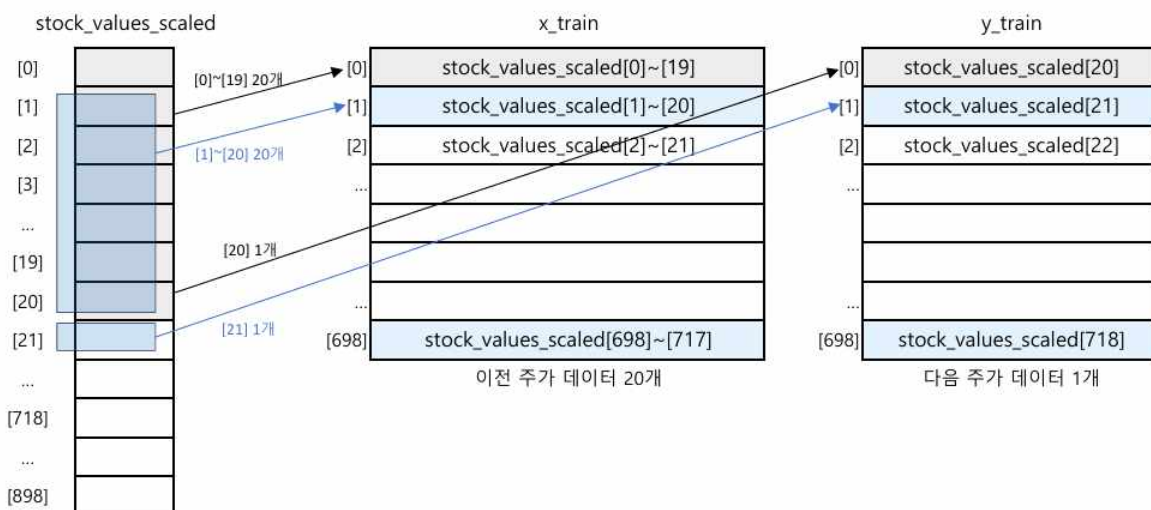
```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
stock_values_scaled = scaler.fit_transform(stock_values)
stock_values_scaled[0]
```

```
array([0.02676564])
```

• 분석 모델 구축

분석 모델에 사용할 학습 데이터 x_train, y_train을 준비한다. 한 달 동안의 주가 데이터를 학습하여 다음 날의 주가를 예측하는 시계열 분석을 진행하기 위해 정규화된 주가 데이터(stock_values_scaled)로 x_train과 y_train을 구성한다. 주식시장은 평일에만 개장하므로 한 달 기간에 대한 실제 주가 데이터는 20개를 사용한다. 일별 시계열 데이터인 20개 주가를 학습하여 다음 21번째 주가를 예측한다.

아래와 그림과 같이 stock_values_scaled[0]~[19]가 첫 번째 x값인 x_train[0]이 되고 그에 대한 출력 y_train[0]은 다음 값인 stock_values_scaled[20]이 된다. 그다음 stock_values_scaled[1]~[20]은 두 번째 x값인 x_train[1]이 되고 y_train[1]의 값은 stock_values_scaled[21]이 된다. 이 작업을 stock_values_scaled의 분할한 학습 데이터 범위 안에서 반복해 학습 데이터를 만든다.



1) 시계열 분석을 위한 학습 데이터 준비하기

- 데이터를 학습용과 평가용으로 8:2 분할하여 각각의 데이터 개수를 계산한다.

```
n_train = int(len(stock_values) * 0.8)
len(stock_values), n_train
```

```
(899, 719)
```

- stock_values_scaled에서 위에서 계산한 학습용 데이터의 개수만큼 추출한다. 학습용 데이터는 2차원 배열(179x1)이다.

```
stock_train = stock_values_scaled[:n_train]
stock_train.shape
```

```
(719, 1)
```

- 학습할 x 데이터 x_train[0]에는 stock_values_scaled[0]~[19]까지 20개의 데이터를 묶어서 추가하고 학습할 출력 데이터 y_train[0]에는 stock_values_scaled[20]을 추가한다. 이 작업을 range(20, 179)에서 반복하므로 699번 반복하게 된다.

```
x_train = []
y_train = []
for i in range(20, len(stock_train)):
    x_train.append(stock_train[i-20:i, 0])
    y_train.append(stock_train[i, 0])
x_train[0]
```

```
array([0.02676564, 0.03257305, 0.02999455, 0.0537923 , 0.06293394,
       0.04438949, 0.03515161, 0.03874694, 0.03223417, 0.03040674,
       0.05544565, 0.0994321 , 0.1014244 , 0.10591279, 0.10515712,
       0.11332785, 0.07373362, 0.08853626, 0.07600077, 0.10605019])
```

- 학습 데이터를 LSTM 모델의 구조로 변경한다. y_train은 배열로 변형하여 y_train1에 저장한다. x_train은 20개의 시계열 값을 하나로 묶은 20행 1열의 배열을 699개 가진 3차원 배열(699x20x1)로 변형하여 x_train1에 저장한다.

```
import numpy as np
```

```
x_train1 = np.array(x_train)
y_train1 = np.array(y_train)
```

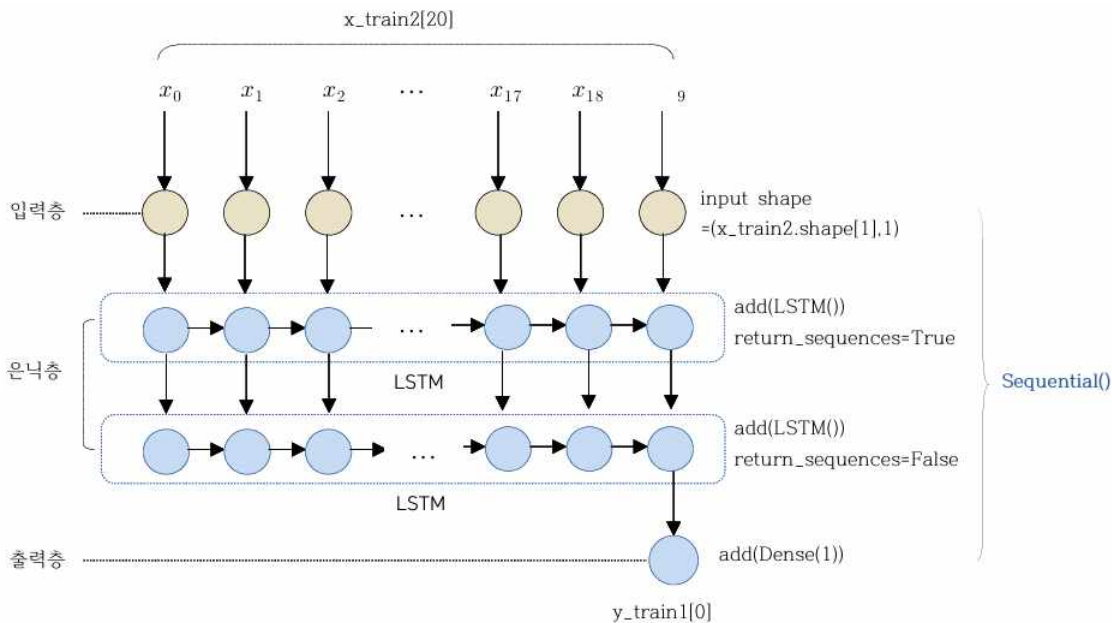
```
x = x_train1.shape[0]
y = x_train1.shape[1]
z = 1
x_train2 = np.reshape(x_train1, (x, y, z))
```

```
print(x_train2.shape)
print(x_train2[0][:5])
```

```
(699, 20, 1)
[[0.02676564]
 [0.03257305]
 [0.02999455]
 [0.0537923 ]
 [0.06293394]]
```

2) LSTM 모델 구축하고 학습하기

딥러닝 모델을 구축할 때 입력층, 은닉층, 출력층의 구조와 개수를 다양하게 구성할 수 있다. 여기서는 케라스 라이브러리를 사용하여 아래 그림과 같이 LSTM 딥러닝 모델을 구축한다. 즉 케라스 시퀀스 모델 객체에 한 층씩 추가하면서 구축한다. 입력층에 대한 정보는 첫 번째 층을 추가할 때 매개변수(input_shape)를 통해 설정한다. 은닉층의 첫 번째 LSTM층을 20개 입력값에 대한 노드 20으로 구성(unit=20)하며 각 노드의 출력을 다음 층으로 전달(return_sequence=True)한다. 이전 층에서 20개의 값을 입력받아 은닉층의 두 번째 LSTM층을 처리하고 마지막 시퀀스 노드의 값 하나만 출력층에 전달(return_sequence=False)한다.



- LSTM 딥러닝 모델 구축에 필요한 모듈을 import하여 시퀀스 모델 객체를 생성하고 위 그림과 같은 모델을 구성한다.

```
from tensorflow import keras
model = keras.Sequential([keras.Input(shape=(x_train2.shape[1], x_train2.shape[2])),
    keras.layers.LSTM(units=20, return_sequences=True),
    keras.layers.LSTM(units=20, return_sequences=False),
    keras.layers.Dense(1)])
```

- 학습 방법에 필요한 손실 함수(mean_squared_error)와 최적화 함수(adam)를 매개변수로 설정(compile())한다. 전체 학습 데이터에 대해 10번 반복(epochs=10)하여 학습을 수행(fit())한다.

```
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(x_train2, y_train1, epochs=10, batch_size=1, verbose=2)
```

```
Epoch 1/10
699/699 - 5s - 8ms/step - loss: 0.0039
Epoch 2/10
699/699 - 3s - 4ms/step - loss: 0.0020
Epoch 3/10
699/699 - 3s - 5ms/step - loss: 0.0014
Epoch 4/10
699/699 - 3s - 5ms/step - loss: 0.0012
Epoch 5/10
699/699 - 4s - 5ms/step - loss: 0.0010
Epoch 6/10
699/699 - 4s - 5ms/step - loss: 8.8827e-04
Epoch 7/10
699/699 - 4s - 5ms/step - loss: 7.8577e-04
Epoch 8/10
699/699 - 4s - 5ms/step - loss: 7.3580e-04
Epoch 9/10
699/699 - 3s - 5ms/step - loss: 6.6070e-04
Epoch 10/10
699/699 - 4s - 6ms/step - loss: 6.3035e-04

<keras.src.callbacks.history.History at 0x2280c7fd000>
```

3) 평가 데이터로 예측 수행하기

학습 데이터 다음 시퀀스의 데이터 180개를 평가 데이터 `x_test`로 구성한다. 이를 위해 학습 데이터 구성과 같은 방법으로 평가 데이터 범위의 데이터를 3차원 구조로 변형한 `x` 입력을 만들고 LSTM 모델에 입력하여 예측값을 구한다.

- 평가 데이터로 정리해야 하는 `stock_values_scaled[699]`부터 데이터인 `stock_values_scaled[898]`까지 총 200개의 데이터를 `stock_test`에 저장한다. `stock_test[0]~stock_test[19]`의 20개 데이터를 묶어서 `x_test[0]`에 추가한다. 이 작업을 `range(20, 200)`에서 반복하여 평가 데이터 180개가 `x_test`에 만들어진다.

```
stock_test = stock_values_scaled[n_train-20:]
x_test = []
for i in range(20, len(stock_test)):
    x_test.append(stock_test[i-20:i, 0])
x_test[0]

array([0.50462582, 0.5221215 , 0.52670152, 0.51103781, 0.50828978,
       0.48777141, 0.49839693, 0.5187322 , 0.39552989, 0.36594305,
       0.36557664, 0.38710268, 0.3829807 , 0.40377393, 0.41302557,
       0.42942195, 0.4393148 , 0.44801679, 0.45589442, 0.44151325])
```

- `x_test`를 3차원 배열(180x20x1)로 변형한다.

```
x_test = np.array(x_test)

x = x_test.shape[0]
y = x_test.shape[1]
z = 1

x_test = np.reshape(x_test, (x, y, z))
print(x_test.shape)
print(x_test[0][:10])

(180, 20, 1)
[[0.50462582]
 [0.5221215 ]
 [0.52670152]
 [0.51103781]
 [0.50828978]
 [0.48777141]
 [0.49839693]
 [0.5187322 ]
 [0.39552989]
 [0.36594305]]
```

- 평가 데이터를 모델에 입력하여 예측한 값을 `predicted_value`에 저장한다.

```
predicted_value = model.predict(x_test)
```

6/6  1s 58ms/step

```
predicted_value[0]
```

```
array([0.45817074], dtype=float32)
```

• 결과 분석 및 시각화

LSTM 모델의 실행 결과를 그래프로 시각화하여 확인해 본다.

1) 그래프에 사용할 데이터 정리하기

그래프에는 원본 주가 데이터를 넣어야 하므로 정규화 작업 이전의 원본 데이터 stock을 사용한다. LSTM 모델에서 구한 predict_value 예측값은 0~1범위로 정규화된 값이므로 역정규화(inverse_transform())를 수행하여 변환 값을 사용한다.

- 정규화 상태의 예측값에 대해 역정규화(inverse_transeform())를 수행하여 원본 범위의 값으로 변환한다.

```
predicted_value = scaler.inverse_transform(predicted_value)
predicted_value[0]
```

```
array([133.5085], dtype=float32)
```

- 학습 구간 그래프에 사용할 데이터(stock_train_vis)와 평가/예측 구간에 사용할 데이터를 stock_test_vis를 구한다.

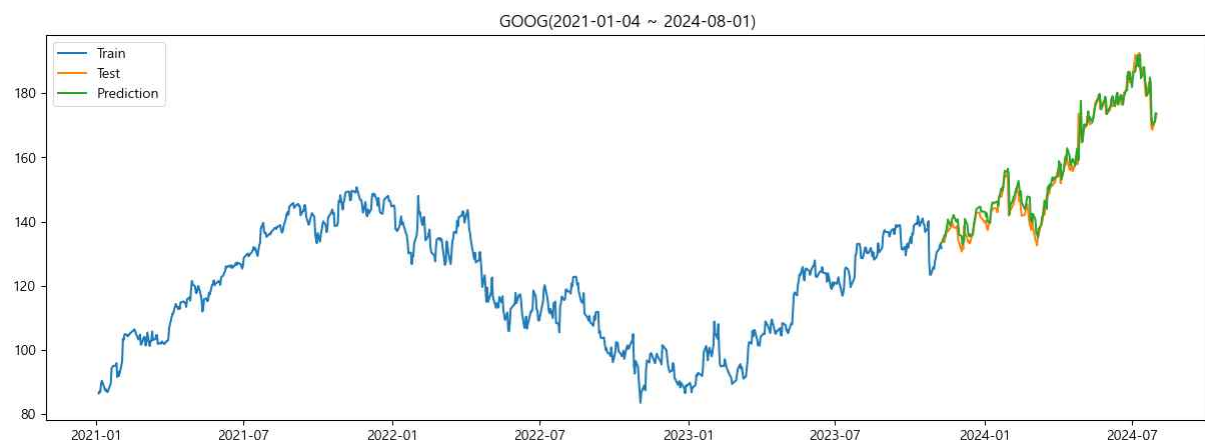
```
stock_train_vis = stock[:n_train]
stock_test_vis = stock[n_train:].copy()
stock_test_vis['Prediction'] = predicted_value
stock_test_vis.head(2)
```

	Open	High	Low	Close	Volume	Adj Close	Prediction
Date							
2023-11-10	131.529999	134.270004	130.869995	134.059998	20872900	133.267822	133.508499
2023-11-13	133.360001	134.110001	132.770004	133.639999	16409900	132.850311	135.893463

2) 그래프 그리기

그래프 그리기에 필요한 모듈을 import한다. 학습데이터, 평가데이터, 예측값에 대한 plot을 그린다. 어제의 주가가 오늘과 내일의 주가에 영향을 미치기 때문에 주가는 시간이 중요한 분석 요소가 되어야 하는 시계열 데이터이다. 오늘까지의 20개 데이터를 가지고 내일의 주가를 예측할 수 있고 이러한 과정을 반복하면서 3일 뒤, 7일 뒤, 한 달 뒤의 주가를 예측할 수 있다.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(15, 5))
plt.plot(stock_train_vis['Close'], label='Train')
plt.plot(stock_test_vis['Close'], label='Test')
plt.plot(stock_test_vis['Prediction'], label='Prediction')
plt.legend()
plt.title(f'{code}({start} ~ {end})')
plt.show()
```



- 다음날 주가 예측

1) 최근 과거 20일 주가 데이터를 추출하여 20개의 값을 하나로 묶는 배열(1x20)로 변환하기 위해 행, 열을 바꿔준다.

```
stock_test = stock_values_scaled[-20:]
print(stock_test.shape)
print(stock_test[:5])
x_test = stock_test.T
print(x_test.shape)
print(x_test)

(20, 1)
[[0.95172663]
 [0.99358801]
 [0.98003107]
 [0.97966473]
 [1.          ]]
(1, 20)
[[0.95172663 0.99358801 0.98003107 0.97966473 1.          0.95090226
 0.94613901 0.95905468 0.93441419 0.90803327 0.87688924 0.87844643
 0.91472019 0.91701019 0.83246306 0.78473941 0.7803425  0.80278467
 0.80947145 0.82128782]]
```

2) LSTM 모델로 변경하기 위해 x_test 데이터를 3차원 배열(1x20x1)로 변환한다.

```
x = x_test.shape[0]
y = x_test.shape[1]
z = 1
x_test = np.reshape(x_test, (x, y, z))
print(x_test.shape)
print(x_test)

(1, 20, 1)
[[[0.95172663]
 [0.99358801]
 [0.98003107]
 ...
 [0.80947145]
 [0.82128782]]]]
```

3) 3차원 구조로 변경한 x_test를 입력하여 예측값을 구한다.

```
predicted_value = model.predict(x_test)
predicted_value

1/1 ————— 0s 51ms/step
array([[0.8197806]], dtype=float32)
```

4) LSTM 모델에서 구한 예측값은 0~1 범위로 정규화된 값이므로 역정규화를 수행하여 변환된 값을 출력한다.

```
predicted_value = scaler.inverse_transform(predicted_value)
predicted_value

array([[172.98546]], dtype=float32)
```

- 향후 특정일 주가 예측

1) 향후 10일에 대한 주가 예측값을 구해 predicted_values 배열에 저장한다.

```
days = 10
stock_test = stock_values_scaled[-20:]
predicted_values = []

for i in range(days):
    x_test = stock_test[-20:]  #최근 데이터 20개만 추출
    x_test = x_test.T  #행렬 변환(20행1열->1행 20열)

    #예측을 위해 3차원 배열로 변환
    x = x_test.shape[0]
    y = x_test.shape[1]
    z = 1
    x_test = np.reshape(x_test, (x, y, z))

    predicted_value = model.predict(x_test)  #days 다음 일에 대한 예측값
    print(predicted_value[0])
    predicted_values.append(predicted_value[0])

stock_test = np.append(stock_test, predicted_value, axis=0)  #예측값 추가
```

```
1/1 ————— 0s 24ms/step
[0.8085247]
1/1 ————— 0s 25ms/step
[0.78822654]
1/1 ————— 0s 25ms/step
[0.76811075]
1/1 ————— 0s 31ms/step
[0.7539066]
1/1 ————— 0s 24ms/step
[0.7448725]
1/1 ————— 0s 24ms/step
[0.7382176]
1/1 ————— 0s 23ms/step
[0.7319459]
1/1 ————— 0s 25ms/step
[0.7253622]
1/1 ————— 0s 25ms/step
[0.7185944]
1/1 ————— 0s 25ms/step
[0.71202004]
```

2) LSTM 모델에서 구한 예측값들은 0~1 범위로 정규화된 값들이므로 역정규화를 수행하여 변환된 값들을 출력한다.

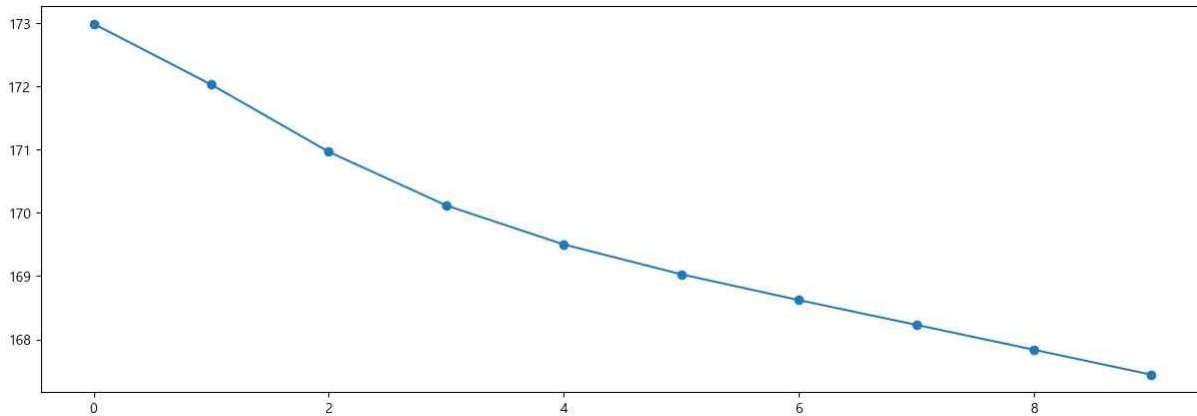
```
predicted_values = scaler.inverse_transform(predicted_values)
predicted_values
```

```
array([[172.98544943],
       [172.02650703],
       [170.96364721],
       [170.11644369],
       [169.50039624],
       [169.02859684],
       [168.62078767],
       [168.22964953],
       [167.83665038],
       [167.44244091]])
```

3) 향후 10일에 대한 주가 예측값들을 시각화 한다.

```
import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
plt.rc('font', size= 10)
plt.rc('axes', unicode_minus=False)
```

```
plt.figure(figsize=(15, 5))
plt.plot(predicted_values, marker='o')
plt.show()
```



4) 주식 개장일을 생성하여 predicted_values index로 지정한다.

- 시작일과 종료일을 입력받아 주식 개장일 생성 후 데이터 타입의 index로 생성하는 함수를 정의한다.

```
import pandas as pd
import exchange_calendars as ecals

def get_open_dates(start, end):
    k = ecals.get_calendar('XKRX')
    df = pd.DataFrame(k.schedule.loc[start:end])
    data_list=[]
    for i in df['open']:
        data_list.append(i.strftime('%Y-%m-%d'))
    date_index = pd.DatetimeIndex(data_list)
    return date_index
```

- '2024-08-01'이후부터 '2025-08-01'이전 주식 개장일을 생성한다.

```
date_index = get_open_dates('2024-08-01', '2025-08-01')
date_index
```

```
DatetimeIndex(['2024-08-01', '2024-08-02', '2024-08-05', '2024-08-06',
                '2024-08-07', '2024-08-08', '2024-08-09', '2024-08-12',
                '2024-08-13', '2024-08-14',
                ...,
                '2025-07-21', '2025-07-22', '2025-07-23', '2025-07-24',
                '2025-07-25', '2025-07-28', '2025-07-29', '2025-07-30',
                '2025-07-31', '2025-08-01'],
              dtype='datetime64[ns]', length=242, freq=None)
```

- 생성된 index(date_index)에서 days 개수만큼 날짜를 추출한다.

```
date_index = date_index[:days]
date_index

DatetimeIndex(['2024-08-01', '2024-08-02', '2024-08-05', '2024-08-06',
               '2024-08-07', '2024-08-08', '2024-08-09', '2024-08-12',
               '2024-08-13', '2024-08-14'],
              dtype='datetime64[ns]', freq=None)
```

- predicted_values의 idnex를 date_index로 지정한다.

```
stock_pred_vis = pd.DataFrame(predicted_values, date_index)
stock_pred_vis.head(2)
```

```

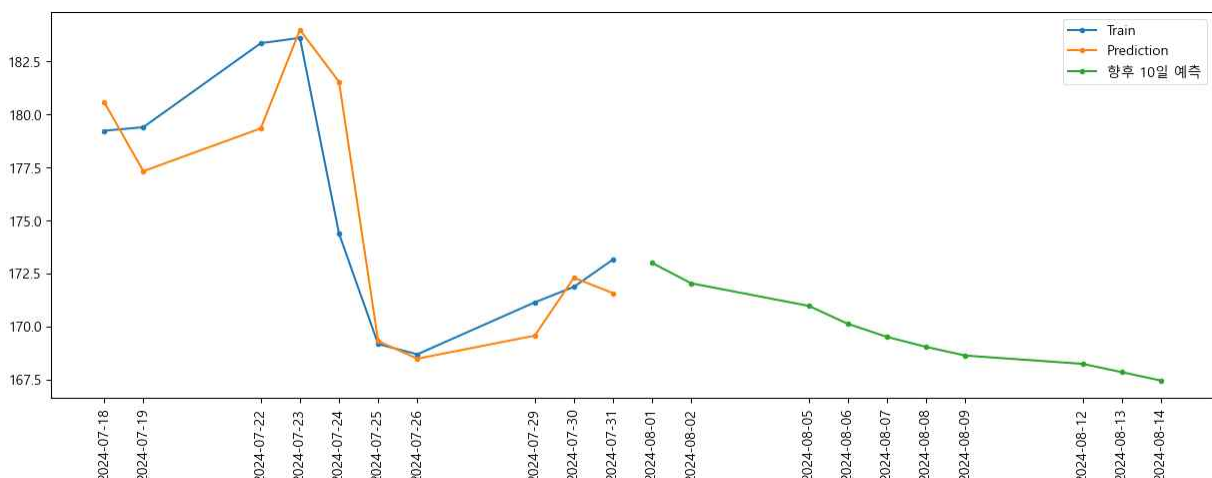
0
2024-08-01    171.756647
2024-08-02    169.540694
```

- 5) x축 눈금을 생성하여 xticks를 설정하고 Test, Prediction값들을 시각화에 추가한다.

```
xticks = stock_test_vis[-days:].index
xticks = xticks.append(stock_pred_vis.index)
xticks

DatetimeIndex(['2024-07-18', '2024-07-19', '2024-07-22', '2024-07-23',
               '2024-07-24', '2024-07-25', '2024-07-26', '2024-07-29',
               '2024-07-30', '2024-07-31', '2024-08-01', '2024-08-02',
               '2024-08-05', '2024-08-06', '2024-08-07', '2024-08-08',
               '2024-08-09', '2024-08-12', '2024-08-13', '2024-08-14'],
              dtype='datetime64[ns]', freq=None)
```

```
plt.figure(figsize=(15, 5))
plt.plot(stock_test_vis['Close'][-days:], label='Train', marker='o', ms=3)
plt.plot(stock_test_vis['Prediction'][-days:], label='Prediction', marker='o', ms=3)
plt.plot(stock_pred_vis, label=f'향후 {days}일 예측', marker='o', ms=3)
plt.xticks(xticks, rotation=90)
plt.legend()
plt.show()
```



• Prophet

Prophet 라이브러리는 페이스북(현 메타)에서 개발한 오픈 소스 시계열 예측 모델이다. 날짜 정보를 x 데이터로 하여 y를 예측하는 간단한 구조를 사용한다. 비즈니스 예측(예: 매출, 사용자 수, 웹 트래픽 등)에 유용하다.

'13.구글주가(Prophet).ipynb' 파일을 생성한 후 Prophet 라이브러리 모듈을 사용하여 100일 후 구글 주가를 예측해 본다.

- prophet 패키지를 설치한다.

```
pip install prophet
```

• 데이터 수집

- 1) 필요한 모듈을 import하고 '2021-01-04'~'2023-08-01' 기간의 구글 주가 데이터를 데이터프레임(goog)에 저장한다.

```
import FinanceDataReader as fdr

code = 'GOOG'
start = '2021-01-04'
end = '2024-08-01'
goog = fdr.DataReader(code, start, end)
```

- 2) 데이터프레임(goog)에서 종가(Close) 칼럼을 추출하고 index 이름을 'Date'로 변경한다.

```
goog = goog[['Close']]
goog.index.name = 'Date'
goog.head(3)
```

	Close
Date	
2021-01-04	86.412003
2021-01-05	87.045998
2021-01-06	86.764503

• 데이터 준비 및 탐색

- 1) 데이터에 대한 기본 정보를 확인한다. 899행과 1개의 열(Close)로 결측치 없어 구성되어있다.

```
goog.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 899 entries, 2021-01-04 to 2024-07-31
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Close    899 non-null     float64
dtypes: float64(1)
memory usage: 14.0 KB
```

```
goog.isnull().sum()
```

```
Close    0
dtype: int64
```

2) index를 reset하여 'Date'를 일반 컬럼으로 변경한다.

```
stock = goog.copy()
stock.reset_index(inplace=True)
stock.head(1)
```

	Date	Close
0	2021-01-04	86.412003

3) Prophet 모델은 입력 데이터의 구조와 열 이름이 정해져있다. 열은 단 2개로 구성되어 있다. 첫 번째 열은 날짜 데이터로 열 이름이 'ds'(datestamp)여야 하고, 두 번째 열은 숫자 데이터로 열 이름이 'y'여야 한다.

```
stock.columns = ['ds', 'y']
stock.head(1)
```

	ds	y
0	2021-01-04	86.412003

• 분석 모델 구축

1) 시계열 분석을 위해 Prophet 모델 객체를 생성하고 주가 데이터를 사용하여 학습을 수행한다.

```
from prophet import Prophet
model = Prophet()
model.fit(stock)
```

```
21:03:34 - cmdstanpy - INFO - Chain [1] start processing
21:03:35 - cmdstanpy - INFO - Chain [1] done processing

<prophet.forecaster.Prophet at 0x2c39a4ac670>
```

2) 이후 100일(periods=100)의 날짜 데이터를 생성(make_future_dataframe)한다. 생성된 future_ds를 확인해 보면 '2024-08-01' 이후 100일인 '2024-11-08'까지의 날짜 데이터가 있다.

```
future_ds = model.make_future_dataframe(periods=100)
future_ds.tail(2)
```

	ds
996	2024-11-06
997	2024-11-07
998	2024-11-08

3) future_ds는 원래 데이터에 100일의 데이터가 추가되어 999개의 행과 날짜 열 ds로 구성되어 있다.

```
future_ds.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 999 entries, 0 to 998
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0    ds      999 non-null    datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 7.9 KB
```

4) 미래 날짜에 대한 Prophet 예측값을 생성한다. Prophet 모델은 예측값, 예측 최솟값, 예측 최댓값을 생성한다.

```
future_y = model.predict(future_ds)
```

```
future_y[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].head()
```

	ds	yhat	yhat_lower	yhat_upper
0	2021-01-04	92.735032	86.682458	98.769747
1	2021-01-05	92.574436	86.882764	98.362979
2	2021-01-06	92.467329	86.530349	98.806576
3	2021-01-07	92.427258	86.228281	98.611946
4	2021-01-08	92.444746	86.428333	98.537646

```
future_y[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail()
```

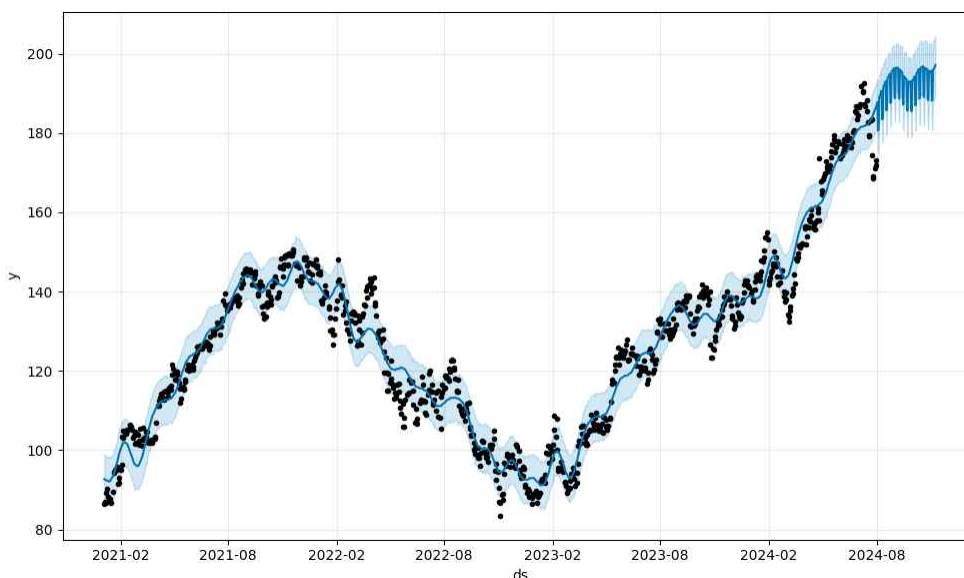
	ds	yhat	yhat_lower	yhat_upper
994	2024-11-04	195.939613	188.508455	203.472248
995	2024-11-05	196.098134	187.946049	203.455392
996	2024-11-06	196.354945	188.409114	203.267035
997	2024-11-07	196.714879	189.055611	203.923638
998	2024-11-08	197.158386	189.682329	204.482437

• 결과 분석 및 시각화

1) 결과 시각화 및 Prophet3요소 시각화하기 : Prophet 라이브러리에서 제공하는 시각화 함수를 사용하여 예측 그래프뿐 아니라 Prophet 3요소에 대한 패턴을 시각화 할 수 있다.

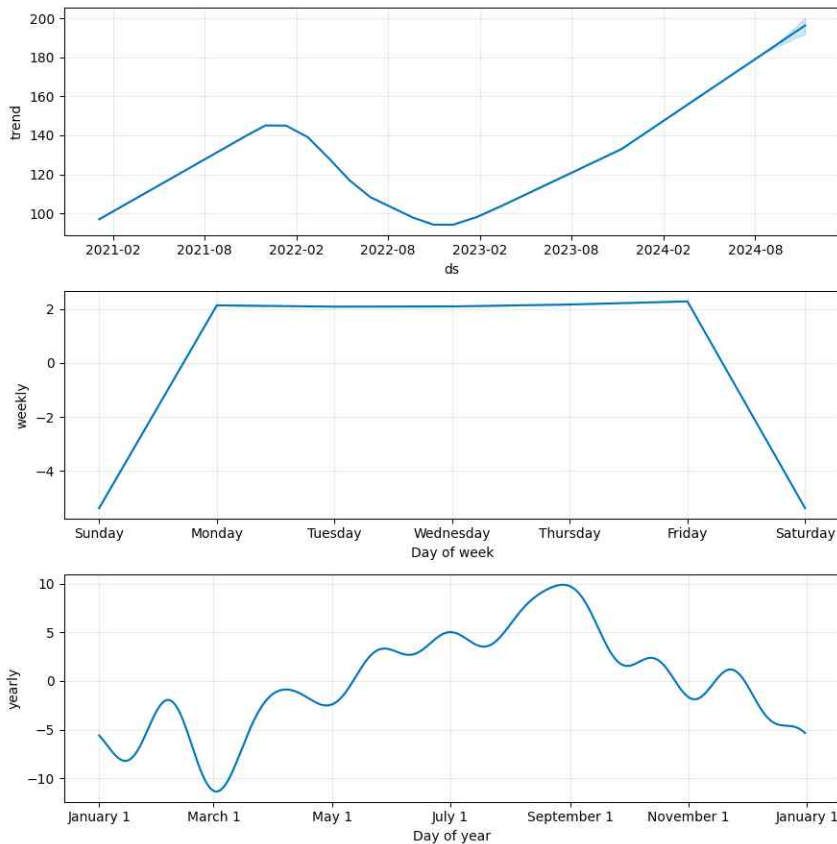
- Prophet 라이브러리에서 제공하는 함수(plot())를 사용하여 그래프로 시각화 한다.

```
fig_trend = model.plot(future_y)
```



- `plot_components()` 함수를 사용하여 트렌드, 주 계절성, 연 계절성 패턴 분석을 그래프로 시각화 한다.

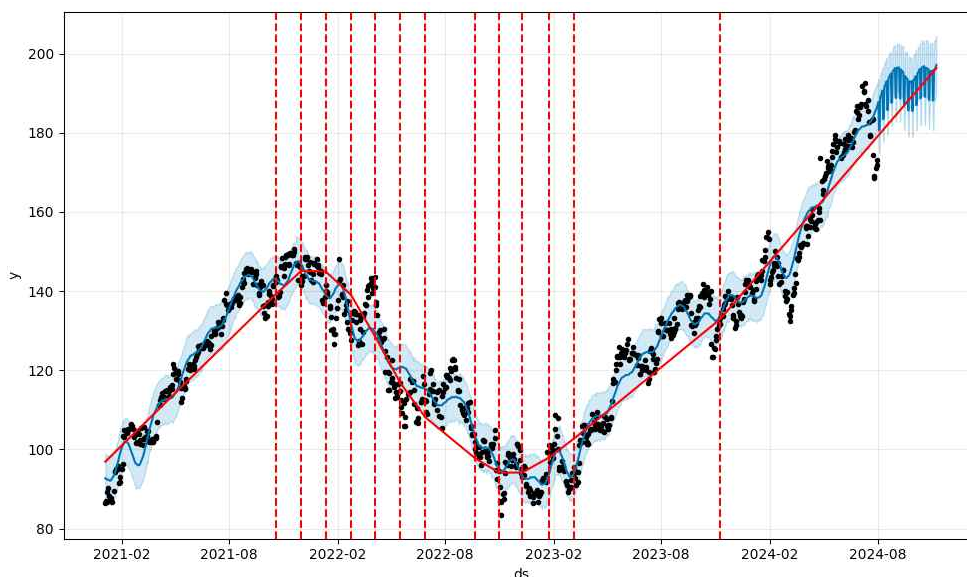
```
fig_components = model.plot_components(future_y)
```



- 2) 트렌드 변화 시점을 반영하여 시각화하기: 트렌드에서 주된 변화가 나타난 시점을 찾아 세로선을 추가한 플롯으로 시각화 한다.

- `add_changepoints_to_plot()` 함수로 트렌드 변화 시점을 추출해서 트렌드 시각화 객체 `fig`에 빨간 세로선을 추가하여 시각화한다.

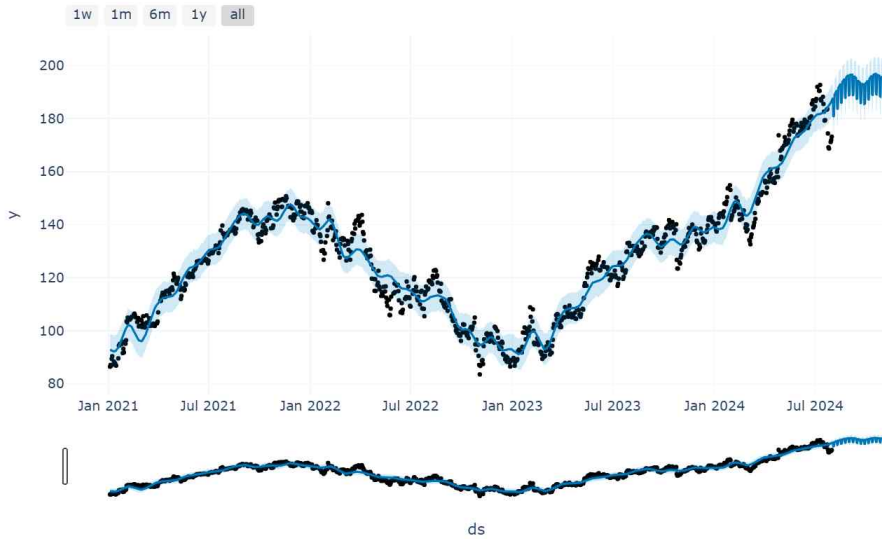
```
from prophet.plot import add_changepoints_to_plot
fig_trend = model.plot(future_y)
fig_trend_changepoints = add_changepoints_to_plot(fig_trend.gca(), model, future_y)
```



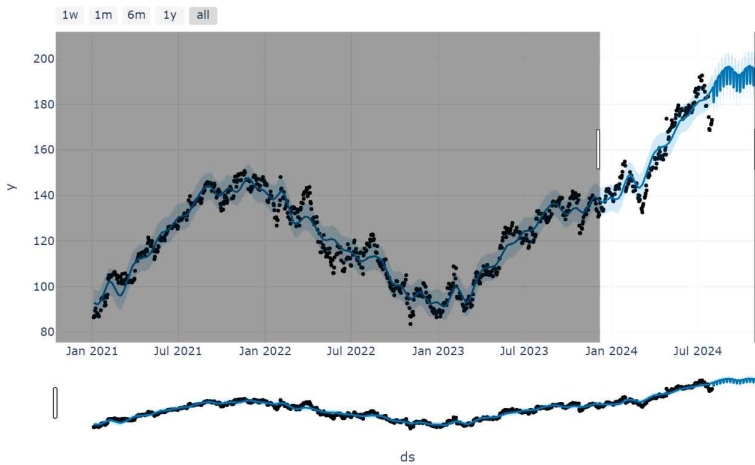
3) Prophet의 트렌드와 계절성(weekly, yearly)에 대한 대화형 시각화 표현하기

- Prophet은 트렌드 플롯과 3요소(trend, yearly, weekly) 플롯에서 마우스로 드래그하여 설정한 영역을 실시간으로 다시 그려주는 대화형 플롯을 제공한다.

```
from prophet.plot import plot_plotly
plot_plotly(model, future_y)
```



- 마우스를 클릭&드래그하여 상세 보기 영역을 설정한다.



- 설정한 영역에 대한 그래프 시각화를 한다.



• 자연어 처리와 텍스트 마이닝

텍스트 마이닝(Text Mining)은 얼핏 보아 자연어처리(NLP)와 비슷한 개념인 것처럼 보이지만, 엄밀히 말해 NLP(자연어처리)와는 다른 분야다. 자연어처리는 인간의 언어인 자연어(Natural Language)를 인공지능이 인식하고 표현할 수 있도록 하는 것에 중점을 두는 분야인 반면, 텍스트 마이닝은 자연어처리 과정을 통해 추출된 단어나 문장 속에서 유의미한 인사이트(Insight)를 찾아내는 작업을 칭한다.

텍스트 마이닝은 비정형의 텍스트 데이터로부터 패턴을 찾아내어 의미 있는 정보를 추출하는 분석 과정 또는 기법을 말한다. 텍스트 마이닝은 데이터 마이닝과 자연어 처리, 정보 검색등의 분야가 결합된 분석 기법을 사용하여 텍스트 데이터로부터 유용한 고급 정보를 찾아내는 과정이다. 텍스트 마이닝의 프로세스는 다음과 같다. 텍스트 전처리에는 토큰화, 불용어 제거, 표제어 추출, 형태소 분석 등의 작업이 포함된다.

텍스트 전처리 → 특성 벡터화 → 머신러닝 모델 구축 및 학습/평가 프로세스 수행

• 감성 분석 모델링

'14.감성분석모델링.ipynb' 파일을 생성한 후 영화 리뷰 데이터에 텍스트 마이닝의 감성 분석 기술을 사용하여 감성 분석 모델을 구축한 뒤 새로운 데이터에 대한 감성을 분석한다. 감성 분석은 텍스트 데이터에 담긴 감정, 의견, 태도 등을 파악하는 자연어 처리 기술이다. 긍정, 부정, 중립 등 감정의 극성을 분석하거나 감정의 강도를 평가하여 제품리뷰 분석, 고객응대, 시장조사 등 다양한 분야에서 활용된다.

한국어 자연어 처리 파이썬 라이브러리 KoNLPy(코엔엘파이)를 아래와 같이 설치한다. 이 라이브러리는 한국어 텍스트 분석, 형태소 분석, 품사 태깅 등 다양한 기능을 제공하여 자연어 처리 작업을 용이하게 해준다.

```
pip install konlpy
```

• 데이터 수집

- <https://github.com/e9t/nsmc>에서 ratings.txt, ratings_test.txt, ratings_train.txt 파일을 다운로드한다.
- ratings.txt 파일은 네이버 영화 페이지에서 리뷰를 크롤링하여 수집한 200K 용량의 데이터 파일이다. ratings_test.txt 파일은 50K 평가용으로 분리한 파일이 이고 나머지 150K를 훈련용으로 준비한 것이 ratings_train.txt 파일이다.
- 파일 내용은 3개의 칼럼(id, document, label)이 탭(\t)로 분리되어 있다. label 칼럼은 강성 분류 클래스 값인데 1~10점의 평점 중에서 중립적인 평점인 5~8점은 제외하고 1~4점을 부정 감성 0으로, 9~10점을 긍정 감성 1로 표시하였다.

• 데이터 준비 및 탐색

1) 훈련데이터를 읽고 데이터프레임(df_train)에 저장한 후 결측치 정보를 확인해 본다.

```
import pandas as pd
df_train = pd.read_csv('data/ratings_train.txt', encoding='utf-8', sep='\t')
df_train.head(3)
```

	id	document	label
0	9976970	아 더빙.. 진짜 짜증나네요 목소리	0
1	3819312	흠...포스터보고 초딩영화줄....오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 ..솔직히 재미는 없다..평점 조정	0
4	6483659	사이몬페그의 익살스런 연기가 돋보였던 영화!스파이더맨에서 늙어보이기만 했던 커스틴 ...	1

- 훈련 데이터 150,000개중 document 칼럼 값이 non-null인 샘플이 149,995개다. 따라서 5개의 결측치가 있다는 것을 알 수 있다.

```
df_train.isnull().sum()
```

```
id          0
document    5
label       0
dtype: int64
```

```
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150000 entries, 0 to 149999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    id         150000 non-null  int64
1    document   149995 non-null  object
2    label      150000 non-null  int64
dtypes: int64(2), object(1)
memory usage: 3.4+ MB
```

2) 결측치 제거하기 위해 document 칼럼이 non-null 인 샘플만 데이터프레임(df_train)에 다시 저장한다.

```
df_train = df_train[df_train['document'].notnull()]
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 149995 entries, 0 to 149999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    id         149995 non-null  int64
1    document   149995 non-null  object
2    label      149995 non-null  int64
dtypes: int64(2), object(1)
memory usage: 4.6+ MB
```

■ 감성 분류 클래스의 구성을 확인한다. 부정 감성(0) 샘플이 75,170, 긍정 감성(1) 샘플이 74,825개로 비슷하게 구성되어 있다.

```
df_train['label'].value_counts()
```

```
label
0    75170
1    74825
Name: count, dtype: int64
```

3) 정확한 분석을 하기 위해 한글 외의 문자는 제거한다. re.sub(r'패턴', '바꿀 문자열', '대상 문자열')는 Python의 re 모듈에서 제공하는 함수로, 문자열에서 특정 패턴을 찾아 다른 문자열로 대체하는 역할을 합니다. r'패턴' 에서 r 접두사는 raw string을 의미한다.

```
import re
pattern = r'[^가-힣]+'

# '가'부터 '힣'까지의 문자를 제외한 나머지는 공백으로 치환한다.
df_train['document'] = df_train['document'].apply(lambda x: re.sub(pattern, ' ', x))
df_train.head()
```

	id	document	label
0	9976970	아 더빙 진짜 짜증나네요 목소리	0
1	3819312	흠 포스터보고 초딩영화줄 오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 솔직히 재미는 없다 평점 조정	0
4	6483659	사이몬페그의 익살스런 연기가 돋보였던 영화 스파이더맨에서 늑어보이기만 했던 커스틴 ...	1

4) 다운로드한 파일 중에서 평가 데이터(ratings_test.txt) 파일을 읽어 데이터프레임(df_test)에 저장한다.

```
df_test = pd.read_csv('data/ratings_test.txt', encoding='utf-8', sep='\t')
df_test.head()
```

	id	document	label
0	6270596	굳 ㅋㅋ	1
1	9274899	GDNTOPCLASSINTHECLUB	0
2	8544678	뭐야 이 평점들은.... 나쁜진 않지만 10점 짜리는 더더욱 아니잖아	0
3	6825595	지루하지는 않은데 완전 막장임... 돈주고 보기에....	0
4	6723715	3D만 아니어도 별 다섯 개 줬을텐데.. 왜 3D로 나와서 제 심기를 불편하게 하죠??	0

```
df_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    id          50000 non-null  int64
1   document    49997 non-null  object
2    label      50000 non-null  int64
dtypes: int64(2), object(1)
memory usage: 1.1+ MB
```

```
df_test.isnull().sum()
```

```
id          0
document     3
label        0
dtype: int64
```

5) document 칼럼이 Null 샘플을 제거하고 한글의 문제를 제거한다.

```
df_test = df_test[df_test['document'].notnull()]
```

```
df_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 49997 entries, 0 to 49999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    id          49997 non-null  int64
1   document    49997 non-null  object
2    label      49997 non-null  int64
dtypes: int64(2), object(1)
memory usage: 1.5+ MB
```

```
df_test.isnull().sum()
```

```
id          0
document     0
label        0
dtype: int64
```

```
df_test['label'].value_counts()
```

```
label
1    25171
0    24826
Name: count, dtype: int64
```

```
df_test['document'] = df_test['document'].apply(lambda x: re.sub(r'^가-항]+', ' ', x))
df_test.head()
```

	id	document	label
0	6270596	굳 껌	1
1	9274899		0
2	8544678	뭐야 이 평점들은 나쁘진 않지만 점 따리는 더더욱 아니잖아	0
3	6825595	지루하지는 않은데 완전 막장임 돈주고 보기에는	0
4	6723715	만 아니었어도 별 다섯 개 줬을텐데 왜 로 나와서 제 심기를 불편하게 하죠	0

• 분석 모델 구축

감성 분류는 긍정과 부정으로 분류하는 작업으로 이진 분류 분석이다. 머신러닝의 로지스틱 회귀분석을 이용해 감성 분류 모델을 생성한다.

1) 특정 벡터화 작업하기: 분류 모델을 구축하기 전에 사용할 텍스트 데이터를 숫자 형태의 벡터 데이터로 변환 작업을 해야 한다. 형태소 단위로 토큰화한 한글 단어에 대해 TF-IDF 방식을 사용하여 벡터화 작업을 한다. 이때 한글 형태소 분석 작업은 konlpy 패키지의 Okt 클래스를 사용한다. Okt 클래스는 트위터에서 만든 한글 형태소 분석기로 0.5.0 버전부터 클래스 이름이 Twitter에서 Okt로 변경되었다.

■ 형태소 분석에 사용할 konlpy 패키지의 Okt 클래스를 import하고 okt 객체를 생성한다.

```
from konlpy.tag import Okt
okt = Okt()
```

■ okt.morphs() 함수를 사용하여 형태소 단위로 토큰화 작업을 수행하는 함수(okt_tokenizer)를 정의한다.

```
def okt_tokenizer(text):
    tokens = okt.morphs(text)
    return tokens
```

■ 사이킷런의 TfidfVectorizer를 이용해 TF-IDF 벡터화에 사용할 tfidf 객체를 생성한다. 실행 시간은 컴퓨터마다 차이가 있겠지만 대략 6~7분 정도 소요된다. 토큰 생성기는 위에서 정의한 okt_tokenizer() 함수로 설정하고 토큰의 단어 크기(ngram_range)는 1~2개 단어로 한다. 토큰의 출현 빈도가 최소(min_df) 3번 이상이고 최대(max_df) 90% 이하인 것만 사용한다. 벡터화 할 데이터(df_train)에 대해 벡터 모델(tfidf)로 학습(fit())하고 벡터로 변환을 수행(transform())한다.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(tokenizer=okt_tokenizer,
                        ngram_range=(1, 2), min_df=3, max_df=0.9, token_pattern=None)
tfidf.fit(df_train['document'])
df_train_tfidf = tfidf.transform(df_train['document'])
```

```
df_train_tfidf.shape
```

```
(149995, 115715)
```

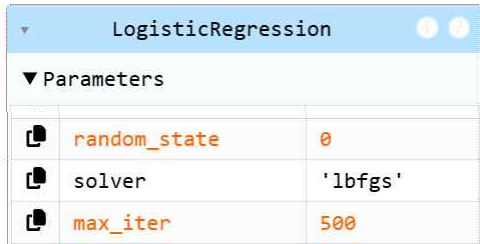
2) 감성 분류 모델 구축하기: 머신 러닝의 로지스틱 회귀 모델을 이용해 긍정과 부정의 감성이진 분류 모델을 생성한다. 로지스틱 회귀 모델의 독립변수 X는 TF-IDF 방식으로 벡터화한 df_train_tfidf가 되고 종속 변수 Y는 감성 클래스 분류값을 가지고 있는 label 컬럼이 된다.

- 사이킷런의 LogisticRegression 클래스에 대해 객체 model을 생성한다.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(random_state=0, max_iter=500)
```

- df_train_tfidf를 독립 변수 X로 하고 label 컬럼을 종속 변수 Y로 하여 로지스틱 회귀 모델(model)을 학습한다.

```
model.fit(df_train_tfidf, df_train['label'])
```



LogisticRegression		
Parameters		
random_state	0	
solver	'lbfgs'	
max_iter	500	

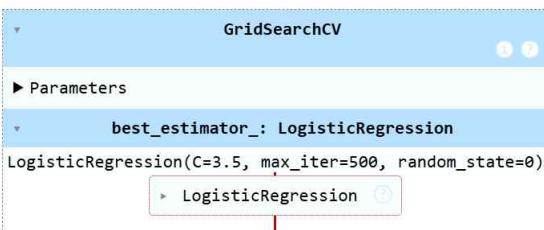
2) 로지스틱 회귀의 하이퍼 매개변수 C의 최적값을 구하기 위해 C값을 다르게 한 여러 모델을 만들고 실행하여 각 성능을 비교해야 한다. 이러한 작업을 수행하는 것이 사이킷런의 GridSearchCV 클래스다. GridSearchCV는 매개변수의 가능한 모든 조합에 대해 내부 모델을 생성하고 교차검증(cross-validation)을 수행하여 최적의 매개변수 조합을 찾는다.

- 하이퍼 매개변수 C에 대해 비교 검사를 할 6개 값([1, 3, 3.5, 4, 4.5, 5])을 params로 하고 교차 검증(cv)을 3, 모델 비교 기준은 정확도로 설정(scoring='accuracy')하여 GridSearchCV 객체 모델(grid_cv)을 생성한다.

```
from sklearn.model_selection import GridSearchCV
params = {'C': [1, 3, 3.5, 4, 4.5, 5]}
grid_cv = GridSearchCV(model, param_grid=params, scoring='accuracy', cv=3, verbose=1)
```

- GridSearchCV 객체에 df_train_tfidf와 label 컬럼으로 grid_cv 모델을 학습한다.

```
grid_cv.fit(df_train_tfidf, df_train['label'])
```



GridSearchCV	
Parameters	
best_estimator_: LogisticRegression	
LogisticRegression(C=3.5, max_iter=500, random_state=0)	
LogisticRegression	

- GridSearchCV로 찾은 최적의 C 매개변수(best_params)와 최고 점수(best_score)를 출력하여 확인한다.

```
print(grid_cv.best_params_, round(grid_cv.best_score_, 4))
```

```
{'C': 3.5} 0.8554
```

- 최적 매개변수가 설정된 모델(best_estimator)을 model_best 객체에 저장한다.

```
model_best = grid_cv.best_estimator_
```

- 분석 모델 평가

평가 데이터를 이용해 감성 분류 모델의 정확도를 확인하고 새로운 텍스트를 입력하여 감성 예측값을 확인해 본다.

1) 테스트 데이터를 벡터화한 후 감성 예측 값을 확인하고 모델 정확도를 계산하여 출력한다.

- 테스트 데이터프레임의 'document'컬럼을 위에서 생성한 tfidf 객체를 적용하여 벡터 변환을 수행(transform())한다.

```
df_test_tfidf = tfidf.transform(df_test['document'])
```

- 감성 분류 베스트 모델(model_best)에 df_test_tfidf 벡터를 사용하여 감성을 예측(predict())한다.

```
test_predict = model_best.predict(df_test_tfidf)
```

- 평가 데이터의 감성 결과 값(df_test['label'])과 감성 예측 값(test_predict)을 기반으로 정확도를 계산(accuracy_score())한다.

```
from sklearn.metrics import accuracy_score
print('감성 분석 정확도: ', round(accuracy_score(df_test['label'], test_predict), 3))
```

```
감성 분석 정확도: 0.858
```

2) 감성 분류 모델에 새로운 텍스트를 직접 입력하여 감성 예측한 후 결과를 출력한다.

- 입력받은 텍스트에 대해 위에서 수행한 것과 같은 전 처리 작업을 수행한다.

```
sentence = '웃자 ^o^ 오늘은 좋은 날이 될 것 같은 예감 100%! ^^*'
```

```
st = re.compile(r'[^가-힣]+' ).findall(sentence)
print(st)
st = [' '.join(st)]
print(st)
```

```
['웃자', '오늘은', '좋은', '날이', '될', '것', '같은', '예감']
['웃자 오늘은 좋은 날이 될 것 같은 예감']
```

- tfidf 객체로 벡터 변환(transform())한 후에 모델에 적용하여 감성 예측(predict())을 한다.

```
st_tfidf = tfidf.transform(st)
st_predict = model_best.predict(st_tfidf)
```

```
if(st_predict==0):
    print(sentence, '->>부정 감성')
else:
    print(sentence, '->>긍정감성')
```

```
웃자 ^o^ 오늘은 좋은 날이 될 것 같은 예감 100%! ^^* ->>긍정감성
```

실행 결과인 감성 예측이 예상과 다를 수도 있다. 우리가 만든 감성 분석 모델은 정확도가 85.8%이지만 아주 기본적인 감성 분석 모델이다. 최적의 매개변수 찾기, 머신러닝 모델의 결합, 딥러닝 모델 적용과 같이 감성 분석 모델의 정확도를 높이기 위한 연구는 자연어 처리 분야에서 중요한 영역이다. 그리고 모델 학습에 사용한 데이터도 완전하지 않다. '영화 리뷰' 데이터는 특정 분야의 데이터이기 때문에 다른 분야 또는 일반적인 단어에 대한 감성 예측에서도 같은 정확도를 발휘할 것으로 보기 어렵다. 분석 성능을 높이기 위해 품질 높은 학습 데이터를 만드는 것도 빅데이터 분석 분야에서 중요한 연구 영역이다.

• 감성분석과 바 차트

이전에 생성한 감성 분류 모델을 이용하여 네이버 뉴스에서 크롤링한 '인공지능' 뉴스 텍스트에 대해 감성을 분석 후 바 차트로 시각화한다.

■ 데이터 수집

감성 분석에 사용할 데이터를 네이버 검색 API로 수집한다. 수집을 수행한 시점으로 최근 1,000개의 뉴스가 수집되어 json 파일에 저장된다.

[data] naver_api.py

```
import requests
import datetime
import json

def getNaver(url, query, start, display):
    headers = {
        'X-Naver-Client-Id': 'xxxxxxxxxxxxxxxxxxxx',
        'X-Naver-Client-Secret': 'xxxxxxxxxxxxxxxx'
    }
    params = {
        'query': query,
        'display': display,
        'start': start
    }

    try:
        response = requests.get(url, headers=headers, params=params)
        if response.status_code == 200:
            data_json = response.json()
            return data_json['items']
    except Exception as e:
        print(e)
        print(f'[{datetime.datetime.now()}] Error for URL:{url}')
        return None

if __name__ == '__main__':
    query = '챗GPT'
    url = 'https://openapi.naver.com/v1/search/news.json'
    display = 100
    start = 1

    results = []
    while (True):
        result = getNaver(url, query, start, display)

        start = start + display
        if (result == None):
            break
        else:
            results.extend(result)

    print(f'전체 데이터 수:{len(results)}')
    with open(f'data/{query}.json', 'w', encoding='utf-8') as json_file:
        json.dump(results, json_file, ensure_ascii=False, indent=4)
```

[data] '챗GPT'.json

```
[
  {
    "title": "정부 AI 인프라 정책에 외면받는 '국산 서버'...&quot;NPU와 함께 풀스택 고려...",
    "originallink": "https://zdnet.co.kr/view/?no=20250721104731",
    "link": "https://n.news.naver.com/mnews/article/092/0002382847?sid=105",
    "description": "(사진=<b>챗GPT</b> 제작) 21일 업계에 따르면 과학기술정보통신부는 두 차례 유찰됐던 국가AI컴퓨팅센터 구축 사업의... ",
    "pubDate": "Mon, 21 Jul 2025 11:19:00 +0900"
  },
  ...
]
```


• 데이터 준비 및 탐색

1) json 파일을 읽고 분석할 칼럼을 추출하여 데이터프레임(df_naver)에 저장한다.

```
df_naver = pd.read_json('data/성형.json')
df_naver = df_naver[['title', 'description']]
df_naver.head(2)
```

	title	description
0	이화여대, 대학혁신지원사업 교육혁신 평가 2년 연속 최우수	구축 △인공지능(AI) 기반 지능형 학생지원시스템(E-벗) 운영 △미래...
1	한수원, '자동예측진단기술' 상용화 기술이전	설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 ...

2) 구성된 데이터프레임은 분석에 사용할 텍스트이므로 전 처리 작업으로 한글 이외의 문자를 제거한다.

```
pattern = r'[^가-힣]'
df_naver['title'] = df_naver['title'].apply(lambda x:re.sub(pattern, ' ', x))
df_naver['description'] = df_naver['description'].apply(lambda x:re.sub(pattern, ' ', x))
df_naver.head()
```

	title	description
0	이화여대 대학혁신지원사업 교육혁신 평가 년 연속 최우수	구축 인공지능 기반 지능형 학생지원시스템 벗 운영 미래...
1	한수원 자동예측진단기술 상용화 기술이전	설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 ...

• 감성 분석

감성 분석을 할 수 있는 텍스트는 뉴스 제목과 뉴스 내용이다. title 칼럼의 텍스트와 description 칼럼의 텍스트에 대해 각각 감성 분석을 수행하고 결과 값을 데이터프레임에 저장한다. 감성 분석은 구축해 놓은 감성 분류 모델에 적용해야 하므로 만들어놓은 tfidf 객체를 통해 특성 벡터화를 수행한다. 그리고 감성 분류 모델을 통해 감성을 예측하고 결과 값을 데이터 프레임에 저장한다.

1) title 칼럼에 대한 감성 분석을 수행한다.

```
df_title_tfidf = tfidf.transform(df_naver['title'])
df_title_predict = model_best.predict(df_title_tfidf)
df_naver['title_label'] = df_title_predict
df_naver.head(2)
```

	title	description	title_label
0	이화여대 대학혁신지원사업 교육혁신 평가 년 연속 최우수	구축 인공지능 기반 지능형 학생지원시스템 벗 운영 미래형 융복합 인재 양성을 위한 ...	1
1	한수원 자동예측진단기술 상용화 기술이전	설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 인공지...	0

2) description 칼럼에 대해서도 같은 작업을 하여 감성 분석을 수행한다.

```
df_description_tfidf = tfidf.transform(df_naver['description'])
df_description_predict = model_best.predict(df_description_tfidf)
df_naver['description_label'] = df_description_predict
df_naver.head(2)
```

	title	description	title_label	description_label
0	이화여대 대학혁신지원사업 교육혁신 평가 년 연속 최우수	구축 인공지능 기반 지능형 학생지원시스템 벗 운영 미래형 융복합 인재 양성을 위한 ...	1	0
1	한수원 자동예측진단기술 상용화 기술이전	설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 인공지...	0	1

• 결과 확인 및 분리

감성 분석 결과 부정감성 및 긍정감성의 개수를 비교해 보면 title 분석결과와 description 분석 결과에 차이가 있다. 단어를 기준으로 분석하기 때문에 단어의 개수가 부족하면 정확도가 떨어진다. 우리가 구축한 감성분류 모델의 정확도가 85.8%였으니 틀린 결과도 있을 것이다. 그리고 분류 모델의 학습 데이터로 사용했던 영화 리뷰의 구성 단어와 분석 데이터인 뉴스를 구성하는 단어의 차이로 인한 오차도 있을 것이다. 빅데이터 분석의 정확도를 높이려면 훈련에 사용할 데이터의 품질, 전처리 작업, 훈련 데이터와 실제 데이터의 특성 차이, 분석 모델의 종류, 모델에 적용한 매개변수 값, 텍스트언어(한글, 영어 등)의 특성 등 고려할 점이 많다.

1) 다음과 같은 과정을 통해 감성 분석 결과를 확인해 본다.

```
df_naver['title_label'].value_counts()
```

```
title_label
0    629
1    371
Name: count, dtype: int64
```

```
df_naver['description_label'].value_counts()
```

```
description_label
0    757
1    243
Name: count, dtype: int64
```

2) 결과 분석과 시각화를 위해 뉴스 본문(description)에 대한 감성분석을 기준으로 긍정감성 데이터와 부정감성 데이터를 분리한다.

```
filt = df_naver['description_label']==0
df_naver_nag = df_naver[filt]
df_naver_nag.head()
```

	title	description	title_label	description_label
0	이화여대 대학혁신지원사업 교육혁신 평가 년 연속 최우수	구축 인공지능 기반 지능형 학생지원시스템 벗 운영 미래형 융복합 인재 양성을 위한 ...	1	0
2	네이버 분기 매출 조 역 원 커머스가 실적 견인	네이버가 올해 분기에 인공지능 기반 신규 서비스와 커머스 부문 호조에 힘입어 견조한...	0	0
3	네이버 분기 연속 최대실적 소버린 매출 견인	하반기 인공지능 서비스 접목 영역을 지속 확대하는 가운데 글로벌 시장 공략도 이어간...	0	0
6	하이닉스 점유율 추가 확대 난관 이젠 수익성 관건	그는 경쟁사 진입 시 더 낮은 가격을 제시해 출혈 경쟁을 하기 보다는 가격과 물량의...	0	0
7	여한구 비관세장벽 헬프데스크 통해 애로사항 적극 정취	한국의 인공지능 데이터 센터 등 미국계 외투기업의 적극적인 투자확대 를 당부했다 제...	0	0

```
filt = df_naver['description_label']==1
df_naver_pos = df_naver[filt]
df_naver_pos.head(3)
```

	title	description	title_label	description_label
1	한수원 자동예측진단기술 상용화 기술이전	설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 인공지...	0	1
4	김홍일장군기념사업회 제 주기 김홍일 장군 추모식	이번 추모식에서는 인공지능 으로 구현한 김홍일 장군이 추모객들에게 감사의 인사를 전...	0	1
5	아주대 연구팀 난수 발생 장치 개발 위한 원천기술 구현	이에 앞으로 암호 보안과 인공지능 연산 기술 등의 발전에 기여할 수 있을 것으로 기...	0	1
11	지자체 브리프 서울시의회 마포구 영등포구	이번 세미나는 한국지방행정연구원 등이 주최한 행사로 인공지능 을 활용한 의정 지원 ...	0	1
14	시흥시 학교복합시설 공모사업서 시흥과학고 학교복합시설 최종 선정	시흥과학고는 바이오 생명과학 및 인공지능 특화 교육과정을 중심으로 운영되며 복합시설...	0	1

```
len(df_naver_nag), len(df_naver_pos)
```

```
(757, 243)
```

• 결과 시각화

감성분석 결과에서 긍정감성 뉴스에 나타난 단어와 부정감성 뉴스에 나타난 단어를 바 차트로 시각화하여 비교해 본다. 바 차트를 작성하기 위해서는 단어별 출현 빈도를 정리한다. 명사 단어를 추출하고 TF-IDF 값을 DTM을 구성한 후에 단어별 합을 계산 하면 바 차트에 사용할 단어별 출현 빈도 값이 된다.

TF-IDF (Term Frequency-Inverse Document Frequency:단어 빈도-역 문서 빈도)는 DTM(문서 단어 행렬) 내의 각 단어에 중요도에 따른 가중치를 부여하는 방법이다. DTM은 문서 집합에서 각 단어의 등장 빈도를 나타내는 행렬인데, TF-IDF는 이 DTM을 기반으로 단어의 중요도를 계산하여, 특정 문서에서 특정 단어가 얼마나 중요한지를 수치화한다. TF-IDF는 문서 유사도 계산, 검색 시스템, 키워드 추출 등 다양한 자연어 처리 작업에 활용된다.

1) 긍정 감성 뉴스에서 형태소 분석을 하여 명사를 추출한다. 출력해 보면 명사가 아닌 것도 있다. 우리가 사용한 Okt 형태소 분석기는 많이 사용되는 분석기 중 하나지만 완벽하지는 않다. 정확하게 한글 형태소를 분류해 내는 형태소 분석기는 아직은 개발되지 않았다.

- 형태소 토큰화를 하여 명사 토큰(okt.nouns())만 추출한 후 리스트를 구성한다.

```
pos_description = df_naver_pos['description']
```

```
pos_description_noun_tk = []
```

```
for d in pos_description:
```

```
    pos_description_noun_tk.append(okt.nouns(d))
```

```
pos_description_noun_tk
```

```
[['설비', '자동', '예측', '진단', '기술', '원격', '발전소', '중요', '설비', '데이터', '수집', '표준화', '인공', '지능', '알고리즘', '통해', '설비', '고장', '예측', '진단', '약', '년', '동안', '자체', '기술', '개발', '시스템', '인증', '과정', '현재', '국내'], ['이번', '추모식', '인공', '지능', '구현', '김홍일', '장군', '추모', '객', '감사', '인사', '전', '식', '중', '김홍일', '장군', '육군사관학교', '제작', '시절', '생도', '입학', '전쟁', '참전', '유공'], ['이', '앞', '암호', '보안', '인공', '지능', '연산', '기술', '등', '발전', '기여', '수', '것', '기대', '아주대', '학교', ...]]
```

- 토큰의 길이가 1인 것은 제외한 후 연결(join())하여 리스트를 구성한다.

```
pos_description_noun_join=[]
```

```
for d in pos_description_noun_tk:
```

```
    d2 = [w for w in d if len(w)>1] #길이가 1보다 큰 토큰만 추출
```

```
    pos_description_noun_join.append(' '.join(d2)) #토큰을 연결하여 리스트에 추가
```

```
pos_description_noun_join
```

```
['설비 자동 예측 진단 기술 원격 발전소 중요 설비 데이터 수집 표준화 인공 지능 알고리즘 통해 설비 고장 예측 진단 약 년 동안 자체 기술 개발 시스템 인 증 과정 현재 국내', '이번 추모식 인공 지능 구현 김홍일 장군 추모 객 감사 인사 전 식 중 김홍일 장군 육군사관학교 제작 시절 생도 입학 전쟁 참전 유공', ...]
```

2) 부정감성 뉴스에도 같은 작업을 수행한다.

```
neg_description = df_naver_nag['description']
```

```
neg_description_noun_tk = []
```

```
for d in neg_description:
```

```
    neg_description_noun_tk.append(okt.nouns(d))
```

```
neg_description_noun_tk
```

```
[['구축', '인공', '지능', '기반', '지능', '학생지원시스템', '벗', '운영', '미래', '용', '복합', '인재', '양성', '위', '교양', '교육', '및', '기초', '소양', '인증', '제', '추진', '교육', '혁신', '교과목', '개발', '및', '교원', '참여', '활성화', '학내', '구성원', '참여', '중심', '교육', '혁신'], ['네이버', '올해', '분기', '인공', '지능', '기반', '신규', '서비스', '커머스', '부문', ' ...]]
```

```
neg_description_noun_join = []
```

```
for d in neg_description_noun_tk:
    d2 = [w for w in d if len(w) > 1]
    neg_description_noun_join.append(' '.join(d2))
```

```
neg_description_noun_join
```

```
['구축 인공 지능 기반 지능 학생지원시스템 운영 미래 복합 인재 양성 교양 교육 기초 소양 인증 추진 교육 혁신 교과목 개발 교원 참여 활성화 학내 구성원 참여 중심 교육 혁신', '네이버 올해 분기 인공 지능 기반 신규 서비스 커머스 부문 호조 견조 실적 플랫폼 경쟁력 강화 기술 고도화 추진 결과 대부분 사업 부문 성장', ... ]]
```

3) 단어별 출현 빈도를 계산하기 위해 단어별 TF-IDF 값으로 DTM을 구성해야 한다. 먼저 긍정 뉴스에 대한 DTM을 구성해 본다. 문서에 나타난 단어의 TF-IDF를 구하는 작업은 문서 단위로 토큰이 연결된 pos_description_noun_join을 사용해야 한다.

- TfidfVectorizer 객체를 생성하고 pos_description_noun_join에 대해 TF-IDF 값을 구하여 DTM을 구성한다.

```
pos_tfidf = TfidfVectorizer(tokenizer=okt_tokenizer,
                             min_df=2, #min_df=2 옵션은 2개 이하로 나온 단어에 대해서 무시한다.
                             token_pattern=None)
pos_dtm = pos_tfidf.fit_transform(pos_description_noun_join)
pos_dtm.getcol(0).sum()
```

```
0.39318667445278443
```

- 단어별 TF-IDF 값의 합을 구하기 위해 DTM의 단어(get_feature_names())마다 칼럼의 합(getcol(idx).sum())을 구한다.

```
pos_vocab = dict()

for idx, word in enumerate(pos_tfidf.get_feature_names_out()):
    pos_vocab[word] = pos_dtm.getcol(idx).sum()
```

```
pos_vocab
```

```
{'가능': 0.39318667445278443,
 '가능성': 1.5162785704730157,
 '가속': 0.5085479442689065,
 '가운데': 1.092871982058254,
 '가입': 0.7921261372523516,
 '가입자': 3.158101210988661,
 '가장': 0.7975363626664012,
 ...}
```

- TF-IDF 값의 합을 구하여 내림차순으로 정렬한다.

```
pos_words = sorted(pos_vocab.items(), key=lambda x:x[1], reverse=True)
pos_words
```

```
[('지능', 11.994666264947831),
 ('인공', 11.900931335669103),
 ('등', 9.613056407963658),
 ('기술', 8.137146404194603),
 ('기업', 8.037369481834865),
 ('네이버', 6.46734106764796),
 ('활용', 6.341261246010406),
 ('산업', 6.030031242412571),
 ...]
```

4) 부정 감성 뉴스인 neg_description_noun_join에 대해서도 같은 작업을 수행한다.

```
neg_tfidf = TfidfVectorizer(tokenizer=okt_tokenizer, min_df=2, token_pattern=None)
neg_dtm = neg_tfidf.fit_transform(neg_description_noun_join)
neg_dtm.getcol(0).sum()
```

```
1.0544145459318743
```

```
neg_vocab = dict()
```

```
for idx, word in enumerate(neg_tfidf.get_feature_names_out()):
    neg_vocab[word] = neg_dtm.getcol(idx).sum()
```

```
neg_vocab
```

```
{'가격': 1.0544145459318743,
 '가능성': 0.9807043855190429,
 '가동': 0.5421095737414525,
 '가상현실': 0.4774388841568328,
 '가속': 0.5483871872636377,
 '가시': 0.5801213055185701,
 '가운데': 3.490056433991576,
 '가임': 1.4952308008316053,
 '가임자': 4.804610756435832,
 '가장': 2.8401141458689834,
 '가전': 2.115145036796477,
 '가족': 2.104655644182511,
 '가지': 1.2164084065494203,
 ...}
```

```
neg_words = sorted(neg_vocab.items(), key=lambda x:x[1], reverse=True)
```

```
neg_words
```

```
[('지능', 36.574425388336856),
 ('인공', 35.431677918620196),
 ('기술', 28.19950699651143),
 ('에너지', 20.692996185238393),
 ('기반', 18.65664393989002),
 ('활용', 18.143090722940507),
 ('전력', 16.06837140926117),
 ('모텔', 15.43565668284175),
 ('사업', 15.036895321168249),
 ('이번', 14.76962019052442),
 ('산업', 14.5099734176221),
 ('교육', 13.24340225570077),
 ('시스템', 13.1456516590845),
 ('분야', 13.055499913902741),
 ...]
```

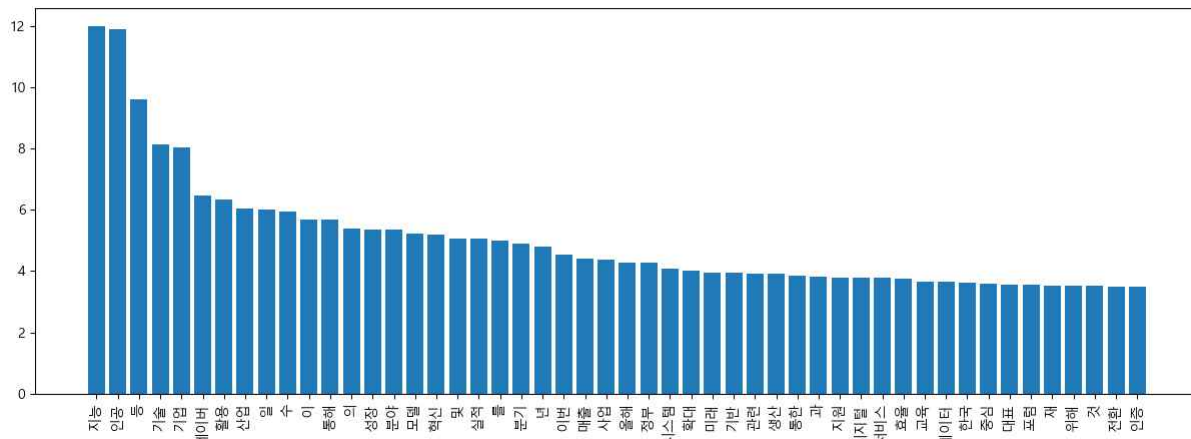
5) DTM 기반 단어 사전의 상위 단어로 바 차트 그리기를 수행한다.

- 바 차트를 그리기 위해 matplotlib 패키지를 import하고 한글을 표시하기 위해 한글 폰트를 설정한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 10
matplotlib.rcParams['axes.unicode_minus'] = False
```

- ```
max=50 #바 차트에 나타낼 단어의 수
x = [i[0] for i in pos_words[:max]]
y = [i[1] for i in pos_words[:max]]

plt.figure(figsize=(15, 5))
plt.bar(x, y)
plt.xticks(x, rotation=90)
plt.title(f'긍정 뉴스의 단어 {max}개\n', size=20)
plt.show()
```



### 부정 뉴스의 단어 50개

| 단어   | 빈도 |
|------|----|
| 지능   | 36 |
| 인공   | 35 |
| 기술   | 28 |
| 에너지  | 21 |
| 기반   | 18 |
| 활동   | 18 |
| 전력   | 16 |
| 모형   | 15 |
| 시장   | 15 |
| 이번   | 14 |
| 산업   | 14 |
| 교육   | 13 |
| 시스템  | 13 |
| 필요   | 13 |
| 데이터  | 13 |
| 오류   | 12 |
| 서비스  | 12 |
| 스마트  | 11 |
| 한국   | 11 |
| 문화   | 11 |
| 개발   | 11 |
| 협업   | 11 |
| 파이낸스 | 11 |
| 기업   | 11 |
| 분기   | 10 |
| 전기차  | 10 |
| 통해   | 10 |
| 대해   | 10 |
| 경험   | 10 |
| 배터리  | 10 |
| 데이터  | 10 |
| 절감   | 10 |
| 전원   | 10 |
| 반도체  | 10 |
| 인터넷  | 10 |
| 미래   | 10 |
| 공개   | 10 |
| 블록   | 10 |
| 주권   | 10 |
| 휴먼   | 10 |
| 노이드  | 10 |
| 시장   | 10 |
| 매출   | 10 |
| 공정   | 10 |
| 확대   | 10 |
| 장관   | 10 |
| 시간   | 10 |
| 투자   | 10 |
| 평가   | 10 |
| 생신   | 10 |
| 형신   | 10 |

- 감성 분석 후 긍정 뉴스와 부정 뉴스에 많이 나오는 단어를 순서별로 출력하는 함수를 작성한다.

```
def naver_pos_neg(file_name):
 #1)파일을 읽어 분석할 칼럼을 추출한다.
 df_naver = pd.read_json(f'data/{file_name}.json')
 df_naver = df_naver[['title', 'description']]

 #2)전처리 작업으로 한글 이외의 문자를 제거한다.
 df_naver['title'] = df_naver['title'].apply(lambda x: re.sub(r'[^\가-힣]+' , ' ', x))
 df_naver['description'] = df_naver['description'].apply(lambda x: re.sub(r'[^\가-힣]+' , ' ', x))

 #3)감성 분석 수행 후 결과값을 저장한다.
 df_title_tfidf = tfidf.transform(df_naver['title'])
 df_title_predict = model_best.predict(df_title_tfidf)
 df_naver['title_label'] = df_title_predict

 df_description_tfidf = tfidf.transform(df_naver['description'])
 df_description_predict = model_best.predict(df_description_tfidf)
 df_naver['description_label'] = df_description_predict

 #4)감성 분석 결과를 분리 저장하기
 filt = df_naver['description_label']==0
 df_naver_nag = df_naver[filt]

 filt = df_naver['description_label']==1
 df_naver_pos = df_naver[filt]

 #5)영태소 토큰화를 하여 명사 토큰만 추출한 후 리스트에 추가한다.
 pos_description = df_naver_pos['description']
 pos_description_noun_tk = []

 for d in pos_description:
 pos_description_noun_tk.append(oka.nouns(d))

 #6)토큰의 길이가 1인 것은 제외한 후 연결(join())하여 리스트를 구성한다.
 pos_description_noun_join=[]
 for d in pos_description_noun_tk:
 d2 = [w for w in d if len(w)>1]
 pos_description_noun_join.append(' '.join(d2))

 #7)부정 감성 뉴스에도 5) ~ 6) 작업을 반복한다.
 neg_description = df_naver_nag['description']
 neg_description_noun_tk = []
 for d in neg_description:
 neg_description_noun_tk.append(oka.nouns(d))

 neg_description_noun_join = []
 for d in neg_description_noun_tk:
 d2 = [w for w in d if len(w) > 1]
 neg_description_noun_join.append(' '.join(d2))

 #8)단어별 출현 빈도를 계산하기 위해 TF-IDF 값으로 DTM을 구성한다.
 pos_tfidf = TfidfVectorizer(tokenizer=oka_tokenizer, min_df=2, token_pattern=None)
 pos_dtm = pos_tfidf.fit_transform(pos_description_noun_join)

 #9)DTM의 단어마다 칼럼의 합(TF-IDF 값의 합)을 구한다.
 pos_vocab = dict()
 for idx, word in enumerate(pos_tfidf.get_feature_names_out()):
 pos_vocab[word] = pos_dtm.getcol(idx).sum()

 #10)TF-IDF 값의 합을 구하여 내림차순으로 정렬한다.
 pos_words = sorted(pos_vocab.items(), key=lambda x:x[1], reverse=True)

 #11)부정 감성에 대해 8) ~ 10) 작업을 반복한다.
 neg_tfidf = TfidfVectorizer(tokenizer=oka_tokenizer, min_df=2, token_pattern=None)
 neg_dtm = neg_tfidf.fit_transform(neg_description_noun_join)

 neg_vocab = dict()
 for idx, word in enumerate(neg_tfidf.get_feature_names_out()):
 neg_vocab[word] = neg_dtm.getcol(idx).sum()
 neg_words = sorted(neg_vocab.items(), key=lambda x:x[1], reverse=True)

 return pos_words, neg_words
```

## • 토픽 분석 LDA모델링

토픽 모델링 (Topic Modeling)은 문서 집합에서 주제를 찾아내는 기술이다. '특정 단어가 자주 등장하는 것이 그 문서의 주제일 가능성이 높다'라는 가정에서 출발한다. 예를 들어 '스타벅스', '카페인', '커피 향기', '모닝커피' 등의 단어가 다른 문서에 비해 자주 등장한다는 것은 해당 문서의 주제가 '커피'일 것으로 예측할 수 있다.

LDA는 대표적인 머신러닝 기반 토픽 모델링 기법이다. 디리클레 분포를 이용해 문서에 잠재된 토픽을 추론하는 확률 모델 알고리즘을 사용한다. 문서의 토픽 분포에 따라서 단어의 분포가 결정된다고 가정하고 하나의 문서에 잠재된 토픽별 단어와 관련된 토픽별 비율을 도출한다.

'15.토픽분석모델링.ipynb' 파일을 생성하고 데이터수집 날짜를 기준으로 최근 1,000개의 네이버 뉴스에서 '인공지능'와 관련된 어떤 토픽이 있는지 분석해 본다. 토픽 분석은 머신러닝 기반의 LDA 토픽 모델을 사용한다.

## • 데이터 준비

이전에 수집한 '인공지능' 네이버 뉴스의 'description'을 토픽 분석용 데이터로 사용한다. 토픽 모델은 단어별 확률 분포를 분석하므로 명사를 추출한 단어(토큰) 상태의 리스트를 준비한다.

1) 네이버 뉴스를 읽어 'description' 칼럼 값만 저장한 후 정확한 분석을 하기 위해 한글 이외의 문자는 제거한다.

```
import pandas as pd
df_naver = pd.read_json('data/인공지능.json')
description = df_naver['description']
description.head()

0 구축 △인공지능(AI) 기반 지능형 학생지원시스템(E-벗) 운영 △미래...
1 설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 ...
2 네이버가 올해 2분기에 인공지능 기반 신규 서비스와 커머스 부문 호조에...
3 하반기 인공지능(AI) 서비스 접목 영역을 지속 확대하는 가운데 글로벌...
4 이번 추모식에서는 인공지능(AI)으로 구현한 김홍일 장군이 추모객들에게...
Name: description, dtype: object
```

```
import re
description = description.apply(lambda x:re.sub(r'[^가-힣st]+', ' ', x))
description

0 구축 인공지능 기반 지능형 학생지원시스템 벗 운영 미래형 융복합 인재 양성을 위한 ...
1 설비 자동예측진단기술은 원격지 발전소 중요 설비의 데이터를 수집해 표준화하고 인공지...
2 네이버가 올해 분기에 인공지능 기반 신규 서비스와 커머스 부문 호조에 힘입어 견조한...
3 하반기 인공지능 서비스 접목 영역을 지속 확대하는 가운데 글로벌 시장 공략도 이어간...
4 이번 추모식에서는 인공지능 으로 구현한 김홍일 장군이 추모객들에게 감사의 인사를 전...
Name: description, dtype: object
```

2) 형태소 분석에 사용할 konlpy 패키지의 Okt 클래스를 import하고 okt 객체를 생성한다.

```
from konlpy.tag import Okt
okt = Okt()
```

3) 형태소 분석 객체(okt)를 사용해 명사 형태소만 추출한다.

```
description_noun_tk = []
for d in description:
 description_noun_tk.append(okt.nouns(d))
print(description_noun_tk)
```

```
[['구축', '인공', '지능', '기반', '지능', '학생지원시스템', '벗', '운영', '미래', '용', '복합', '인재', '양성', '위', '교양', '교육', '및', '기초', '소양', '인중', '제', '추진', '교육', '역진', '교과목', '개발', '및', '교원', '참여', '활성화', '학내', '구성원', '중심', '교육', '역진'], ['설비', '자동', '예측', '진단', '기술', '원격', '발전소', '중요', '설비', '데이터', '...]]
```



4) 토큰 길이가 1보다 큰 것만 추출한다.

```
description_noun_tk2 = []
for d in description_noun_tk:
 item = [i for i in d if len(i) > 1]
 description_noun_tk2.append(item)
print(description_noun_tk2)
```

```
[['구축', '인공', '지능', '기반', '지능', '학생지원시스템', '운영', '미래', '복합', '인재', '양성', '교양', '교육', '기초', '소양', '인증', '추진', '교육', '역신', '교과목', '개발', '교원', '참여', '활성화', '학내', '구성원', '참여', '중심', '교육', '역신'], ['설비', '자동', '예측', '진단', '기술', '원격', '발전소', '중요', '설비', '데이터', '수집', '표준화', '인공', '지능', ...]]
```

## • 분석 모델 구축

토픽 분석을 위해 파이썬의 `gensim` 패키지에서 제공하는 LDA 토픽 모델을 사용한다.

### • `gensim`에서 사용할 BoW를 생성

LDA 토픽 모델링은 Bag of words(BoW)을 데이터로 사용한다. BoW는 단어의 순서는 고려하지 않고 단어들의 출현 빈도(frequency)만 수치화해 표현하는 가장 기초적인 임베딩(embedding) 기술이다.

BoW를 생성하기 위해서 우선 텍스트를 다루는 `gensim` 패키지인 `corpora`를 사용해 토큰화된 단어와 `gensim` 내부 id와 매칭 시키는 단어 사전(dictionary)으로 만든다. `gensim`의 LDA에서 사용하는 BoW의 형태는 (단어번호: index, 빈도)로 이뤄진 리스트 자료이다. Dictionary 클래스에 `doc2bow` 메서드를 이용해 (index, 빈도) 형태의 리스트 BoW를 생성할 수 있다.

그리고 사전(dictionary)는 필터링할 수 있다. 빈도가 1인 단어가 포함되면 단어가 너무 많아 분석이 비효율적이고, 토픽 네이밍도 쉽지가 않다. 그리고 너무 지나치게 흔한 단어도 의미가 없을 수 있다. 그래서 빈도가 2 이상인 단어는 포함시키고 전체의 50% 이상을 차지하는 단어는 제외하도록 한다.

1) `gensim` 패키지를 설치한다.

```
pip install gensim
```

2) `gensim` 패키지와 `gensim` 패키지의 `corpora`를 import 한다.

```
import gensim
from gensim import corpora
```

3) 토큰 단어와 `gensim` 내부 아이디를 매칭 시키는 사전을 생성한다. 사전 토큰중 빈도2이상은 포함하고 전체 50%이상 단어는 제거한다.

```
dictionary = corpora.Dictionary(description_noun_tk2)
dictionary.filter_extremes(no_below=2, no_above=0.5)
print(dictionary)
```

```
Dictionary<1648 unique tokens: ['개발', '교양', '교육', '구성원', '구축']...>
```

4) Dictionary 클래스에 `doc2bow` 메서드를 이용해 카운터 기반의 벡터(단어번호, 빈도)로 이뤄진 리스트 BoW를 생성한다.

```
corpus = [dictionary.doc2bow(word) for word in description_noun_tk2]
print(corpus)
```

```
[[[0, 1), (1, 1), (2, 3), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (9, 1), (10, 1), (11, 1), (12, 1), (13, 1), (14, 2), (15, 1), (16, 2), (17, 1)], [(0, 1), (12, 1), (18, 1), (19, 1), (20, 1), (21, 2), (22, 1), (23, 1), (24, 1), (25, 3), (26, 1), (27, 1), (28, 1), (29, 2), (30, 1), (31, 1), (32, 1), (33, 1), (34, 2), (35, 1), (36, 1), (37, 1)], [(5, 1), (15, 1), (21, 1), (38, 1), (39, 1), (40, 1), (41, 1), (42, 1), (43, 1), (44, 1), (45, 2), (46, 1), (47, 1), (48, 1), (49, 1), (50, 1), (51, 1), (52, 1), (53, 1), (54, 1), (55, 1)], [(43, 1), (46, 1), (48, 1), (52, 1), (56, 1), (57, 1), (58, 1), (59, 1), (60, 1), (61, 1), (62, 1), (63, 1), (64, 3), (65, 1), (66, 1), (67, 1), (68, 1), (69, 1), (70, 1), (71, 1)], [(72, 1), (73, 1), (74, 1), (75, 1)], [(21, 1), (27, 1), (76, 1), (77, 1), (78, 1), (79, ...]]
```

## • LDA 모델링

gensim의 LDA는 `gensim.models.LdaModel`을 사용해 모델링을 한다. gensim의 LDA 모델의 경우 옵션이 매우 많다. 그중 토픽 수(`num_topics`)와 알고리즘 반복 회수(`passes`) 정도만 설정해 준다. 하지만 반복 회수는 임의로 정할 수 있지만, 토픽 수는 임의로 정하기가 쉽지 않다. 토픽 수는 이론적으로 철저히 검토한 후 토픽 수를 결정할 수도 있지만, 통계치에 의존해 결정할 수도 있다. 보통 혼잡도(`perplexity`)와 응집도(`coherence`)를 이용하지만, 응집도가 혼잡도보다는 토픽 수를 결정하는데 보다 유용하다고 알려져 있다.

`gensim.models.LdaModel`을 이용하여 모델링할 경우 옵션은 아래와 같다.

- `corpus` : 변환된 특성 벡터, 즉 `dictionary.doc2bow()` 의 실행 결과
- `id2word` : 참조할 특성 집합, 생성한 Dictionary 객체
- `passes` : 알고리즘 반복 횟수
- `num_topics` : 토픽의 수

```
model = gensim.models.ldamodel.LdaModel(corpus, id2word=dictionary, num_topics=3, passes=5)
```

## • Coherence 모델링(토픽 수 결정)

모델링을 하는 경우 우선 적으로 토픽 수를 결정해야 하는데 토픽 수를 미리 아는 것은 사실상 불가능하다. 그래서 토픽 수를 미리 결정하기보다는 토픽 수에 따라 응집도(`coherence`)를 측정하고 응집도가 최대치가 되는 토픽 수를 최종적으로 선택하는 것이 보다 수월하다. 여기서 응집도는 토픽을 구성하는 단어들의 관련성이 얼마나 높은지를 측정한다.

gensim에서는 `CoherenceModel` 클래스를 이용해 응집도를 구할 수 있다. 사용하는 옵션은 아래와 같다.

- LDA모델명 : LAD 설정 모델을 저장한 이름
- `texts` : 토큰화된 단어 집합
- `dictionary` : 참조할 특성 집합, 생성한 Dictionary 객체
- `coherence` : `u_mass`, `c_v`, `c_uci`, `c_npmi` 중 선택

1) 토픽수를 2~10으로 늘려가면서 응집도를 계산해 응집도가 가장 높은 토픽수를 선택한다. 응집도 값은 `get_coherence()`를 이용한다.

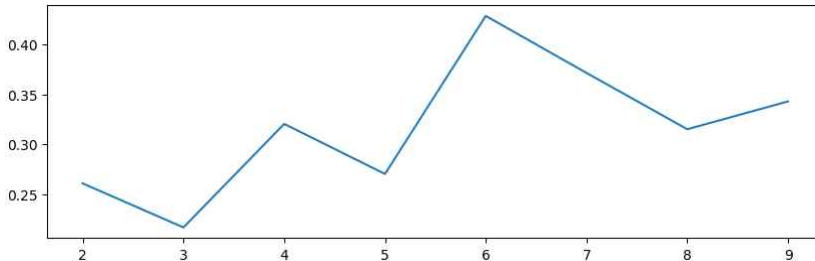
```
from gensim.models import CoherenceModel

coherence_score=[]
for i in range(2, 10):
 model=gensim.models.ldamodel.LdaModel(corpus=corpus, id2word=dictionary,
 num_topics=i, passes=5)
 coherence_model = CoherenceModel(model, texts=description_noun_tk2,
 dictionary=dictionary, coherence='c_v')
 coherence_lda = coherence_model.get_coherence()
 print('k=', i, '\nCoherence Score: ', coherence_lda)
 coherence_score.append(coherence_lda)
```

```
k= 2
Coherence Score: 0.2607584250111947
k= 3
Coherence Score: 0.2165710249351669
k= 4
Coherence Score: 0.3204318910024093
k= 5
Coherence Score: 0.2702095558792161
k= 6
Coherence Score: 0.4287217655948984
k= 7
Coherence Score: 0.3715590654248651
k= 8
Coherence Score: 0.315076005201264
k= 9
Coherence Score: 0.3430005462086633
```

2) 토픽 수별 응집도 그래프를 그려보면 더 확실히 토픽수가 6일 때 LDA 모델의 적합성이 가장 높다는 것을 확인 할 수 있다.

```
import matplotlib.pyplot as plt
x = [x for x in range(2, 10)]
y = coherence_score
plt.figure(figsize=(10, 3))
plt.plot(x, y)
plt.show()
```



### • 최종 LDA 모형 선택

응집도가 가장 높은, 즉 모델의 적합성이 가장 높은 토픽 수가 6이라는 사실을 확인하였다. 따라서 토픽 수 6을 기준으로 최종 LDA 모델을 설정한다. 그런 후 `print_topics()` 함수를 이용해 토픽별 단어를 확인할 수 있다. `print_topics()`는 기본적으로 토픽 당 10개의 단어가 표시된다. 토픽 당 5개의 단어만 보고 싶다면 `print_topics(num_words=5)` 수정하면 된다.

1) 토픽 수를 5로 설정하여 토픽 모델객체(`final_model`)을 생성한다.

```
final_model = gensim.models.ldamodel.LdaModel(corpus=corpus, id2word=dictionary,
 num_topics=5, passes=5)
```

2) 토픽 모델 객체에 저장된 토픽 분석 결과를 `print_topics()` 함수를 사용하여 출력해 본다.

결과를 살펴보면, 0~5까지 토픽 번호가 할당되어 있고, 단어 앞에 붙은 수치는 해당 단어가 토픽에 미치는 기여도를 나타낸다. 토픽 번호는 랜덤으로 정해지고 실행 시스템에 따라 설치된 패키지 버전이 다를 수 있어 토픽 모델링 결과는 실행할 때마다 약간씩 다를 수 있다.

```
final_model.print_topics(num_words=5)
```

```
[(0, '0.037*기술' + 0.035*에너지 + 0.019*학습 + 0.018*시스템 + 0.018*배터리'),
 (1, '0.015*활용' + 0.013*한국 + 0.012*평가 + 0.012*사업 + 0.011*매출'),
 (2, '0.019*분야' + 0.011*사회 + 0.010*활용 + 0.009*추진단 + 0.009*이스타항공'),
 (3, '0.023*네이버' + 0.022*전력 + 0.019*모델 + 0.016*서비스 + 0.015*오픈'),
 (4, '0.026*교육' + 0.014*활용 + 0.014*이번 + 0.012*위원회 + 0.012*국정'),
 (5, '0.032*기술' + 0.014*보이스피싱 + 0.013*기반 + 0.012*이번 + 0.010*분야')]
```

3) 토픽에 대한 레이블은 자동으로 만들어지지 않기 때문에 주요 단어를 고려하여 각 토픽 내용을 설명하는 레이블(네이밍)을 결정한다.

우리가 수행한 토픽 모델링의 결과는 토픽마다 단어가 중복되어 단어만 보고 토픽의 내용을 결정하기에는 어려움이 있다. 머신 러닝을 사용한 분석은 데이터가 많을수록 정확도가 올라간다. 그런 의미에서 우리가 분석한 뉴스 데이터 1,000개는 머신러닝 기반 모델에 적용하기에는 데이터수가 부족하다. 이번 프로젝트는 상용화할 수 있는 수준의 정확한 분석보다는 머신러닝 기반의 텍스트 마이닝 과정과 관련 패키지 사용 방법의 학습을 목적으로 한다.

| 토픽번호 | 주요 단어 상위 20개                                                                                           | 토픽 레이블   |
|------|--------------------------------------------------------------------------------------------------------|----------|
| 0    | 0.021*검색, 0.013*모델, 0.013*활용, 0.012*대화, 0.010*성형, 0.010*미나, 0.009*거래, 0.009*시장, 0.009*사진, 0.008*매매     | 검색 모델 활용 |
| 1    | 0.033*활용, 0.025*지능, 0.024*인공, 0.017*교육, 0.014*직원, 0.011*검색, 0.011*위에 + 0.010*성형 + 0.010*참가자 + 0.009*엔진 | 인공지능 활용  |
| 2    | 0.029*오픈, 0.016*개발, 0.009*울트먼, 0.008*모델, 0.007*서비스, 0.007*지난, 0.007*사용자, 0.006*데이터, 0.006*미나, 0.006*사용 | 오픈 개발 모델 |

## • LDA 모델 시각화

토픽별 해당 단어들을 보고 단어들의 공통점을 찾아내 토픽에 대한 네이밍을 하는 경우 토픽 수가 많거나 토픽에 해당된 단어들이 많으면 한눈에 들어오지가 않기 때문에 시각화를 실시해 토픽별 단어 분포를 확인하는 것이 토픽 네이밍에 매우 유용하다.

LDA 시각화를 위해 파이썬에서 가장 많이 사용되는 패키지는 pyLDAvis 이다. pyLDAvis는 gensim 함수를 지원하기 때문에 코드는 매우 간단하다. 먼저 시각화를 위한 데이터(gensimvis.prepare)를 만들어 주고 이를 pyLDAvis로 시각화하면 된다.

## • 분석 결과 시각화하기

토픽 분석 결과를 시각화하기 위해 pyLDAvis 패키지의 pyLDAvis.gensim.prepare() 함수를 사용한다. 토픽 분석 결과를 가지고 있는 LDA 모델명, corpus명, dictionary명을 매개변수로 사용한다.

1) pyLDAvis 패키지를 설치한다.

```
pip install pyLDAvis
```

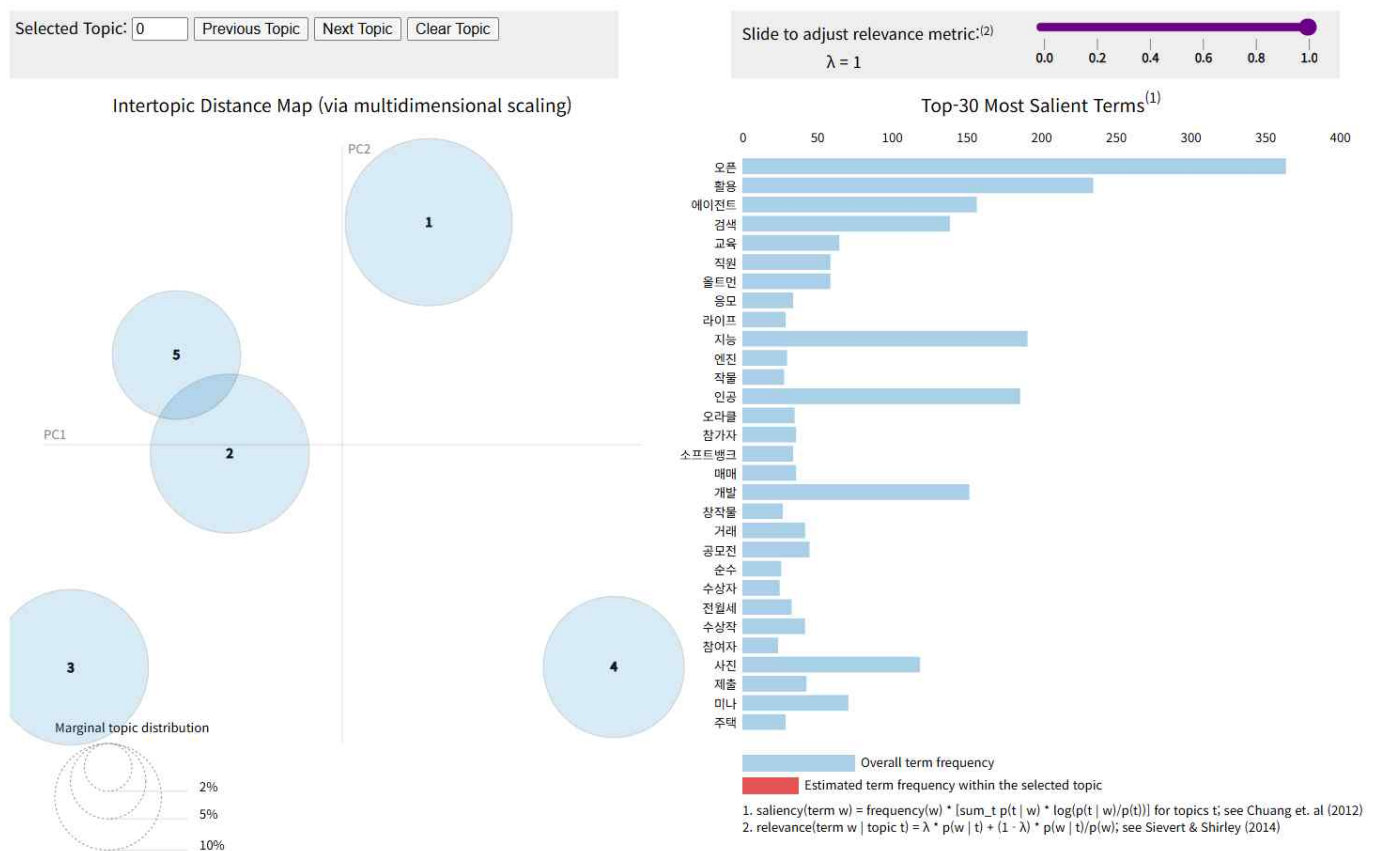
2) pyLDAvis.gensim.prepare() 함수를 사용하여 시각화 데이터 객체를 생성한다.

```
import pyLDAvis.gensim
prepared_data = pyLDAvis.gensim.prepare(final_model, corpus, dictionary)
```

3) pyLDAvis 객체는 display() 함수에 의해 출력된다.

화면의 왼쪽 영역에는 토픽간 거리 지도가 있고 오른쪽 영역에는 토픽에서 관련성 높은 30개 단어에 대한 바 차트가 있다. 왼쪽 영역에서 각 토픽을 나타내는 원이 다른 원에 포함되어 있거나 많이 겹쳐져 있다면 토픽의 수를 다르게 하여 LDA 모델을 다시 실행해야 한다.

```
pyLDAvis.display(prepared_data)
```



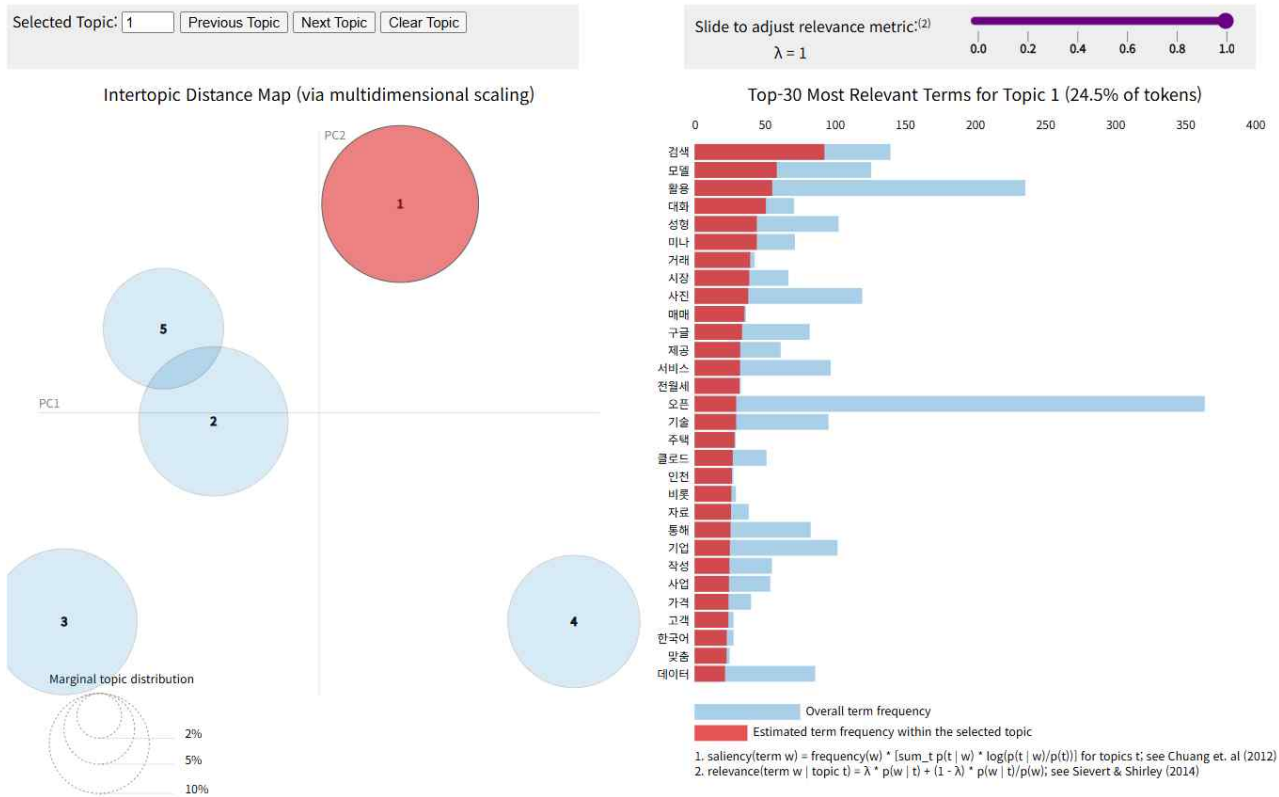
4) pyLDAvis를 사용하여 시각화한 결과를 html 파일로 저장한다.

```
pyLDAvis.save_html(prepared_data, 'data/토픽시각화.html')
```

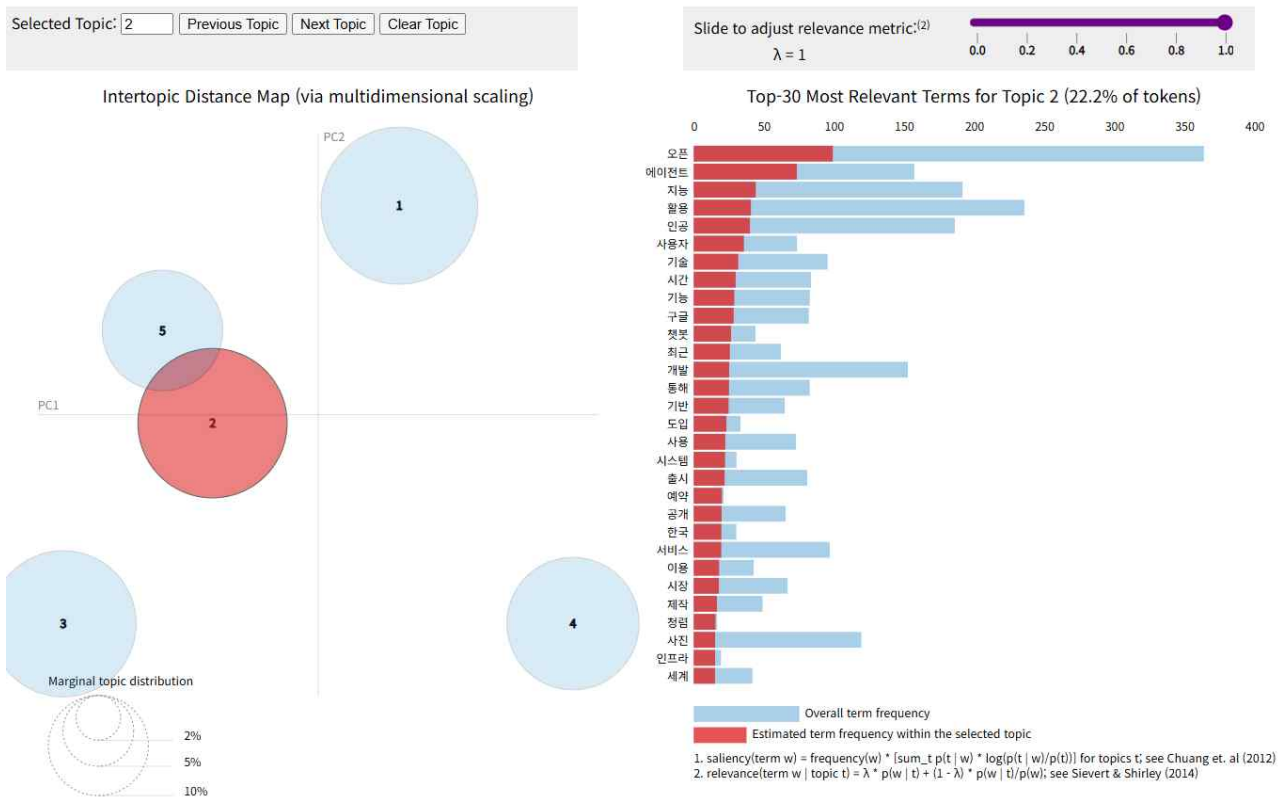
• 토픽 레이블(네이밍) 및 결과 해석

`print_topics()` 함수를 사용하여 출력한 토픽 번호 및 단어 목록과 `pyLDAvis` 시각화를 통해 나타난 토픽의 번호 및 단어 목록이 100% 일치하지는 않는다. 각 토픽에서 출력된 주요 단어만으로는 토픽을 정확히 분석하기가 어려우므로 주요 단어가 포함된 뉴스의 내용을 확인하는 추가 작업이 필요하다.

1) 토픽 1의 경우 '검색', '모델', '활용' 등 '챗GPT의 검색 모델 활용'을 토픽 레이블(네이밍)로 추론할 수 있다.



2) 토픽 2의 경우 '오픈', '지능', '활용', '인공' 등 '인공 지능 활용'에 대한 담론을 구성하고 있다고 결론지을 수 있다.



## • 기타 인공지능 예제

### • 인공지능 이미지 분류

이미지 분류는 이미지를 정해진 카테고리에 따라 AI가 분류해 주는 기술입니다. 이미지는 0~255 정수 범위의 값을 가지는 Width(너비) x Height(높이) x 3의 크기의 3차원 배열이다. 3은 Red, Green, Blue로 구성된 3개의 채널을 의미한다.

- [image] 폴더를 생성 후 이미지 분류.ipynb 파일을 생성하고 필요한 라이브러리를 import 한다.

```
import numpy as np
from PIL import Image #PIL(Python Image Library)
import matplotlib.pyplot as plt
```

### • 이미지 화면에 출력

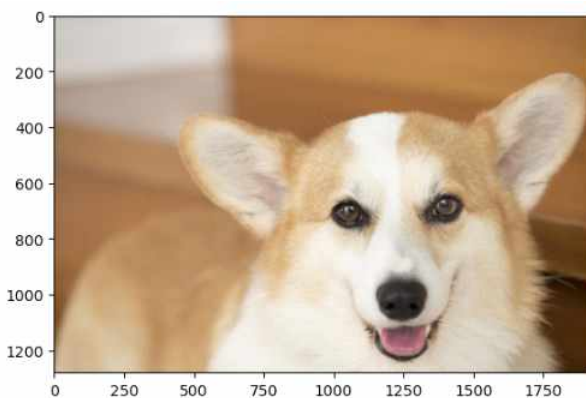
imshow()는 주로 Python의 Matplotlib 라이브러리에서 사용되는 함수로 2D 배열의 값을 색상으로 변환하여 이미지를 표시하는데 사용된다. 즉 배열 데이터를 이미지 형태로 시각화하는 함수이다.

- 프로젝트 폴더의 이미지 파일들을 Open 한 후 numpy array 함수를 이용해 3차원 배열로 변경한다.

```
files = ['./data/dog.png', './data/cat.png']
images = [np.array(Image.open(file)) for file in files]
```

- 배열(images)에 저장된 이미지를 imshow() 함수를 이용하여 출력한다.

```
for image in images:
 plt.imshow(image)
 plt.axis('off')
 plt.show()
```



- 이미지 분류를 위한 패키지 설치

- OpenCV(Open Source Computer Vision) 오픈 소스 라이브러리를 설치한다. 이 라이브러리는 컴퓨터가 사람의 눈처럼 인식할 수 있게 처리해주는 역할을 하기도 하며 우리가 많이 사용하는 카메라 Application에서도 OpenCV가 사용된다.

```
pip install opencv-python
```

- 구글(Google)에서 개발한 텐서플로(TensorFlow) 라이브러리를 설치한다. 텐서플로는 딥러닝 프로그램을 쉽게 구현할 수 있도록 다양한 기능을 제공해 주는 라이브러리다.

```
pip install tensorflow
```

- TensorFlow TypeError가 발생하는 경우 아래와 같이 protobuf 패키지를 3.20.x 이하로 다운 그레이드 하고 'Restart' 한다.

```
pip install protobuf==3.20.*
```

- 위에서 설치한 라이브러리를 import 한다.

```
import cv2
import tensorflow as tf
```

- ResNet50(레즈넷50)을 이용한 이미지 분류

ResNet-50은 이미지 분류에 뛰어난 합성곱 신경망(CNN)입니다. 마치 사진을 분석하고, 그 안의 사물과 장면을 식별하고, 그에 따라 분류할 수 있는 고도로 훈련된 이미지 분석가와 같으며 아래와 같은 특징이 있다.

- 1) 계층(Layer) 50개로 구성된 합성곱 신경망이다.
- 2) ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공한다.
- 3) 입력 제약이 매우 크고 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있는 단점이 있다.

- 사전에 훈련된 keras ResNet50 모델을 생성한다.

```
resnet50=tf.keras.applications.resnet.ResNet50(weights='imagenet', input_shape=(224, 224, 3))
```

- 이미지 파일을 resnet50 모델에 대입하기 위해 이미지 크기를 변경 후 이미지의 형태도 변경한다.

```
file = 'data/cat.png'
image = np.array(Image.open(file))
image_resized = cv2.resize(image, (224, 224))
image_resized_reshape = image_resized.reshape([1, 224, 224, 3])
```

- 크기와 형태가 변경된 이미지를 resnet50 모델을 사용하여 어떤 이미지인지 예측한다.

```
predicted = resnet50.predict(image_resized_reshape)
```

```
1/1 ————— 0s 191ms/step
```

- 예측결과를 decode\_predictions 라이브러리를 이용하여 decoding 한다.

```
decoded_predict = tf.keras.applications.imagenet_utils.decode_predictions(predicted)
```

- decoding한 예측 결과를 출력한다.

```
decoded_predict
```

```
[(['n02123394', 'Persian_cat', np.float32(0.5924032)),
 ('n02123045', 'tabby', np.float32(0.13372304)),
 ('n02123159', 'tiger_cat', np.float32(0.077767216)),
 ('n02124075', 'Egyptian_cat', np.float32(0.015219235)),
 ('n02127052', 'lynx', np.float32(0.01208303))]]
```

```
for i, pred in enumerate(decoded_predict[0]):
 print(f'{i}위: {pred[1]} ({pred[2]*100:.2f}%)')
```

```
0위: Persian_cat (59.24%)
1위: tabby (13.37%)
2위: tiger_cat (7.78%)
3위: Egyptian_cat (1.52%)
4위: lynx (1.21%)
```

- 이미지 예측 함수를 작성한 후 특정 이미지를 적용하여 결과를 출력한다.

```
def predict_image(file):
 image = np.array(Image.open(file))
 plt.imshow(image)
 plt.axis('off')
 plt.show()

 image_resized = cv2.resize(image, (224, 224))
 image_resized_reshape = image_resized.reshape([1, 224, 224, 3])
 predicted = resnet50.predict(image_resized_reshape)
 decoded_predict = tf.keras.applications.imagenet_utils.decode_predictions(predicted)

 for i, pred in enumerate(decoded_predict[0]):
 print(f'{i}위: {pred[1]} ({pred[2]*100:.2f}%)')
```

```
predict_image('../data/dog.png')
```



```
1/1 ————— 0s 140ms/step
0위: Siberian_husky (37.08%)
1위: Eskimo_dog (23.64%)
2위: white_wolf (17.68%)
3위: Cardigan (12.10%)
4위: Samoyed (2.56%)
```



- streamlit을 이용한 웹페이지 작성
- 웹페이지를 쉽고 빠르게 생성할 수 있는 streamlit 라이브러리를 설치한다.

```
pip install streamlit
```

[image] app.py

```
import streamlit as st
import numpy as np
import cv2
import tensorflow as tf
from PIL import Image

def predict_image(file):
 image = np.array(Image.open(file))
 st.image(image, use_container_width=False) #use_container_width=True 칼럼 너비에 맞춰 이미지의 크기가 자동으로 조정된다.
 image_resized = cv2.resize(image, (224, 224))
 image_resized_reshape = image_resized.reshape([1, 224, 224, 3])

 resnet50=tf.keras.applications.resnet.ResNet50(weights='imagenet', input_shape=(224, 224, 3))
 predicted = resnet50.predict(image_resized_reshape)
 decoded_predict = tf.keras.applications.imagenet_utils.decode_predictions(predicted)
 return decoded_predict[0]

st.title('이미지 분류 인공지능 페이지')
file = st.file_uploader('이미지를 올려 주세요.', type=['jpg', 'png'])
if file:
 with st.spinner('기다려 주세요...'):
 decoded_predict = predict_image(file)
 for i, pred in enumerate(decoded_predict):
 st.success(f'{i}위: {pred[1]} ({pred[2]*100:.2f}%)')
```

- app.py 파일을 브라우저에 실행한다.

```
streamlit run app.py
```

이미지를 올려 주세요.



cat.png 4.3KB



0위: Persian\_cat (59.24%)

1위: tabby (13.37%)

2위: tiger\_cat (7.78%)

3위: Egyptian\_cat (1.52%)

4위: lynx (1.21%)

- Flask을 이용한 웹페이지 작성

- 웹페이지를 쉽고 빠르게 생성할 수 있는 streamlit 라이브러리를 설치한다.

```
pip install flask
```

- 'app2.py' 파일을 생성하고 Flask 라이브러리를 활용하여 백엔드 페이지를 작성한다.

```
[image] app2.py

from flask import Flask, render_template, request
import numpy as np
import cv2
import tensorflow as tf
from PIL import Image

def predict(file):
 image = np.array(Image.open(file))
 image_resized = cv2.resize(image, (224, 224))
 image_resized_reshape = image_resized.reshape([1, 224, 224, 3])

 resnet50=tf.keras.applications.resnet.ResNet50(weights='imagenet', input_shape=(224, 224, 3))
 predicted = resnet50.predict(image_resized_reshape)
 decoded_predict = tf.keras.applications.imagenet_utils.decode_predictions(predicted)

 result = ''
 for i, pred in enumerate(decoded_predict[0]):
 result += f'<h5>{i+1}위: {pred[1]} ({pred[2]*100:.2f}%)</h5>'
 print(result)
 return result

app = Flask(__name__, template_folder='templates')

@app.route('/')
def index():
 return render_template('index.html', title='이미지 분류')

@app.route('/predict', methods=['POST'])
def classification():
 file = request.files['file']
 result = predict(file)
 return result

if __name__ == '__main__':
 app.run(port=5000, debug=True)
```

- app2.py 파일을 브라우저에 실행한다.

```
python app2.py
```

파일 선택 cat.png

분류 결과

1위: Persian\_cat (59.24%)  
2위: tabby (13.37%)  
3위: tiger\_cat (7.78%)  
4위: Egyptian\_cat (1.52%)  
5위: lynx (1.21%)

- [templates] 폴더에 index.html 파일을 생성하고 html 프론트엔드 페이지를 작성한다.

[image]-[templates] index.html

```
<html>
<head>
 <script src="http://code.jquery.com/jquery-1.9.1.js"></script>
 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css" rel="stylesheet">
 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.min.js"></script>
 <title>{{title}}</title>
 <style>
 #loading {
 position: absolute; display: block; left: 40%; top: 40%; display: none;
 }
 </style>
</head>
<body>
 <div class="my-5 row justify-content-center">
 <h1 class="text-center mb-5">{{title}}</h1>
 <div class="col-md-6 p-5">

 <form name="frm">

 <div class="input-group mt-3">
 <input name="file" type="file" class="form-control" accept="image/*">
 <button class="btn btn-secondary">분류 결과</button>
 </div>
 </form>
 <hr>
 <div id="results"></div>
 </div>
 </div>
</body>
<script>
 $(frm.file).on("change", function(e){ #이미지 파일을 선택한 경우
 $("#image").attr("src", URL.createObjectURL(e.target.files[0]));
 });
 $(frm).on("submit", function(e){ #폼이 Submit된 경우
 $("#loading").show();
 e.preventDefault();
 if($(frm.file).val() == "") {
 alert("분류할 이미지를 선택하세요.");
 return;
 }
 const file=$(frm.file)[0].files[0];
 const formData = new FormData();
 formData.append("file", file)
 $.ajax({
 type:"post",
 url:"/predict",
 data:formData,
 processData:false,
 contentType:false,
 success:function(data){
 $("#results").html(data);
 $("#loading").hide();
 }
 });
 });
</script>
</html>
```

## • 인공지능 스피커

인공지능 스피커는 음성 비서 기능을 갖춘 스피커로, 사용자의 음성 명령을 인식하고 다양한 서비스를 제공한다. 음악 재생, 정보 검색, 일정 관리, 스마트 홈 제어 등 여러 기능을 수행할 수 있다. 기존 스피커에 인공지능 기술을 결합하여 사용자의 편의성을 높인 것이 특징이다.

## • Python 가상머신

가상머신은 물리적 머신의 자원(CPU, 메모리, 저장 공간 등)을 가상화하여 여러 개의 독립적인 가상 환경을 만들 수 있게 해준다. Python 가상머신은 특정 프로젝트에 필요한 Python 인터프리터와 라이브러리들을 격리된 공간에 설치하고 관리할 수 있도록 지원한다. 이렇게 함으로써 프로젝트마다 다른 버전의 패키지를 설치하고 관리할 수 있으며, 프로젝트 간의 패키지 충돌을 방지할 수 있다.

## • Python 가상머신 생성, 실행, 종료

- 프로젝트 폴더로 이동한 후 아래 명령으로 Python 가상머신을 생성한다. -m 옵션은 module을 의미한다.

```
C:\data\python> python -m venv myenv
```

- 아래 명령으로 가상머신(myenv)을 실행한다. S를 입력한 후 Tab키를 누르면 폴더명이 자동 입력된다.

```
C:\data\python> .\myenv\Scripts\activate
```

- VS-Code의 인터프리터(interpreter) Path를 가상머신 Path로 변경한다.

```
[Ctrl + Shift + P] - [Python: Select Interpreter] - [Enter interpreter path] -[Find] -[myenv/Scripts/python.exe]
```

```
[VS-Code 오른쪽 아래 Python 버전] - [Enter interpreter path] -[Find] -[myenv/Scripts/python.exe]
```

- 아래 명령으로 가상머신을 종료하고 VS-Code의 interpreter도 Global Python Interpreter로 변경한다.

```
(myenv) C:\data\python> deactivate
```

## • 가상머신 환경에 패키지 설치

- 가상머신(myenv)에 google에서 제공하는 TTS(Text To Speech) 패키지를 설치한다.

```
(myenv) C:\data\python> pip install gtts
```

- pip list 명령으로 설치된 패키지를 확인하면 가상머신 환경에는 설치되었지만 Global 환경에는 없는 것을 확인할 수 있다.

```
(myenv) C:\data\python> pip list
```

```
C:\data\python> pip list
```

- 가상머신(myenv)에 설치된 패키지들의 이름과 버전을 확인한다.

```
(myenv) C:\data\python>pip freeze
```

- 다른 PC에서 이 프로젝트에서 사용한 같은 버전의 패키지를 사용하기 위해 패키지 버전을 텍스트 파일에 저장한다.

```
(myenv) C:\data\python>pip freeze > requirements.txt
```

- 가상머신(myenv) 실행을 종료한 후 폴더를 완전 삭제한다.

```
(myenv) C:\data\python>deactivate
```

```
C:\data\python> rmdir /s myenv
```

- 가상머신(myenv)를 생성하여 이 가상머신에 이전에 사용한 패키지(requirements.txt)들을 설치한다.

```
C:\data\python>python -m venv myenv
```

```
C:\data\python> .\myenv\Scripts\activate
```

```
(myenv) C:\data\python> pip install -r requirements.txt
```

- 가상머신에서 전역(Global) 패키지 사용

가상머신을 생성할 때 전역(Global) 공간에 설치된 전역 패키지들을 사용하고 싶을 경우 --system-site-packages 옵션을 사용한다.

- 기존에 설치된 가상머신(myenv)을 삭제한다.

```
(myenv) C:\data\python>deactivate
```

```
C:\data\python> rmdir /s myenv
```

- 가상머신(myenv)에서 전역 패키지들을 사용하고 싶은 경우 아래 옵션을 사용하여 가상머신을 생성한다.

```
C:\data\python> python -m venv myenv --system-site-packages
```

- 가상머신(myenv)을 실행한 후 gtts 패키지를 설치하면 가상머신(myenv)에만 gtts 패키지가 설치된다.

```
C:\data\python> .\myenv\Scripts\activate
```

```
(myenv) C:\data\python> pip install gtts
```

- 가상머신(myenv)에서 사용할 수 있는 패키지 목록을 확인하면 전역패키지 목록도 출력된다.

```
(myenv) C:\data\python> pip list
```

- 가상머신(myenv)에서 패키지 목록을 확인할 경우 --local 옵션을 사용하면 가상머신(myenv)에 설치된 패키지만 출력된다.

```
(myenv) C:\data\python> pip list --local
```

- 전역공간에서 패키지 목록을 확인하면 가상머신에서 설치한 패키지는 보이지 않는다.

```
(myenv) C:\data\python>deactivate
```

```
C:\data\python> pip list
```

- 텍스트 입력 음성 출력

- 가상머신(myenv)에 google에서 제공하는 TTS(Text To Speech) 패키지를 설치한다.

```
(myenv) C:\data\python> pip install gtts
```

- 가상머신(myenv)에 음성을 출력하는 패키지를 설치한다. (최신버전이 문제가 있어서 특정 버전 설치)

```
(myenv) C:\data\python>pip install playsound==1.2.2
```

- 영어 문장을 음성으로 Play한다.

```
[speaker] textEnToSpeech.py
```

```
from gtts import gTTS
from playsound import playsound

file_name = 'speaker/sample.mp3'
text = 'Can i help you?'
tts_en = gTTS(text=text, lang='en')
tts_en.save(file_name)
playsound(file_name)
```

- 한글 문장을 음성으로 Play 한다.

```
[speaker] textKoToSpeech.py
```

```
from gtts import gTTS
from playsound import playsound

file_name = 'speaker/sample2.mp3'
text = '파이썬을 배우면 이런 것도 할 수 있어요.'
tts_ko = gTTS(text=text, lang='ko')
tts_ko.save(file_name)
playsound(file_name)
```

- 텍스트 파일(sample\_en.txt)을 읽어 음성으로 Play 한다.

```
[speaker] sample_en.txt
```

```
Imagine that you have just arrived at a hotel after a tiring 7-hour overnight flight.
At the front desk, the staff person welcomes you, gives you a warm smile,
confirms your reservation quickly and hands you your room key.
They tell you the complimentary breakfast is still being served in the dining room.
```

```
[speaker] textFileToSpeech.py
```

```
from gtts import gTTS
from playsound import playsound

file_name = 'speaker/sample3.mp3'
with open('speaker/sample_en.txt') as file:
 text = file.read()

tts_en = gTTS(text=text, lang='en')
tts_en.save(file_name)
playsound(file_name)
```

- 음성 입력 텍스트 출력

- 음성을 텍스트로 변경하는 STT(Speech To Text) 패키지를 설치한다.

```
(myenv) C:\data\python> pip install SpeechRecognition
```

- 마이크 사용을 위한 PyAudio(play audio) 패키지를 설치한다.

```
(myenv) C:\data\python> pip install PyAudio
```

- 마이크를 통해 음성을 입력하면 텍스트로 출력한다.

```
[speaker] speechToText.py
```

```
import speech_recognition as sr
r = sr.Recognizer()

#마이크로부터 음성 듣기
with sr.Microphone() as source:
 print('듣고 있어요.')
 audio = r.listen(source)

try:
 #구글 API로 인식하므로 네트워크가 연결되어있어야 함(하루 50회로 제한)
 #text = rn.recognize_google(audio, language='en-US')
 text = r.recognize_google(audio, language='ko')
 print(text)
except sr.UnknownValueError:
 print('인식 실패')
except sr.RequestError as e:
 print(f'요청 실패 : {e}')
```

- 아래 사이트로 이동하여 sample3.mp3 파일을 smaple3.wav 파일로 변환한다.

```
https://cloudconvert.com/mp3-converter
```

- 파일로부터 음성 파일(sample3.wav)을 듣고 텍스트로 변경 후 출력한다.

```
[speaker] seechFileToText.py
```

```
import speech_recognition as sr
r = sr.Recognizer()

#파일로부터 음성 불러오기 (가능 파일:wav, aiff/aiff-c, flac 불가능 파일: mp3)
with sr.AudioFile('speaker/sample3.wav') as source:
 audio = r.record(source)

try:
 #구글 API로 인식하므로 네트워크가 연결되어있어야 함(하루 50회로 제한)
 text = r.recognize_google(audio, language='en-US')
 print(text)
except sr.UnknownValueError:
 print('인식 실패')
except sr.RequestError as e:
 print(f'요청 실패 : {e}')
```

- 묻고 답하는 인공지능 스피커

1) 인공지능 컴퓨터가 '무엇을 도와드릴까요?' 라고 문자로 출력하며 말한다. (프로그램 종료 시 Ctrl+C)

[speaker] apSpeaker.py ①

```
import time, os
import speech_recognition as sr
from gtts import gTTS
from playsound import playsound

#1) 문자를 입력 받아 소리 내어 읽기(TTS)
def speak(text):
 print('[인공지능] ' + text)
 file_name = 'voice.mp3'
 tts_ko = gTTS(text=text, lang='ko')
 tts_ko.save(file_name)
 playsound(file_name)
 if os.path.exists(file_name):
 os.remove(file_name)

#2) 음성을 듣고 인식하여 문자로 변환(STT)
def listen(recognizer, audio):
 pass

#3) 문자를 입력 받아 대답하기
def answer(input_text):
 pass

speak('무엇을 도와드릴까요?')

while True:
 time.sleep(0.1)
```

2) 사용자가 말하면 인공지능 컴퓨터는 음성을 듣고 텍스트로 변환하여 출력 후 answer() 함수를 실행한다.

[speaker] apSpeaker.py ②

```
#2) 음성을 듣고 인식하여 문자로 변환(STT)
def listen(recognizer, audio):
 try:
 text= recognizer.recognize_google(audio, language='ko')
 print('[홍길동] ' + text)
 answer(text)
 except sr.UnknownValueError:
 print('인식 실패')
 except sr.RequestError as e:
 print(f'요청 실패: {e}')

#3) 문자를 입력 받아 대답하기
def answer(input_text):
 pass

speak('무엇을 도와드릴까요?')

r = sr.Recognizer()
m = sr.Microphone()
stop_listening = r.listen_in_background(m, listen) #백드라운드에서 마이크를 통해 음성이 인식 될 때까지 계속 대기한다.

while True:
 time.sleep(0.1)
```



3) answer() 함수에서는 사용자가 말한 키워드에 따라 인공지능이 대답해 준다.

[speaker] apSpeaker.py ③

```
import time, os
import speech_recognition as sr
from gtts import gTTS
from playsound import playsound

#1) 문자를 입력 받아 소리 내어 읽기(TTS)
def speak(text):
 print('[인공지능] ' + text)
 file_name = 'voice.mp3'
 tts_ko = gTTS(text=text, lang='ko')
 tts_ko.save(file_name)
 playsound(file_name)
 if os.path.exists(file_name):
 os.remove(file_name)

#2) 음성을 듣고 인식하여 문자로 변환(STT)
def listen(recognizer, audio):
 try:
 text= recognizer.recognize_google(audio, language='ko')
 print('[홍길동] ' + text)
 answer(text)
 except sr.UnknownValueError:
 print('인식 실패')
 except sr.RequestError as e:
 print(f'요청 실패: {e}')

#3) 문자를 입력 받아 대답하기
def answer(input_text):
 answer_text = ''
 if '안녕' in input_text:
 answer_text = '안녕하세요? 반갑습니다.'
 elif '날씨' in input_text:
 answer_text = '오늘의 서울 기온은 20도 입니다. 맑은 하늘이 예상됩니다.'
 elif '환율' in input_text:
 answer_text = '원 달러 환율은 1380원입니다.'
 elif '삼성전자' in input_text:
 answer_text = '삼성전자의 주가는 한주가 71800원입니다.'
 elif '날짜' in input_text:
 answer_text = '오늘은 8월 15일 금요일입니다.'
 elif '고마워' in input_text:
 answer_text = '별 말씀요.'
 elif '종료' in input_text:
 answer_text = '다음에 또 만나요'
 stop_listening(wait_for_stop=False)
 else:
 answer_text = '다시 한 번 말씀해 주시겠어요?'
 speak(answer_text)

speak('무엇을 도와드릴까요?')

r = sr.Recognizer()
m = sr.Microphone()
stop_listening = r.listen_in_background(m, listen) #백드라운드에서 마이크를 통해 음성이 인식 될 때까지 계속 대기한다.

while True:
 time.sleep(0.1)
```

## • 웹 소켓을 이용한 채팅 프로그램

웹 소켓(WebSocket)은 웹 애플리케이션에서 클라이언트와 서버 간의 지속적인 양방향 통신을 가능하게 하는 프로토콜이다. 기존 HTTP 프로토콜의 요청-응답 방식과 달리, 연결을 유지하며 실시간으로 데이터를 주고받을 수 있어 채팅, 실시간 게임, 주식 시세 업데이트 등 다양한 분야에서 활용된다.

### 1) 웹 소켓 라이브러리를 설치한다.

```
pip install flask-socketio
```

### 2) 채팅 서버 프로그램을 작성한다.

[chat] server.py

```
from flask import Flask, render_template, request, session
from flask_socketio import SocketIO, join_room, emit, leave_room

app = Flask(__name__, template_folder='templates')
app.secret_key='mysecret'
socketio = SocketIO(app)

@app.route('/')
def index():
 return 'Hello'

#1)채팅 클라이언트 페이지를 출력한다.
@app.route('/chat/<room>')
def chat(room):
 uid = request.args['uid']
 session['uid'] = uid
 session['room'] = room
 return render_template('chat.html', title='채팅방', room=room)

#2)클라이언트에서 입장을 요청하면 입장 후 상태 메시지를 클라이언트에게 전송한다.
@socketio.on('joined', namespace='/chat')
def joined(message):
 uid=session['uid']
 room=session['room']
 join_room(room)
 emit('status', {'msg':uid + '님 입장하셨습니다.'}, room=room)

#3)클라이언트에서 전송한 text를 받아 다시 클라이언트에 메시지 전송한다.
@socketio.on('text', namespace='/chat')
def text(message):
 uid = session['uid']
 room = session.get('room')
 msg = message.get('msg')
 emit('message', {'msg': f'{uid} : {msg}'}, room=room)

#4)클라이언트가 퇴장 요청이오면 입장한 채팅룸을 퇴장한다.
@socketio.on('left', namespace='/chat')
def left(message):
 room=session['room']
 leave_room(room)
 emit('status', {'msg':session['uid'] + '님 퇴장하셨습니다.'}, room=room)

if __name__ == '__main__':
 socketio.run(app, debug=True, port=5000)
```

### 3) 채팅 클라이언트 프로그램을 작성한다.

[chat]-[templates] chat.html

```
<html>
<head>
 <script src="http://code.jquery.com/jquery-1.9.1.js"></script>
 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css" rel="stylesheet">
 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.min.js"></script>
 <script src="https://cdn.socket.io/4.0.1/socket.io.min.js"
 integrity="sha384-LzhRnpGmQP+IOvWruF/lgkcqD+WDVt9fU3H4BWmwP5u5LTmkUGafMcpZKNObVMLU"
 crossorigin="anonymous"></script>
 <title>{{title}}</title>
</head>
<body>
 <div class="my-5 row justify-content-center">
 <div class="col-md-6">
 <div class="card">
 <div class="card-header"><h4>{{title}} [{{room}}]</h4></div>
 <div class="card-body">
 <div id="results"></div><hr>
 <form name="frm" class="input-group mt-3">
 <input name="text" class="form-control">
 <button class="btn btn-secondary">전송</button>
 </form>
 </div>
 </div>
 <button id="left" class="btn btn-primary px-5 mt-3">퇴장하기</button>
 </div>
 </div>
</body>
<script>
 //1)웹소켓에 접속한다.
 const socket = io.connect("http://" + document.domain + ":" + location.port + "/chat");

 //2)웹소켓 접속 후 서버에게 채팅룸 입장을 요청한다.
 socket.on("connect", function(){
 socket.emit('joined', {});
 });

 //3)웹서버로부터 status 요청이 온 경우 메시지를 출력한다.
 socket.on('status', function(data){
 $("#results").append('<div>${data.msg}</div>');
 });

 //4)웹서버에게 입력한 text를 전송한다.
 $(frm).on("submit", function(e){
 e.preventDefault();
 const text = $(frm.text).val();
 if(text == "") {
 alert("보내실 내용을 입력하세요!"); $(frm.text).focus(); return;
 }
 socket.emit('text', {msg: text});
 $(frm.text).val("");
 });

 //5)웹서버로부터 message 요청이 온 경우 메시지를 출력한다.
 socket.on('message', function(data){
 $("#results").append('<div>${data.msg}</div>');
 });

 //6)웹서버에게 퇴장을 요청한다.
 $("#left").on("click", function(){
 socket.emit('left', {}, function(){
 socket.disconnect(); location.href='/';
 });
 });
</script>
</html>
```

- 웹 소켓을 이용한 인공지능 프로그램

1) 서버 프로그램을 작성한다.

```
[chat] app2.py
```

```
from flask import Flask, render_template, session
from flask_socketio import SocketIO, emit, join_room, leave_room
from scrapping import answer

app = Flask(__name__)
app.secret_key = 'mysecret'
socketio = SocketIO(app)

@app.route('/')
def index():
 return 'Hello'

@app.route('/chat/<uid>')
def chatting(uid):
 #혼자만 이용하는 방이므로 방 이름을 아이디로 정한다.
 session['room'] = uid
 return render_template('chat.html', title='채팅방', room=uid)

@socketio.on('joined', namespace='/chat')
def joined(message):
 room=session['room']
 join_room(room)
 emit('status', {'msg': room + '님 입장하셨습니다.'}, room=room)

#클라이언트에서 메시지를 전송한 경우
@socketio.on('text', namespace='/chat')
def text(message):
 room = session.get('room')
 #클라이언트에게 메시지를 보내 출력시킨다.
 emit('message', {'msg':session.get('room') + " : " + message['msg']}, room=room)
 #클라이언트 질문에 대한 대답 결과를 보내 출력시킨다.
 answer_text = answer(message['msg'])
 emit('message', {'msg':'인공지능' + " : " + answer_text}, room=room)

@socketio.on('left', namespace='/chat')
def left(message):
 room=session['room']
 leave_room(room)

if __name__ == '__main__':
 socketio.run(app, debug=True, port=5000)
```

2) 스크래핑에 필요한 라이브러리를 설치한다.

```
pip install requests
```

```
pip install BeautifulSoup4
```

3) 인공지능이 대답할 스크래핑 결과를 구하는 함수를 작성한다.

[chat] app2.py

```
import requests
from bs4 import BeautifulSoup
import time

def create_soup(url):
 res = requests.get(url)
 soup = BeautifulSoup(res.text, 'lxml')
 with open('soup.html', 'w', encoding='utf-8') as file:
 file.write(res.text)
 return soup

def answer(input_text):
 answer_text = ''
 if '안녕' in input_text:
 answer_text = '안녕하세요? 반갑습니다.'
 elif '날씨' in input_text:
 temp = weather_temp()
 answer_text = '오늘의 ' + temp + '입니다.'
 elif '환율' in input_text:
 rate = exechage_rate()
 answer_text = '1달러 환율은 ' + rate + '입니다.'
 elif '주식' in input_text:
 answer_text = stock(input_text)
 elif '고마워' in input_text:
 answer_text = '별말씀을요.'
 else:
 answer_text = '다시 한 번 말씀해 주시겠어요?'
 time.sleep(2)
 return answer_text

def weather_temp():
 url = 'https://search.naver.com/search.naver?where=nexearch&sm=top_h ty&fbm=0&ie=utf8&query=오늘의날씨'
 soup = create_soup(url)
 temp = soup.find('div', attrs={'class': 'temperature_text'})
 if temp:
 temp = temp.get_text()
 else:
 temp = '없음'
 return temp

def exechage_rate():
 url='https://search.naver.com/search.naver?where=nexearch&sm=top_h ty&fbm=0&ie=utf8&query=환율'
 soup = create_soup(url)
 rate = soup.find_all('span', attrs={'class': 'nb_txt_pronunciation'})
 if rate:
 rate = rate[1].get_text()
 else:
 rate = '없음'
 return rate

def stock(input_text):
 index = input_text.find('주식')
 query=input_text[:index+2]
 url='https://search.naver.com/search.naver?where=nexearch&sm=top_h ty&fbm=0&ie=utf8&query='+ query
 soup = create_soup(url)
 price = soup.find('div', attrs={'class': re.compile('^spt_con')}).find('strong')
 if price:
 price = price.get_text()
 else:
 price = '없음'
 return price
```

- Flask로 웹페이지 만들기

- 홈페이지를 작성한다. (실행: Flask run 또는 python app.py)

[web] app.py

```
from flask import Flask, render_template

app = Flask(__name__, template_folder='templates')

@app.route('/')
def index():
 return render_template('index.html', title='홈페이지')

if __name__ == '__main__':
 app.run(port=5000, debug=True)
```

[web]-[templates] index.html

```
<html>
<head>
 <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
 <link rel="stylesheet" href="/static/css/style.css"/>
 <script src="http://code.jquery.com/jquery-1.9.1.js"></script>
 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css" rel="stylesheet">
 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.min.js"></script>
 <script src="https://cdnjs.cloudflare.com/ajax/libs/handlebars.js/3.0.1/handlebars.js"></script>
 <title>{{title}}</title>
</head>
<body>
 <div class="container">
 {%include 'header.html'%}
 {%include 'menu.html'%}
 {%block main_area%}
 {%endblock%}
 {%include 'footer.html'%}
 </div>
</body>
</html>
```

[web]-[templates] header.html

```
<div class="row mt-5">
 <div class="col">
 <h1 class="text-center mb-5">인공지능(머신러닝)</h1>
 <hr>
 </div>
```

[web]-[templates] menu.html

```
<div>
 선형회귀
 다항회귀
 다중선형회귀
 로지스틱회귀
 K-평균
</div>
<hr>
```

[web]-[templates] footer.html

```
<div class="row my-5">
 <div class="col">
 <hr>
 <h4 class="text-center">Copyright 2023 홍길동 All rights reserved.</h4>
 </div>
</div>
```

- 선형회귀 페이지를 작성한다.

[web] aiRoute.py

```
from flask import Blueprint, render_template

aiBluePrint = Blueprint('ai', __name__)

@aiBluePrint.route('/page1') #/ai/page1
def ai_page1():
 return render_template('ai/page1.html')
```

[web]-[templates] page1.html

```
{%extends 'index.html'%}
{%block main_area%}
 <div>
 <h3 class="text-center mb-5">선형회귀</h3>
 </div>
{%endblock%}
```

[web] app.py

```
from flask import Flask, render_template
import aiRoute

app = Flask(__name__, template_folder='templates')
app.register_blueprint(aiRoute.aiBluePrint, url_prefix='/ai')

@app.route('/')
def index():
 return render_template('index.html', title='홈페이지')

if __name__ == '__main__':
 app.run(port=5000, debug=True)
```

- 선형회귀 모델생성 후 시간을 입력하여 점수를 예측한다.

[web] aiRoute.py

```
from flask import Blueprint, render_template, request

import pandas as pd
from sklearn.linear_model import LinearRegression

aiBluePrint = Blueprint('ai', __name__)

@aiBluePrint.route('/page1') #/ai/page1
def ai_page1():
 return render_template('ai/page1.html')

@aiBluePrint.route('/linear') #/ai/linear?hours=5
def ai_linear():
 hours = int(request.args['hours'])
 dataset = pd.read_csv('./data/LinearRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values
 reg = LinearRegression()
 reg.fit(X, y)
 y_predict = reg.predict([[hours]])
 return str(round(y_predict[0], 2))
```

- 선형회귀 모델 그래프를 출력한다.

[web] aiRoute.py

```
from flask import Blueprint, render_template, request, send_file

import pandas as pd
from sklearn.linear_model import LinearRegression

from io import BytesIO
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False

#서버에서 GUI없이 그래프를 생성할 경우 사용한다.
matplotlib.use('Agg')

aiBluePrint = Blueprint('ai', __name__)

@aiBluePrint.route('/page1') #/ai/page1
def ai_page1():
 return render_template('ai/page1.html')

@aiBluePrint.route('/linear')
def ai_linear():
 hours = int(request.args['hours'])
 dataset = pd.read_csv('../data/LinearRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values
 reg = LinearRegression()
 reg.fit(X, y)
 y_predict = reg.predict([[hours]])
 return str(round(y_predict[0], 2))

@aiBluePrint.route('/linear/graph') #/ai/linear/graph
def ai_linear_graph():
 dataset = pd.read_csv('../data/LinearRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values
 reg = LinearRegression()
 reg.fit(X, y)

 plt.figure(figsize=(10, 5))
 plt.scatter(X, y, color='blue')
 plt.plot(X, reg.predict(X), color='green')
 plt.title('Score by hours')
 plt.xlabel('hours')
 plt.ylabel('score')

 #그래프를 이미지 파일로 생성한다.
 img = BytesIO()
 plt.savefig(img, format='png', dpi=200)
 plt.close()
 img.seek(0) #파일의 처음 위치로 돌아간다.
 return send_file(img, mimetype='image/png')
```



- 선형회귀 웹페이지를 작성한다.

[web]-[templates]-[ai] page1.html

```
{%extends 'index.html'%}
{%block main_area%}
 <div>
 <h3 class="text-center mb-5">선형회귀</h3>
 <div class="row justify-content-center">
 <form name="frm" class="col-6">
 <div class="input-group">
 <input name="hours" class="form-control" placeholder="공부시간">
 <button class="btn btn-primary">점수예측</button>
 </div>
 </form>
 <div class="mt-3">
 <h5 id="predict" class="text-center"></h5>
 </div>
 <div class="mb-3 text-center">

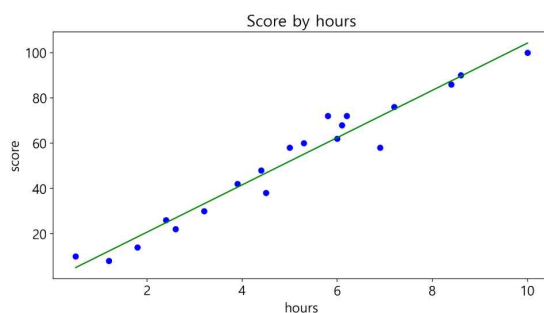
 </div>
 </div>
 </div>
</div>
<script>
 $(frm).on("submit", function(e){
 e.preventDefault();
 const hours=$(frm.hours).val();
 $.ajax({
 type:"get",
 url:"/ai/linear",
 data: {hours: hours},
 success:function(data){
 $("#predict").html(`${hours}시간 공부했을 때 예측점수는 ${data}점 입니다.`)
 }
 })
 });
</script>
{%endblock%}
```

- 다음은 선형회귀 모델링한 그래프와 공부시간을 입력하면 점수를 예측하는 결과이다.

## 선형회귀

점수예측

10시간 공부했을 때 예측점수는 104.22점 입니다.



- 다항회귀 페이지를 작성한다.

[web] aiRoute.py

```
from flask import Blueprint, render_template, request, send_file

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

from io import BytesIO
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
matplotlib.use('Agg') #서버에서 GUI없이 그래프 생성가능
...

@apiBlueprint.route('/page2') #/ai/page2
def ai_page2():
 return render_template('./ai/page2.html')

@apiBlueprint.route('/poly') #/ai/poly?hours=5
def ai_poly():
 hours = float(request.args['hours'])
 dataset = pd.read_csv('../data/PolynomialRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values
 poly_reg = PolynomialFeatures(degree=2)
 X_poly = poly_reg.fit_transform(X)
 reg = LinearRegression()
 reg.fit(X_poly, y)
 y_predict= reg.predict(poly_reg.fit_transform([[hours]]))
 return str(round(y_predict[0], 2))

@apiBlueprint.route('/poly/graph') #/ai/poly/graph
def ai_poly_graph():
 dataset = pd.read_csv('../data/PolynomialRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values
 poly_reg = PolynomialFeatures(degree=2)
 X_poly = poly_reg.fit_transform(X)
 reg = LinearRegression()
 reg.fit(X_poly, y)

 plt.figure(figsize=(10, 5))
 plt.scatter(X, y, color='blue')
 X_range = np.arange(min(X), max(X), 0.1)
 X_range=X_range.reshape(-1, 1)

 plt.plot(X_range, reg.predict(poly_reg.fit_transform(X_range)), color='green')
 plt.title('Score by hours (genius)')
 plt.xlabel('hours')
 plt.ylabel('score')

 img = BytesIO()
 plt.savefig(img, format='png', dpi=200)
 plt.close()
 img.seek(0)
 return send_file(img, mimetype='image/png')
```

```
{%extends 'index.html'%}
{%block main_area%}
<div>
 <h3 class="text-center mb-5">다항회귀</h3>
 <div class="row justify-content-center">
 <form name="frm" class="col-6">
 <div class="input-group">
 <input name="hours" class="form-control" placeholder="공부시간">
 <button class="btn btn-primary">점수예측</button>
 </div>
 </form>
 <div class="mt-3">
 <h5 id="predict" class="text-center"></h5>
 </div>
 <div class="mb-3 text-center">

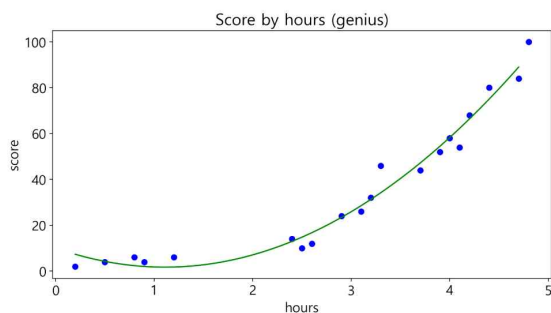
 </div>
 </div>
</div>
</div>
<script>
 $(frm).on("submit", function(e){
 e.preventDefault();
 const hours=$(frm.hours).val();
 $.ajax({
 type:"get",
 url:"/ai/poly",
 data: {hours: hours},
 success:function(data){
 console.log(data);
 $("#predict").html(`${hours}시간 공부했을 때 예측점수는 ${data}점 입니다.`)
 }
 })
 });
</script>
{%endblock%}
```

- 다음은 다항회귀 모델링한 그래프와 공부시간을 입력하면 점수를 예측하는 결과이다.

## 다항회귀

점수예측

7시간 공부했을 때 예측점수는 236.83점 입니다.



- 다중 선형회귀 페이지를 작성한다.

[web] aiRoute.py

```
from flask import Blueprint, render_template, request, send_file
...

@aiBlueprint.route('/page3') #/ai/page3
def ai_page3():
 return render_template('./ai/page3.html')

@aiBlueprint.route('multi') #/ai/multi?hours=5&absence=2&place=1
def ai_multi():
 hours = float(request.args['hours'])
 absence = int(request.args['absence'])
 place = int(request.args['place'])

 if place == 2:
 p1=1
 p2=0
 elif place == 1:
 p1=0
 p2=1
 else:
 p1=0
 p2=0

 dataset = pd.read_csv('../data/MultipleLinearRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values

 #원-핫 인코딩
 from sklearn.compose import ColumnTransformer
 from sklearn.preprocessing import OneHotEncoder

 ct=ColumnTransformer(transformers=[('encoder', OneHotEncoder(drop='first'), [2])], remainder='passthrough')
 X = ct.fit_transform(X)
 reg = LinearRegression()
 reg.fit(X, y)
 y_predict = reg.predict([[p1, p2, hours, absence]])
 return str(round(y_predict[0], 2))
```

- 다음은 다중 선형회귀로 공부시간과 결석횟수를 입력하면 점수를 예측하는 결과이다.

### 다중선형회귀

장소	집	▼
공부시간	6	
결석횟수	2	
점수예측		
예측점수:58.2점입니다.		

```

{%extends 'index.html'%}
{%block main_area%}
 <style>
 span {
 width: 150px;
 }
 </style>
 <div class="row justify-content-center">
 <h3 class="text-center mb-5">다중선형회귀</h3>
 <form name="frm" class="col-md-8 col-lg-6 card p-3">
 <div class="input-group mb-3">
 장소
 <select name="place" class="form-select">
 <option value="2">집</option>
 <option value="1">도서관</option>
 <option value="0">카페</option>
 </select>
 </div>
 <div class="input-group mb-3">
 공부시간
 <input name="hours" class="form-control" value="0">
 </div>
 <div class="input-group mb-3">
 결석횟수
 <input name="absence" class="form-control" value="0" type="number" step="1">
 </div>
 <div>
 <button class="btn btn-primary w-100">점수예측</button>
 </div>
 <div class="text-center my-3">
 <h5 id="predict"></h5>
 </div>
 </form>
 </div>
 <script>
 $(frm).on("submit", function(e){
 e.preventDefault();
 let place=$(frm.place).val();
 let hours=$(frm.hours).val();
 let absence=$(frm.absence).val();
 $.ajax({
 type:"get",
 url:"/ai/multi",
 data:{place:place, hours:hours, absence:absence},
 success:function(data){
 console.log(data)
 $("#predict").html('예측점수: ${data} 점입니다.')
 }
 });
 });
 </script>
{%endblock%}

```

- 로지스틱 회귀 페이지를 작성한다.

[web] aiRoute.py

```
from flask import Blueprint, render_template, request, send_file

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
...

@aiBlueprint.route('/page4') #/ai/page4
def ai_page4():
 return render_template('./ai/page4.html')

@aiBlueprint.route('/logistic') #/ao/logistic?hour=5
def ai_logistic():
 hours = float(request.args['hours'])
 dataset = pd.read_csv('../data/LogisticRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values

 classifier = LogisticRegression()
 classifier.fit(X, y)

 y_predict=classifier.predict([[hours]])
 p_predict=classifier.predict_proba([[hours]])
 y=int(y_predict[0])

 p=round(float(p_predict[0,1])*100,2)
 result = {"y":y, "p":p}
 return result

@aiBlueprint.route('/logistic/graph') #/ai/logistic/graph
def ai_logistic_graph():
 dataset = pd.read_csv('../data/LogisticRegressionData.csv')
 X = dataset.iloc[:, :-1].values
 y = dataset.iloc[:, -1].values

 classifier = LogisticRegression()
 classifier.fit(X, y)
 X_range = np.arange(min(X), max(X), 0.1)
 p = 1/(1 + np.exp(-(classifier.coef_ * X_range + classifier.intercept_)))
 p = p.reshape(-1)

 plt.figure(figsize=(10, 5))
 plt.scatter(X, y, color='blue')
 plt.plot(X_range, p, color='green')
 plt.plot(X_range, np.full(len(X_range), 0.5), color='red')
 plt.title('Probability by hours')
 plt.xlabel('hours')
 plt.ylabel('p')

 img = BytesIO()
 plt.savefig(img, format='png', dpi=200)
 plt.close()
 img.seek(0)
 return send_file(img, mimetype='image/png')
```

```
{%extends 'index.html'%}
{%block main_area%}
<div>
 <h3 class="text-center mb-5">로지스틱회귀</h3>
 <div class="row justify-content-center">
 <form name="frm" class="col-6">
 <div class="input-group">
 <input name="hours" class="form-control" placeholder="공부한시간">
 <button class="btn btn-primary">합격예측</button>
 </div>
 <div class="text-center my-3"><h5 id="predict"></h5></div>
 </form>
 <div class="mt-3"><h5 id="predict" class="text-center"></h5></div>
 <div class="mb-3 text-center"></div>
 </div>
</div>
<script>
 $(frm).on("submit", function(e){
 e.preventDefault();
 hours = $(frm.hours).val();
 if(hours == ''){
 alert("시간을 입력하세요!"); $(frm.hours).focus(); return;
 }
 $.ajax({
 type:"get",
 url:"/ai/logistic",
 dataType:'json',
 data:{hours:hours},
 success:function(data){
 if(data.y==1) $("#predict").html(`합격을 예측합니다. (합격률:${data.p}%)`);
 else $("#predict").html(`불합격을 예측합니다. (합격률:${data.p}%)`);
 }
 });
 });
</script>
{%endblock%}
```

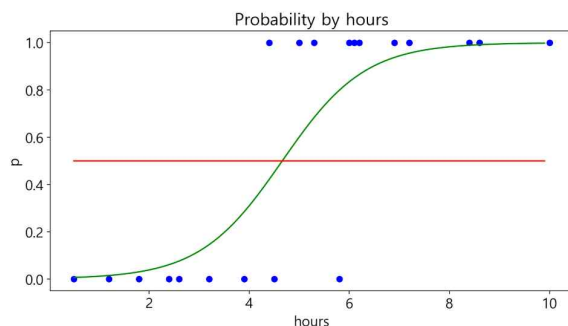
- 다음은 로지스틱회귀로 공부시간을 입력하면 합격여부를 예측한다.

## 로지스틱회귀

4

합격예측

불합격을 예측합니다. (합격률:30.98%)



- K-Mean 모델을 이용하여 최적의 클러스터를 구한 후 클러스터 수로 군집화 한다.

[web] kmeanRoute.py

```
from flask import Blueprint, render_template, request, send_file

import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

from io import BytesIO
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
matplotlib.use('Agg') #서버에서 GUI없이 그래프 생성가능

kmeanBluePrint = Blueprint('kmean', __name__)

@kmeanBluePrint.route('/page5') #/kmean/page5
def ai_page5():
 return render_template('./ai/page5.html')

@kmeanBluePrint.route('/cluster/graph') #/kmean/cluster/graph
def ai_cluster_route():
 plt.cla()
 dataset = pd.read_csv('../data/KMeansData.csv')
 X = dataset.iloc[:, :].values
 sc = StandardScaler()
 X = sc.fit_transform(X)

 inertia_list = []
 for i in range(1, 11):
 kmenas = KMeans(n_clusters=i, init='k-means++', random_state=0, n_init=10)
 kmenas.fit(X)
 inertia_list.append(kmenas.inertia_)

 plt.plot(range(1, 11), inertia_list)
 plt.title('Elow Method')
 plt.xlabel('n_cluster')
 plt.ylabel('inertia')

 img = BytesIO()
 plt.savefig(img, format='png', dpi=100)
 plt.close()
 img.seek(0)
 return send_file(img, mimetype='image/png')
```

[web] app.py

```
from flask import Flask, render_template
import aiRoute
import kmeanRoute

app = Flask(__name__, template_folder='templates')
app.register_blueprint(aiRoute.aiBlueprint, url_prefix='/ai')
app.register_blueprint(kmeanRoute.kmeanBlueprint, url_prefix='/kmean')

@app.route('/')
def index():
 return render_template('index.html', title='홈페이지')

if __name__ == '__main__':
 app.run(port=5000, debug=True)
```



[web] kmeanRoute.py

```
from flask import Blueprint, render_template, request, send_file

import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

from io import BytesIO
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
matplotlib.use('Agg') #서버에서 GUI없이 그래프 생성가능

kmeanBlueprint = Blueprint('kmean', __name__)

@kmeanBlueprint.route('/page5') #/kmean/page5
def ai_page5():
 return render_template('./ai/page5.html')

@kmeanBlueprint.route('/cluster/graph') #/kmean/cluster/graph
def ai_cluster_route():
 plt.cla()
 dataset = pd.read_csv('../data/KMeansData.csv')
 X = dataset.iloc[:, :].values
 ...

@kmeanBlueprint.route('/graph/<num>') #/kmean/graph/4
def kmean(num):
 plt.cla()
 dataset = pd.read_csv('../data/KMeansData.csv')
 X = dataset.iloc[:, :].values
 sc = StandardScaler()
 X = sc.fit_transform(X)
 K = int(num)

 kmeans = KMeans(n_clusters=K, random_state=0, n_init=10)
 y_kmeans = kmeans.fit_predict(X)

 centers = kmeans.cluster_centers_
 X_org = sc.inverse_transform(X)
 centers_org = sc.inverse_transform(centers)

 for cluster in range(K):
 plt.scatter(X_org[y_kmeans==cluster, 0], X_org[y_kmeans==cluster, 1], s=100, edgecolor='black')
 plt.scatter(centers_org[cluster,0], centers_org[cluster,1], s=300, edgecolor='black', color='yellow', marker='s')
 plt.text(centers_org[cluster,0], centers_org[cluster,1], cluster, va='center', ha='center')

 plt.title('Scores by hours')
 plt.xlabel('hours')
 plt.ylabel('score')

 img = BytesIO()
 plt.savefig(img, format='png', dpi=200)
 plt.close()
 img.seek(0)
 return send_file(img, mimetype='image/png')
```

```

{%extends 'index.html'%}
{%block main_area%}
 <div>
 <h3 class="text-center mb-5">K-평균</h3>
 <div class="text-center">
 <div class="row m-5">
 <div class="col-6">
 <button class="btn btn-primary" id="btn_cluster">Cluster</button>
 <div class="text-center mt-3"></div>
 </div>
 <div class="col-6">
 <button class="btn btn-primary" id="btn_kmean">K-Mean</button>
 <select name="cluster" class="p-2" id="num">
 <option>1</option>
 <option>2</option>
 <option>3</option>
 <option selected>4</option>
 <option>5</option>
 </select>
 <div class="text-center mt-3"></div>
 </div>
 </div>
 </div>
 </div>
</div>
<script>
 $("#btn_cluster").on("click", function(){
 $("#graph").attr("src", "/kmean/cluster/graph")
 });

 $("#btn_kmean").on("click", function(){
 const num = $("#num").val();
 $("#graph2").attr("src", "/kmean/graph/" + num)
 });
</script>
{%endblock%}

```

- K-평균의 실행 결과는 아래와 같다.

## K-평균

